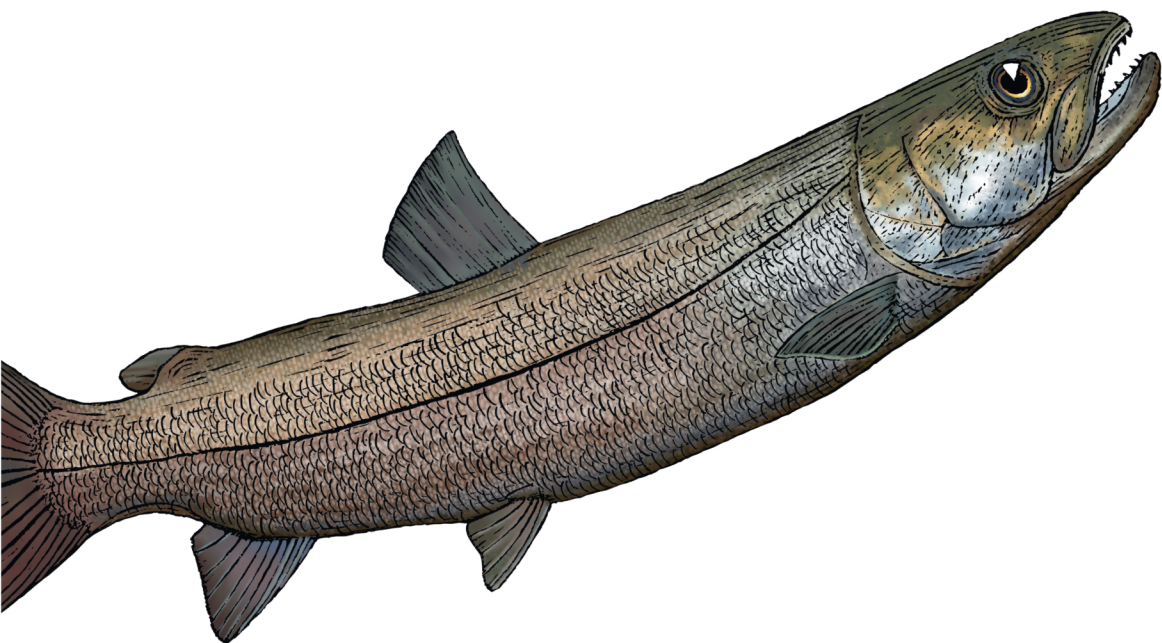


O'REILLY®



Потоковые базы данных

Объединение пакетной
и потоковой обработки

SPRINT
book

Хьюберт Дюлей
Ральф М. Дебусманн

Streaming Databases

Unifying Batch and Stream Processing

Hubert Dulay, Ralph Matthias Debusmann

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Потоковые базы данных

Объединение пакетной и потоковой обработки

Хьюберт Дюлей и Ральф М. Дебусманн

SPRINT
book

2025

Выпущено
при поддержке

КРОК

Дюлей Хьюберт, Ральф М. Дебусманн

Потоковые базы данных

Перевел с английского С. Черников

Научный редактор Меркушин Д.

ББК 32.973.233.02

УДК 004.65

Дюлей Хьюберт, Дебусманн Ральф М.

Д95 Потоковые базы данных. — Астана: «Спринт Бук», 2026. — 272 с.: ил.

ISBN 978-601-12-3659-1

В наши дни приложения реального времени стали нормой. Но для построения корректно работающей модели требуется, чтобы данные обрабатывались на лету и анализировались с низкой задержкой. Из этой практической книги инженеры, архитекторы и аналитики данных узнают, как использовать потоковые базы данных для создания решений, действующих в режиме реального времени.

Авторы, Хьюберт Дюлей и Ральф М. Дебусманн, познакомят вас с основами потоковых баз данных и кроме всего прочего расскажут, как эти базы данных позволяют оптимизировать инфраструктуру. Вы узнаете о различиях между потоковыми базами данных, потоковой обработкой и аналитической обработкой в реальном времени (OLAP), а также о том, когда следует использовать push- и pull-запросы и как обслуживать синхронные и асинхронные данные, обрабатываемые потоковыми базами данных.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

ISBN 978-1098154837 англ.

Authorized Russian translation of the English edition of Streaming Databases.
ISBN 978-1098154837 © 2024 Hubert Dulay and Ralph M. Debusmann.
This translation is published and sold by permission of O'Reilly Media, Inc.,
which owns or controls all rights to publish and sell the same.

ISBN 978-601-12-3659-1

© Перевод на русский язык ТОО «Спринт Бук», 2025
© Издание на русском языке, оформление ТОО «Спринт Бук», 2025

Изготовлено в России. Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес
010000, Казахстан, город Астана, район Алматы, проспект Рахымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 10.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 27.08.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 21,930. Тираж 700. Заказ 0000.

Краткое содержание

https://t.me/IT_Portal

Введение	12
Предисловие	15
От издательства	18
Глава 1. Основы потоковой обработки	20
Глава 2. Платформы обработки потоков	40
Глава 3. Предоставление данных в режиме реального времени	61
Глава 4. Материализованные представления.....	81
Глава 5. Поточковые базы данных	103
Глава 6. Согласованность	124
Глава 7. Появление других гибридных систем данных	158
Глава 8. Zero-ETL или Near-Zero-ETL.....	174
Глава 9. Плоскость потоковой обработки	197
Глава 10. Модели развертывания.....	215
Глава 11. Будущее данных в режиме реального времени	233
Об авторах	267
Иллюстрация на обложке.....	268
Алфавитный указатель	269

Оглавление

Введение	12
Предисловие	15
Условные обозначения.....	15
Использование примеров кода	16
Благодарности от Хьюберта.....	17
Благодарности от Ральфа	17
От издательства	18
О научном редакторе русскоязычного издания	18
Словарь терминов.....	18
Глава 1. Основы потоковой обработки	20
Выворачиваем базу данных наизнанку	22
Вынос функциональности базы данных за ее пределы	22
Механизм упреждающей записи.....	23
Платформы потоковой обработки.....	25
Материализованные представления	29
Пример практического применения: анализ потоков цифровых следов пользователей.....	30
Изучение транзакций и событий.....	31
Предметно-ориентированное проектирование	31
Обогащение контекста	33
Захват изменения данных.....	33
Коннекторы	35
Промежуточное программное обеспечение коннекторов	36
Встроенные коннекторы	37
Специально разработанные коннекторы	37
Резюме	39

Глава 2. Платформы обработки потоков	40
Преобразования с состоянием.....	42
Конвейеры обработки данных.....	45
Ограничения ELT	48
Обработка потоков с помощью ELT	49
Потоковые процессоры	50
Популярные потоковые процессоры.....	51
Новые потоковые процессоры.....	51
Эмуляция материализованных представлений в Apache Spark.....	53
Два типа потоков.....	53
Поток с добавлением	55
Изменение данных с Debezium	56
Материализованные представления	57
Резюме	60
Глава 3. Предоставление данных в режиме реального времени	61
Ожидаемый результат работы в режиме реального времени.....	62
Выбор аналитического хранилища данных.....	63
Топик в качестве источника	65
Преобразования при поглощении	66
OLTP и OLAP	68
ACID	69
Строчная и столбцовая оптимизации	70
Запросы в секунду и конкурентность.....	71
Индексирование.....	72
Предоставление результатов анализа	76
Синхронные запросы.....	76
Асинхронные запросы	77
Запросы push и pull.....	78
Резюме	80
Глава 4. Материализованные представления.....	81
Представления, материализованные представления и инкрементные обновления.....	82
Захват изменения данных.....	84
Запросы push и pull.....	86

CDC и Upsert	91
Объединение потоков	94
Apache Calcite	95
Пример использования цифрового следа	99
Резюме	101
Глава 5. Потокowe базы данных	103
Определение потоковой базы данных	104
Потоковая база данных на основе столбцов	107
Потоковая база данных на основе строк	108
Граничные базы данных, подобные потоковым	111
Выразительность SQL	111
Возможности отладки потоковой обработки	114
Преимущества отладки в потоковых базах данных	115
SQL не панацея	115
Реализации потоковых баз данных	116
Архитектура потоковой базы данных	117
Конвейеры ELT с потоковыми базами данных	121
Резюме	122
Глава 6. Согласованность	124
Воображаемый пример	125
Транзакции	126
Анализ транзакций	127
Сравнение согласованности между системами обработки потоков	128
Flink SQL	128
ksqlDB	132
Proton (Timeplus)	135
RisingWave	138
Materialize	140
Pathway	142
Выход за рамки согласованности в конечном счете	145
Почему согласованные в конечном счете потоковые процессоры не справляются с рассмотренным примером?	145
Как внутренне согласованные системы обработки потоков выполняют рассматриваемый пример?	149

Как можно исправить системы обработки потоков с согласованностью в конечном счете, чтобы выполнить рассматриваемый пример?	152
Согласованность и задержка.....	156
Резюме	157
Глава 7. Появление других гибридных систем данных	158
Плоскости данных.....	159
Гибридная транзакционно-аналитическая база данных.....	161
Другие гибридные базы данных	165
Мотивы создания гибридных систем	165
Влияние PostgreSQL на гибридные базы данных.....	167
Выполнение аналитики вблизи границы плоскостей данных	168
Гибридные базы данных нового поколения.....	169
Потоковые БД OLTP нового поколения	170
Потоковые БД RTOLAP нового поколения.....	172
Базы данных HTAP нового поколения.....	172
Резюме	173
Глава 8. Zero-ETL или Near-Zero-ETL.....	174
Модель ETL.....	174
Zero-ETL	175
Near-zero-ETL	178
PeerDB	178
Proton.....	180
Встроенные OLAP	181
Гравитация и репликация данных	186
Сокращение аналитических данных.....	186
Лямбда-архитектура	187
Гибридные таблицы Apache Pinot	189
Конфигурации конвейеров.....	194
Резюме	196
Глава 9. Плоскость потоковой обработки	197
Гравитация данных.....	198
Компоненты потоковой плоскости.....	200
Инфраструктура потоковой плоскости	202

Операционная аналитика	203
Сетка данных	206
Столпы сетки данных	207
Трудности применения сетки данных	209
Потоковая сетка данных с потоковой плоскостью и потоковыми БД	210
Локальность данных	211
Репликация данных	212
Резюме	214
Глава 10. Модели развертывания	215
Согласованная потоковая база данных	216
Согласованный потоковый процессор и RTOLAP	218
Потоковая БД OLAP с согласованностью в конечном счете	219
Потоковый процессор с согласованностью в конечном счете и RTOLAP	220
Потоковый процессор с согласованностью в конечном счете и HTAP	221
ksqlDB	222
Инкрементное обслуживание представлений	223
Обертка сторонних данных Postgres Multicorn	224
Когда следует использовать потоковые процессоры на основе кода	225
Когда следует использовать технологии Lakehouse/Streamhouse	225
Технологии кэширования	226
Где выполнять обработку и запросы в общем случае	227
Четыре вопроса «Где?»	227
Аналитический сценарий использования	228
Последствия	230
Резюме	232
Глава 11. Будущее данных в режиме реального времени	233
Слияние плоскостей данных	234
Графовые базы данных	235
Memgraph	236
thatDot/Quine	237
Векторные базы данных	240
Milvus 2.x — потоковая передача как основа	241
Базы данных RTOLAP — добавление векторного поиска	243

Инкрементное обслуживание представлений (IVM)	244
pg_ivm	245
Hydra	245
Epsio	246
Feldera	247
PeerDB	248
Обертывание данных и Postgres Multicorn	250
Классические базы данных	253
Хранилища данных	255
BigQuery	255
Redshift	256
Snowflake	257
Lakehouse	259
Delta Lake	260
Apache Paimon	261
Apache Iceberg	262
Apache Hudi	263
OneTable или XTable	264
Взаимоотношения потоковой обработки и хранилищ Lakehouse	264
Резюме	266
Об авторах	267
Иллюстрация на обложке	268
Алфавитный указатель	269

Введение

Множество инженеров мечтают стоять у истоков новой категории программных систем. Я очень рад, что мне удалось поучаствовать в работе над системой управления базами данных (СУБД) `ksqlDB` в самом начале, еще до того, как она получила свое название, и прежде, чем потоковые базы данных стали общеизвестными. Меня сразу заинтересовала новость о том, что Ральф и Хьюберт пишут книгу о потоковых базах данных (streaming databases).

Что же такое потоковые базы данных (БД)? Существует множество видов систем баз данных, от традиционных реляционных БД до XML, графовых, объектных, векторных и NoSQL. Многие из них хорошо изучены и используются в работе уже пару десятилетий. Потоковая обработка — гораздо менее устоявшийся подход, хотя последние десять лет она становится все более распространенной, в первую очередь из-за популярности Apache Kafka, которая де-факто является платформой потоковой передачи сообщений.

Потоковая обработка всегда считалась сложной, и освоить ее могли только крупные организации, имеющие в штате экспертов в этой области. То же самое происходило с обработкой данных и вычислениями 50 лет назад, до изобретения SQL и реляционных систем баз данных, позволяющих неспециалистам работать с данными в компьютерных системах. Сейчас SQL — это *лингва франка* для обработки данных.

Потоковые базы данных — новый этап эволюции потоковой обработки. Они объединяют хорошо зарекомендовавшие себя методы работы систем баз данных с новыми парадигмами из мира потоковых технологий. Благодаря этому получилось упростить обработку потоков и дать неспециалистам возможность работать с данными так, как будто они запрашивают их в состоянии покоя¹.

Системы баз данных предназначены для решения особых задач. Две основные категории систем баз данных — *системы обработки транзакций в режиме реального времени* (OnLine Transaction Processing, OLTP) и *системы аналитической обработки в режиме реального времени* (OnLine Analytical Processing, OLAP) — изначально не были рассчитаны на работу с приложениями такого масштаба, который позволяет современный Интернет.

¹ Термин будет пояснен в дальнейшем. — *Примеч. пер.*

С появлением концепции больших данных в начале III тысячелетия были изобретены новые системы, например MapReduce, чтобы удовлетворить возросшие требования к масштабированию. Однако эти системы были разработаны техническими специалистами для технических специалистов.

С появлением озер данных, первенца эпохи больших данных, быстро пришло осознание того, что необходим SQL, дающий возможность неспециалистам максимально эффективно использовать эти новые технологии. В результате SQL был вновь введен в обиход, и сегодня все современные озера данных используют его для выполнения запросов.

Потоковая обработка данных как второе дитя эпохи больших данных следовала той же тенденции — изначально такие системы создавались профессионалами для профессионалов без поддержки SQL. Вскоре появились технологии SQL и баз данных, позволяющие неспециалистам использовать новые потоковые системы. Эти разработки привели к появлению потоковых баз данных и последующим волнам инноваций.

Сегодня все больше людей осознают значимость потоковых баз данных в сфере потоковой обработки и технологий БД, и им требуется руководство по использованию таких БД в существующих системах. В этой книге рассказывается о том, как потоковая обработка добавляет новую плоскость данных между *операционной* (OLTP) и *аналитической* (OLAP) плоскостями. Она носит название *потоковой* и открывает богатые возможности для развития систем передачи данных. В книге Хьюберт и Ральф описывают три различные отправные точки для создания потоковых баз данных:

- системы обработки потоков на основе технологий баз данных и SQL;
- системы баз данных, расширяемые для внедрения концепций потоковой обработки;
- озера данных (уже использующие язык SQL), расширяемые для реализации возможностей потоковой передачи данных.

Эти три отправные точки привели к появлению множества различных потоковых БД. Но у каждой из них есть ограничения, и они оптимизированы для различных сценариев применения. В связи с этим возникает вопрос: какую систему использовать в том или ином случае и каковы компромиссы?

Если верить предсказанию Джея Крепса о том, что компании будут становиться все более ориентированными на программное обеспечение, нас ждет захватывающее будущее в области обработки данных и его основу составят потоковые БД. Они совместно с потоковым SQL предлагают некоторые упрощения, благодаря чему гораздо больше неспециалистов осваивают потоковую обработку. И это приведет к повсеместному распространению потоковых данных.

Мы все еще находимся в начале эры потоковых БД, и очень интересно наблюдать за современными тенденциями, изучать новые системы.

Эта книга послужит отличной отправной точкой для знакомства с передовыми достижениями и многообразием вариантов, характерным для первых дней новой эры. Еще больше информации о потоковых базах данных можно получить из подкаста Хьюберта и Ральфа на Spotify, который называется просто *Hubert's Podcast*. При подготовке материалов они взяли интервью у многих людей, работающих в сфере потоковых технологий и данных, и книга сама по себе стала жемчужиной.

*Матиас Дж. Сакс, технический руководитель
команды разработчиков Kafka Streams в компании
Confluent Apache Committer и член комитета
управления проектом (Kafka, Flink, Storm)*

Рино, Невада, май 2024 года

Предисловие

В этой книге мы выйдем за рамки традиционной пакетной обработки и плавно войдем в динамичный мир потоковых данных. Если вы уже работаете в сфере потоковой обработки, то мы поможем рассмотреть ее со стороны базы данных. Потоковые БД позволяют преодолеть разрыв между данными в состоянии покоя и данными в движении.

Черпая вдохновение в основополагающей работе Мартина Клеппмана о выворачивании базы данных наизнанку, мы рассмотрим перенос потоковых систем обратно в базу данных. Благодаря такому изменению парадигмы сможем сначала разобраться в сложных слоях потоковой обработки, а затем найти знакомые абстракции, которые сделают потоковую обработку в режиме реального времени более доступной и понятной для разработчиков независимо от их уровня знаний об этих технологиях.

Мы изучим основные принципы построения потоковых баз данных. Покажем, как они позволяют разработчикам решать задачи обработки данных в режиме реального времени в рамках привычной среды. Уделяя особое внимание практичности и юзабилити, расскажем, как потоковые БД предоставляют доступ к данным и их аналитику в режиме реального времени.

Эта книга поможет как опытным инженерам, так и начинающим разработчикам раскрыть весь потенциал потоковых БД и окунуться в будущее обработки данных.

Условные обозначения

В этой книге используются следующие условные обозначения.

Курсив

Курсивом выделены новые термины и важные понятия.

Моноширинный шрифт

Применяется для листингов программного кода, а также внутри абзацев для обозначения таких элементов программы, как имена переменных или функций, базы данных, типы данных, переменные среды, операторы, ключевые слова, пути, имена файлов и их расширений.

Жирный моноширинный шрифт

Показывает команды или другой текст, который пользователь должен ввести самостоятельно.

Курсивный моноширинный шрифт

Применяется для оформления текста, который должен быть заменен значениями, предоставляемыми пользователем или определяемыми контекстом.

Шрифт без засечек

Применяется для обозначения папок, каталогов, URL и адресов электронной почты.



Так обозначаются замечания общего характера.



Так обозначаются предупреждения или предостережения.

Использование примеров кода

Дополнительный материал (примеры кода, упражнения и т. д.) можно загрузить по адресу <https://github.com/hdulay/streaming-databases>.

Если у вас есть технический вопрос или проблема с использованием примеров кода, отправьте электронное письмо по адресу support@oreilly.com.

В общем случае все примеры кода из книги вы можете использовать в своих программах и документации. Вам не нужно обращаться в издательство за разрешением, если вы не собираетесь воспроизводить существенные части программного кода. Если вы разрабатываете программу и используете в ней несколько фрагментов кода из книги, вам не нужно обращаться за разрешением. Но для продажи или распространения примеров из книги вам потребуется разрешение от издательства O'Reilly. Вы можете отвечать на вопросы, цитируя данную книгу или примеры из нее, но для включения существенных объемов программного кода из книги в документацию вашего продукта потребуется разрешение.

Мы рекомендуем, но не требуем добавлять ссылку на первоисточник при цитировании. Под ссылкой на первоисточник мы подразумеваем указание авторов, издательства и ISBN.

За получением разрешения на использование значительных объемов программного кода из книги обращайтесь по адресу permissions@oreilly.com.

Благодарности от Хьюберта

Прежде всего я хотел бы поблагодарить свою жену Бет и детей Астера и Нико за то, что они поддерживали меня, пока я писал эту книгу. Без них это было бы непросто. Хочу также сказать спасибо Ральфу: он стал для меня замечательным учителем и талантливым соавтором, благодаря чему мы составили отличную писательскую пару.

Когда мы только начинали писать эту книгу, мы опросили многих экспертов-новаторов и ведущих специалистов в области потоковой обработки и аналитики в режиме реального времени и, что еще важнее, в сфере их внедрения. Спасибо Сету Висману, Арджуну Нараяну и Фрэнку Макшерри за их советы, а также Нихилу Бенешу, который разыскал меня в компании Curgent. Спасибо Уиллу Пламмеру за то, что обратился к нам, Джове Чжуну — за рецензию на книгу, а также Ган Тао и Тин Вангу — за постоянную поддержку. Я также хотел бы поблагодарить Инцзюнь Ву за мудрость и за рецензию на книгу. Спасибо Адриану Косовски, Анупу Сурендрану и Бобуру Умурзокову за их постоянное сотрудничество и поддержку. Спасибо Ходжату Джафарпуру и Монишу за приятные беседы. Спасибо Михая Будию и Леониду Рыжику за предварительный разговор с нами и цитату: «Все базы данных — это потоковые базы данных». Спасибо Мике Уайлду, Ричарду Артулу и Райану Уорлу за интересные беседы. Спасибо Робину Феру и Нико Крубелу за рецензии на книгу. Спасибо Матиасу Саксу за то, что написал введение к этой книге. Спасибо Рите Фернандо, благодаря которой писать для O'Reilly легко и весело. И наконец, спасибо другим специалистам по потоковой обработке и базам данных, которые обеспечивают своим клиентам потоковую передачу и аналитику в режиме реального времени.

Благодарности от Ральфа

Прежде всего я хотел бы поблагодарить Беа за поддержку (не только в подготовке этой книги), моих родителей и детей Софи, Стеллу и Селену. Огромное спасибо Хьюберту за возможность стать соавтором этой книги и прекрасное время, проведенное вместе за ее написанием. Хочу также поблагодарить его коллег из Migros, с которыми я имел удовольствие сотрудничать и обсуждать темы, связанные с этой книгой, особенно Мартина Маггли, Джейсона Нгуена, Симона Хофера, Александра Ровнера, Андре Пехштейна, Эрика Видо и Филиппа Джуда де Капитани. Спасибо всем, кто принял участие в создании нашей книги — предоставил ценные идеи и отзывы, а также участвовал во вдохновляющих дискуссиях. Помимо уже упомянутых, сюда входят Джейми Брэндон, Паван Кешавамурти, Джаннис Полисос, Флоран Рамиер, Майкл Розам и Ярослав Ткаченко, причем этот список далеко не полный.

От издательства

Мы выражаем огромную благодарность компании КРОК за помощь в работе над русскоязычным изданием книги и их вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@sprintbook.kz (издательство SprintBook, компьютерная редакция).

Мы будем рады узнать ваше мнение!

О научном редакторе русскоязычного издания

Денис Меркушин работает в компании КРОК, два года руководит командой Java-разработчиков. Почти десять лет создает информационные системы, регулярно участвует в трансформации процессов разработки внутри компании. Преподаватель в BrainZ by CROC, предлагающей программы обучения в сфере IT для студентов и школьников.

Словарь терминов

В этой книге используется множество терминов. Перевод части из них устоялся в индустрии, но какие-то термины разные эксперты могут переводить по-своему. При этом в рамках данной книги хочется сфокусироваться на сути терминов, а не на нюансах их перевода. Надеемся, что каждый читающий сможет понять, о чем идет речь, и следить за смыслом текста. Для большего удобства приводим словарь терминов.

На языке оригинала	В книге	Синонимы
(Analytical) Plane	(Аналитическая) плоскость	(Аналитический) слой, область
Append-only stream	Поток только с добавлением	Поток с поддержкой только добавления, дополняемый поток
Data reduction	Сокращение данных	Редукция, уменьшение данных
Eventual consistency	Согласованность в конечном счете	Конечная согласованность, отложенная согласованность
First-class citizen	Элемент первого класса	Полноценный элемент, полноправный элемент
In-memory	In-memory	В памяти, в оперативной памяти
Middleware	Промежуточное программное обеспечение	Посредническое программное обеспечение, посредник, промежуточный слой
Partition	Раздел	Партиция, разбиение
Persistence layer	Слой хранения данных	Слой постоянства, уровень сохранения, слой работы с БД, слой персистентности
Replica	Копия	Реплика
Sorted index	Отсортированный индекс	Упорядоченный индекс, индекс сортировки
Snapshot	Моментальный снимок	Снимок, снапшот, копия состояния
Stateful transformation	Преобразование с состоянием	Сохранение состояния при трансформации, состояние — зависимая трансформация
Stream processing platform	Платформа обработки потоков	Платформа потоковой обработки
Stream processor	Потоковый процессор	Потоковый обработчик, обработчик потоков

Основы потоковой обработки

https://t.me/IT_Portal

Путешествие героя всегда начинается с зова. Так или иначе должен появиться проводник, который скажет: «Оглянись, ты в Стране снов. Проснись. Пора отправляться в путешествие. Существует целый аспект твоего сознания, твоего существа, который еще не затронут. Ты здесь чувствуешь себя как дома? Что ж, для тебя этого маловато». И так все начинается.

Джозеф Кэмпбелл. Reflections on the Art of Living: A Joseph Campbell Companion

Концепция потоковой БД родилась из более чем десятилетнего опыта деятельности в сфере обработки и предоставления данных. Корни эволюции, приведшей к появлению потоковых баз данных, уходят глубже в историю развития систем управления базами данных, обработки данных и меняющихся требований цифровой эпохи. Для изучения этой эволюции мы совершим историческое путешествие по ключевым вехам развития потоковых БД.

Появление Интернета и взрывной рост объема цифровых данных в конце XX века привели к необходимости создания более масштабируемых и гибких решений для управления данными. Хранилища данных и ориентированные на пакетную обработку фреймворки, такие как Hadoop, создавались в ту эпоху для решения задач в ходе работы с большими объемами данных.

Термин «большие данные» использовался и продолжает использоваться для обозначения не только объема данных, но и всех решений хранения и обработки данных огромного объема. Большие данные не могут поместиться на одном компьютере или сервере. Приходится разделять их объем на более мелкие части одинакового размера и хранить их на нескольких компьютерах. Такие системы, как Hadoop и MapReduce, стали популярными благодаря возможности распределенного хранения и обработки данных.

Это привело к возникновению идеи применения распределенных потоков для перемещения больших объемов данных в Hadoop. Платформа Apache Kafka стала одним из сервисов обмена сообщениями для работы с большими данными. Платформа не только обеспечивала возможность переноса данных из системы в систему, но и предоставляла доступ к данным в движении — в режиме реального времени. Эта разработка породила новую волну спроса на решения для потоковых сценариев использования, работающих в режиме реального времени.

Были созданы новые технологии, такие как Apache Flink и Apache Spark, и они смогли оправдать повышенные ожидания. Это распределенные фреймворки для пакетной и потоковой обработки, способные работать с данными на многих серверах и выдавать аналитические результаты. В сочетании с Kafka это трио обеспечило решение для поддержки потоковой аналитики в режиме реального времени. Более подробно о потоковых процессорах мы поговорим в главе 2.

В середине 2010-х годов появились более простые и совершенные парадигмы потоковой передачи данных, позволяющие увеличить масштаб обработки данных в режиме реального времени. В их число вошли два новых фреймворка для обработки потоков — Apache Kafka Streams (KStreams) и Apache Samza. В KStreams и Samza впервые были воплощены материализованные представления, которые сделали поток более похожим на базу данных.

Мартин Клеппман пошел еще дальше в объединении баз данных и потоковой обработки. В своей презентации *Turning the Database Inside-Out* (<https://oreil.ly/EPsaz>) в 2015 году он описал способ реализации потоковой обработки с использованием внутренних возможностей базы данных, выведенных за ее пределы в виде потоков в режиме реального времени. Этот подход позволил создать более масштабируемые, отказоустойчивые системы обработки потоков в режиме реального времени.

Одной из проблем потоковой обработки была и остается бóльшая сложность ее применения по сравнению с пакетной. Здесь меньше абстракций и гораздо больше сложных технологий. Чтобы реализовать потоковую обработку данных для своих целей, инженеры по обработке данных должны были учесть порядок данных, согласованность для точной обработки, отказоустойчивость, надежность, масштабируемость и многое другое. Эти условия стали препятствием для команд по работе с данными, пытающихся использовать потоковую обработку. В результате большинство из них предпочли и дальше использовать базы данных для преобразования и пакетной обработки, что привело к снижению требований к производительности.

С этой книгой мы попытаемся сделать потоковую обработку более доступной для тех, кто привык работать с базами данных. Мы начнем, как и Клеппман, с рассказа о том, как вывернуть базу данных наизнанку.

Выворачиваем базу данных наизнанку

Мартин Клеппман — выдающийся разработчик программного обеспечения, который выступил с наводящим на размышления докладом *Turning the Database Inside-Out* («Выворачивая базу данных наизнанку»). Он представил вычислительную среду Apache Samza как новый способ осуществления потоковой обработки с использованием внутренних возможностей базы данных и их реализацией в потоках реального времени. Его идейное лидерство привело к изменению парадигмы — внедрению материализованных представлений в потоковую обработку.

На самом деле это скрытая попытка взять известную архитектуру баз данных и вывернуть ее наизнанку.

Мартин Клеппман. Turning the Database Inside-Out
(<https://oreil.ly/EPsaz>)

Однако потоковая обработка по-прежнему сложна, и поэтому многие инженеры по обработке данных все еще используют БД для преобразования данных и их пакетной обработки. Даже если это означает несоблюдение требований *соглашения об уровне услуг* (Service Level Agreement, SLA).

По мере развития повествования в книге мы попытаемся сделать потоковую передачу и обработку более доступными для инженеров по обработке данных, перенеся эти процессы в БД. Но прежде чем мы сможем это сделать, необходимо понять, почему Клеппман решил разобрать базу данных на части и в своей новой парадигме выбрал конкретные функции базы данных для получения возможности обработки данных в режиме реального времени.

Вынос функциональности базы данных за ее пределы

Клеппман выделил две важные функциональные возможности базы данных: *механизм упреждающей записи* (Write-Ahead Log, WAL) и материализованное представление. Как оказалось, эти функции сами по себе обладают потоковыми характеристиками, которые обеспечивают лучший способ обработки данных в режиме реального времени.

Механизм упреждающей записи

WAL — это механизм, позволяющий базам данных обеспечивать долговечность и согласованность данных. Жесткие диски для записи данных из БД не предлагают транзакций. Поэтому перед СУБД стоит задача — обеспечить выполнение транзакций на устройстве без такой возможности. Механизмы WAL предоставляют способ их реализации без использования транзакционной памяти.

Транзакция в базе данных — это последовательность одной или нескольких операций над БД, выполняемых как единое целое. Это могут быть операции вставки (INSERT), изменения (UPDATE) или удаления (DELETE) данных (рис. 1.1).

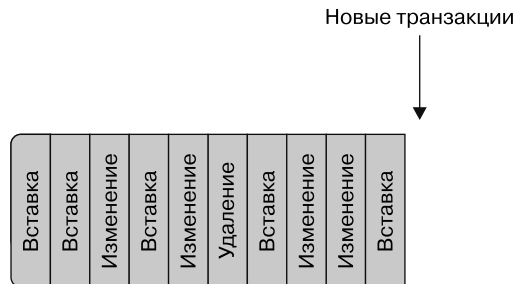


Рис. 1.1. Механизм упреждающей записи с фиксацией событий изменений в БД

Механизм WAL работает как буфер — его можно перезаписывать по мере внесения изменений. Он сохраняет изменения на диске (рис. 1.2).

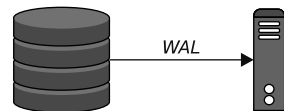


Рис. 1.2. Запись БД на диск с помощью механизма упреждающей записи

При сохранении транзакций на диске выполняются следующие действия.

1. Клиент начинает транзакцию, используя оператор `BEGIN`.
2. База данных заносит в WAL запись с указанием начала транзакции.
3. Клиент вносит изменения в данные базы данных.
4. Клиент фиксирует транзакцию, применяя оператор `COMMIT`.
5. База данных записывает в WAL запись с указанием фиксации транзакции.
6. Изменения, внесенные транзакцией, записываются на диск.

Когда транзакция начинается, база данных заносит в WAL запись, указывающую на начало транзакции. После этого СУБД приступает к внесению изменений

в данные базы. Однако изменения не записываются на диск до тех пор, пока транзакция не будет зафиксирована. Кроме того, в случае сбоя в работе БД или обрыва питания изменения можно воспроизвести из журнала и БД будет восстановлена до согласованного состояния¹.

WAL предоставляет механизм для захвата транзакций базы данных в режиме реального времени, позволяя внешним системам подписываться на него. Один из таких случаев — аварийное восстановление баз данных. Считывая WAL, можно реплицировать данные во вторичную базу данных. Если основная база данных выходит из строя, ее клиенты могут переключиться на вторичную БД — копию основной (рис. 1.3).

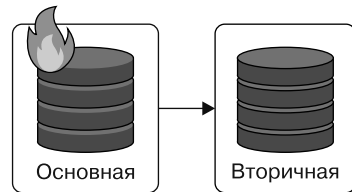


Рис. 1.3. Механизм WAL используется для репликации данных из основной базы данных во вторичную для защиты в случае выхода из строя основной БД

Поскольку журналы WAL получают транзакции в режиме реального времени, они, естественно, обладают идеальной семантикой для потоковой передачи. Клиенты могут подписаться на WAL и передавать свои транзакции на потоковую платформу, чтобы другие системы могли их использовать. И другие системы также могут создавать копии, представляющие исходную основную базу данных. Семантика конструкции WAL имитируется в потоковых платформах, таких как Kafka, в их реализации хранения данных. Потоковые платформы расширяют базы данных WAL извне, чтобы его могли использовать другие приложения и системы.

Есть и другие концепции WAL, связанные с потоковой обработкой. После фиксации транзакций журнал WAL очищается не сразу. В нем используется механизм под названием «контрольная точка» — его действие заключается в периодическом перемещении транзакций WAL в основные файлы данных. Установка контрольных точек служит нескольким целям, одна из которых — обеспечить постоянную запись некоторых зафиксированных изменений в файлы данных. Это позволяет сократить *воспроизводимый* при восстановлении после сбоя объем данных и, следовательно, ускоряет процесс восстановления. Кроме того, со временем по мере фиксации транзакций журнал WAL увеличивается. Контрольные точки помогают контролировать размер журнала, перемещая часть его содержимого в файлы данных. Таким образом, журнал WAL не становится слишком большим и не занимает слишком много места на диске. Контрольная

¹ Существуют и другие типы алгоритмов восстановления, помимо описанных нами (накат изменений/воспроизведение). Изменения в актуальных данных можно применить и до выполнения операции COMMIT (учитывая, что WAL содержит старое значение), а незафиксированные транзакции — откатить.

точка и воспроизведение транзакций — это функциональные возможности, применяющиеся в потоковой обработке по очень похожим причинам.

Конструкция WAL, обычно существующая внутри базы данных, может быть представлена во внешних потоковых платформах, таких как Kafka. Они обеспечивают подобную WAL семантику при репликации данных от системы к системе.

Платформы потоковой обработки

Платформы потоковой обработки данных, такие как Apache Kafka, — это распределенные, масштабируемые и отказоустойчивые системы для обработки потоков данных в режиме реального времени. Они представляют собой мощную инфраструктуру для поглощения больших объемов непрерывных данных из различных источников, их хранения и обработки.

В большинстве потоковых платформ есть конструкция, называемая *разделом* (partition). Эти конструкции имитируют журналы WAL из баз данных. Транзакции добавляются в разделы, как и транзакции в журналы WAL. Потоковые платформы могут содержать множество разделов для распределения потоковой нагрузки, что позволяет горизонтально масштабировать эти платформы. Разделы группируются в абстракции — *топики* (topic). Приложения либо публикуют транзакции в них, либо потребляют из них.

Публикуя транзакцию на потоковой платформе, мы делаем это для всех подписчиков. Это называется *моделью публикации и подписки*, которая позволяет нескольким отдельным потребителям использовать эти транзакции.

Для других потоковых платформ названия этих конструкций могут быть иными.

В табл. 1.1 перечислены некоторые альтернативные потоковые платформы. Apache Kafka — самая популярная из современных платформ такого типа. В ней абстракция этих конструкций называется топиком, а разделы более низкого уровня — разделами.



В книге мы будем использовать понятия «топик» (topic) и «раздел» (partition) как названия конструкций потоковой платформы для хранения потоковых данных в режиме реального времени.

Поскольку Kafka — самая популярная потоковая платформа, в последней графе табл. 1.1 указано, поддерживает ли платформа Kafka-клиентов. Это позволит заменить Kafka в приложениях на другую потоковую платформу, совместимую с Kafka.

Таблица 1.1. Альтернативные потоковые платформы

Название	Описание	Реализация	Топик	Раздел	Совместимость с Kafka
Memphis	Альтернатива традиционным брокерам сообщений с открытым исходным кодом нового поколения	GoLang	Station (Станция)	Stream (Поток)	Нет
Apache Pulsar	Платформа распределенного обмена сообщениями и потоковой передачи данных с открытым исходным кодом. Разработана в компании Yahoo!	Java	Topic (Топик)	Ledger (Леджер)	Да. Обертка Pulsar Kafka поддерживает большинство операций, предлагаемых API Kafka
Redpanda	Потоковая платформа с открытым исходным кодом. Разработана для обеспечения высокопроизводительной, масштабируемой и надежной обработки потоков данных в режиме реального времени	C++	Topic (Топик)	Partition (Раздел)	Да
WarpStream	Совместимая с Kafka потоковая платформа, созданная непосредственно на основе S3	GoLang	Topic (Топик)	Partition (Раздел)	Да
Gazette	Легкая потоковая платформа с открытым исходным кодом	GoLang	Selector (Селектор)	Journal (Журнал)	Нет
Pravega	Потоковый процессор, обеспечивающий абстракцию потокового хранения для непрерывно генерируемых и неограниченных данных	Java	Stream (Поток)	Stream Segment (Сегмент потока)	Доступен адаптер для Kafka

Деление на разделы — это механизм платформ для масштабирования. Чем больше разделов содержится в топике, тем больше можно разделить нагрузку от данных. Это позволяет большему количеству экземпляров потребителей обрабатывать транзакции параллельно. Для распределения транзакций по разделам используется присвоенный транзакции ключ. На рис. 1.4 журнал WAL из базы данных считывается и сохраняется в топике потоковой платформы — на более высоком уровне абстракции, чем просто диск.

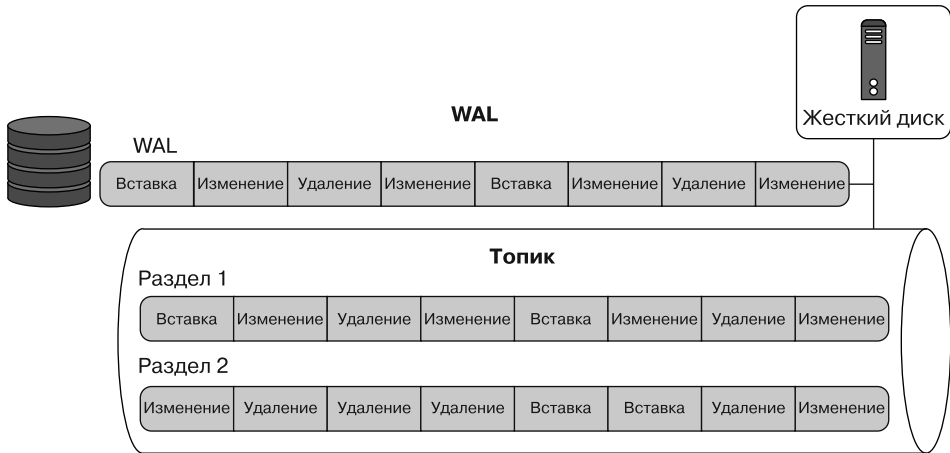


Рис. 1.4. Топики в потоковой платформе могут имитировать журнал WAL и передавать его другим системам для создания копий исходной базы данных

Вместо хранения данных для последующего запроса потоковая платформа реконструирует WAL и распределяет транзакции по отдельным разделам¹. Реконструкция WAL открывает доступ к транзакциям для других систем данных, которые смогут создать копию основной базы данных.

Разделы — это неизменяемые журналы с возможностью только добавления данных, которые используются потоковыми платформами для сбора и обслуживания транзакций. С помощью смещений на них может подписаться множество потребителей. Смещение соответствует индексу или позиции транзакции в разделе². Каждый потребитель топика содержит указатель смещения для отслеживания своей позиции в разделе. Это дает возможность потребителям читать и обрабатывать транзакции в разделе в своем темпе. Побочным эффектом такого подхода стала необходимость хранения транзакций в разделах дольше, чем базы данных хранят свои транзакции в журналах WAL. По умолчанию в Kafka срок хранения составляет семь дней, что дает медленным потребителям достаточно времени на обработку транзакций в топике. Время хранения можно настроить, чтобы еще увеличить его.

Что касается рис. 1.4, то подход к публикации транзакций в топике должен сильно отличаться от записи на диск. Важно отметить, что при публикации транзакций в топике потоковых платформ они все еще рассматриваются как

¹ Транзакции могут охватывать несколько записей. В этом случае ключ для распределения данных по разделам не совпадает напрямую с первичным ключом в исходной базе данных.

² Для простоты описания наша транзакция содержит только одну запись.

потоковые. Для лучшего понимания воспользуемся метафорой с водой. Когда вы пьете воду из крана, она кажется вам свежей. То же самое происходит и с потоковыми платформами. Когда вы потребляете транзакции из топика, они тоже считаются свежими. И наоборот, если принести в дом литр воды и не пить ее некоторое время, она будет считаться несвежей. В несвежей или застоявшейся воде размножаются бактерии, и она считается нечистой. Литр такой воды больше похож на пакетные данные.

В то же время, если не открывать краны больше месяца, в воде, поступающей из них, может появиться ржавчина или мусор — это говорит о том, что вода стала несвежей. В таком случае вода в кране не всегда будет свежей. Как правило, в потоковых платформах предусмотрен механизм, защищающий их от хранения несвежих транзакций. Чтобы избежать публикации несвежих транзакций, к топикам применяется политика хранения. Транзакция может быть удалена после окончания периода хранения, настроенного пользователем потоковой платформы.

Суммируя, напомним, что основные базы данных OLTP при хранении данных на жестких дисках ведут запись в WAL. Эти журналы можно использовать для репликации данных во вторичную базу данных OLTP в сценариях аварийного восстановления системы. Потоковые платформы, такие как Kafka, подходят для предоставления журналов базы данных WAL в другие системы с помощью абстрагированных по топикам разделов. Такие системы подписываются на топик, чтобы получить возможность построить свою копию таблиц из оригинальной первичной базы данных OLTP точно так же, как это выполнялось для вторичной базы данных OLTP (рис. 1.5). Таким образом, потоковые платформы можно использовать для того, чтобы сделать журналы WAL, ранее скрытые в системах баз данных OLTP, общедоступными и превратить их в инструмент для синхронизации систем БД всей организации.

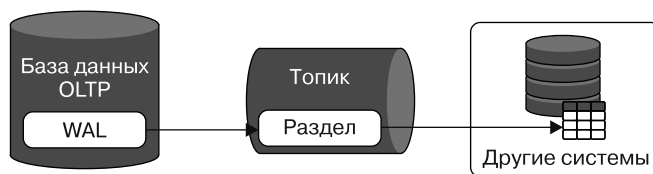


Рис. 1.5. Разделы в топике могут хранить транзакции из исходной базы данных OLTP и публиковать их для других систем с целью создания копий таблиц

Используя аналогичный подход, можно создавать материализованные представления в платформах обработки потоков.

Материализованные представления

В типичных базах данных OLTP *материализованные представления* — это специальные типы объектов БД, которые хранят результаты предварительно вычисленного запроса или агрегации. Обычные представления виртуальны и динамически генерируют свои результаты на основе базовых данных. А материализованные представления хранят фактические данные, а значит, физически хранят в базе данных.

Цель материализованных представлений — повысить производительность сложных запросов или агрегаций, предварительно вычислив и сохранив результаты. Когда запрос ссылается на материализованное представление, база данных может быстро получить предварительно вычисленные данные из этого представления вместо того, чтобы пересчитывать их на лету из таблиц. Это может значительно сократить время выполнения запроса и повысить общую производительность базы данных, особенно если запросы большие и ресурсоемкие.

Материализованные представления обычно приходится обновлять вручную, чтобы сохранить актуальность хранимых результатов. В примере 1.1 показано, как обновить материализованное представление в базе данных Postgres — популярной базе данных OLTP.

Пример 1.1. Обновление материализованного представления в Postgres

```
REFRESH MATERIALIZED VIEW CONCURRENTLY product_sales;
```

Из-за возможности обновления материализованных представлений хранимые данные всегда будут актуальными, то есть данными реального времени. Благодаря этой особенности материализованные представления органично вписываются в потоковые фреймворки.

Ранее мы рассмотрели, что потоковые платформы могут хранить транзакции из журналов WAL баз OLTP. Эти разделы имитируют конструкцию WAL, поэтому другие системы могут создавать копии таблиц исходной базы данных OLTP. Этот же подход можно применить в потоковых процессорах для построения табличных структур (рис. 1.6).

Подробнее о потоковой обработке поговорим в главе 2. А главу 3 мы посвятили материализованным представлениям, поскольку их важность для потоковых БД очень велика. Чтобы лучше объяснить принципы работы потоковых БД, нужно создать простой пример практического применения и проследить ход его выполнения до конца. Попутно мы определим каждую систему, необходимую для достижения цели сценария использования.

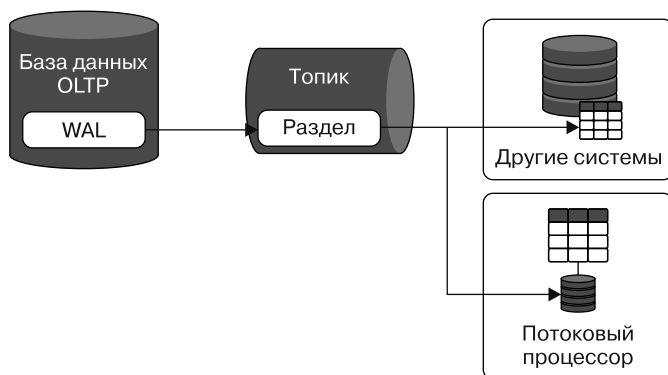


Рис. 1.6. Копии можно создавать в потоковых процессорах так же, как и в других системах

Пример практического применения: анализ потоков цифровых следов пользователей

Начнем с определения простого варианта применения. Этот пример поможет лучше понять, как использовать потоковые базы данных в режиме реального времени и какие преимущества они дают при проектировании решений для работы в данном режиме.

Здесь мы будем использовать данные о *потоке цифровых следов пользователей* (clickstream). Этот поток представляет собой последовательность записанных событий, которые фиксируют действия пользователей и их взаимодействие с системой во время навигации по сайту, приложению или цифровой платформе. Он обеспечивает подробную запись щелчков кнопкой мыши, просмотров страниц и других действий пользователей во время их онлайн-сессий.

Данные о цифровом следе могут использоваться для различных целей, таких как персонализация, целевая реклама, сегментация пользователей, обнаружение мошенничества и оптимизация коэффициента конверсии. Они играют важнейшую роль в веб-аналитике, маркетинговой аналитике, исследовании пользовательского опыта и других дисциплинах на основе данных. На рис. 1.7 клиент щелкает левой кнопкой

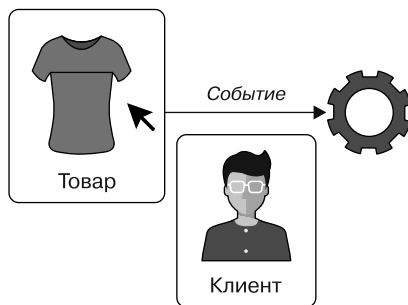


Рис. 1.7. Пользователь щелкает левой кнопкой мыши на зеленой футболке. Таким образом генерируется событие щелчка, фиксируемое микросервисом

кой мыши на товаре, генерируя событие нажатия, которое перехватывается микросервисом. Событие щелчка кнопкой мыши отправляется в последующее аналитическое потребление.

В нашем примере покупатель, 24-летний мужчина, живущий в городе Вудстоке, штат Нью-Йорк, щелкает на зеленой футболке, используя приложение для телефона. Наша цель — предоставить конечным пользователям данные о цифровом следе, чтобы они могли проводить аналитику и делать выводы для принятия решений на основе данных.

Допустим, в этом примере требуется фиксировать события щелчка мышью и связывать их с существующими клиентами. Это поможет аналитикам обеспечить целевой маркетинг и создать более персонализированный опыт.

Мы называем данные, поступающие в журнал WAL в базе данных OLTP, *транзакциями*. В этом примере щелчки мышью, поступающие из приложения пользователя, будут называться *событиями*. В конечном счете они оба окажутся в потоковой платформе вроде Kafka, и тогда мы сможем объединить их.

Изучение транзакций и событий

До сих пор мы называли транзакцией операции с данными, поступающими из БД. Это операции вставки, изменения и удаления, которые произошли и были записаны в журнал WAL, а затем перенесены в топик потоковой платформы. Такие транзакции также можно назвать *событиями изменений* или просто *событиями*. Это *события вставки, обновления и удаления*, точно так же как и событие щелчка в приложении.

Несмотря на то что все это события, очень важно понимать, что это разные типы событий. Одно из них возникает при изменении таблиц в базе данных, а другое — при выполнении действий в приложении. Чтобы понять, чем они различаются, необходимо кратко рассказать о предметно-ориентированном проектировании (Domain-Driven Design, DDD).

Предметно-ориентированное проектирование

В программном обеспечении инженеры моделируют свои приложения с помощью объектов, существующих в их бизнес-домене. Например, если часть вашего бизнеса включает в себя клиентов, то вы создадите в приложении объект для представления *клиента*. Это необходимо проделать для каждого объекта, входящего в бизнес-домен.

Давайте построим модель с описанием объектов в нашем примере. Клиенты и товары — это объекты, представляющие собой часть модели домена приложения.

Эти объекты называются *сущностями*. Они существуют в базе данных OLTP и подвергаются таким событиям изменения, как вставка, изменение и удаление.

Такие события, как щелчок, фиксируют взаимодействие между объектами в приложении. В нашем примере клиент щелкает на товаре. *Клиент* и *товар* выступают в роли объектов, а *щелчок на товаре* — действие. Это показано на рис. 1.7.

Для описания этих отношений можно использовать структуру предложения. В предложении есть *подлежащее*, *сказуемое* и *существительное*. *Подлежащее* в предложении — это, как правило, сущность, выполняющая действие. *Сказуемое* описывает действие. И наконец, *существительное*, или *объект*, — это сущность, к которой применяется действие. В нашем примере предложение выглядит так:

Клиент щелкнул на товаре.

События щелчка обычно содержат гораздо больше информации, поэтому предложение можно расширить, добавив сведения:

Клиент с IP-адресом 111.11.1111 щелкнул на товаре 12345 21/07/2023 в 11:20 по восточному времени.

Обратите внимание на то, что мы не знаем ни имени покупателя или товара, ни его местоположения или возраста. Мы также не знаем ни типа товара, ни его цвета. Необходимо обогатить события потока цифрового следа информацией о клиенте и товаре, прежде чем передавать их для анализа.

У вас может возникнуть вопрос: «Почему событие щелчка не может также храниться в базе данных?» Этот вопрос вполне обоснован. Почему бы не использовать журнал WAL для считывания событий щелчка вместе с сущностями? Одна из главных причин — в базе данных OLTP может закончиться место. Просто представьте, сколько раз покупатель щелкает кнопкой мыши на элементах в приложении, и вам станет понятно, что хранить все эти данные в БД OLTP будет неразумно. Хотя сущности обычно меняются очень медленно, их можно удалять или обновлять. А события щелчков неизменяемы, и их можно только вставить в таблицу. Эта модель также называется моделью только с добавлением (*append-only*). События щелчков лучше фиксировать с помощью микросервиса, выполняющего запись напрямую в потоковую платформу.

Еще одно важное отличие — то, что событие действия обогащается, а события сущности используются для обогащения. Важно понимать различия между событиями действия и событиями изменения сущности. Каждый тип будет обрабатываться по-разному по мере прохождения через конвейер потоковых данных до момента передачи конечному пользователю.

Обогащение контекста

Все виды аналитического потребления нуждаются в контексте возникновения события. Событие щелчка, как уже отмечалось, содержит только информацию, связанную с щелчком кнопкой мыши на области экрана, но не сведения о клиенте или товаре. Как правило, информация о сущности недоступна во время такого события. При ее наличии процессы сбора и обогащения данных о потоке цифрового следа в приложении стали бы неэкономичными и немасштабируемыми из-за размера данных и возникающей задержки.

Лучший способ обогатить событие щелчка — выполнить его в конвейере данных в режиме реального времени. Эта дополнительная информация поможет принимать более взвешенные решения. Например, знание того, что покупатель любит зеленые рубашки и что это мужчина в возрасте 20 лет, позволит принимать более разумные решения и сделает приложение более персонализированным.

В нашем примере событие щелчка связано с двумя другими сущностями в бизнес-домене — клиентом и выбранным товаром. Сочетание этих подробностей о сущностях с событием щелчка создаст более подробный контекст для аналитики в режиме реального времени. Обоснованная аналитика может рассказать больше о событии и о том, как быстро реагировать на возникающие из-за событий ситуации, например, принять решение об увеличении запасов мужских зеленых рубашек.

Мы поговорили о том, что сущности из домена приложения существуют в базе данных OLTP. Обсудили также, что изменения в этих сущностях записываются в журнал WAL. Но мы не рассматривали процесс превращения событий в WAL в топик потоковой платформы, где другие системы могут потреблять события изменений и строить свои копии сущностей в приложении. Копия позволит обогатить событие щелчка информацией о товаре и клиенте в потоковом процессе ниже по потоку обработки. Создание такой копии называется *захватом изменения данных*.

Захват изменения данных

Техника *захвата изменений данных* (Change Data Capture, CDC) применяется в базах данных и системах интеграции данных для фиксации и отслеживания изменений, вносимых в данные в режиме реального времени. Основная цель CDC — выявить и зафиксировать все транзакции модификаций (вставки, изменения или удаления), произведенные в определенных таблицах, и сделать такие события доступными для использования другими системами или процессами.

При выполнении CDC можно либо подписаться на поток выполненных транзакций, либо сделать *моментальный снимок* (snapshot). Но моментальный снимок не является событием изменения, как это было в журналах WAL. В терминологии баз данных снимок означает копию базы данных (или таблицы в БД), сделанную в определенный момент времени, как при съемке фотоаппаратом. Потоки из баз данных похожи на сжатое видео, где каждый кадр представляет собой не картинку или моментальный снимок, а изменение пикселей от одного кадра к другому.



Сжатие видео, которое для экономии времени обработки обеспечивает только изменения в каждом кадре, а не моментальные снимки, называется дельта-кодированием. При использовании этого метода кодирования сохраняются только различия между последовательными кадрами. Это позволяет значительно уменьшить размер видеофайла, сохранив при этом исходное качество видео.

CDC можно реализовать несколькими способами.

Прослушивание журнала WAL

Этот подход мы обсуждали в данной главе. Он считается предпочтительным способом фиксации изменений в базе данных. Выполняется в режиме реального времени и, естественно, является потоковым.



WAL-подход к захвату транзакций изменений обычно используется в реляционных базах данных OLTP, таких как PostgreSQL и MySQL. Мы рассматриваем его, потому что он похож на конструкции потоков в потоковых платформах. Некоторые транзакционные базы данных NoSQL могут не придерживаться этого подхода и содержать другой механизм захвата изменений.

Сравнение моментальных снимков

Этот способ подразумевает сравнение одного моментального снимка таблицы с предыдущим для определения изменений. Это действие может быть трудоемким, особенно если таблица большая. Кроме того, данный подход не соответствует парадигме реального времени. Моментальные снимки делаются через определенные промежутки времени. Изменения, включающие возврат, происходящий между интервалами сравнения, будут потеряны. Подозрительные изменения и события возврата могут иногда оставаться незамеченными.

Сравнение временных меток обновлений

Этот подход сохраняет временную метку последнего пакета изменений и фильтрует записи с временными метками обновлений после нее. Такой подход требует наличия в таблице графы, фиксирующей дату последнего обновления. Ее необходимо обновлять при каждом изменении записи. Этот подход также не работает в режиме реального времени.

К счастью, в большинстве баз данных OLTP есть определенный способ чтения своих журналов WAL. Некоторые БД OLTP содержат также встроенную поддержку для передачи событий на потоковые платформы или в другие системы. Например, СУБД CockroachDB предоставляет возможность создать канал изменений от себя:

- к Kafka;
- Google Cloud Pub/Sub;
- облачному хранилищу (Amazon S3, Google Cloud Storage, Azure Storage);
- Webhook.

Это избавляет клиента от необходимости подписываться на WAL в CockroachDB, и вместо этого СУБД передает события изменений в Kafka напрямую (пример 1.2). Это наиболее предпочтительная схема работы, поскольку она значительно снижает архитектурную сложность конвейера потоковых данных.

Пример 1.2. Создание потока изменений из CockroachDB в Kafka

```
CREATE CHANGEFEED FOR TABLE customer, product INTO 'kafka://localhost:9092';
```

Наличие этой функции в базах данных OLTP принципиально приближает их к потоковым БД, о которых мы поговорим в главе 5.



Даже если бы вы были самым Мартином Клеппманом, главы 1–4 все равно критически важно прочесть перед главой 5. Не пропускайте их, потому что там содержится основополагающая информация для введения потоковых баз данных в главе 5.

Напомним, что этот механизм уменьшает сложность архитектуры. Базы данных OLTP без этой функции требуют дополнительных компонентов, называемых *коннекторами* (connectors). Они необходимы для извлечения данных и их публикации в топике потоковой платформы.

Коннекторы

В потоковой обработке различают два основных типа коннекторов.

Source-коннекторы

Source-коннекторы считывают данные из системы — источника данных (например, базы данных) и предоставляют их в виде потока событий.

Sink-коннекторы

Sink-коннекторы потребляют данные из потока событий и записывают их в систему-накопитель (опять же в базу или хранилище данных, озеро данных и т. д.).

Два типа коннекторов показаны на рис. 1.8. В большинстве случаев source-коннекторы преобразуют *данные в состоянии покоя* в потоковые данные (они же *данные в движении*), а sink-коннекторы выполняют обратное преобразование.

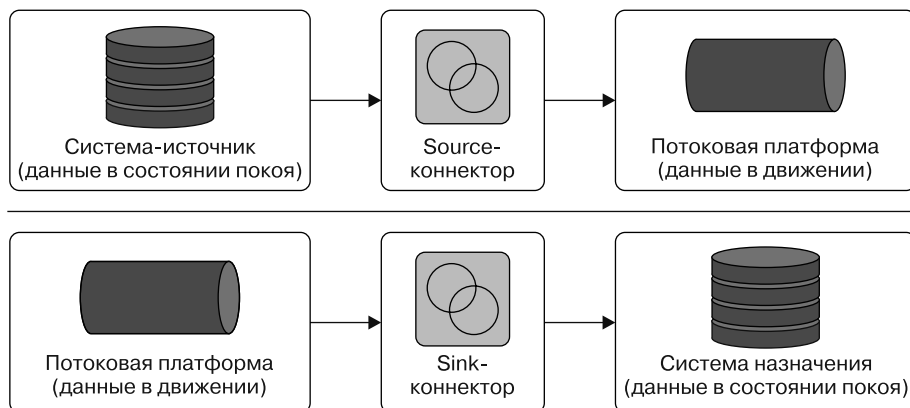


Рис. 1.8. Source-коннекторы (вверху) и sink-коннекторы (внизу)

Под данными в состоянии покоя подразумевается то, что они находятся в базе данных или файловой системе и не перемещаются. Такие данные обычно обрабатываются с помощью технологий пакетной обработки или микропакетов¹. У набора данных, переданного из одной системы-источника в другую, есть начало и конец. Приложения для обработки пакетных данных можно запустить с помощью планировщика заданий, например Сron, и обработка данных закончится, когда завершится набор данных.

Эта концепция противоположна потоковой передаче, или данным в движении. У данных в таком состоянии нет ни начала, ни конца. Приложения для обработки потоковых данных постоянно работают и слушают, когда в поток поступают новые данные. Теперь рассмотрим некоторые возможности реализации коннекторов источника (source) и назначения (sink).

Промежуточное программное обеспечение коннекторов

Промежуточные варианты ПО коннекторов, такие как Kafka Connect, Meroxa, Striim или StreamSets, сразу предоставляют большое количество коннекторов и часто имеют возможность расширения для подключения дополнительных источников и назначений. Промежуточное ПО также обеспечивает горизонтальное

¹ До сих пор ведутся споры о том, к чему ближе микропакеты — к потоковой или пакетной обработке.

масштабирование, мониторинг и другие необходимые функции, особенно для развертывания в производстве.

Компонент Kafka Connect представляет собой часть проекта Apache Kafka. Это распределенный кластер, в котором коннекторы Kafka развернуты для параллельной работы. Такие типы развертывания создают сложности в потоковой архитектуре. Эти кластеры громоздки, и их обслуживание сопряжено с большими трудностями.

Если у вас большое количество источников и точек назначения данных, кластеры часто становятся дорогостоящими и потребляют много ресурсов. Делегирование этой интеграции лучше решать путем встраивания коннекторов в сами системы.

Встроенные коннекторы

Все большее число баз данных предлагают встроенные коннекторы для потоковых платформ. Примером может служить упоминавшаяся система CockroachDB. В еще большем количестве баз данных реализованы встроенные коннекторы, то есть они могут сами потреблять данные из потока событий. В качестве примеров можно назвать Apache Druid, Apache Pinot, ClickHouse, StarRocks, Apache Doris и Rockset.

Наличие у баз данных возможностей решения задач интеграции с потоковыми платформами приближает их к тому, чтобы стать потоковыми БД. Если вы предоставите базам данных возможность передавать данные в потоковые платформы, потоковая передача естественным образом станет элементом первого класса (first-class citizen) в БД.

Специально разработанные коннекторы

Коннекторы могут быть специально разработаны, например, путем реализации специального микросервиса. Преимуществом такого подхода будет его гибкость, а недостатком — необходимость изобретать колесо. Ведь зачастую нет смысла создавать коннекторы с нуля, особенно при таком обилии существующих мощных и масштабируемых коннекторов с открытым исходным кодом (пример — коннекторы Debezium для промежуточного ПО Kafka Connect).

На рис. 1.9 мы проиллюстрировали три способа реализации коннекторов (для простоты здесь показаны только source-коннекторы — соответствующие реализации sink-коннекторов будут зеркальным отражением этой иллюстрации).

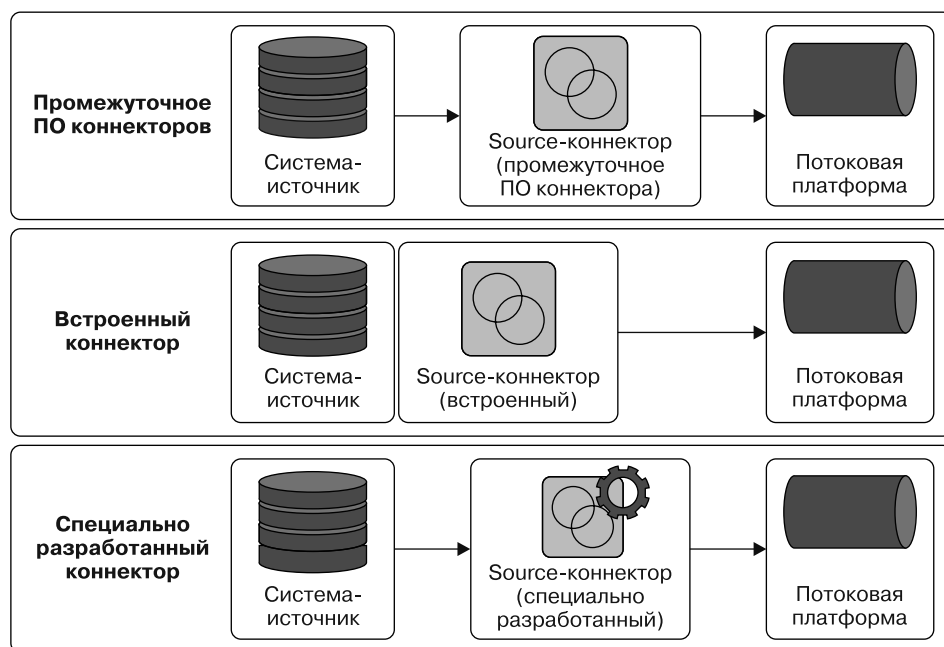


Рис. 1.9. Варианты реализации коннекторов с помощью промежуточного ПО для встроенных или специально разработанных коннекторов



В оставшейся части книги абстрагируемся от фактической реализации коннекторов. Когда мы говорим о коннекторе, это может быть коннектор на основе промежуточного ПО, встроенный или специально разработанный.

Вернемся к нашему примеру. Здесь требуется обогатить события щелчков мышью информацией о товаре и клиенте. Скорее всего, эти данные будут находиться в транзакционной БД, или БД обработки транзакций в режиме реального времени (OLTP). Чтобы сделать их доступными в виде потока событий, необходимо использовать source-коннектор для этой базы данных.



Базу данных OLTP также называют операционной. Такой тип БД предназначен для обработки большого количества транзакций. Они обеспечивают быстрый доступ к данным и их изменение, что важно для приложений, требующих обработки данных в режиме реального времени.

На рис. 1.10 видно, что информация о товарах и клиентах хранится в базе данных OLTP. Два source-коннектора БД считывают данные из нее и записывают их

в топики (топик товара и топик клиента). События щелчка мышью записываются в топик событий щелчка.

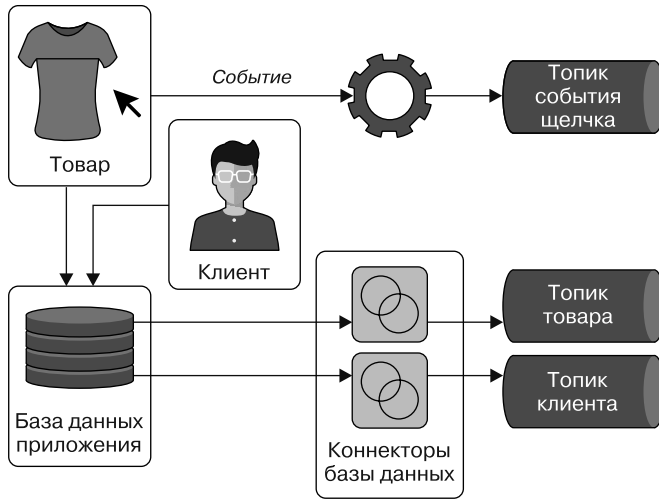


Рис. 1.10. Данные о клиентах и товарах в базе данных

Резюме

В этой главе мы познакомились с некоторыми основополагающими концепциями потоковой передачи данных, представив Мартина Клеппмана и его подход к выворачиванию базы данных наизнанку. Таким образом мы определили две функциональные возможности заложить основу потоковой обработки: база данных с WAL и материализованное представление.

Мы узнали, что топики в потоковых платформах — это предоставленные для доступа WAL-файлы БД, на которые могут подписаться другие системы. Затем эти системы могут создавать копии таблиц из исходной базы данных с помощью CDC или других форм коннекторов и выполнять обработку данных в режиме реального времени.

В главе 2 продолжим рассматривать пример использования потока цифрового следа и доведем его до следующего этапа — платформы обработки потоков, на которой будет происходить обогащение.

ГЛАВА 2

Платформы обработки потоков

В главе 1 мы представили простой пример практического применения — получение данных для потребителей в режиме реального времени. Также рассмотрели коннекторы и то, как они могут преобразовывать данные в состоянии покоя в данные в движении (или потоки событий), а затем публиковать их в топиках на потоковых платформах.

Потоки событий теперь можно читать, но, скорее всего, они еще не переведены в формат, пригодный для использования потребителями. События, как правило, нуждаются в очистке и подготовке, прежде чем подвергнуться аналитической обработке. Для того чтобы события были достаточно полезными для дальнейшего анализа, их также необходимо обогатить контекстом. Аналитическая обработка в значительной степени зависит от точности и надежности данных. Решение таких проблем, как недостающие значения, несоответствия, дубликаты и выбросы, повышает качество данных. В итоге будут получены более надежные и точные аналитические результаты.

Как показано на рис. 2.1, подготовка данных о событиях может существенно повлиять и на производительность аналитических запросов. Оптимизация расположения, индексации и разбиения данных позволяет повысить эффективность их поиска и обработки. К подготовке относятся также методы денормализации данных, столбцовое хранение и стратегии индексирования, адаптированные к аналитической рабочей нагрузке. Хорошо подготовленные данные позволяют сократить время обработки и быстрее получить результат анализа. О перечисленных методах подготовки данных мы расскажем в главе 4 при обсуждении способов предоставления аналитических данных потребителям.

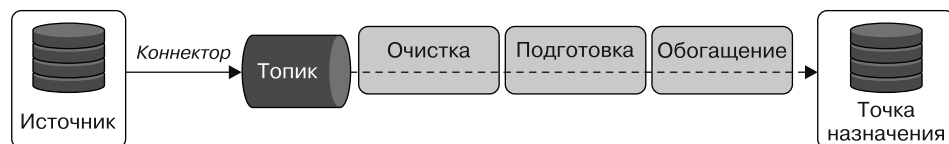


Рис. 2.1. Очистка, подготовка и обогащение данных о событиях до того, как они попадут к адресату

Подготовка данных о событиях играет важнейшую роль также в обеспечении руководства данными и соблюдения нормативных требований. Эти концепции предполагают соблюдение мер безопасности данных, анонимизацию конфиденциальной информации и соблюдение правил конфиденциальности. Правильно подготовив данные, организации могут обеспечить их целостность, защиту конфиденциальности и соблюдение правовых и этических требований.

В целом подготовка данных в конвейере обработки необходима для обеспечения их точности, согласованности и хорошей структуры для эффективной аналитической обработки. Так повышается надежность аналитических результатов, улучшаются производительность и возможность извлекать значимые сведения.

На рис. 2.1 пунктирная линия от топики до точки назначения представляет платформу потоковой обработки. Роль платформы потоковой обработки заключается в выполнении задач по очистке, подготовке и обогащению данных о событиях в качестве этапа предварительной обработки. Все это выполняется, прежде чем данные попадут в целевое хранилище.



В этой книге задачи по очистке, подготовке и обогащению данных о событиях будем называть преобразованиями в конвейере потоковых данных.

Задачи преобразования, как правило, требуют больших затрат ресурсов и трудоемких процессов. Лучше всего выполнять их как часть этапа предварительной обработки в конвейере данных перед записью в хранилище, предоставляющее данные потребителям. Лучше проводить преобразования как можно раньше. Это связано с тем, что их постепенное выполнение с одновременным просмотром небольших объемов данных требует гораздо меньше ресурсов, чем последующая работа с большими объемами данных в покое.

Как правило, целевые хранилища данных представляют собой хранилища аналитической обработки в режиме реального времени, или OLAP. Они дают возможность пользователям выполнять аналитические запросы к данным. Хранилища данных в режиме реального времени OLAP, или RTOLAP, оптимизированы для предоставления аналитических данных в режиме реального времени.

Лучше всего в OLAP избегать задач с высокой загрузкой процессора для обслуживания аналитики потребителей. OLAP должны резервировать свои ресурсы в целях быстрого реагирования на аналитические запросы для ответа на вопросы, связанные с бизнесом, или обеспечения быстрого понимания данных клиентами. Это еще одна причина выполнять предварительную обработку в конвейере как можно раньше.



OLAP — это хранилища данных, оптимизированные для аналитического чтения. Как правило, они являются колоночными (columnar-based) и используются для работы с пользовательскими дашбордами (информационными панелями) и приложениями. Базы данных OLTP собирают информацию из приложений, откуда поступают события. Они оптимизированы для записи и поиска по одной строке. Обычно базы данных OLTP служат источником для извлечения CDC, описанного в главе 1.

В приведенном примере мы обогатим поток цифрового следа информацией о покупателе и товаре. Это обеспечит контекст для аналитики щелчков кнопкой мыши, например местоположение покупателя, тип товара, его цвет и многое другое. Такая информация позволяет делать более точные прогнозы, чтобы помочь предприятиям значительно сократить временные затраты, потери товарно-материальных ценностей и повысить уровень удовлетворенности клиентов.

Некоторые преобразования довольно сложны и требуют запоминания информации, например, если в них необходимо выполнять объединения (joining), как в нашем случае, или агрегирование. Эту информацию нужно где-то сохранить, чтобы не забыть. Это место называется *хранилищем состояний*. А такие сложные преобразования называются преобразованиями с состоянием.

Преобразования с состоянием

В главе 1 мы завершили конвейер данных записью событий в топик потоковой платформы. Это позволило сохранить события в потоке, на который могут подписаться потребители. Не путайте это с *загрузкой*, или буквой «L» в названии подхода ELT. На данном этапе все, что мы технически сделали, — извлекли события из их источника и поместили в топик. Что касается процесса ELT, то мы все еще находимся на стадии E (Extract — «извлечение»).

Потоковые платформы, такие как Kafka, не преобразуют данные таким образом, что они могут хранить состояние. Следовательно, более сложные преобразования, например агрегацию и объединение, нельзя выполнить в Kafka напрямую — для этого нужны дополнительные компоненты, такие как Kafka Streams.

Конвейеры данных следуют двум типам подходов. Они представляют собой общие подходы для интеграции и обработки данных, но различаются последовательностью операций и могут быть ограничены в возможностях потоковой обработки:

- извлечение, преобразование и загрузка (Extract, Transform and Load, ETL);
- извлечение, загрузка и преобразование (Extract, Load and Transform, ELT).

В подходе ETL данные сначала извлекаются из исходных систем, которые могут быть базами данных OLTP, файлами, устройствами или другими источниками данных. Процесс извлечения заключается в получении необходимых событий из этих источников. Извлечение — это работа source-коннекторов, описанных в главе 1.

В аббревиатуре ETL буква «Т» требует, чтобы потоковые данные временно сохранялись в хранилище. В нем находится *состояние* преобразования. Другими словами, преобразования с состоянием требуют сохранения и обновления некоторой информации или контекста между несколькими элементами входных данных. Приведем несколько примеров.

Скольльзящее среднее

Чтобы рассчитать скользящее среднее, необходимо отслеживать сумму и количество предыдущих элементов данных. По мере поступления новых данных состояние обновляется путем добавления нового значения и удаления старого. Это позволяет вычислять среднее значение в скользящем окне.

Анализ данных сеанса

В веб-аналитике или обработке событий анализ данных сеанса подразумевает группировку связанных событий в сеансы на основе определенного набора критериев (например, активность пользователя в течение конкретного временного промежутка). Для этого необходимо хранить информацию о состоянии текущих сеансов, например время начала, время окончания и список событий, относящихся к каждому сеансу.

Дедупликация

Для удаления дубликатов событий из потока часто требуется вести учет ранее просмотренных событий. По мере поступления новых событий они сравниваются (обычно с помощью первичных ключей) с сохраненным состоянием, чтобы выявить и отфильтровать дубликаты.

Оконные агрегации.

Агрегатные функции, такие как `sum`, `count` или `maximum`, которые работают с окнами¹ данных, должны содержать состояние для накопления значений в каждом окне. По мере поступления новых данных вы соответствующим образом обновляете состояние и периодически получаете агрегированные результаты.

¹ Термин «окно» в данном случае означает один из поднаборов данных, на который разделился весь набор. — *Примеч. пер.*

Модели машинного обучения

В сценариях, где требуется постоянно обновлять и уточнять модели машинного обучения на основе потоковых данных, сами модели требуют состояния. Состояние модели содержит информацию об изученных параметрах, весах и других важных данных, и она обновляется с каждой новой точкой данных.

Эти примеры демонстрируют, как сохранение состояния в потоковых преобразованиях позволяет выполнять вычисления, охватывающие множество элементов данных. Таким образом можно получить возможность проводить более сложный и содержательный анализ потоковых данных. Чистые потоковые платформы, например Kafka, не поддерживают такой возможности без развертывания дополнительных компонентов экосистемы, таких как микросервисы на основе библиотеки Kafka Streams.



В некоторые потоковые платформы встроены преобразования без сохранения состояния. Они называются функциями или преобразованиями одного сообщения и выполняются без сохранения состояния. К ним относятся преобразования строк, такие как капитализация, или простые математические операции между столбцами. Они не считаются преобразованиями с состоянием в ETL или ELT.

Пока что мы выполнили только пункт Е для извлечения данных. Простое помещение событий в топик не считается загрузкой (L) в ETL или ELT, поскольку потоковая платформа не может выполнять преобразования с сохранением состояния. Кроме того, она не может напрямую обслуживать потребителей или пользовательские приложения. Помните, что эти потребители нуждаются в очистке, подготовке и обогащении событий — и ни один из этих этапов еще не завершен, когда события изначально попадают на потоковую платформу.

Именно здесь на помощь приходят платформы обработки потоков. Обратите внимание на то, что во всей книге мы используем термины «платформа обработки потоков», «система обработки потоков», «фреймворк обработки потоков» и «потоковый процессор» как взаимозаменяемые. В платформах обработки потоков есть механизмы для хранения состояния и выполнения сложных преобразований.

Эти преобразования с сохранением состояния выполняются в конвейере данных. Мы называем конвейером данных процесс извлечения, преобразования и загрузки (ETL) в хранилище данных OLAP. В нашем случае это конвейер потоковых данных (потоковый ETL). Конвейеры данных играют важнейшую роль в аналитике в режиме реального времени. Они отвечают за сбор данных из различных источников, их преобразование в пригодный для анализа формат и оперативную передачу в аналитические инструменты и приложения.

Конвейеры обработки данных

Возвращаясь к метафоре с водой из главы 1, конвейеры потоковых данных можно сравнить с трубами в вашем доме. На рис. 2.2 показано, что вода в трубах может содержать минералы и ее необходимо фильтровать или кипятить перед употреблением. Подобно водопроводным трубам, конвейеры потоковых данных направляют данные о событиях через ряд преобразований, прежде чем те попадут к потребителю аналитических данных.

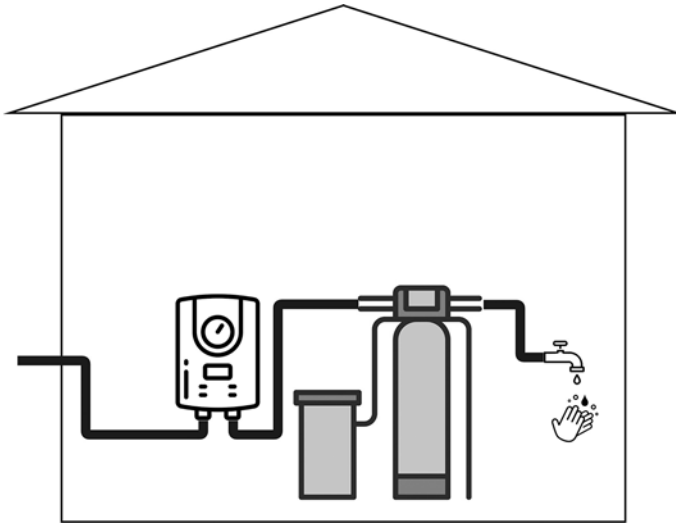


Рис. 2.2. Конвейеры потоковых данных похожи на систему водопровода

Одна из функций конвейера данных — перемещение данных из операционной плоскости данных в аналитическую. Операционная плоскость данных — это место, где располагаются приложения, включая микросервисы и базы данных OLTP. Аналитическая плоскость данных — это место, где располагаются аналитические системы, такие как хранилища, озера данных, хранилища Lakehouse, — всевозможные хранилища данных OLAP. Конвейеры данных находятся между операционной и аналитической плоскостями данных, преобразуя данные и интегрируя их из одной в другую (рис. 2.3).

После того как данные извлечены, они проходят через ранее упомянутые процессы *преобразования* и переводятся в формат, подходящий для целевой системы, или хранилища данных, или OLAP-хранилища данных (рис. 2.4). Преобразованные данные обычно временно хранятся в промежуточном хранилище.

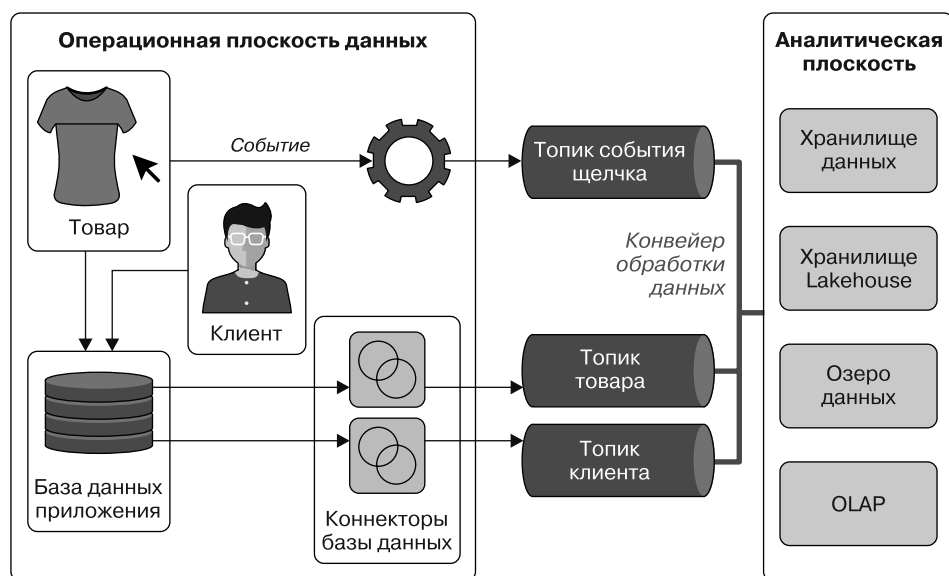


Рис. 2.3. Данные переходят из операционной плоскости в аналитическую с помощью конвейеров потоковых данных



Рис. 2.4. Конвейер ETL, предназначенный для извлечения данных из исходной базы данных OLTP, выполняет преобразования с учетом состояния и записывает их в хранилище данных OLAP

Наконец, операция загрузки из ETL *загружает* преобразованные данные в целевую систему, например хранилище данных. Это означает вставку данных в соответствующие таблицы или структуры в целевой системе.

Вернемся к простому случаю, когда покупатель щелкает на изображении зеленой футболки в приложении. На рис. 2.5 показано, где мы сейчас находимся.

Сначала мы получаем данные из приложения на телефоне (см. рис. 2.5, *слева*). События щелчков поступают в микросервис, направляющий событие в топик потоковой платформы (топик события щелчка).

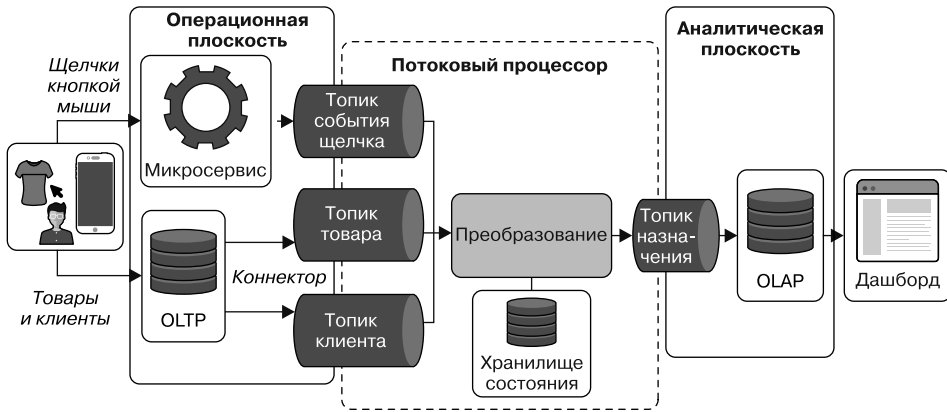


Рис. 2.5. Конвейер потоковых данных ETL с коннектором и топиками источника и назначения

Информация о товаре и клиенте отправляется в базу данных OLTP. Стрелка от телефона к базе данных OLTP будет представлять собой транзакции, совершенные с клиентом или товаром. Транзакции — это вставка, изменение или удаление. Это транзакции из главы 1, которые связаны с CDC. Транзакции в журнале WAL в базе данных OLTP потребляются коннектором и публикуются в соответствующих топиках (топик товара, топик клиента).

Все три топика — события щелчков, товары и клиенты — потребляются потоковым процессором. Он реализует преобразование событий с учетом состояния. Результат работы потокового процессора публикуется в топике назначения, который затем потребляется хранилищем данных OLAP. Далее хранилище данных OLAP передает аналитические данные на дашборд для аналитиков данных.

На рис. 2.5 показано, что ETL отлично поддерживает потоковую передачу, поскольку преобразование выполняется в платформе потоковой обработки. В конце концов данные находятся в движении. Преобразование выполняется перед загрузкой данных в целевое хранилище (или OLAP), которое предоставляет данные потребителям. Такой подход позволяет избежать повторного выполнения предварительной обработки в целевом хранилище данных для каждого обслуживаемого запроса. Помните, что задачи предварительной обработки трудоемки и требуют больших ресурсов. В хранилище данных необходимо зарезервировать эти ресурсы для предоставления данных, а не для их преобразования.

Ограничения ELT

В ELT задачи преобразования и загрузки переносятся друг на друга, что делает этот подход более гибким, но при этом накладывает некоторые ограничения. Преобразование выполняется в целевой системе вместо преобразования в конвейере данных. Без необходимости хранить состояние преобразования конвейеры данных ELT намного проще и не используют обработку потоков с состоянием. После загрузки в целевую систему (скорее всего, в базу данных, способную выполнять преобразования) данные удаляются из потока и переходят в состояние покоя. Конвейер данных больше не рассматривается как механизм реального времени, поскольку преобразование должно выполняться по расписанию или запускаться через определенный промежуток времени. Такой подход считается пакетным.

SQL-запрос запускается на выполнение в целевом хранилище данных. Он содержит логику преобразования данных и соответствует букве «Т» в аббревиатуре ELT (Transformation — «преобразование»). Давайте разберемся в этом подробнее.

На рис. 2.6 показан процесс ELT.

1. Данные извлекаются из базы данных OLTP.
2. Данные загружаются в хранилище OLAP.
3. SQL-код для преобразования данных выполняется после завершения загрузки.

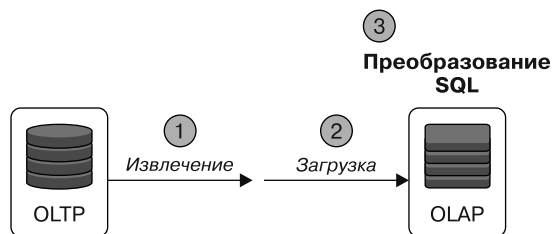


Рис. 2.6. Конвейер данных ELT, преобразующий данные в целевом хранилище

Вызов SQL-запроса в целевом хранилище данных будет инициирован только после завершения этапа загрузки. Это означает, что через конвейер обработки проходят пакетные данные, потому что у пакетов есть конец. Если бы данные представляли собой поток, он бы никогда не заканчивался, а значит, было бы сложно определить, когда нужно выполнить SQL-команды (рис. 2.7).

При использовании потоковой передачи данных в режиме реального времени конвейер должен откладывать сохранение данных в место назначения до тех пор,

пока преобразования не будут завершены и данные не будут готовы к передаче. На первый взгляд, ETL совсем не подходит для потоковой обработки.

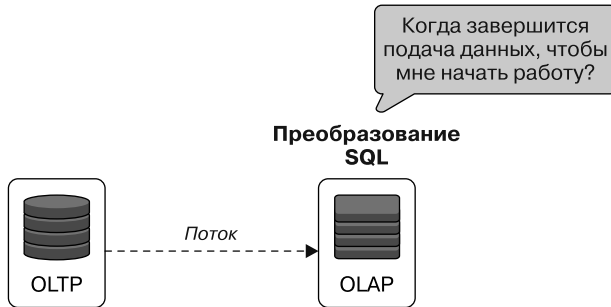


Рис. 2.7. Конвейер данных ETL не справляется с преобразованием событий

Обработка потоков с помощью ETL

Однако есть способ применения ETL для поддержки потоковых приложений в режиме реального времени. Это можно сделать, только используя возможности современных потоковых платформ и фреймворков для обработки данных в режиме реального времени. ETL может применяться в потоковых системах реального времени следующим образом.

Извлечение

В ETL этап извлечения включает в себя получение данных из потоковых источников, таких как очереди сообщений, потоки событий или потоки данных в режиме реального времени. Эти источники постоянно производят данные в режиме реального времени, и ETL может извлекать их для дальнейшей обработки.

Загрузка

Извлеченные данные загружаются в целевую систему, поддерживающую поглощение и хранение потоковых данных. Это может быть потоковая платформа или хранилище данных для обработки высокоскоростных и объемных потоков данных в режиме реального времени. Целевая система должна быть способна эффективно принимать потоковые данные и хранить их таким образом, чтобы их можно было обрабатывать в режиме реального времени.

Преобразование

В ETL этап преобразования следует за загрузкой данных в целевую систему, такую как фреймворки для обработки данных в режиме реального времени, например Apache Flink, Apache Kafka Streams, или в любую платформу для обработки потоков с контролем состояния.

На рис. 2.8 показано, как может работать ELT, используя в качестве цели платформу обработки потоков. В этом случае поток может быть коннектором, извлекающим данные из БД OLTP, или потоковой платформой, такой как Kafka. Однако поскольку во многих платформах обработки потоков, таких как Flink, нет слоя хранения данных (persistence layer) (исключая их хранилища состояний), результат этапа преобразования должен быть перенесен, например, в базу данных для выполнения запросов. В потоковых БД есть слой хранения данных, и они также могут выступать в роли целевой системы для реализации этого архитектурного паттерна. Мы вернемся к этому вопросу, когда будем говорить о потоковых базах данных в главе 5.

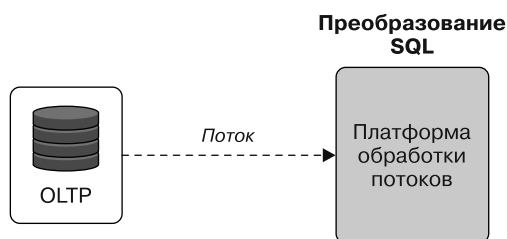


Рис. 2.8. ELT с платформой обработки потоков в качестве цели

Благодаря использованию потоковых платформ и процессоров с сохранением состояния механизм ELT может эффективно обрабатывать потоковые задачи реального времени. Ключевыми факторами при выборе между ETL и ELT стали масштабируемость и производительность целевой системы, эффективный прием потоковых данных, а также возможность выполнять преобразования и аналитику потоковых данных с учетом состояния.

Потоковые процессоры

Потоковые процессоры — это программные платформы или инструменты для обработки непрерывных потоков данных в режиме реального времени. Самое главное — они могут выполнять преобразования с сохранением состояния, поскольку содержат встроенное хранилище состояний¹. Эти преобразования данных о событиях требуются потребителям для получения необходимых аналитических данных.

¹ Существуют (как правило, более старые) потоковые процессоры, такие как Apache Storm, без встроенной поддержки состояния. Но в данном случае мы ориентируемся на большинство потоковых процессоров, получивших такую функциональность.

Популярные потоковые процессоры

Приведем подборку популярных потоковых процессоров.

- Apache Kafka Streams — это библиотека обработки потоков для языков программирования на базе JVM. Она представляет собой часть проекта Apache Kafka. Библиотека позволяет разработчикам создавать приложения и микросервисы для работы в режиме реального времени, которые потребляют, обрабатывают и создают потоки данных в Kafka и из него.
- Apache Flink — это потоковый процессор с поддержкой пакетной и потоковой обработки. Его можно подключить к большому количеству источников и точек назначения, включая Kafka, Pulsar и Kinesis, а также базы данных, например, MongoDB и Elasticsearch. В процессоре Flink пакетная передача рассматривается как частный случай потоковой (потоковая передача с ограниченными данными). В отличие от Kafka Streams, Flink — это не библиотека, он работает на собственном кластере.
- Spark Structured Streaming — это компонент Apache Spark для потоковой обработки. Он поддерживает большое количество коннекторов и тоже работает на кластере. От других потоковых процессоров, таких как Kafka Streams, Flink и Samza, его отличает использование мини-пакетов вместо собственной обработки потоков — в Spark потоковая передача выступает как частный случай пакетной, а не наоборот, как во Flink.
- Apache Samza — потоковый процессор, разработанный компанией LinkedIn. Он поддерживает Kafka, Azure Event Hubs, Kinesis и HDFS, и он кластерный, как и Flink.
- Apache Beam — это не сам потоковый процессор, а унифицированная модель программирования и набор *средств разработки программного обеспечения* (Software Development Kit, SDK) для создания конвейеров обработки данных. Он предоставляет абстракцию, позволяющую разработчикам писать задания по обработке данных для выполнения на различных распределенных процессорах, таких как Apache Flink, Apache Spark, Apache Samza и Google Cloud Dataflow.

Новые потоковые процессоры

Продолжим рассказ подборкой более новых потоковых процессоров.

Quix Streams

Библиотека обработки потоков на базе C#/Python, сопоставимая с Kafka Streams для JVM. Quix Streams поддерживает также потоковые структуры

данных data frame. Они ведут себя аналогично элементам data frame, например, в Pandas или Spark, но при этом инкрементно обновляются внутри системы.

Bytewax

Библиотека обработки потоков на базе Python, сравнимая с Kafka Streams. Она использует актуальный поток данных в качестве базового механизма обработки потоков.

Pathway

Библиотека обработки потоков на основе Python. Она создана на базе *дифференциального потока данных* (Differential Dataflow, DD) в качестве базового механизма обработки потоков.

Estuary Flow

Потоковый процессор, поддерживающий большое количество коннекторов. Также кластерный.

Эти фреймворки для обработки потоков предоставляют такие возможности, как обработка событий, работа с окнами, управление состоянием, повышение отказоустойчивости, масштабируемость и интеграция с различными источниками и поглотителями данных. Они дают возможность разработчикам создавать конвейеры обработки данных в режиме реального времени и разрабатывать приложения с возможностью реагировать на данные по мере их поступления. Это делает описанные фреймворки ценными инструментами для работы с потоковыми данными в самых разных случаях.



В эти списки включены только потоковые процессоры, но не потоковые базы данных. Последние также могут выполнять преобразования с учетом состояния. Мы познакомимся с ними в главе 5.

Хотя эти потоковые процессоры очень мощные, не все они поддерживают материализованные представления. Такие представления отображают данные в виде, напоминающем базу данных. Эта возможность очень полезна для реализации аналитических сценариев в режиме реального времени.

На момент написания книги материализованные представления поддерживали следующие потоковые процессоры:

- Kafka Streams (KTable, GlobalKTable);
- Samza (Table);
- Pathway (Table).

Эмуляция материализованных представлений в Apache Spark

На момент выхода этой книги, насколько нам известно, в Apache Spark не было встроенной поддержки создания материализованных представлений. Однако Apache Spark предлагает различные функции и оптимизации для достижения преимуществ, аналогичных преимуществам материализованных представлений. Вот примеры.

- Можно использовать механизм кэширования Spark для кэширования промежуточных или конечных результатов дорогостоящих вычислений. Благодаря кэшированию результатов в памяти или на диске последующие запросы могут обращаться к предварительно вычисленным данным, что потенциально повышает производительность.
- Другой подход в Spark заключается в задействовании структур DataFrame или Dataset для определения многократно используемых представлений или преобразований, которые можно применять к входным данным. Эти представления можно сохранять и повторно использовать в нескольких приложениях Spark, обеспечивая абстракцию и оптимизацию данных.
- Кроме того, Spark интегрируется с внешними хранилищами данных и системами, например Apache Hive, Apache HBase или другими БД. А в них может присутствовать встроенная поддержка материализованных представлений. В таких случаях можно использовать возможности базового хранилища данных для создания материализованных представлений и управления ими, а Spark применять уже для обработки данных и аналитических задач.

Стоит отметить, что экосистема Apache Spark постоянно развивается и в новых версиях могут появляться новые функции и усовершенствования, расширяющие ее возможности. Поэтому всегда рекомендуется обращаться к официальной документации Apache Spark и примечаниям к релизу для получения актуальной информации о материализованных представлениях или подобных оптимизациях.

Два типа потоков

Чтобы лучше понять материализованные представления, необходимо разобраться в типах потоков в потоковом процессоре. Через него проходят два типа потоков данных в режиме реального времени: *потоки изменений* (change) и *потоки только для добавлений* (append-only). Лучший способ описать эти потоки — привести примеры того, как они создаются (рис. 2.9).

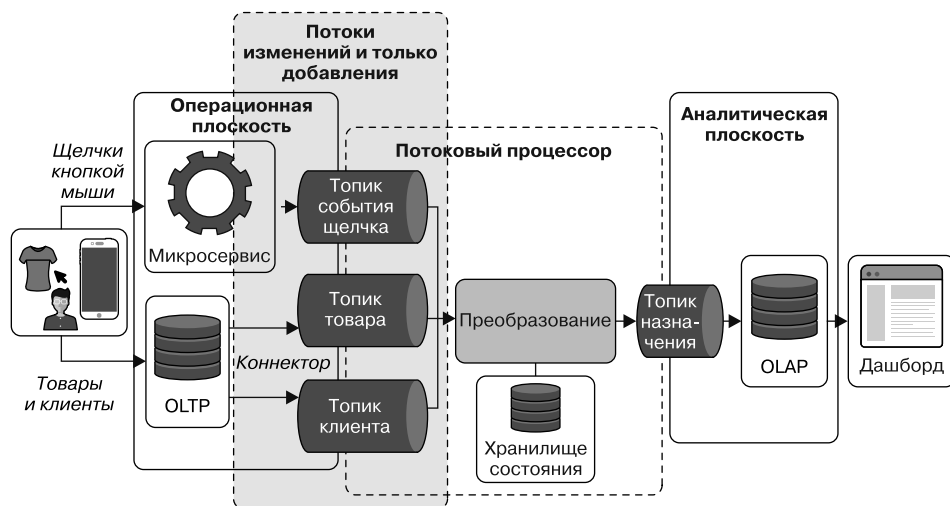


Рис. 2.9. События щелчка — это потоки добавлений, а потоки изменений — потоки CDC

На рис. 2.9 выделены следующие топики: события щелчка, товара и клиента. Первый предназначен только для добавления данных — он содержит дискретные, отдельные события. Как мы сделали такой вывод? Щелчки кнопкой мыши всегда уникальны. Каждое такое событие отличается от другого, даже если оно совершается на одной и той же футболке одним и тем же клиентом. Единственное отличие — время нажатия. В потоке только для добавлений (append-only) события связаны с временной меткой, указывающей на точку возникновения события во времени. Информация временной метки дает возможность упорядочить и последовательно выстроить события в потоке. Это относится и к другим типам событий, а не только к щелчкам мышью.

Потоки такого типа, как правило, не поступают из баз данных. В примере на рис. 2.9 данные о цифровом следе отправляются в микросервис, выступающий в качестве прокси для топика. Если бы щелчки мышью записывались в базу данных OLTP, в ней очень быстро закончилось бы место, потому что каждое событие уникально. То есть мы вызывали бы операцию *вставки* в таблицу OLTP для каждого щелчка мышью в приложении. Следовательно, использование БД OLTP для событий щелчка мышью было бы плохим вариантом — здесь явно лучше задействовать поток событий только для добавления.

И наоборот, данные о потоке изменений почти всегда поступают из событий CDC. Они представляют собой транзакции вставки, изменения и удаления, применяемые к таблице в базе данных OLTP. Количество записей такого типа будет увеличиваться не так быстро, как данные потока цифрового следа, поскольку некоторые события будут включать обновление или удаление существующих в таблице записей.

Кроме того, данные о клиентах могут изменяться нечасто. Клиенты редко меняют свои имена, электронные адреса или номера телефонов. В товарах будет гораздо больше изменений, но все равно не так уж много по сравнению с щелчками кнопкой мыши в приложении. Кроме того, потоки изменений, как правило, представляют собой изменения в данных с несколькими измерениями (dimensional data). А измерения медленно меняются в базе данных, поскольку обычно представляют свойства, которые нечасто и небыстро модифицируются для описываемых ими сущностей.

Если наборы данных о товарах и клиентах помещаются в базу данных OLTP, это означает, что они должны поместиться в хранилище состояний на платформе потоковой обработки. Конечно, если у хранилища состояний соответствующий размер. Это дает возможность создать материализованное представление в потоковом процессоре, и его можно использовать для обогащения данных потока цифрового следа.

В главе 1 мы рассказывали о том, как базы данных OLTP могут поддерживать аварийное восстановление путем репликации основной базы данных во вторичную. Именно это мы и сделали — реплицировали таблицу из базы данных OLTP в хранилище состояний на платформе обработки потоков.

На платформе обработки потоков потоки изменения и добавления будут находиться в структурах с разными именами в зависимости от используемого потокового процессора. В случае потоков только с добавлением эти структуры могут называться *потоками*. Для потоков изменений они могут называться просто *таблицами*. Обогащение потока цифрового следа можно выполнить с помощью команд SQL.

Поток с добавлением

Рассмотрим пример 2.1. В нем данные потока цифрового следа из Kafka попадают в гипотетический потоковый процессор (для примера используем синтаксис, аналогичный существующим потоковым процессорам). Этот запрос создаст исходную таблицу `click_events`, и она станет потоком только для добавления.

Пример 2.1. Создание исходной таблицы из топика Kafka

```
CREATE SOURCE click_events (  
  id integer,  
  ts long, ❶  
  url varchar, ❷  
  ipAddress varchar, ❸  
  sessionId varchar,  
  referrer varchar,  
  browser varchar  
)
```

```
WITH (
    connector='kafka',
    topic='clicks',
    properties.bootstrap.server='kafka:9092',
    scan.startup.mode='earliest'
)
ROW FORMAT JSON;
```

- ❶ Временная метка.
- ❷ Содержит идентификатор товара, который необходимо проанализировать.
- ❸ Переменная `ipAddress` для идентификации клиента.

Изменение данных с Debezium

SQL-запросы в примерах 2.2 и 2.3 также получают данные из Kafka, но в этом случае это данные Debezium CDC о товарах и информация о клиентах. Debezium — это популярный коннектор различных СУБД. Он может считывать журнал WAL базы данных OLTP, захватывать изменения в таблице и публиковать их в Kafka. Обратите внимание на ключевые слова `before`, `after` и `op` в SQL-запросах. Они указывают значения до и после изменений, а также тип операции, выполняемой над записью.

Пример 2.2. Загрузка и создание таблицы для товаров

```
CREATE SOURCE products (
    before ROW<id long, name varchar, color varchar, barcode long>,
    after ROW<id long, name varchar, color varchar, barcode long>,
    op varchar,
    source <...>
)
WITH (
    connector='kafka',
    topic='products',
    properties.bootstrap.server='kafka:9092',
    scan.startup.mode='earliest'
)
ROW FORMAT JSON
```

Пример 2.3. Загрузка и создание таблицы для клиентов

```
CREATE SOURCE customers (
    before ROW<id long, name varchar, email varchar, ipAddress varchar>,
    after ROW<id long, name varchar, email varchar, ipAddress varchar>,
    op varchar,
    source <...>,
)
```

```
WITH (
  connector='kafka',
  topic='customers',
  properties.bootstrap.server='kafka:9092',
  scan.startup.mode='earliest'
)
ROW FORMAT JSON;
```

Чтобы обогатить данные потока цифрового следа информацией о товаре и клиенте, необходимо объединить эти три таблицы. Следует провести множество преобразований, чтобы перенести данные CDC в структуру, которая позволит объединять их с данными цифрового следа. Пока предположим, что эта работа уже выполнена. Более подробно мы рассмотрим этот процесс в последующих главах, когда начнем использовать потоковую базу данных. Если таблицы товаров и клиентов были преобразованы из формата Debezium в стандартную таблицу, их все равно нельзя объединить с таблицей цифрового следа. Таблицы товаров и клиентов будут содержать дубликаты записей для каждого идентификатора (ID). Они возникают из-за нескольких изменений, внесенных в одну запись. Это означает, что объединение таблиц приведет к появлению еще большего количества дубликатов записей. В табл. 2.1 видно, что значение цвета изменялось несколько раз. Но нам требуется, чтобы в нее проецировалась только последняя запись. При этом на каждый идентификатор будет проецироваться только одна запись.

Таблица 2.1. Таблица товаров с предыдущими изменениями столбца цвета

ID	Название	Цвет	Штрихкод
1	Футболка	Зеленый	123456
1	Футболка	Зеленоватый	123456
1	Футболка	Зеленый лаймовый	123456

Материализованные представления

Термин «материализоваться» определяется как создание чем-то самого себя. Обычно материализующееся нечто делает это самостоятельно. Эта концепция важна для понимания материализованных представлений.

В примере 2.4 создаются материализованные представления для каждого из наборов данных CDC: товаров и клиентов. Каждое материализованное представление будет содержать только последнюю версию каждой записи¹.

¹ Предполагается, что здесь id товара и id клиента используются в качестве первичных ключей.

Материализованное представление сокращает количество записей, сохраняя только последнюю запись для каждого товара или идентификатора клиента. По сути, именно для этого и используется материализованное представление — оно сокращает поток изменений. Такое сокращение — это агрегация, например подсчет, среднее, минимальное, максимальное значения и т. д. В нашем случае материализованное представление сокращает записи по их ID, фиксируя последнее состояние записи.

Пример 2.4. Создание исходной таблицы из топика Kafka

```
CREATE MATERIALIZED VIEW mv_products AS select * from products;
CREATE MATERIALIZED VIEW mv_customers AS select * from customers;
```

Материализованные представления позволяют применить к ним операцию JOIN с событиями потока цифрового следа, проецируя только последние записи для каждого потока изменений товара и клиента. Мы не создаем материализованное представление для потока цифрового следа, потому что его нельзя агрегировать. Помните, что потоки только с добавлением — это отдельные уникальные события, как и потоки цифрового следа.

События щелчков нельзя агрегировать. В примере 2.5 показан процесс объединения трех таблиц и записи их в топик назначения.

Пример 2.5. Создание исходной таблицы из топика Kafka

```
CREATE SINK http_enrich AS
SELECT
    E.*,
    C.*,
    P.*
FROM click_events E
JOIN CUSTOMERS C ON C.ip=E.ip and
JOIN PRODUCTS P ON P.product_id=E.product_id ❶
WITH (
    connector='kafka',
    topic='click_customer_product',
    properties.bootstrap.server='kafka:9092',
    type='upsert',
    primary_key='id'
);
```

❶ Значение идентификатора товара было извлечено из URL щелчка кнопкой мыши и помещено в отдельный столбец `product_id`.

Поскольку потоки изменений можно агрегировать, их результаты достаточно малы для того, чтобы поместиться в хранилище состояний в платформе обработки потоков.

Выгодно выполнять эти преобразования над потоками перед их загрузкой в хранилище данных OLAP для предоставления клиентам. Потребительские запросы, вызываемые на OLAP, должны содержать только аналитическую бизнес-логику без каких-либо преобразований, выполненных в процессе предварительной обработки.

Логику запросов для преобразования событий необходимо отделить от запросов, направленных на бизнес-аналитику. Мы применим эту практику, чтобы отделить логику преобразования от аналитической логики с использованием запросов push и pull. Подробнее о материализованных представлениях и запросах типа push и pull поговорим в главе 4.

С этого момента мы будем рассматривать только потоковые процессоры с поддержкой построения материализованных представлений. С материализованными представлениями в потоковом процессоре можно работать почти так же, как с материализованными представлениями в базе данных. Основное отличие заключается в том, что в потоковом процессоре данные берутся из потоковых данных, а материализованные представления обновляются непрерывно, а не периодически либо вручную.

На рис. 2.10 показаны выполненные преобразования. Они были заданы на языке SQL и выполнены на платформе обработки потоков. Материализованные представления для товара и клиентов постоянно работают, создавая последние записи из потоков изменений Debezium.



Рис. 2.10. Преобразования, выполняемые в потоковом процессоре

Резюме

В этой главе мы рассмотрели распространенные способы преобразования данных и то, как они выполняются в различных схемах конвейера данных: ETL и ELT. Мы указали, что для выполнения более сложных преобразований у потокового процессора должна быть возможность хранить состояние.

- Платформы обработки потоков должны поддерживать обработку потоков с состоянием, чтобы сохранять состояние сложных преобразований.
- ETL подходит для использования в потоковой обработке, поскольку данные предварительно обрабатываются до загрузки в хранилище данных.
- ELT записывает события в целевое хранилище данных перед преобразованием данных. Преобразования определяются как операторы SQL, запускаемые на выполнение в целевой базе данных. Это вынуждает использовать семантику пакетной обработки.
- ELT может поддерживать потоковую обработку, меняя целевую систему на платформу для обработки потоков.
- Материализованные представления делают возможными нетривиальные преобразования с сохранением состояния на платформах обработки потоков. Для их применения требуются платформы обработки потоков, способные хранить состояние преобразования.
- Платформа обработки потоков с состоянием записывает результаты преобразований в топик назначения.

В следующей главе мы обсудим различные способы предоставления преобразованных данных потребителям.

Предоставление данных в режиме реального времени

В главе 2 платформа обработки потоков преобразовывала данные и помещала их в топик назначения. Теперь предварительно обработанные данные находятся в топике на потоковой платформе. На рис. 3.1 топик назначения и хранилище данных OLAP выделены в аналитической плоскости.

Следующее, что нам нужно делать, — предоставлять потребителям данные в режиме реального времени. В этой главе мы поговорим о доставке обогащенных данных в режиме реального времени конечному пользователю. Этот этап конвейера данных представляет собой последний шаг, который потоковые данные проходят перед тем, как попасть к конечному пользователю.

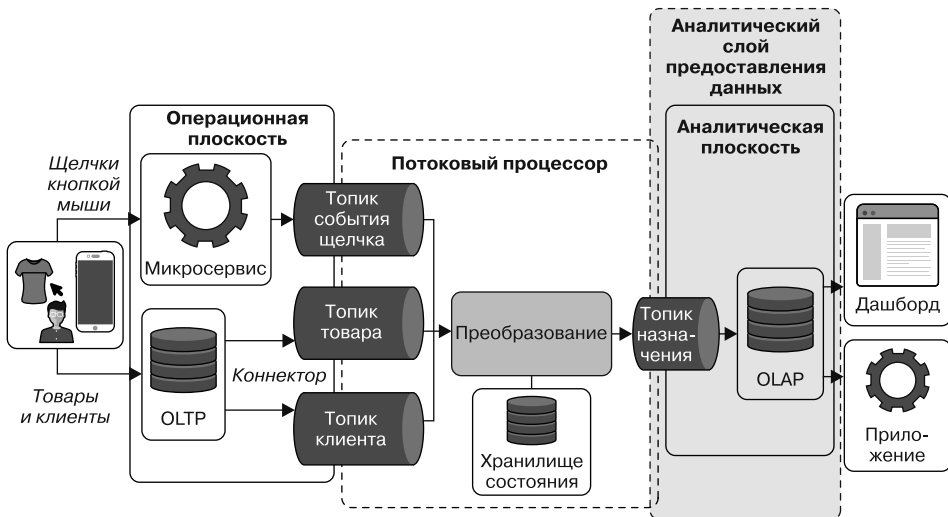


Рис. 3.1. Предварительно обработанные данные доступны в топике назначения для аналитической подачи в режиме реального времени на пользовательские дашборды или в приложения

Ожидаемый результат работы в режиме реального времени

Для предоставления аналитики в режиме реального времени определенным потребителям (людям и приложениям) необходимо рассмотреть набор соглашений об уровне обслуживания (SLA). В примере с использованием потока цифрового следа требования к конечному пользователю или приложению не указывались. Однако поскольку аналитику планируется предоставлять в режиме реального времени, следует учитывать некоторые параметры.

Задержка

Измеряет время выполнения аналитического запроса или вычисления и получения результатов. В аналитике реального времени низкая задержка имеет решающее значение для предоставления пользователям практически мгновенных выводов. Показатели SLA могут определять допустимые пороги задержки, например среднее или максимальное время отклика, для обеспечения своевременной доставки результатов аналитики.

Пропускная способность и конкурентность

Измеряет количество аналитических запросов или вычислений, которые могут быть обработаны за определенный промежуток времени. Этот показатель указывает на способность системы обрабатывать конкурентные запросы и особенно важен для сценариев с высокой интенсивностью работы. Показатели SLA могут задавать целевую пропускную способность, например количество запросов в секунду или вычислений в минуту, чтобы обеспечить достаточную пропускную способность для аналитических рабочих нагрузок в режиме реального времени.

Свежесть данных

Показывает, насколько актуальны результаты аналитики по отношению к базовым потокам данных. Этот показатель измеряет запаздывание между моментом создания данных и моментом, когда они становятся доступными для анализа. В показателях SLA могут быть указаны приемлемые требования к свежести данных, например максимальное запаздывание в секундах или минутах, чтобы обеспечить пользователям доступ к самой свежей информации.

Точность

Измеряет правильность и точность результатов аналитики. Показатели SLA могут определять допустимые коэффициенты ошибок, доверительные интервалы или критерии проверки для обеспечения точности аналитики в режиме реального времени.

При предоставлении потребителям аналитики в режиме реального времени описанные параметры больше всего влияют на требования обработки в режиме реального

времени. Существуют также показатели, связанные с определением вместимости, устойчивости данных и безопасности. Мы подробнее остановимся на них в последующих главах. К другим показателям SLA относятся следующие.

Доступность

Измеряет процент времени, в течение которого система аналитики в режиме реального времени работает и доступна для пользователей. Показатели SLA могут определять желаемое время бесперебойной работы или простоя системы, принимая во внимание запланированные окна технического обслуживания и неожиданные простои. Высокая доступность критически важна для обеспечения постоянного доступа к аналитическим возможностям в режиме реального времени.

Согласованность

Показатели обеспечивают согласованность результатов при выполнении различных запросов и обращении к разным репликам системы.

Масштабируемость

Измеряет способность системы аналитики в режиме реального времени справляться с растущими объемами данных, запросами пользователей и сложностью вычислений. Среди показателей SLA могут быть указаны контрольная производительность или пороговые значения пропускной способности. Это гарантирует возможность горизонтального или вертикального масштабирования системы для удовлетворения растущих потребностей.

Безопасность и конфиденциальность

Показатели SLA могут включать требования к безопасности и конфиденциальности для обеспечения защиты данных, контроля доступа, шифрования и соответствия нормативным требованиям. К ним относятся меры по предотвращению несанкционированного доступа, защите конфиденциальной информации и обеспечению целостности данных.

Нам нужно выбрать наиболее подходящую БД для предоставления данных. Она должна удовлетворять условиям соглашения SLA, связанным с обработкой в режиме реального времени, — показателям задержки, конкурентности, свежести и точности.

Выбор аналитического хранилища данных

Несколько типов хранилищ данных могут удовлетворять требованиям SLA для аналитики в режиме реального времени в зависимости от конкретных требований и ограничений конкретного варианта применения. И в нашем примере не было указано SLA для аналитики в режиме реального времени. Но в случаях использования потоков цифрового следа требуются запросы с очень низкой задержкой

для отслеживания изменений в запасах товаров и реакции на них, такой как динамическое пополнение запасов, динамическое ценообразование, перераспределение и распределение запасов, а также предиктивная аналитика и т. д.

Приведем несколько примеров хранилищ данных, удовлетворяющих этим строгим требованиям SLA в режиме реального времени.

- Базы данных in-memory, такие как Redis, SingleStore (бывший MemSQL), Hazelcast или Apache Ignite, хранят данные в памяти для быстрого доступа и обработки. Эти БД отличаются крайне низкой задержкой и высокой пропускной способностью, что делает их подходящими для аналитики в режиме реального времени, требующей практически мгновенных ответов.
- Хранилища данных RTOLAP, такие как Apache Pinot, Apache Druid, ClickHouse, StarRocks и Apache Doris. Как правило, это распределенные столбцовые хранилища данных. Они организуют данные по столбцам, а не по строкам, обеспечивая эффективную аналитику и обработку запросов. Эти БД могут обрабатывать большие объемы данных, обеспечивают отличную масштабируемость и высокую доступность, что делает их хорошо подходящими для масштабной аналитики в режиме реального времени.
- Хранилища данных с гибридной транзакционно-аналитической обработкой (Hybrid transactional/analytical processing, HTAP), такие как TiDB или SingleStore (бывший MemSQL), поддерживают как транзакционную обработку в режиме реального времени, так и аналитику в рамках одной системы. Эти базы данных позволяют выполнять аналитику в режиме реального времени непосредственно на оперативных данных, сводя к минимуму их перемещение и уменьшая время задержки.

При предоставлении данных из хранилища, озера данных или хранилища в озере данных не удастся достичь условий SLA в режиме реального времени, необходимых для некоторых из описанных ранее сценариев использования для целей инвентаризации.



Существует мнение, что термин «база данных» относится к базам данных OLTP. Если вы помните, базы данных OLTP обычно транзакционные и строчные. Они существуют в операционной плоскости данных, которая обслуживает записи в пользовательских приложениях. Те, кто согласен с этим предположением, предпочитают для систем обслуживания аналитических запросов использовать термин «хранилище данных». Мы будем придерживаться этих определений и можем чередовать их в тексте книги.

Некоторые из таких хранилищ данных обычно развертываются в аналитической плоскости, что совпадает с хранилищами данных в стиле OLAP. Другие будут развернуты только в операционной плоскости, например базы данных HTAP (о них мы подробнее поговорим в главе 7). В этой главе будем использовать то, что традиционно применяется в аналитических приложениях реального

времени, — хранилища данных RTOLAP. Способность получать данные из топика потоковой платформы, такой как Kafka, дает им возможность работать в режиме реального времени.

Топик в качестве источника

На рис. 3.1 было показано, что данные в режиме реального времени для нашего сценария были предварительно обработаны и помещены в топик назначения на потоковой платформе типа Kafka. Потоковые платформы предоставляют модель публикации и подписки для распространения потоковых данных. Это означает, что другие системы, домены и сценарии использования могут потреблять те же данные из топика назначения. Предварительная обработка на платформе обработки потоков может подготовить данные только к тому моменту, когда все потребители смогут их использовать, причем данные будут приведены к общему формату, соответствующему общему знаменателю для потребителей. Таким образом, специфичная подготовка данных для конкретного потребителя должна реализовываться самим потребителем. Этот сценарий показан на рис. 3.2.

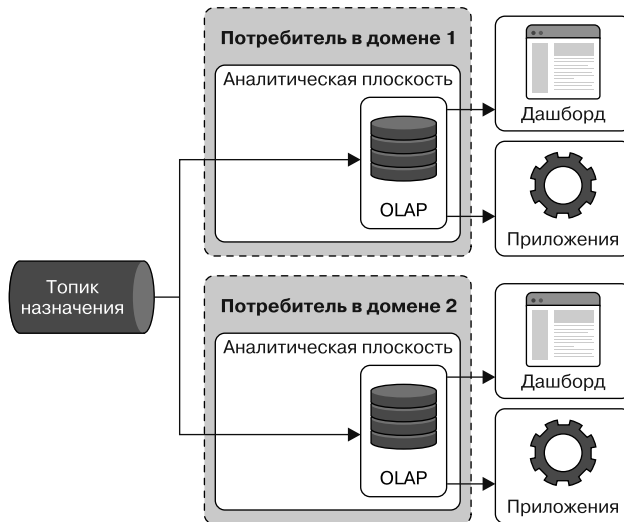


Рис. 3.2. Топик назначения может быть использован несколькими потребителями домена, требующими различных форматов временных меток

Допустим, потребителю в домене 1 нужны временные метки данных в виде секунд, а потребителю в домене 2 требуются данные в формате ГОД-МЕСЯЦ-ДЕНЬ. Инженер, реализовавший преобразование в платформе обработки потоков, может принять решение о предоставлении временной метки в миллисекундах.

И тогда оба потребителя смогут получить свои предпочтительные значения временной метки. Это удовлетворит и любого будущего потребителя, которому потребуются данные в миллисекундах.

В этих сценариях запросы должны реализовать преобразование, чтобы привести значение временной метки к нужному формату. К сожалению, это преобразование замедлит выполнение запроса. Кроме того, его нужно будет делать при каждом выполнении запроса. И в итоге это может нарушить SLA по задержке. Далее это вызовет замедление работы приложения или дашборда и создаст неудовлетворительные условия для потребителей данных. Любая подготовка данных, оставленная на усмотрение потребителя, всегда будет влиять на SLA при обслуживании аналитики в режиме реального времени. Этот сценарий показан на рис. 3.3.

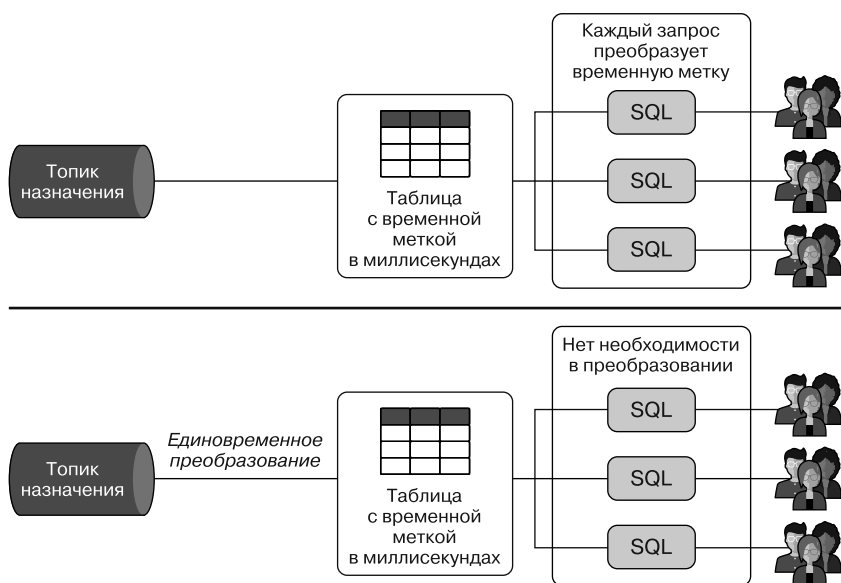


Рис. 3.3. Преобразование временных меток: по каждому запросу (*сверху*), только один раз (*снизу*)

Преобразования при поглощении

Во многих RTOLAP существует возможность предварительной обработки потоковых данных перед тем, как они попадут в постоянное хранилище. Она позволяет RTOLAP преобразовывать потоковые данные еще в процессе их передачи. В случае использования временных меток данные из топика могут проходить через набор преобразований без состояний, например преобразование миллисекунд в формат ГОД-МЕСЯЦ-ДЕНЬ.

Преобразования при поглощении похожи на потоковый процессор без статических данных. Это важно иметь в виду в последующих главах при обсуждении материализованных представлений. Потребители в топике вынуждены рассматривать эти данные как базовый формат, из которого можно извлечь свои предпочтительные форматы.

Большинство хранилищ данных RTOLAP могут выполнять только предварительную обработку данных в режиме реального времени без состояния перед их хранением. Но некоторые из них уже рассматривают возможность выполнения преобразований с учетом состояния как часть процесса поглощения данных.

Apache Pinot — пример хранилища RTOLAP с возможностью выполнять преобразования с учетом состояния при поглощении данных. В Pinot содержится обширный список индексов, благодаря которому запросы к Pinot выполняются с очень низкой задержкой. Один из индексов называется *индексом звездного де-рева*. Его преимущество заключается в возможности предварительной обработки агрегаций с хранением состояния. По сути, это преобразование с сохранением состояния, происходящее при поглощении. Подробнее об индексах и их важности мы поговорим позже в этой главе.

В хранилищах данных RTOLAP все чаще используют преобразования при поглощении с сохранением состояния, чтобы перевести данные из базового формата в предпочтительный. В будущих вариантах применения такие преобразования в RTOLAP могут объединяться с другими таблицами, чтобы обеспечить денормализованное представление в RTOLAP. Представления такого типа гораздо более производительны при обслуживании аналитических запросов, поскольку объединение выполняется только один раз перед сохранением данных. Это более оптимально, чем если бы пользователь при каждом запросе отправлял запрос на объединение.



Денормализованное представление, также называемое One Big Table (ОБТ), относится к способу организации данных с объединением и консолидацией информации из нескольких таблиц в одно представление или таблицу. Как правило, данные делятся на несколько таблиц. Каждая из них представляет определенную сущность или отношение для обеспечения целостности данных и минимизации избыточности и как итог — уменьшения объема хранения. Однако в некоторых аналитических сценариях может оказаться более эффективным объединить связанные данные в денормализованное представление. Оно упрощает составление запросов и анализ, поскольку вся необходимая информация доступна в одном месте, что устраняет необходимость в сложных объединениях таблиц. Денормализованные представления будут вносить избыточность и потенциально влиять на целостность данных, если ими не управлять особенно внимательно.

Преобразования при поглощении и потоковая обработка ведут себя одинаково, поскольку это асинхронные процессы, выполняющиеся в фоновом режиме. Асинхронные процессы не взаимодействуют с пользователем, а, наоборот, определяются и запускаются им, но не возвращают результаты напрямую — пользователь их не ждет. Мы вернемся к асинхронным и синхронным процессам позже в этой главе.

Преобразования при поглощении не только форматируют данные, но и оптимизируют их для быстрых аналитических запросов. Одна из таких оптимизаций — применение столбцовых форматов, чем пользуются многие OLAP-системы, такие как Pinot. Столбцовые форматы обычно не используются в базах данных OLTP, где применяются строчные. Подробнее об этом мы расскажем далее в главе.

OLTP и OLAP

Для наглядности при изучении способов предоставления данных можно сравнить базы данных OLTP и хранилища данных OLAP.

Напомним, что базы данных OLTP используются для транзакционных рабочих нагрузок в операционной плоскости. Они передают записи обратно в пользовательское приложение и предназначены для эффективной обработки транзакционных операций и управления ими в режиме реального времени. В первую очередь они применяются для поддержки повседневной деятельности, связанной с частым взаимодействием с данными. К таким операциям относятся создание (вставка), получение, обновление и удаление записей данных. Они также известны как акроним CRUD: создание (create), чтение, а вернее получение данных (read), модификация (update), удаление (delete).

Основная задача базы данных OLTP — обеспечить целостность и согласованность данных при обработке транзакций. Она позволяет получить быстрый и надежный доступ к данным для выполнения таких оперативных задач, как обработка заказов, управление запасами, финансовые операции и взаимодействие с клиентами. Базы данных OLTP оптимизированы для обработки большого количества конкурентных транзакций, часто с одновременным доступом к системе нескольких пользователей.

У таких БД обычно нормализованная структура данных, что означает минимальную избыточность данных и обеспечение их согласованности. Для них приоритетны транзакционные возможности, поэтому они поддерживают свойства ACID (atomicity — «атомарность», consistency — «согласованность», isolation — «изоляция», durability — «устойчивость») для обеспечения надежной и безопасной обработки данных.

ACID

ACID — это акроним, определяющий набор свойств для обеспечения надежности и согласованности данных при выполнении транзакций в базе данных OLTP.

Атомарность

Представьте себе транзакцию как единое, неделимое действие. Атомарность означает, что транзакция рассматривается целиком и либо все ее изменения применяются успешно, либо ни одно из них. Это гарантирует, что состояние базы данных останется неизменным, даже если во время транзакции что-то пойдет не так.

Согласованность

Согласованность гарантирует, что транзакция переводит базу данных из одного корректного состояния в другое. Это означает, что любые правила, ограничения или отношения, определенные в базе данных, сохраняются во время транзакции и после нее. Проще говоря, база данных сохраняет разумное и ожидаемое состояние, поэтому, когда множество клиентов извлекают из нее записи, все они получают одинаковое представление о БД, а все записи одинаковы и, следовательно, согласованны.

Изоляция

Изоляция означает, что несколько транзакций могут происходить конкурентно, не мешая друг другу. Каждая транзакция изолирована от других до своего завершения. Это предотвращает конфликты и гарантирует, что каждая транзакция обращается к согласованному представлению базы данных.

Устойчивость

Устойчивость гарантирует, что после успешного завершения транзакции ее изменения будут сохранены навсегда и переживут любые последующие сбои или падения системы. Изменения становятся постоянной частью базы данных, и данные надежно сохраняются для дальнейшего использования.

Соответствие стандарту ACID гарантирует, что транзакции базы данных надежны, последовательны, изолированы друг от друга и устойчивы. Так обеспечивается прочная основа целостности и надежности данных.

В противоположность им хранилища данных OLAP используются для аналитических нагрузок в аналитической плоскости. В них применяются иные методы оптимизации. Каждый тип хранилища данных оптимизирован для решения поставленных задач.

Строчная и столбцовая оптимизации

Ранее в этой главе мы познакомились со строчными и столбцовыми форматами. Они представляют собой два способа организации и хранения данных в БД.

В *строчном формате* данные хранятся и упорядочиваются строка за строкой. Это означает, что все столбцы конкретной строки хранятся вместе, затем хранятся данные следующей строки и т. д. Этот формат похож на то, как данные традиционно представляются в электронных таблицах. Он оптимизирован для транзакционной обработки, когда отдельные строки часто обновляются или вставляются.

В *столбцовом формате* данные хранятся и упорядочиваются столбец за столбцом. Это означает, что все значения определенного столбца хранятся вместе, затем хранятся значения следующего столбца и т. д. Этот формат оптимизирован для аналитической обработки, где запросы часто включают агрегирование, фильтрацию и анализ определенных столбцов или подмножеств данных. Больше различий — в табл. 3.1.

Таблица 3.1. Различия строчной и столбцовой оптимизации

Свойство	Строчные	Столбцовые
Хранение данных	В строчном формате данные каждой строки хранятся вместе. Такие форматы влияют на способ доступа к данным и их чтения во время запросов. Аналитические запросы не рекомендуются	В столбцовом формате данные каждого столбца хранятся вместе. Такие форматы обеспечивают лучшую оптимизацию для поддержки аналитических запросов
Производительность запросов	Строчные форматы обычно лучше подходят для транзакционных операций, связанных с получением целых строк или их модификацией	Столбцовые форматы лучше всего подходят для аналитических операций, поскольку они позволяют быстрее получать данные и обрабатывать определенные столбцы или подмножества данных
Сжатие	Строчные форматы предлагают некоторые методы сжатия, такие как сжатие с подавлением нулевых значений, кодирование длин серий, сжатие с использованием словаря и сжатие данных по определенному типу. Методы сжатия приводят к дополнительным вычислительным затратам во время операций чтения и записи данных, поэтому выбор и настройка алгоритмов сжатия должны быть сбалансированы в зависимости от конкретной рабочей нагрузки и требований к производительности системы БД	Столбцовые форматы часто обеспечивают лучший коэффициент сжатия. Поскольку столбцы обычно содержат повторяющуюся информацию, методы сжатия могут применяться более эффективно, что приводит к снижению требований к хранению данных

Свойство	Строчные	Столбцовые
Эффективность запросов	Строчные форматы могут потребовать считывания целых строк, даже если нужны всего несколько столбцов, что может привести к увеличению объема ввода-вывода на диске и снижению производительности запросов	Столбцовые форматы предназначены для чтения и обработки только необходимых для конкретного запроса столбцов. Это позволяет минимизировать объем ввода-вывода на диске и повысить производительность запросов

Когда потоковые данные в режиме реального времени поступают на уровень предоставления клиентам, ожидается, что они уже подготовлены для быстрых аналитических запросов, поскольку этот уровень находится в аналитической плоскости. Столбцовое хранилище обеспечивает наилучшую производительность и эффективность аналитических запросов.

Запросы в секунду и конкурентность

Количество запросов в секунду (Queries Per Second, QPS) — это показатель того, насколько хорошо хранилище данных OLAP может возвращать результаты запросов. Это, в свою очередь, указывает на объем запросов, которые OLAP может обрабатывать в течение секунды. В конечном счете так измеряется количество конкурентных запросов, которые могут быть вызваны одновременно.

В примере использования потока цифрового следа мы не указывали, сколько конечных пользователей могут просматривать аналитику. Предполагается, что на каждого конечного пользователя приходится хотя бы один запрос. Мы указали, что это пример с работой в режиме реального времени, то есть свежесть данных на дашборде должна обеспечиваться в режиме реального времени. Как правило, такие дашборды требуют высокой частоты обновления, чтобы диаграммы на них отображались в режиме реального времени. Для каждого обновления дашборда существует запрос.

Для 1000 пользователей, просматривающих аналитику цифрового следа с частотой обновления дашборда 5 с, формула QPS будет выглядеть следующим образом:

$$1000 \times 1 \text{ запрос / обновление каждые } 5 \text{ с} = 200 \text{ QPS.}$$

Это означает, что задержка запроса должен составлять 5 мс. Можно преобразовать 200 запросов/с в миллисекунды/запрос, взяв обратную величину:

$$1 / 200 = 5 \text{ мс / запрос.}$$

Необходимо протестировать один запрос, чтобы проверить, сможет ли хранилище данных OLAP вернуть результат в течение 5 мс. Это требование трудновыполнимо для 1000 конечных пользователей. Если дашборд открывают сторонние пользователи или используют в приложении на телефоне, то число конечных пользователей может превысить 1000. И, скорее всего, никто не хотел бы ограничивать свой бизнес только 1000 пользователей. Поэтому предстоит масштабировать OLAP-кластер для размещения большего их количества или упростить запрос, чтобы он выполнялся менее чем за 5 мс. А также можно применить лучшие стратегии индексирования.

Индексирование

Индексирование играет решающую роль в улучшении QPS для баз данных OLAP. Стратегическое применение индексов поможет улучшить показатель QPS без масштабирования OLAP-кластера, которое потребует больше оборудования и больших затрат.

Все хранилища данных RTOLAP используют набор индексов для помощи в работе с QPS. Индексы полезны по нескольким причинам. Например, индексирование позволяет базе данных быстрее находить и извлекать нужные данные. Создавая индексы на часто запрашиваемых столбцах, БД может избежать сканирования всего набора данных и напрямую обращаться к индексированным значениям. Это значительно сокращает время отклика запроса и, следовательно, улучшает показатель QPS.

Индексы также предоставляют ценную статистику и метаданные. Оптимизатор запросов может задействовать их для определения наиболее эффективного плана выполнения запроса. Это помогает оптимизировать распределение ресурсов, стратегии поиска данных и операции объединения, что в итоге повышает общую производительность запросов и улучшает показатель QPS.

Техника повышения производительности с названием «обрезка» (pruning) позволяет OLAP отсекал фрагменты данных, которые не содержат результатов. Этот подход сужает пространство поиска на основе конкретных значений столбцов. Такой выборочный доступ избавляет от необходимости сканировать весь набор данных (или пропускать большие подмножества данных), позволяя БД обрабатывать больший объем запросов за заданный промежуток времени, что также улучшает показатель QPS.

Индексы обеспечивают создание структуры данных, которая хранит данные так, что количество операций ввода-вывода на диск при обработке запросов уменьшается. Сокращая количество необходимых считываний с диска, индек-

сирование минимизирует время доступа к данным. В результате показатель QPS улучшается.

В качестве конкретного примера использования индексов в хранилище данных RTOLAP можно привести Apache Pinot. Эта СУБД поддерживает несколько типов индексов для оптимизации производительности запросов. В табл. 3.2 приведены некоторые из наиболее часто используемых в Apache Pinot индексов.

Таблица 3.2. Индексы, наиболее часто используемые в Apache Pinot

Индекс	Описание	Использование
Растровый индекс	Bitmap-индексы эффективны для столбцов с низкой кардинальностью (столбцы с небольшим количеством различных значений). Они создают битовый массив для каждого отдельного значения, указывая на его наличие или отсутствие в каждой строке. Bitmap-индексы очень эффективны для операций фильтрации и агрегирования	Столбцы с низкой кардинальностью
Отсортированный индекс	Отсортированные индексы хранят значения столбца в отсортированном порядке. Они особенно полезны для запросов на основе диапазонов и позволяют эффективно выполнять операции фильтрации и сортировки	Даты, временные метки
Инвертированный индекс	Инвертированные индексы подходят для столбцов с высокой кардинальностью (столбцы с большим количеством различных значений). Они сопоставляют каждое уникальное значение с соответствующим набором строк, содержащих это значение. Такие индексы эффективны при поиске по равенству и префиксу	Столбцы с высокой кардинальностью
Прямой индекс	Прямые индексы хранят исходные значения столбца в порядке их появления в данных. Они полезны для запросов на получение исходных значений без какой-либо фильтрации или агрегирования	Для столбцов без фильтрации или агрегирования
Текстовый индекс	Текстовые индексы предназначены для сценариев полного текстового поиска. Они обеспечивают эффективный поиск по ключевым словам, фразам или другим текстовым критериям	Текстовые столбцы для эффективного поиска

Эти индексы можно использовать по отдельности или в комбинации в зависимости от характера данных и специфики запросов. Выбор подходящего индекса для каждого столбца играет решающую роль в оптимизации производительности запросов в Apache Pinot. Важно отметить, что, хотя индексирование может значительно повысить производительность запросов и улучшить QPS, необходимо соблюдать баланс между количеством и размером индексов, поскольку

чрезмерное индексирование может негативно повлиять на операции вставки, изменения и удаления. Правильная стратегия индексирования и периодическое обслуживание индексов имеют решающее значение для оптимальной работы баз данных OLAP.

Как уже говорилось в этой главе, индекс звездного дерева может предварительно обрабатывать операции агрегирования с удержанием состояния. По сути, это преобразование с сохранением состояния, происходящее во время поглощения данных.

Метод индексирования звездного дерева используется для сортировки и предварительной группировки данных по нескольким полям на основе их кардинальности. Он предназначен для ускорения агрегирования и вычислительных запросов к большим наборам данных. Индекс звездного дерева работает как древовидная структура, где каждый уровень соответствует полям, используемым для фильтрации и агрегирования данных в вашем наборе данных (рис. 3.4). Корень дерева представляет собой *звезду* для объединения показателей этих полей. Каждый путь от корня к узлу листа — это комбинация значений полей и соответствующих им агрегированных показателей.

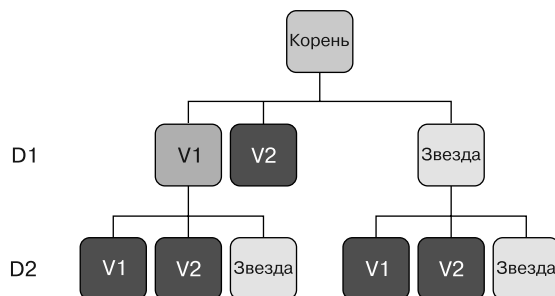


Рис. 3.4. Предварительно обработанные данные доступны в топике назначения для аналитической подачи в режиме реального времени на пользовательские дашборды или в приложения

Рассмотрим на простом примере, как работает индекс звездного дерева.

- Запрос начинается с корневого узла и идет вниз к первому измерению D1. Измерения звездного дерева упорядочены по кардинальности сверху вниз. В данном случае D1 — это столбец с наибольшей кардинальностью.
- Если предикат SQL имеет вид `where D1 = V2`, то записи агрегируются по узлу V2 в измерении D1:

Путь запроса = Корень → Сканирование V2.

- Если предикат SQL имеет вид `where D1 = V1`, то запрос переходит к узлу звезды во втором измерении под `V1` и возвращает предварительно агрегированное значение:

Путь запроса = Корень $\rightarrow$ `V1` $\rightarrow$ Звезда с предварительной агрегацией.

- Если предикат SQL имеет вид `where D1 = V1 and D2 = V1`, запрос сканирует записи `V1` под измерением `D2`:

Путь запроса = Корень $\rightarrow$ `V1` $\rightarrow$ Сканирование `V1`.

- Если предикат SQL имеет вид `where D2 = V2`, путь запроса идет к узлу звезды в измерении `D1`, поскольку `D1` не фильтруется. Затем он переходит к узлу `V2` и сканирует его записи:

Путь запроса = Корень $\rightarrow$ Звезда $\rightarrow$ Сканирование `V2`.

- И наконец, для запросов без предиката:

Путь запроса = Корень $\rightarrow$ Звезда $\rightarrow$ Звезда.

Определение порога ограничения количества записей для сканирования — это часть конфигурации звездного дерева. Если вы зададите порог разбиения 100 000, индекс звездного дерева гарантирует, что ваши запросы никогда не просканируют более 100 000 записей.

Индекс звездного дерева строится после создания сегмента на этапе его генерации. Сегмент в Apache Pinot — это логическая абстракция, представляющая собой фрагмент табличных данных с некоторым количеством строк. В сегменте хранятся данные для всех столбцов таблицы в столбцовом виде, а также словари и индексы для этих столбцов.



Сегменты похожи на шарды или разделы в реляционных базах данных и могут рассматриваться также как разделы на основе времени. Они позволяют распределенной архитектуре Pinot масштабироваться горизонтально, разбивая большие объемы данных на более мелкие управляемые фрагменты. После этого они распределяются между несколькими узлами.

Звездное дерево агрегирует данные по мере их поступления, чтобы входящие запросы могли использовать их в своих задачах. Эта оптимизационная деятельность похожа на предварительную обработку на платформе обработки потоков. Предварительная обработка в хранилище данных OLAP становится все более распространенной, поскольку очень трудно избежать работы по оптимизации в столбцовые форматы, оптимизации индексов и простых преобразований вроде

переформатирования временных меток. БД OLAP *чрезвычайно зависимы* от реализации потоковой обработки данных в режиме реального времени.

Предоставив возможность определять в OLAP преобразования при поглощении данных с сохранением состояния, пользователи могут создавать дополнительную логику предварительной обработки. Она будет выполняться один раз, а не при каждом вызове аналитического запроса (рис. 3.5).



Рис. 3.5. Пользователи хранилища данных OLAP могут определить преобразование при поглощении и аналитический запрос

Пользователи могут определить отдельную команду для поглощения данных в SQL или набор функций на этапе поглощения, чтобы упростить операции SQL на этапе аналитических запросов, выполнить их с меньшей задержкой и в итоге повысить показатель QPS хранилища данных OLAP. Эта техника также снизит потребность в масштабировании OLAP-кластера, поскольку объем работы аналитических запросов был уменьшен.

Предоставление результатов анализа

Показатель QPS указывает на хорошую производительность запросов, вызываемых пользователями или приложениями. Аналитические запросы, особенно в режиме реального времени, будут иметь высокую частоту обновления, что может вызвать нагрузку на систему OLAP. Это явление также называется *частыми опросами* или *поллингом*.

Синхронные запросы

Ранее мы описывали аналитические запросы, называемые *pull-запросами*. Это означает, что пользователь или приложение (клиент) отправляет запрос и извлекает данные из OLAP-системы. Работа выполняется по схеме «запрос — ответ».

Клиенты отправляют запросы через драйвер и диалект. Последний обеспечивает взаимодействие клиента и OLAP с использованием варианта SQL, характерного для системы OLAP.

Асинхронные запросы

Аналитические данные также могут предоставляться в асинхронном режиме. Этот тип запросов называется *push-запросом*, поскольку данные «вытаскиваются» клиентам. Преимущество push-запросов заключается в том, что измерение QPS больше не нужно, потому что оно относится исключительно к производительности pull-запросов. Клиенты push-запросов получают уведомления о новых аналитических данных, поэтому частые опросы не требуются.

Push-запросы, скорее всего, обслуживаются из топиков потоковой платформы. Клиенты могут подписаться на эти топики и, получив уведомление, повторно заполнить свои приложения или дашборды в режиме реального времени.

Такие сервисы, как Akivity/Zilla, предоставляют асинхронные API, которые позволяют клиентам подписываться на асинхронные аналитические данные. Для этого применяются события, отправляемые сервером (Server-Sent Events, SSE), или протоколы WebSocket. SSE и WebSocket — это технологии для связи в режиме реального времени между клиентами (обычно веб-браузерами) и серверами. Хотя они оба позволяют обновлять информацию в режиме реального времени, между ними есть ключевые различия.

Однонаправленная связь

SSE позволяют серверу передавать данные браузеру клиента, обеспечивая односторонний канал связи. Клиент получает обновления или события от сервера, но не может отправлять данные обратно по тому же самому соединению.

Формат потока событий

SSE используют специальный формат потока событий для передачи данных клиенту. Сервер отправляет данные в виде серии событий, а клиент обрабатывает эти события с помощью JavaScript. Формат потока событий включает в себя типы событий, данные и необязательные поля, такие как идентификаторы или временные метки.

Протокол HTTP

SSE построены на основе протокола HTTP и используют стандартные HTTP-соединения. При этом устанавливается долговременное соединение между клиентом и сервером. Для нескольких обновлений обычно задействуется одно HTTP-соединение.

Совместимость с браузерами

SSE поддерживаются большинством современных браузеров, но в старых браузерах или специфических конфигурациях могут быть ограничения. Для базовой функциональности SSE не требуются дополнительные плагины или библиотеки браузера.

Двунаправленная связь

Протоколы WebSocket поддерживают полнодуплексную связь, позволяя клиенту и серверу отправлять данные друг другу через одно соединение. Это обеспечивает связь в режиме реального времени в обоих направлениях.

Произвольный формат данных

У WebSocket гибкий формат данных с поддержкой передачи произвольных двоичных или текстовых данных. Они не используют особый формат потока событий, как SSE.

Протокол WebSocket

Механизмы WebSocket используют специализированный протокол WebSocket на основе TCP. Он обеспечивает постоянный двунаправленный канал связи с низкой задержкой.

Совместимость с браузерами

WebSocket поддерживаются большинством современных веб-браузеров, но в старых браузерах или специфических конфигурациях могут быть ограничения. Функциональность WebSocket требует встроенной их поддержки или использования библиотек либо фреймворков WebSocket.

SSE подходят для сценариев с передачей периодических уведомлений, обновлений и событий в режиме реального времени, а также потоковых данных сервером клиенту. Механизмы WebSocket хорошо подходят для приложений, требующих двунаправленной интерактивной связи в режиме реального времени. К ним относятся чат-приложения, многопользовательские игры или инструменты совместного редактирования.

Выбор между SSE и WebSocket зависит от конкретных требований приложения. SSE проще в реализации и широко поддерживается браузерами, в то время как WebSocket предлагают более продвинутую функциональность и возможности двунаправленного обмена данными.

Запросы push и pull

Синхронные запросы похожи на pull-запросы, а асинхронные соответствуют push-запросам. Важно понять этот факт сейчас, а более очевидным он станет в главе 4 при обсуждении материализованных представлений.

При использовании push-запросов данные активно передаются клиенту или приложению, когда в базе данных происходят изменения. Это позволяет обновлять информацию в режиме реального времени и получать немедленный доступ к новой информации, не выполняя частые опросы или запросы вручную.

Поскольку данные отправляются сразу же, как только становятся доступными, push-запросы могут обеспечивать меньшую задержку, чем pull-запросы. Не нужно ждать, пока клиент запросит данные, — они доставляются заранее.

Push-запросы хорошо сочетаются с событийно-ориентированными архитектурами. Они позволяют системам реагировать на события и запускать действия на основе полученных данных, обеспечивая более оперативный и событийно-ориентированный подход.

В то же время pull-запросы обеспечивают бóльшую гибкость, поскольку дают клиенту или приложению возможность получать данные по запросу, когда это необходимо. Данные извлекаются по запросу, что позволяет лучше контролировать время и частоту их получения.

Такие запросы сокращают ненужный сетевой трафик, поскольку клиент получает данные только при необходимости. Это может быть полезно, когда пропускная способность сети ограничена или извлекаемые данные нечасто меняются.

Pull-запросы могут быть более эффективными с точки зрения ресурсов, поскольку серверу не нужно активно рассылать данные всем клиентам или приложениям. Это избавляет от необходимости постоянно отслеживать и передавать данные, что позволяет снизить нагрузку на сервер и потребление ресурсов. Однако частые запросы могут вызвать очень высокое потребление ресурсов, поскольку хранилищу данных может потребоваться слишком часто просматривать большие объемы данных.

Выбор между запросами push и pull зависит от конкретных требований и сценария использования. Push-запросы обычно предпочтительны, когда важны обновления в режиме реального времени и немедленная доступность данных. А pull-запросы обеспечивают бóльшую гибкость и эффективность в сценариях, когда данные могут извлекаться по требованию и не слишком часто.

Но что, если получится совместить преимущества запросов push и pull? Сделать это можно следующим образом.

1. Клиент отправляет pull-запрос, чтобы получить текущее состояние таблицы в OLAP-системе.
2. Затем подписывается на изменения в таблице, то есть обновляет только изменившиеся записи.

Как и в случае с WAL в базе данных OLTP, клиент может сначала вызвать pull-запрос, чтобы получить моментальный снимок данных, а затем подписаться на журнал WAL таблицы, чтобы получать инкрементные изменения. Эта идея показана на рис. 3.6, она будет важна в главе 5 при обсуждении потоковых баз данных.

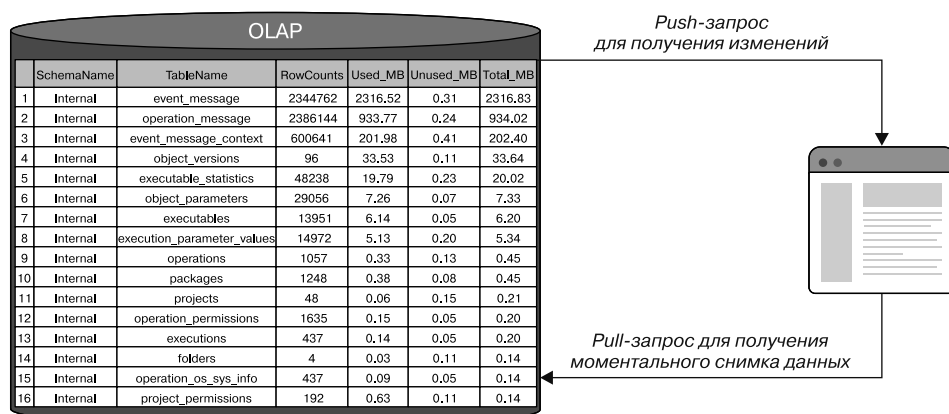


Рис. 3.6. Pull-запрос вызывается дашбордом и автоматически подписывается на изменения в представлении

К сожалению, хранилища данных OLAP не поддерживают асинхронные push-запросы. Эти хранилища обычно считаются тупиковыми в том смысле, что из них трудно извлечь необработанные значения. Они оптимизированы для обслуживания аналитических результатов, представляющих собой агрегированные или обобщенные данные.

Резюме

В этой главе мы представили хранилища данных OLAP и сравнили их с базами данных OLTP. Рассмотрели также методы оптимизации для хранилищ данных OLAP, которые используются для предоставления аналитических запросов с низкой задержкой. Рекомендуется всегда помнить о следующих важных концепциях:

- важности преобразований при поглощении данных с учетом состояния и о том, как они готовят данные для оптимизированных аналитических запросов;
- push- и pull-запросах и различных способах работы с аналитическими данными.

В главе 4 эти понятия будут представлять особую важность, поскольку мы подробно рассмотрим концепцию материализованных представлений.

Материализованные представления

В предыдущих главах мы лишь вкратце рассказали о структурах данных, называемых материализованными представлениями. Но перед изучением потоковых баз данных очень важно понять и изучить эту концепцию. Материализованные представления в базах данных впервые появились в начале 1990-х годов. Изначально они были разработаны как функциональная возможность повышения производительности запросов в некоторых БД OLTP за счет предварительного вычисления и хранения результатов сложных запросов. Материализованные представления позволяют хранить результаты запроса в виде физической таблицы. Для поддержания данных в актуальном состоянии ее можно периодически или по требованию обновлять. Этот подход позволяет сократить накладные расходы на многократное выполнение дорогостоящих запросов, так как дает пользователям возможность получать данные из материализованного представления.

При потоковой обработке материализованные представления обновляются не только периодически или по требованию — они всегда обновляются асинхронно в фоновом режиме. При поступлении новых данных материализованное представление немедленно обновляется, а результаты сохраняются. В предыдущих главах мы уже обращали внимание на эту закономерность. Асинхронное обновление соответствует потоковому, а синхронное — пакетному.

В видеоролике Мартина Клеппмана *Turning the Database Inside-Out* говорится, что материализованные представления — это не только предварительно обработанные данные, но и данные, непосредственно созданные на основе записей в журнал транзакций. Материализованные представления значительно повлияли на потоковую обработку. В них была представлена концепция предварительно вычисленных и постоянно инкрементно обновляемых результатов запросов. Эти структуры данных решают некоторые проблемы потоковой обработки и обеспечивают такие преимущества, как повышение производительности запросов, уменьшение дублирования данных и упрощение аналитики.

Представления, материализованные представления и инкрементные обновления

В материализованных представлениях логика обработки для генерации определенных результатов запроса отделена от основного конвейера обработки. В результате такого разделения может получиться более модульный и управляемый код. Он поможет упростить сопровождение и расширение системы обработки потоков.

Чтобы понять, что такое материализованные представления, сначала следует разобраться с *традиционными представлениями*. И те и другие находятся в базе данных. Традиционные представления, или просто представления, определяются оператором SQL. Он выполняется, когда клиент запускает операцию получения данных из представления. Результаты представления не сохраняются. Это увеличивает время ожидания для запросов выбора данных из представления, поскольку результаты предварительно не обрабатываются. Чтобы лучше понять этот процесс, давайте снова воспользуемся аналогией. У нас есть умный бурундук по имени Саймон (рис. 4.1).

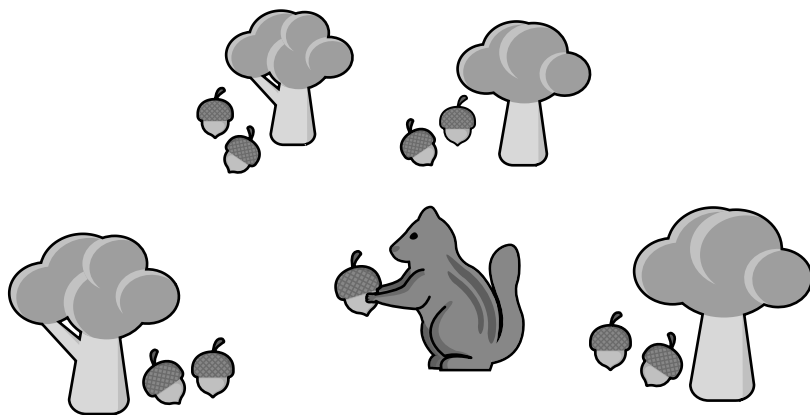


Рис. 4.1. Бурундук считает желуди, иллюстрируя традиционное представление

Спросим Саймона: «Сколько желудей сейчас у нас во дворе?» Саймон выбегает во двор и считает желуди, а затем возвращается и сообщает их количество. Если снова спросить его, сколько желудей во дворе, он снова бежит пересчитывать желуди и называет количество. Оба раза пришлось ждать, пока Саймон сосчитает желуди, даже если их количество не изменилось. Это похоже на традиционное представление и математически показано на рис. 4.2.

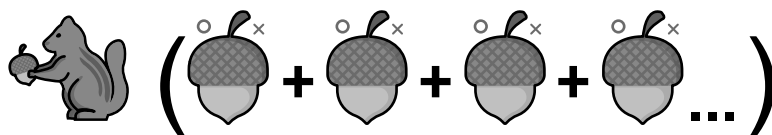


Рис. 4.2. Умного бурундука можно представить в виде функции, которая агрегирует желуди во дворе и возвращает их количество

Очевидно, что такой подход неэффективен. Поэтому поручим Саймону написать общее количество желудей на бумаге и разместить ее в рамке. Затем спросим у него текущее количество желудей, и Саймон не сможет ответить, потому что будет слишком занят поиском изменений в количестве желудей во дворе. Поэтому добавим другого бурундука и назовем его Элвин. Он не так умен, как Саймон, и просто будет называть число в рамке. Эта аналогия похожа на материализованное представление.

В ней бурундуки — это операторы SQL. Рамка во втором сценарии — это хранилище материализации представлений для сохранения предварительно посчитанных результатов. В этом же сценарии Саймон, предварительно подсчитавший желуди, умнее Элвина, озвучивающего значение в рамке (рис. 4.3). Элвин сообщает значения с малой задержкой и без особых усилий может одновременно обслуживать множество клиентов.

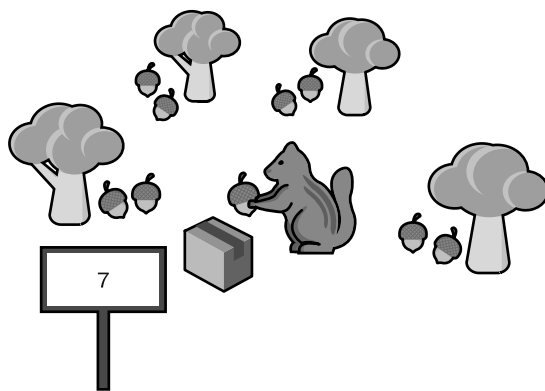


Рис. 4.3. Описание материализованного представления с использованием бурундуков

Важная часть аналогии с материализованным представлением заключается в том, что Саймон не считает все желуди — он ищет инкрементные изменения в их количестве. Для этого необходимо указать, сколько желудей было убрано со двора и сколько добавлено (или упало с деревьев).

Инкрементные изменения — это процесс внесения небольших целенаправленных изменений в существующие данные, а не пересчет всего набора данных с нуля. Эти обновления обычно применяются для того, чтобы поддерживать согласованность и актуальность данных с течением времени без вычислительных затрат на повторную обработку всего набора данных.

Математически инкрементная функция представлена на рис. 4.4. X представляет собой текущее состояние желудей во дворе, а Δ — инкрементное изменение их количества. X уже хранится в памяти, а умный бурундук фиксирует значение ΔX и добавляет к нему текущее состояние X . После этого происходит переход к следующему состоянию.

Чтобы фиксировать инкрементные изменения, Саймон всегда должен следить за новыми изменениями асинхронно — аналогично процессам при потоковой обработке.

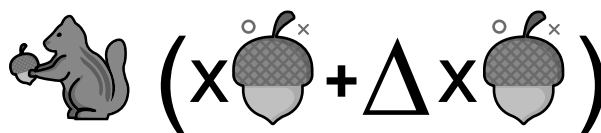


Рис. 4.4. Умный бурундук вносит инкрементные изменения в общее количество желудей во дворе

Вспомните процесс CDC (захват изменений данных) из главы 1. Он представляет собой яркий пример инкрементных изменений. Итак, CDC — это техника для захвата и отслеживания изменений, внесенных в базу данных или источник данных с течением времени путем чтения журнала WAL в базе данных OLTP. Вместо того чтобы обрабатывать весь набор данных с нуля, CDC выявляет и фиксирует только инкрементные модификации: вставки, изменения и удаления.

Захват изменения данных

Существует взаимосвязь между CDC и материализованными представлениями. Последние выполняют тяжелую работу — предварительные вычисления, отслеживая инкрементные изменения и сохраняя результаты. А процесс CDC предварительно предоставляет инкрементные изменения из WAL в базу данных OLTP. Это означает, что можно использовать материализованное представление для предварительной обработки CDC с инкрементными изменениями.

Возвращаясь к аналогии с бурундуком, мы попросили Саймона посчитать количество желудей во дворе. Давайте немного расширим этот пример и определим, что там много видов желудей. У каждого желудя есть свойства (или атрибуты):

- цвет;
- местоположение (широта, долгота).

Цвет желудей может меняться по мере старения, и их могут перемещать или забирать другие животные. Саймон следит за этими изменениями, вставляя, изменяя или удаляя каждый желудь из списка на бумаге в рамке. Таким образом, когда клиент запрашивает список, он видит только последний статус каждого желудя во дворе.

Технически этот сценарий проиллюстрирован на рис. 4.5. Отметим несколько важных моментов на схеме.

- Слева находится журнал WAL из основной/OLTP базы данных. Он реплицируется для создания копии основной базы данных.
- Используя коннектор CDC, WAL также записывается в топик потоковой платформы. Этот топик публикует журнал WAL основной базы данных, и на него могут подписаться другие системы.
- Sink-коннекторы могут потреблять данные из топика и создавать реплики в других системах баз данных.
- Поточковые процессоры могут создавать одну и ту же реплику базы данных в своем кэше.

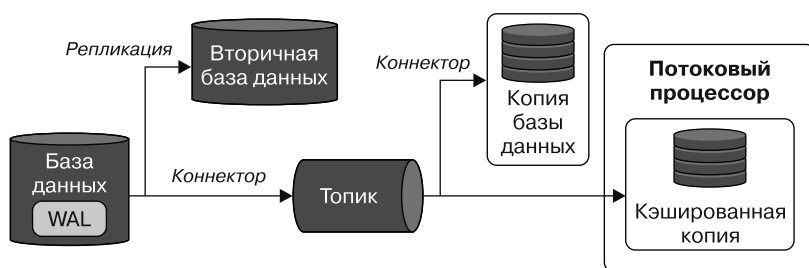


Рис. 4.5. Репликация с использованием инкрементных изменений

С помощью этой техники можно создать копию основной базы данных OLTP из пользовательского приложения в любом последующем хранилище данных или механизме потоковой обработки. Мы сосредоточимся на механизме потоковой

обработки прежде всего потому, что он удовлетворяет требованиям работы в режиме реального времени и не заставляет использовать семантику пакетной обработки.

В главе 3 мы познакомились с запросами push и pull. Если применить аналогию с бурндуком, то Саймон — это push-запрос, а Элвин — pull-запрос.



Когда мы говорим об умных (Саймон) и более простых (Элвин) бурндуках, подразумевается сложность SQL-запроса. Саймон может выполнять сложные преобразования и агрегации, в то время как Элвин выполняет простые SQL-запросы низкой сложности.

Запросы push и pull

Давайте расширим аналогию с бурндуком. С помощью push-запроса (он же Саймон) можно запросить результат Элвина без задержки, возникающей при синхронном вычислении результата.

Вернемся к исходному варианту использования, где Саймон подсчитывает количество желудей во дворе. Напомним, что Саймон работает асинхронно, следит за изменениями количества желудей и записывает все обновления в рамке. В каком-то смысле Саймон передает (push) результат в рамку. Элвин синхронно предоставляет содержимое рамки клиенту. Аналогично во время запроса Элвин извлекает результат из рамки и передает его клиенту. Подведем итоги:

- Саймон — это асинхронный push-запрос;
- Элвин — это синхронный pull-запрос.

Саймон выполняет большую часть работы по вычислению результатов, чтобы Элвин мог сосредоточиться на предоставлении результатов с низкой задержкой, как только к нему поступит запрос. Такая схема работает очень хорошо, но есть один недостаток — гибкость клиента, инициирующего запрос, недостаточна для постановки более сложных вопросов. При формировании аналитических данных в режиме реального времени он использует только количество желудей. А если клиенту нужен подсчет средних и максимальных значений или объединение нескольких таблиц? В этом случае push-запрос сводит на нет его возможность задавать более развернутые вопросы.

Для увеличения гибкости запросов придется пойти на компромисс с задержкой, поскольку мы указываем механизму предоставления данных выполнять больше работы (рис. 4.6). Если запрос инициирован пользовательским приложением, необходимо, чтобы он выполнялся с наименьшей задержкой, поскольку предпо-

лагается, что с приложением будут работать множество конечных пользователей. И наоборот, если требуется максимальная гибкость с возможностью нарезать данные на фрагменты и получать более обширные сведения, то следует ожидать, что эти запросы будут выполнять только несколько опытных конечных пользователей.

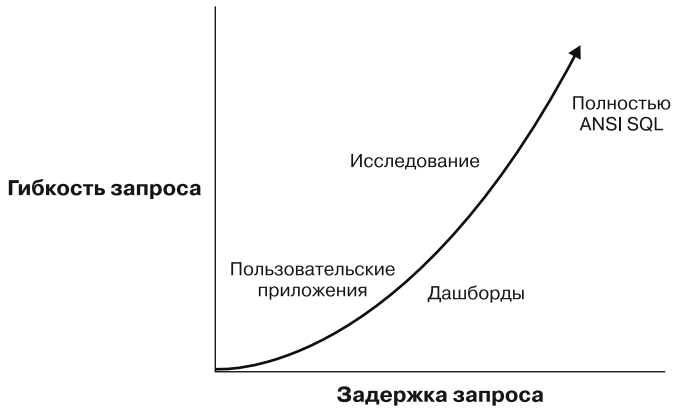


Рис. 4.6. Диаграмма компромиссов pull-запросов при увеличении гибкости и соответствующие примеры применения

Если задуматься, то приложения с требованиями самых низких задержек больше всего выиграют от использования push-запросов. На рис. 4.7 показано, как можно установить баланс между push- и pull-запросами.

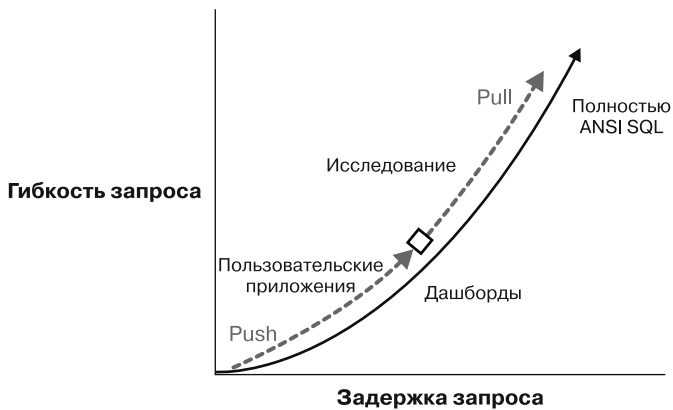


Рис. 4.7. Когда время ожидания запроса приближается к нулю, предпочтение отдается push-запросам

Квадрат в центре представляет собой материализованное представление. Оно уравнивает тяжелую нагрузку push-запросов с гибкостью pull-запросов. Способы установки баланса разных видов запросов зависят от сценария использования. Если флажок перемещается вниз по линии, то материализованное представление обеспечивает менее гибкие запросы, но производительность возрастает. И наоборот, чем выше флажок, тем более гибкими становятся запросы, но при этом растет задержка их выполнения. Запросы push и pull работают совместно для поиска правильного баланса между задержкой и гибкостью (рис. 4.8).



Рис. 4.8. Совместная работа pull- и push-запросов для обеспечения баланса между задержкой и гибкостью

А есть ли способ обеспечить одновременно высокую гибкость и низкую задержку, не прибегая к двум SQL-запросам? Это можно осуществить с помощью материализованных представлений, передающих изменения в журнале WAL. Для клиента это будет работать так.

1. Клиент отправляет push-запрос, и тот создает материализованное представление.
2. Затем клиент подписывается на изменения в материализованном представлении точно так же, как выполняется подписка на журнал WAL.

При таком подходе клиент отправляет push-, а не pull-запрос. Чтобы добиться гибкости, необходимой для запросов ad hoc (нестандартных моментальных), клиенту позволяется вносить изменения в push-запрос. Кроме того, подписка на изменения материализованного представления избавляет от проблем с задержкой запросов, поскольку инкрементные изменения передаются клиенту по мере их поступления. Это означает, что клиенту больше не нужно инициировать запрос и ждать его результата — таким образом снижается время ожидания. Достаточно одного SQL-запроса, чтобы клиент начал получать аналитические данные в режиме реального времени.

Сегодня эту схему трудно реализовать, поскольку push- и pull-запросы обычно обрабатываются в отдельных системах. Push-запрос зачастую выполняется в потоковом процессоре, а pull-запрос — в OLAP-системе обслуживания конечных пользователей. Более того, запросы обычно составляются разными командами инженеров. Инженеры по работе с потоковыми данными пишут push-запросы,

а аналитики или разработчики пользовательских приложений инициируют pull-запросы.

Чтобы избежать этой дилеммы, вам понадобится система с поддержкой возможностей:

- обработки потоков, таких как создание материализованных представлений;
- предоставления материализованных представлений топикам потоковой платформы, подобно журналам WAL;
- оптимального хранения данных для их предоставления;
- обеспечения синхронных и асинхронных методов предоставления данных.

Эти функциональные возможности доступны только в потоковых базах данных. Они способны объединить платформы потоковой обработки и базы данных, используя один и тот же механизм обработки SQL для данных как в движении, так и в состоянии покоя. Более подробно поговорим об этом в главе 5.

Наиболее распространенным решением для аналитики в режиме реального времени считается применение платформы обработки потоков, например Apache Flink, и хранилища данных RTOLAP, например Apache Pinot (рис. 4.9).

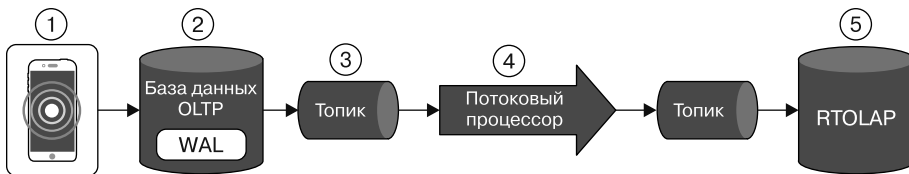


Рис. 4.9. Общее решение для аналитики в режиме реального времени

На рис. 4.9 показан путь перемещения данных из БД OLTP в систему RTOLAP для предоставления клиенту. Давайте рассмотрим эту архитектуру подробнее.

1. Сущности представлены в виде таблиц в базе данных OLTP в соответствии с предметно-ориентированным проектированием.
2. Приложение вставляет, изменяет или удаляет записи в таблице. Эти изменения записываются в журнал WAL базы данных.
3. Коннектор CDC считывает данные WAL и записывает изменения в топик потоковой платформы. Она предоставляет для доступа WAL базы OLTP, публикуя изменения в топиках/разделах, имитирующих структуру WAL. Потребители могут считывать их для создания копий таблиц из исходной базы данных OLTP.

4. Поточковый процессор — одна из систем для считывания топика и построения внутренних копий таблиц с помощью материализованных представлений. Когда материализованное представление обновляется асинхронно, оно выводит свои изменения в другой топик.
5. Хранилище данных RTOLAP считывает топик с выводом материализованного представления и оптимизирует данные для аналитических запросов.

На рис. 4.9 потоковый процессор выполняет push-запрос на этапе 4, а pull-запрос вызывается на этапе 5. И снова каждый запрос выполняется в отдельной системе, и авторами их выступают разные инженеры.

На рис. 4.10 более детально показаны сложность и разделение запросов на push и pull. Запрос push выполняет трудоемкую задачу сложных преобразований и сохраняет результат в материализованном представлении. Оно записывает свои изменения в локальное хранилище в топик потоковой платформы, которая выставляет материализованное представление на слой предоставления данных, содержащий систему RTOLAP.

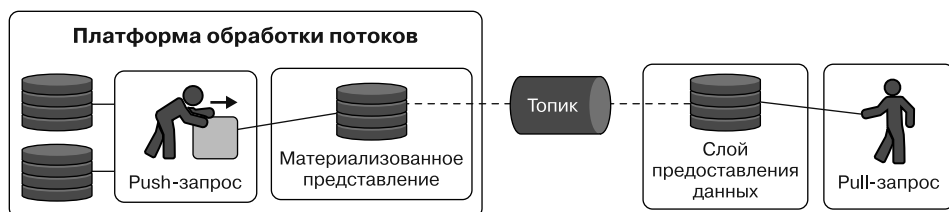


Рис. 4.10. Pull-запрос извлекает результаты из двух сохраненных таблиц через представление

В результате у конечного пользователя системы RTOLAP нет гибкости в определении логики предварительной обработки, необходимой для того, чтобы запрос выполнялся с низкой задержкой (рис. 4.11).

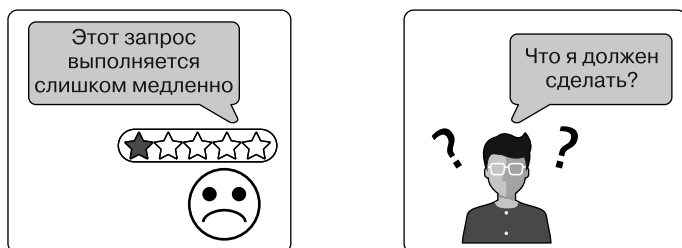


Рис. 4.11. Конечный пользователь пытается заставить инженера по обработке данных оптимизировать запрос

Избежать подобных сценариев можно, если конечный пользователь будет составлять запрос и предоставлять логику оптимизации потоковых данных. К сожалению, такие ситуации возникают очень часто из-за современного состояния потоковых архитектур.

Проблема усугубляется при попытке напрямую реплицировать данные CDC в систему RTOLAP.

CDC и Upsert

Термин *upsert* образован слиянием слов *update* (изменить) и *insert* (вставить). Он описывает логику приложения при вставке и/или обновлении таблицы базы данных¹. Upsert описывает логику, при которой приложение проверяет, существует ли запись в таблице базы данных. Если запись существует, вызывается оператор обновления. Если же записи не существует, приложение вызывает оператор вставки, чтобы добавить ее в таблицу.

Данные CDC содержат инкрементные модификации, такие как вставка, изменение и удаление данных. Логика upsert обрабатывает два из трех типов модификаций в потоке CDC (к операции delete мы вернемся позже)².

Операции upsert могут косвенно улучшить производительность и точность запросов выборки в определенных сценариях. Хотя сами эти операции в основном направлены на модификацию данных, они могут положительно влиять на производительность и точность запросов выборки, поддерживая целостность данных и оптимизируя их хранение. Операции upsert могут способствовать реализации этих улучшений следующим образом.

Целостность и точность данных

Upsert помогают поддерживать целостность данных, обеспечивая их точность и согласованность и предотвращая дублирование записей. Когда запросы выборки получают данные из БД с помощью правильных операций upsert, они с большей вероятностью вернут точную и надежную информацию.

Упрощенные pull-запросы

Выборка из таблицы с правильными операциями upsert упрощает запросы при поиске. А необходимость выполнять дедупликацию или фильтрацию последних записей усложняет SQL-код и увеличивает время задержки при его выполнении.

¹ Значение слова, полученного в результате соединения двух или более слов или их частей, оказывается некоторой комбинацией их значений.

² Во множестве СУБД операция UPDATE состоит из этапов DELETE и INSERT. Следовательно, в таких системах UPSERT включает в себя также операцию DELETE.

Операции upsert ведут себя как push-запросы, помогая оптимизировать и упростить pull-запросы. Это один из факторов контроля баланса между запросами push и pull. Чтобы лучше понять это, рассмотрим сценарий работы CDC на рис. 4.12.

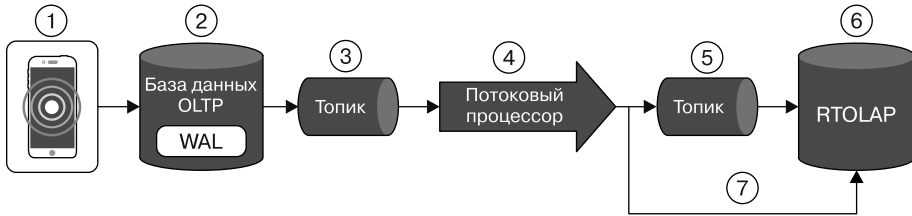


Рис. 4.12. Этапы, описывающие репликацию в хранилище данных RTOLAP

1. Транзакция из приложения отправляется для вставки, изменения или удаления записи в таблице базы данных OLTP. Предположим, что сценарий использования заключается в обновлении запасов зеленых футболок, поэтому в вопросе речь идет о таблице товаров — Products.
2. Обновление записывается в журнал WAL базы данных OLTP.
3. Предположим, что коннектор, считывающий WAL, только что был запущен. Ему потребуется сделать моментальный снимок таблицы Products для получения текущего статуса.
 - Если у коннектора нет этого снимка, последующие системы не смогут создать точную зеркальную копию таблицы Products.
 - Сделав моментальный снимок таблицы, коннектор создает начальные события. Они логически эквивалентны вставке для каждой записи в таблице Products.
 - Как только этот снимок будет доступен в топике, можно будет создать копию таблицы. Нельзя создавать копии только с инкрементными изменениями.
4. Если топик используется впервые, то при запуске потоковый процессор считывает его с самого начала. В противном случае он начинает считывание с сохраненного смещения. Считывание топика с начала позволяет потоковому процессору построить копию таблицы Products. И снова отметим: невозможно создать копию таблицы только с инкрементными изменениями.
 - Сложные преобразования реализуются в потоковом процессоре. Они потребуют от него создания материализованного представления, которое представляет собой копию таблицы Products.

- Операции преобразования выполняются над или между табличными конструкциями, такими как материализованные представления. Если преобразование не требуется, создавать материализованное представление не нужно и поток может проходить напрямую от входного топика к выходному.
5. Выходной топик похож на входной тем, что в нем хранится снимок данных для создания последующих копий. Однако в потоковом процессоре он подвергнется преобразованиям. Для данных CDC содержимое топиков в этом конвейере должно обеспечивать создание копий в дальнейшем.
 6. Если хранилище данных RTOLAP считывает данные из топика напрямую, ему придется самостоятельно обрабатывать логику операции `upsert`. Для этого ему также потребуется понять данные в топике, чтобы определить операции вставки, изменения и удаления. И впоследствии их можно будет применить к существующей внутренней таблице.
 7. Данный этап — альтернатива этапу 6. В этом случае потоковый процессор отправляет данные непосредственно в хранилище данных RTOLAP. Для RTOLAP без поддержки операций `upsert` потоковый процессор должен будет выполнить логику `upsert` вместо системы RTOLAP.

Поскольку операции `upsert` по определению поддерживают только вставку и изменение, то удаление обычно не производится. Некоторые системы реализуют операцию `upsert` с логикой удаления. Другие, например Apache Pinot, помечают запись как удаленную с сохранением возможности восстановления ее предыдущих версий. В этих случаях важно использовать RTOLAP-реализацию `upsert`, которая требует, чтобы хранилище RTOLAP выполняло считывание непосредственно из топика вывода. Некоторые RTOLAP могут не предоставлять функциональность удаления, и эту работу придется выполнять в потоковом процессоре.



На этапе 3 рассказывается о том, как сделать моментальный снимок таблицы `Product` в топике. В первой главе мы говорили о том, что у топиков есть период хранения, по истечении которого старые записи уничтожаются. Для данных CDC необходим другой тип топика — сжатый. В нем процесс усечения данных сохраняет только последнюю запись каждого первичного ключа. Это позволяет сохранять более старые данные, обеспечивая материализацию последующих копий таблиц, включая накопленные записи.

В итоге мы получаем две точки возможной реализации логики `upsert` — в системе RTOLAP или в потоковом процессоре. Более простой и предпочтительный подход — считывание данных из топика вывода хранилищем RTOLAP с последующим применением им же логики `upsert`. Топик вывода также служит

буфером в случаях, когда потоковый процессор производит данные быстрее, чем RTOLAP может их потреблять.

Upsert подчеркивает сложность того, что двум системам реального времени приходится *бороться за владение* такой сложной логикой или *избегать ее*. Эти проблемы приведут к дальнейшим разногласиям между инженерами по обработке данных и конечными пользователями аналитических систем.

Сформировать концепцию CDC в потоковой обработке может оказаться непростой задачей, потому что этот процесс задействован в большом количестве конструкций и сложной логике. Например, он связан с журналами WAL в базе данных OLTP, для сохранения истории требует наличия сжатых топиков на потоковых платформах, нуждается в применении upsert для упрощения и ускорения pull-запросов, а также должен быть материализован в представлениях. Сложность увеличивается, когда между OLTP-источником и RTOLAP-хранилищем данных задействовано несколько систем для создания копии таблицы Products. Существуют способы консолидации этих систем, позволяющие сократить избыточность и снизить сложность. И потоковые базы данных — один из способов добиться такой консолидации.

Преобразования с обогащением потребуют объединения нескольких потоков в потоковом процессоре. Вспомните два типа потоков: потоки изменений и потоки только с добавлением. Потоки изменений содержат данные об изменениях для таких сущностей в бизнес-домене, как товары и клиенты. Потоки только с добавлением содержат события, например данные цифрового следа из приложения. Давайте еще раз пройдемся по конвейеру потоковых данных, чтобы понять, как это реализовать.

Объединение потоков

Операции преобразования выполняются над табличными конструкциями, содержащими потоки изменений (материализованные представления) и потоки только с добавлением. Последние похожи на потоки изменений, в которых единственными разрешенными изменениями будут операции вставки. Фактически можно рассматривать все табличные конструкции в базах данных как последовательности изменений, входящих в табличную структуру и исходящих из нее.

Потоки только с добавлением не передаются в материализованные представления преимущественно потому, что последние должны хранить результаты. Поскольку в этих потоках доступны только операции вставки и их размер постоянно растет, в какой-то момент место для хранения закончится. И события цифрового следа не записываются в базу данных, потому что она тоже закончится.

Оба вида потоков представлены в виде табличных конструкций, которые во многих потоковых системах называются по-разному. В этой книге применительно к таблицам в потоковом процессоре мы будем использовать следующие понятия.

Таблицы только с добавлением

Табличная конструкция, в которой хранятся потоки только с добавлением. Эти конструкции не задействуют хранилище состояний. Они представляют данные, проходящие через потоковый процессор.

Таблицы изменений

Эта табличная конструкция представляет материализованные представления. Таблицы изменений используют хранилище состояний.

Точно так же необходимо разделить топики потоковой платформы. Тип потоковых данных в топиках определяет, как их можно обработать или представить в табличной форме. Для определения топиков в потоковой платформе мы применяем следующие понятия.

Топики только с добавлением

Топики, содержащие данные только с добавлением.

Топики изменений

Топики, содержащие события изменения или события CDC. Некоторые инженеры Kafka также называют их *топиками таблиц*.

На основе этих понятий можно лучше описать, как объединяются потоки, поскольку логика может быть запутанной. Следует использовать SQL в качестве языка для определения объединений и преобразований, потому что SQL — это универсальный язык для выполнения манипуляций с данными, а его механизм должен объединять потоки и базы данных. Применение движка SQL для управления данными в движении и в состоянии покоя приводит к созданию потоковой базы данных.

Apache Calcite

Начнем с объединения таблицы с добавлением и таблицы изменений, описанных в главе 2. SQL-код в примере 4.1 создан на основе фреймворка управления данными Apache Calcite. Он используется для создания баз данных с применением реляционной алгебры — формального математического способа описания операций над реляционными базами данных. То есть это набор правил и символов для выполнения манипуляций и запросов данных в таблицах. Таблицы также называются отношениями (от Relations).

Множество частей механизма Apache Calcite построены на основе математических операций, но в нем отсутствуют некоторые ключевые функции: хранение данных, алгоритмы их обработки и репозиторий для хранения метаданных. Apache Calcite послужит универсальным строительным блоком для создания базы данных с нуля. Фактически он применяется во множестве существующих систем реального времени: Apache Flink, Apache Pinot, Apache Kylin, Apache Druid, Apache Beam, Apache Hive и многих других.

«Calcite намеренно не вмешивается в процесс хранения и обработки данных... Это делает его отличным выбором для посредничества между приложениями и одним или несколькими местами хранения данных и механизмами их обработки. Он также играет роль прекрасной основы для создания базы данных — просто добавьте данные».

Документация Apache Calcite

Именно это мы и будем делать — просто добавлять данные. Вернемся к примеру с потоком цифрового следа. Здесь существуют три источника данных, каждый в своем топике потоковой платформы.

Пример 4.1. Объединение с таблицами топиков

```
CREATE SINK clickstream_enriched AS
SELECT
    E.*,
    C.*,
    P.*
FROM CLICK_EVENTS E ❶
JOIN CUSTOMERS C ON C.ip=E.ip and ❷
JOIN PRODUCTS P ON P.product_id=E.product_id ❸
WITH (
    connector='kafka',
    topic='click_customer_product',
    properties.bootstrap.server='kafka:9092',
    type='upsert',
    primary_key='id'
);
```

❶ **CLICK_EVENTS** — таблица с добавлением, полученная из топика с добавлением. Она содержит события щелчка из пользовательского приложения.

❷ **CUSTOMERS** — таблица изменений, полученная из топика изменений. Она содержит события изменений из базы данных OLTP, захваченные с помощью коннектора CDC.

❸ **PRODUCTS** — таблица изменений, полученная из топика изменений. Она также содержит поступившие через коннектор CDC события изменений из базы данных OLTP. Здесь следует предположить, что значение идентификатора товара было извлечено из URL-адреса щелчка мышью и помещено в отдельный столбец `product_id`.

Платформы потоковой обработки могут представлять данные в топиках в виде табличных структур, если в них есть поддержка SQL-кода. Это позволит использовать язык SQL и инструменты наподобие Calcite для определения сложных преобразований. Пример 4.1 представляет собой внутреннее объединение совпадающих записей во всех трех таблицах — **CLICK_EVENTS**, **CUSTOMERS** и **PRODUCTS**.

Выходом любого потокового SQL-кода для агрегирования или объединения потоков станет материализованное представление. В данном случае выполняется объединение.

CLICK_EVENTS

Таблица с добавлением, содержащая события щелчка кнопкой мыши.

CUSTOMERS

Таблица изменений/материализованное представление клиентов.

PRODUCTS

Еще одна таблица изменений/материализованное представление товаров.

Далее приведем список свойств различных типов объединений таблиц.

Таблица с добавлением и таблица с добавлением

В этом случае всегда применяется оконный режим, иначе в хранилище состояний закончится место.

Таблица изменений и таблица изменений

Окно не требуется, поскольку результат объединения может поместиться в хранилище состояний, если его размер будет соответствующим.

Таблица изменений и таблица с добавлением

Здесь также требуется оконный режим, иначе в хранилище состояний закончится место¹.

¹ В Kafka Streams и ksqlDB для таблицы с добавлением можно использовать материализованные представления KTable или GlobalKTable. В этом случае окно не требуется, поскольку на выходе снова получается поток.

Стоит отметить, что, когда поток только для добавления становится частью операции объединения, необходимо применять оконный режим для ограничения объема данных в хранилище состояний.

При выполнении SQL-операции левого внешнего объединения потоков из таблицы с добавлением и таблицы изменений результат определяется таблицей с добавлением.

В SQL такое объединение выглядит следующим образом:

```
SELECT ...  
FROM append_table_stream  
LEFT JOIN change_table_stream ON join_condition;
```

Здесь параметры `append_table_stream` и `change_table_stream` представляют собой два входных потока, которые мы объединяем, а параметр `join_condition` задает условие сопоставления двух потоков.

Левый поток (`append_table_stream`) управляет результатом и определен первым в операторе `FROM`. Результат будет содержать все события из левого потока, и для каждого из них будут добавляться совпадающие события из правого потока (`change_table_stream`) на базе условия `join_condition`¹.

Давайте проиллюстрируем это на примере двух потоков: *щелчков кнопкой мыши* и *клиентов*. Каждое событие в первом потоке представляет собой щелчок кнопкой мыши с идентификатором клиента, а каждое событие во втором потоке — клиента с его идентификатором. Чтобы объединить два потока по идентификатору клиента, нужно написать SQL-запрос следующим образом:

```
SELECT k.product_id, c.customer_name  
FROM click k  
LEFT JOIN customers c ON k.customer_id = c.customer_id;
```

В этом примере `click` выступает в роли левого потока, и он управляет результатом объединения. Запрос извлекает соответствующее имя клиента из потока `customers` на основе совпадающего идентификатора для каждого события в потоке `click`.

Важно отметить, что при потоковой обработке объединение происходит непрерывно и динамично. По мере поступления новых событий во входные потоки результат объединения постоянно обновляется и выдается в виде потока результатов. Это позволяет выполнять обработку и анализ потоковых данных в режиме реального времени с помощью команд SQL.

¹ Такова логика работы обычного `left join`. — *Примеч. науч. ред.*

Пример использования цифрового следа

Давайте взглянем немного шире и пошагово разберем всю схему, приведенную на рис. 4.13.

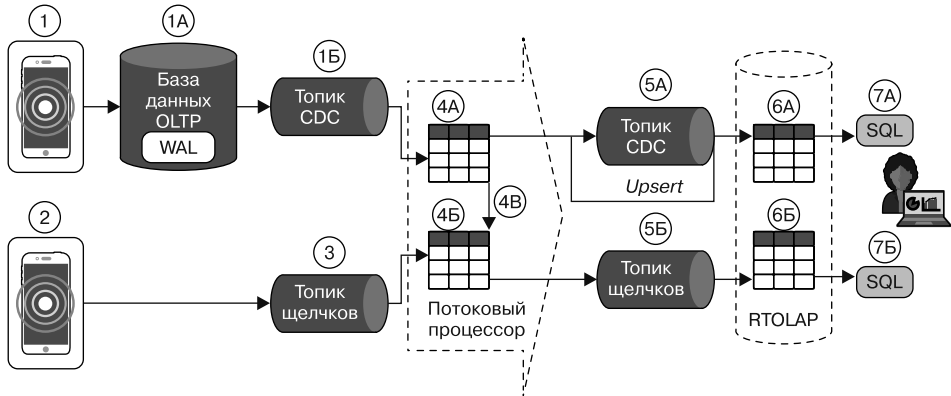


Рис. 4.13. Путь CDC и событий только с добавлением от приложения к RTOLAP

1. Клиент обновляет информацию о себе.
 - А. Информация сохраняется в базе данных OLTP.
 - Б. Процесс CDC запускается в БД OLTP, фиксирует изменения в таблице `CUSTOMERS` и записывает их в топик CDC. Этот топик сжатый, и его можно считать копией таблицы `CUSTOMERS`. Он позволит другим системам создавать свои копии таблицы `CUSTOMERS`.
2. Тот же покупатель щелкает на товаре в приложении электронной коммерции.
3. Событие щелчка записывается в топик. Мы не сохраняем эти события в БД OLTP, потому что они представляют собой только операции вставки. Их хранение в базе данных OLTP может привести к тому, что в ней закончится место для хранения.
4. Поточковый процессор считывает данные из CDC и топика щелчков мышью.
 - А. Это сообщения из топика `CUSTOMERS` таблицы изменений в потоковом процессоре. Они находятся в хранилище состояний, а его вместимость зависит от размера окна. В случае использования потоков Kafka или `ksqlDB` сообщения полностью хранятся в представлении `KTable`.
 - Б. Это сообщения из топика `CLICK_EVENTS` таблицы с добавлением в потоковом процессоре.

- В. Выполняется левое внешнее объединение сообщений таблицы с добавлением `CLICK_EVENTS` и таблицы изменений `CUSTOMERS`. Его результатом выступает поток `CLICK_EVENTS`, обогащенный соответствующей информацией из потока `CUSTOMER` (если она существует).
5. Поточковый процессор записывает свои выходные данные в указанные далее топики.
 - А. Это топик изменений, содержащий изменения CDC `CUSTOMER`. Он будет лишним, поскольку топик на этапе 1Б содержит те же данные. Но мы оставили его на диаграмме, чтобы она была сбалансированной.
 - Б. Это топик с добавлением, содержащий исходные данные потока `CLICK_EVENT`, дополненные данными потока `CUSTOMER`.
 6. Топики попадают в хранилище данных `RTOLAP` для предоставления клиентам в режиме реального времени.
 - А. Это копия исходной таблицы `CUSTOMERS` в базе данных `OLTP`, созданная на основе топика изменений.
 - Б. Эта таблица содержит обогащенные данные потока `CLICK_EVENTS`.
 7. Пользователь инициирует запросы к хранилищу данных `RTOLAP`.
 - А. Он может напрямую запросить таблицу `CUSTOMERS`.
 - Б. Он может запрашивать обогащенные данные потока `CLICK_EVENTS`. Ему не требуется самостоятельно выполнять их объединение, поскольку эта операция уже выполнена в потоковом процессоре.

Вы можете реализовать операцию объединения либо в потоковом процессоре, либо на стороне пользователя. В данном случае мы решили предварительно объединить данные потоков `CLICK_EVENTS` и `CUSTOMER`, чтобы повысить производительность запроса с точки зрения пользователя. Тяжелую работу по объединению выполняет потоковый процессор, так что хранилище `RTOLAP` может сосредоточиться на быстрых запросах с низкой задержкой. В таком сценарии потоковый процессор создает материализованное представление, которое записывается в топик на этапе 5Б (см. рис. 4.13). Хранилище `RTOLAP` строит копию материализованного представления в себе из топика с этапа 5Б. В БД `RTOLAP`, возможно, придется реализовать схему хранения с удалением старых обогащенных данных потока `CLICK_EVENTS`, чтобы избежать переполнения хранилища.

В качестве альтернативы можно было бы обойтись без потокового процессора и позволить `RTOLAP` осуществить объединение при выполнении запроса пользователем. Такой подход не потребует создания материализованного представления и избавит от необходимости управлять еще одной сложной потоковой системой. Но в этом виде запрос будет медленным и создаст большую нагрузку на систему `RTOLAP`.

Как же уменьшить сложность архитектуры, но при этом получить производительность материализованных представлений? Именно в этой ситуации можно объединить потоковую обработку с базами данных реального времени, используя потоковые базы данных.

Резюме

Я сделаю очень смелое заявление: все виденные вами раньше базы данных — потоковые.

*Muxay Будуй. Building a Streaming
Incremental View Maintenance Engine with
Calcite. Март 2023 года*

Традиционно обработка потоков и БД рассматривались как разные сущности: системы обработки потоков работали с непрерывно поступающими данными в режиме реального времени, а БД управляли постоянными данными, которые можно запросить. Однако материализованные представления бросают вызов этому разделению, преодолевая разрыв между двумя системами.

Материализованные представления позволяют создавать предварительно вычисленные постоянные сводки данных из потоковых источников. Эти структуры данных служат кэшем для хранения результатов вычислений или агрегаций в удобном для запросов виде. Это означает, что вместо применения систем обработки потоков для анализа в режиме реального времени можно использовать материализованные представления для хранения и запроса обобщенных данных без необходимости их постоянной обработки.

Материализованные представления обладают рядом преимуществ, поскольку объединяют преимущества потоковой обработки и баз данных. Они позволяют выполнять сложную аналитику на потоковых данных более эффективно и масштабируемо. Вместо того чтобы обрабатывать весь набор данных для каждого запроса, материализованные представления хранят предварительно вычисленные результаты, что позволяет быстрее и оперативнее выполнять запросы.

Более того, они способствуют беспрепятственной интеграции парадигм потоковой и пакетной обработки. Материализованные представления можно использовать для хранения промежуточных результатов конвейеров потоковой обработки, обеспечивая мост между непрерывным потоком данных и пакетной аналитикой, обычно выполняемой в базах данных. Такая интеграция позволяет унифицировать модели обработки и упростить общую архитектуру систем с интенсивным использованием данных.

Итак, материализованные представления стирают границы между обработкой потоков и базами данных, позволяя использовать постоянные доступные для запросов сводки потоковых данных. На основе преимуществ обеих систем обеспечиваются эффективная и масштабируемая аналитика в режиме реального времени, интеграция собранных ранее данных и данных реального времени, а также слияние парадигм потоковой и пакетной обработки. Применение материализованных представлений открывает широкие возможности для создания интеллектуальных и отзывчивых систем данных. Они способны справиться с динамическим характером потоковых данных, обеспечивая при этом быстрые и гибкие запросы.

Мы также представили две конструкции, имеющиеся в базах данных OLTP, которые приближают их к потоковым технологиям.

Журнал WAL

Конструкция, фиксирующая изменения в таблицах базы данных.

Материализованное представление

Асинхронный запрос предварительной обработки и хранения данных для обеспечения запросов с низкой задержкой.

В главе 1 мы впервые узнали о фразе Мартина Клеппмана «вывернуть базу данных наизнанку». И действительно сделали это.

1. Взяли конструкцию WAL в OLTP и опубликовали ее на потоковой платформе, например Kafka.
2. Использовали функциональность материализованных представлений и ее имитацию на платформе обработки потоков с состоянием. Это избавило БД OLTP от выполнения сложных преобразований и позволило сосредоточиться на обработке транзакций и обслуживании данных путем их передачи в потоковый слой.

Так мы сформировали базу для разговора о потоковых базах данных в следующей главе.

Здесь мы снова перевернем представление о потоковой парадигме, вернув WAL и материализованные представления в базу данных. Иными словами, вывернем потоковые архитектуры наизнанку.

Потоковые базы данных

В электронной таблице можно поместить в ячейку формулу (например, сумму ячеек в другом столбце), и при изменении входного значения формулы ее результат будет автоматически пересчитан. Именно это нам нужно на уровне системы данных: когда запись в базе данных изменяется, требуется, чтобы любой индекс для этой записи автоматически обновлялся, а все кэшированные представления или агрегаты, зависящие от этой записи, автоматически обновлялись. Вы не должны беспокоиться о технических деталях того, как происходит обновление, а можете просто верить, что оно работает правильно.

Мартин Клеппман. Высоконагруженные приложения

В предыдущей главе мы узнали, что значит вывернуть базу данных наизнанку, как метко выразился Мартин Клеппман. Этот процесс включал в себя извлечение WAL из базы данных во *входные потоки изменений*, создание *материализованных представлений* поверх них и запись обработанных данных обратно в *выходные потоки изменений*. В отличие от материализованных представлений в классических базах данных, таких как Oracle или Postgres, где интервалы обновления составляют от нескольких минут до нескольких часов, материализованные представления в платформах обработки потоков, таких как Flink, Kafka Streams, ksqlDB или Samza, могут обновляться непрерывно — при каждом новом изменении.

Идея выворачивания базы данных наизнанку позволила нам создавать материализованные представления с более свежими данными, чем когда-либо прежде. Однако по сравнению с простой классической установкой БД это повлекло за собой дополнительные сложности. Для того чтобы постоянно обновляемые материализованные представления, создаваемые Flink, Kafka Streams, ksqlDB или Samza, *имели смысл*, выходные потоки изменений должны были попадать в дополнительную внешнюю базу данных, например в БД RTOLAP, такую как

Druid, Pinot, ClickHouse или Rockset. Таким образом, с архитектурной точки зрения выворачивание базы данных наизнанку заставило нас развернуть и эксплуатировать три системы: потоковую платформу, потоковый процессор и внешнюю базу данных — вместо одной классической базы данных¹. И что еще сложнее, реализовать это могли только высокооплачиваемые и редкие специалисты по обработке потоков, а не обычные эксперты по базам данных. Поэтому с глобальной точки зрения эта идея осталась за бортом: ее применяют только в таких случаях, как обнаружение мошенничества, где низкая задержка действительно имеет первостепенное значение.

Мы считаем, что идея вывернуть базу данных наизнанку стала первым важным шагом на пути к созданию моста между миром потоковых технологий и миром баз данных и проложила путь к значительным достижениям в области обработки потоков. Но весь путь еще не пройден.

В этой, центральной, главе книги мы поможем вам совершить путешествие от обработки потоков с учетом состояния, материализованных представлений на основе потоков и хранилищ состояний к новому понятию материализованных представлений, основанному на их первоначальном определении из мира баз данных. Мы покажем вам, как новые *потокowe базы данных*, такие как ksqlDB, Materialize, RisingWave и Timeplus, собираются положить последние кирпичики на мост между миром потоковой обработки и миром баз данных. И сделаем следующий логический шаг после выворачивания базы данных наизнанку — снова вывернем ее вовнутрь. Для начала рассмотрим компоненты, знакомые вам по многим диаграммам.

Определение потоковой базы данных

На представленных ранее диаграммах в основном содержатся компоненты, показанные на рис. 5.1. Пока не будем обращать внимания на коннекторы для чтения из систем и записи в них.



Рис. 5.1. Части системы потоковой обработки

¹ Если внешняя база данных «из коробки» не поддерживает чтение из потоковой платформы, потребуется четвертый архитектурный компонент — промежуточное ПО для коннекторов (Kafka Connect, Striim, StreamSets, HVR и т. д.).

Слева направо на рис. 5.1 показаны следующие компоненты.

База данных

Может относиться к одному из трех описанных ранее типов хранилищ: OLTP, RTOLAP и внутренние хранилища состояния в потоковом процессоре. Их различия определяют, как хранить и запрашивать данные.

Топик

Представляет собой конструкцию, имитирующую WAL в базе данных OLTP. В них публикуются потоки данных для других баз данных и потоковых процессоров.

Потоковые процессоры

Это приложения для преобразования потоков данных. В них хранится внутреннее состояние.

Создание материализованного представления

Это процесс предварительного вычисления результата и сохранения его в базе данных. Материализованные представления создаются в БД или потоковом процессоре, и в обоих случаях необходим слой хранения данных.

Можно организовать эти компоненты, чтобы построить топологию потоков, представляющую собой поток данных. Предложенный ранее поток выглядит следующим образом.

Пример	Описание
База данных → Топик → Потоковый процессор → Топик → База данных	Простой поток

Ранее наше внимание было сосредоточено на типичном аналитическом потоке реального времени. Мы использовали его в качестве решения для примера обработки событий щелчков.

Пример	Описание
OLTP → Топик → Потоковый процессор → Топик → RTOLAP	БД OLTP передает данные в RTOLAP
Микросервис → Топик → Потоковый процессор → Топик → RTOLAP	Микросервис захватывает события щелчков, поступающие в ту же RTOLAP

Но нет никаких препятствий для реализации решения, где выходом будет другая или даже та же самая база данных OLTP. Это важное различие станет

очевидным, когда мы начнем говорить о разных типах потоковых баз данных и материализованных представлений далее в этой главе.

Пример	Описание
OLTP → Топик → Поточковый процессор → Топик → OLTP	Местом назначения потока служит другая или та же база данных OLTP

Во всех этих потоках точное местоположение материализованного представления неоднозначно. Есть три компонента, в которых материализуется результат потока.

- Поточковый процессор создает материализованное представление и сохраняет результат в своем внутреннем хранилище состояний.
- Изменения в материализованном представлении передаются в топик вывода.
- Ни внутреннее хранилище состояния потокового процессора, ни топик вывода нельзя запросить напрямую. Таким образом, целевая база данных переносит изменения в собственное материализованное представление, соответствующее таблице. Эта таблица оптимизирована для аналитических запросов конечных пользователей.

Единственное, что мы знаем наверняка, — то, что материализованное представление не считается элементом первого класса (first-class citizen), потому что охватывает все три компонента, изображенные в середине рис. 5.2.

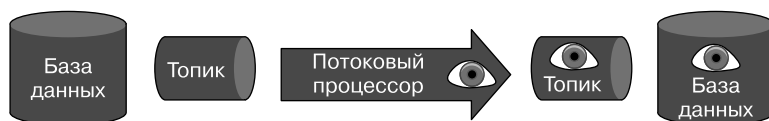


Рис. 5.2. Материализованное представление распространяется на несколько систем

Что, если объединить эти три компонента? Мы получим пример потоковой базы данных. Если объединить потоковый процессор и базу данных, то больше не понадобится топик для отображения изменений в материализованном представлении хранилища состояний потокового процессора. И тогда хранилище состояния потокового процессора и база данных станут единым целым (рис. 5.3).

Эта потоковая БД будет способна обрабатывать запросы push и pull. Напомним, что push-запрос — это асинхронный процесс, он выполняется в фоновом режиме как потоковый процессор, а pull-запрос — это аналитический запрос, инициируемый конечным пользователем. Это означает, что один и тот же механизм обработки SQL должен поддерживать запросы push и pull.

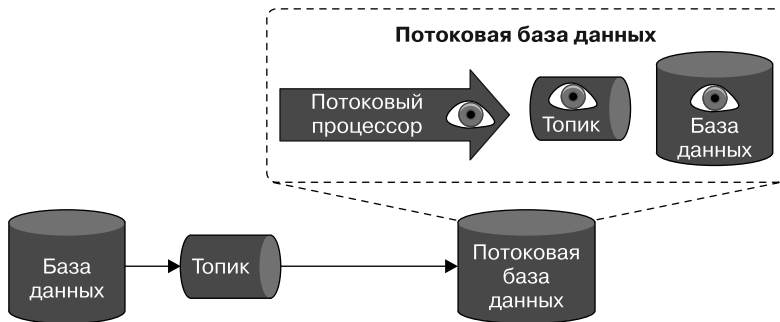


Рис. 5.3. Потоковая база данных формируется при сближении потокового процессора и БД

Консолидация хранилища в одном месте решает только половину проблемы. Гораздо сложнее создать единый механизм SQL с поддержкой обработки данных в состоянии покоя и в движении. Стороне потоковой обработки базы данных может потребоваться перейти от использования своего внутреннего хранилища состояния к реальной БД при объединении и агрегировании данных. Кроме того, объектам потоковой базы данных может потребоваться различать таблицы данных в движении и состоянии покоя.

Потоковая база данных на основе столбцов

Возвращаясь к сценарию использования потока цифрового следа, можно сказать, что конечным пунктом потокового конвейера было хранилище данных RTOLAP. Обновим эту диаграмму, чтобы задействовать столбцовую потоковую базу данных (рис. 5.4).

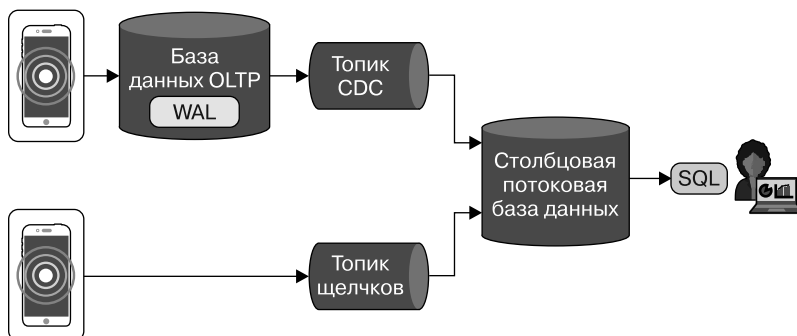


Рис. 5.4. Вариант использования потока цифрового следа, в котором потоковая БД применяется для аналитических запросов

Это решение дает возможность пользователям потоковой базы данных создавать запросы push и pull с помощью SQL-кода в одном месте. Push-запрос будет представлять собой SQL-запрос для создания материализованного представления, как в примере 5.1.

Пример 5.1. Создание материализованного представления из двух потоков данных

```
create materialized view CUSTOMER_CLICKS
as select * from CLICK_EVENTS E
join CUSTOMERS C on C.ip = E.ip
```

Приложение также может вызвать pull-запрос, как в примере 5.2.

Пример 5.2. Приложение инициирует pull-запрос, возвращающий данные о клиенте

```
select * from CLICK_EVENTS E
join CUSTOMERS C on C.ip = E.ip
where C.id = '123'
```

Код из примера 5.1 вызывается тем, что ранее было механизмом потокового процессора, а код из примера 5.2 — тем, что ранее было механизмом SQL в хранилище RTOLAP для выполнения операции с низкой задержкой.

Потоковая база данных на основе строк

На рис. 5.5 показано, что потоковая база данных на основе строк может даже стать местом назначения конвейера потоковых данных. Базы данных на основе строк, такие как БД OLTP, оптимизированы для быстрых операций чтения и записи, связанных с транзакционными данными (например, с транзакциями CRUD — создание, получение, модификация, удаление). Системы OLAP оптимизированы для аналитических запросов, таких как агрегирование. Зачем использовать БД на основе строк в качестве места назначения, если в данном случае речь идет о выполнении аналитических запросов?



Использовать БД OLTP для аналитических нагрузок обычно не рекомендуется из-за ряда фундаментальных проблем, связанных с производительностью, блокировкой и конкурентностью, использованием ресурсов и т. д.

Решение загадки из предыдущего абзаца заключается в том, что нет необходимости выполнять аналитические запросы к потоковым базам данных на основе строк. И все еще есть возможность опубликовать результаты в топике вывода и впоследствии передать их в базу данных RTOLAP для аналитических запро-

сов, как показано на рис. 5.5, *справа*. Но консолидация систем не уменьшает сложности. Происходит лишь замена одного потокового процессора другим — на этот раз потоковой базой данных на основе строк, и материализованное представление снова растягивается на несколько систем: потоковую базу данных на основе строк, топик и хранилище RTOLAP.

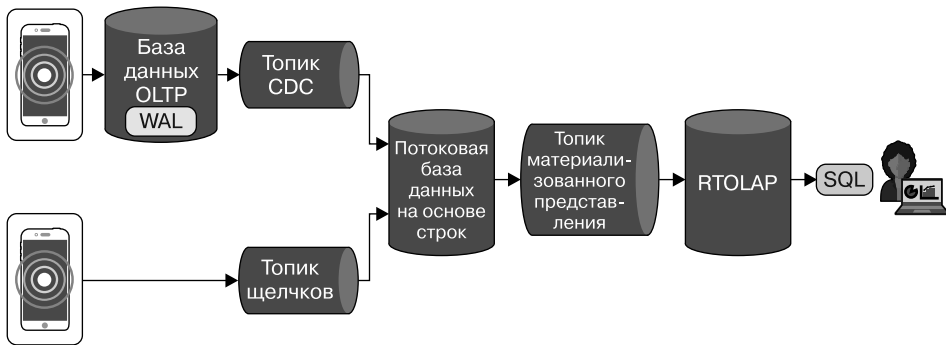


Рис. 5.5. Потоковая база данных на основе строк заменяет потоковый процессор

По своей природе потоковая передача представляет собой отправку данных строка за строкой, что лучше подходит для работы с потоковыми базами данных на основе строк. Лучшим решением для таких БД считается хранение преобразованных данных внутри приложения с помощью парадигмы Command Query Responsibility Segregation (CQRS).

Это архитектурный паттерн, разделяющий операции чтения данных (запросы) и операции изменения данных (команды) на отдельные компоненты. Суть CQRS состоит в том, чтобы создать отдельные модели для чтения и записи данных. Это позволяет оптимизировать производительность и масштабируемость, поскольку операции чтения и записи можно независимо оптимизировать под конкретные требования. CQRS помогает повысить скорость реакции системы, улучшить масштабируемость и обеспечить лучшее соответствие моделей данных их предполагаемому применению.

На рис. 5.6 данные в топике вывода записываются обратно в исходную БД OLTP. Лучшим решением будет обновление на основе топика вывода отдельной базы данных, используемой клиентским приложением только для чтения. Этот паттерн обеспечивает конечную согласованность (eventual consistency) между базами данных OLTP для записи и чтения. Кроме того, можно использовать доступную приложению потоковую базу данных в качестве БД только для чтения в паттерне CQRS (рис. 5.7).

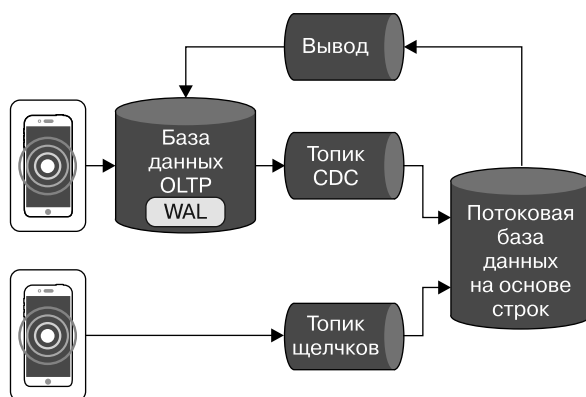


Рис. 5.6. Паттерн CQRS реализован с помощью потоковой базы данных на основе строк

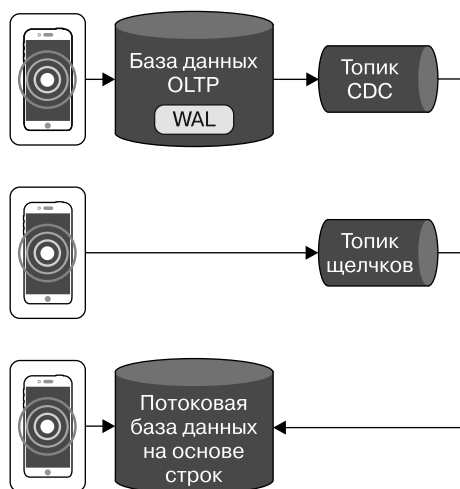


Рис. 5.7. Паттерн CQRS, где потоковая БД используется приложением в качестве БД только для чтения

Потоковые базы данных на основе строк значительно приближают потоковую передачу к границе — ее часто называют *веб-границей*.



Понятие «веб-граница» обычно относится к внешнему слою или границе веб-приложения либо сервиса. Это точка взаимодействия между приложением и внешним миром, включая пользователей, клиентов и другие системы. Веб-граница отвечает за обработку входящих запросов, их выполнение и маршрутизацию к соответствующим компонентам в архитектуре приложения.

Граничные базы данных, подобные потоковым

До сих пор мы объединяли потоковый процессор, топик и базу данных назначения в потоковую БД на основе столбцов (рис. 5.8, *справа*).

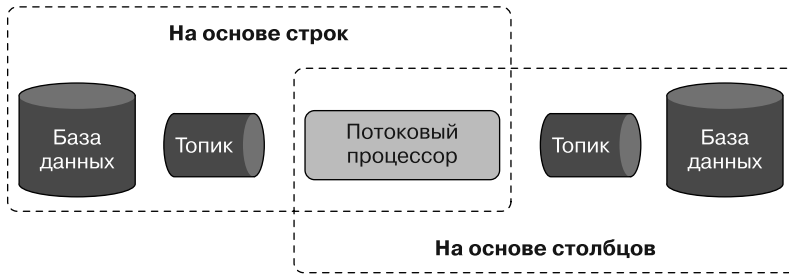


Рис. 5.8. Конвейер данных в режиме реального времени для размещения потоковых БД

Начинают появляться новые базы данных — более близкие к приложениям и веб-границе. Пример отражен в левой части этой диаграммы. Подробнее мы поговорим о них в главе 7 при обсуждении гибридных баз данных с транзакционной и аналитической обработкой — БД HTAP.

До этого момента мы подробно рассказывали о двух типах потоковых баз данных — на основе строк и столбцов. И в том и в другом случае возникает сложная задача конвергенции механизмов обработки SQL-кода в потоковом процессоре и базе данных назначения. У этих двух SQL-механизмов может быть различная семантика с ограничениями возможностей SQL. Это связано с тем, что их базовый конвергентный SQL-механизм не поддерживает такие возможности.

Выразительность SQL

Под выразительностью SQL-кода понимается способность механизма SQL кратко и эффективно с помощью лаконичного синтаксиса представлять сложные операции с данными и запросы. Другими словами, этот параметр показывает, насколько хорошо SQL может передать замысел запроса или операции в удобном для понимания и сопровождения виде. Объединение SQL-механизмов потокового процессора и БД OLAP или OLTP может вызвать разные проблемы. Они возникают из-за фундаментальных различий в их конструкции, сценариях использования и характеристиках производительности.

Несоответствие производительности

Потоковые процессоры оптимизированы для обработки высокоскоростных потоков данных в режиме реального времени, а БД OLAP предназначены для

сложных аналитических запросов к накопленным данным. Слияние их SQL-механизмов может породить проблемы с производительностью, если объединенный механизм обработки будет пытаться сбалансировать требования реального времени к потоковым данным и ресурсоемкость OLAP-запросов.

Задержка

Потоковая обработка требует низкой задержки для обработки в режиме реального времени, в то время как базы данных OLAP часто отдают приоритет оптимизации запросов, а не низкой задержке отклика. Попытка добиться одновременно низкой задержки для потоковой передачи данных и высокой производительности для аналитических запросов в рамках одного SQL-механизма может оказаться непростой задачей.

Распределение ресурсов

Требования к ресурсам для потоковых процессоров и БД OLAP различаются. Потокowe процессоры должны обрабатывать данные по мере их поступления, что может привести к нехватке ресурсов при выполнении OLAP-запросов, требующих значительного количества вычислительных ресурсов и памяти. Правильное распределение ресурсов критически важно для того, чтобы избежать возникновения узких мест.

Различия в моделировании данных

Потоковые процессоры обычно работают с необработанными или полуструктурированными данными. А БД OLAP для эффективного выполнения запросов требуют структурированных, предварительно обработанных и хорошо смоделированных данных. Слияние двух SQL-механизмов может вызвать конфликты в подходах к моделированию данных.

Согласованность данных

Потоковые процессоры часто работают с данными в движении, в то время как базы данных OLAP работают с данными в состоянии покоя. Обеспечение согласованности данных между этими двумя состояниями может оказаться сложной задачей. Особенно ярко это проявляется при объединении SQL-механизмов для обработки обоих состояний.

Сложность

Объединение возможностей потоковой обработки и БД OLAP может привести к созданию более сложной системы. Повышенная сложность влияет на удобство обслуживания и отладки и общую стабильность системы.

Объем и хранение данных

У потоковых процессоров может быть более короткий период хранения данных из-за их большого объема и требований к обработке в режиме реального

времени. В базах данных OLAP обычно хранятся накопленные данные для анализа за более длительные периоды. Принять решение о том, как обеспечить хранение и интеграцию данных, может оказаться непростой задачей.

Оптимизация запросов

Базы данных OLAP часто предоставляют расширенные методы оптимизации запросов для сложных аналитических запросов. Поточковые процессоры не способны предложить такой же уровень оптимизации, что может привести к неоптимальной производительности для OLAP-запросов.

Эволюция схемы

Потоковые процессоры могут более гибко подходить к изменению схемы хранения, чем БД OLAP. Последние, как правило, требуют четко определенных схем хранения данных. Слияние их SQL-механизмов может привести к трудностям в работе с изменяющимися схемами данных.

Сопровождение и обновления

Управление обновлениями и обслуживание комбинированного SQL-механизма обработки потоковых и OLAP-нагрузок могут оказаться более сложными, поскольку обновления должны учитывать требования обоих вариантов использования.

Чтобы избежать этих подводных камней, необходимо тщательное архитектурное планирование, тестирование производительности и глубокое понимание специфики применения каждого SQL-механизма.

Объединение SQL-механизмов OLTP и потокового процессора может быть проще, чем подобное объединение в случае с OLAP и потоковым процессором. Это связано с присущими системам OLTP и OLAP различиями в характеристиках хранения и обработки данных.

Формат данных

Базы данных OLTP обычно используют модель хранения на основе строк. Она хорошо подходит для хранения отдельных транзакционных записей. Поточковые процессоры также работают с данными в формате строк, обрабатывая события в режиме реального времени. Такое согласование в модели данных может способствовать более плавной интеграции и совместимости.

Характер работы в режиме реального времени

И OLTP-системы, и потоковые процессоры в той или иной степени работают с данными в режиме реального времени. Хотя требования к обработке данных могут различаться, общая направленность на работу с данными в режиме реального времени может упростить объединение их SQL-механизмов.

Обработка транзакций

И OLTP, и потоковые процессоры могут включать в себя транзакционную обработку, хотя и с разным уровнем сложности. Этот общий аспект может способствовать лучшей интеграции, когда речь идет о согласованности и обновлении данных.

Ориентация на события

Потоковые процессоры по своей природе управляются событиями, что вполне соответствует обновлениям в режиме реального времени в базах данных OLTP. Такая совместимость упрощает процесс интеграции.

Если объединение SQL-механизмов между OLTP и потоковым процессором может быть проще из-за этих общих характеристик, то объединение с БД OLAP сложнее из-за фундаментальных различий в стратегиях хранения, обработки и оптимизации запросов. Опять же тщательное архитектурное планирование и учет специфических требований каждой системы имеют решающее значение для успешного достижения интеграции в любом из сценариев.

Снижение сложности инфраструктуры и унифицированный SQL-движок упростят разработку для инженеров, особенно при отладке материализованных представлений.

Возможности отладки потоковой обработки

Инженерам по обработке данных всегда нужно проверять логику своего SQL-кода при написании конвейеров данных. Если материализованное представление распространяется на три различные распределенные системы (потоковый процессор, топик и OLAP), отлаживать его становится очень сложно. Хотя теоретически можно выполнить его отладку, просматривая только входные и выходные топики, на практике при отладке нужно просматривать одновременно топики и внешнюю базу данных.

Потоковые базы данных облегчают отладку за счет предоставления более продвинутой формы материализованных представлений по сравнению с хранилищами состояний (ключ/значение) классических потоковых процессоров. Потоковые базы данных сохраняют материализованные представления в хранилище на основе строк или столбцов, что упрощает проверку результатов в одном месте. Отладка также может быть выполнена быстрее, поскольку данные уже проиндексированы для более сложных запросов *ad hoc*. А потоковые процессоры, такие как Flink, сначала требуют записи результатов в базу данных, чтобы проверить их достоверность с помощью запросов *ad hoc*.

Преимущества отладки в потоковых базах данных

Далее перечислены преимущества отладки в потоковых базах данных.

Знакомый интерфейс SQL

Во многих потоковых базах данных предлагается SQL-подобный язык запросов для определения операций потоковой обработки. Если вы уже знакомы с SQL, то благодаря этому отладка может оказаться более простой.

Более простая логика

Потоковые базы данных часто предоставляют абстракцию более высокого уровня для упрощения сложных задач обработки потоков. Это может привести к упрощению логики, что, в свою очередь, облегчит отладку.

Интегрированная экосистема

Потоковые базы данных часто выступают частью более крупной экосистемы данных. Они могут помочь достичь лучшей интеграции с другими инструментами обработки данных и решениями для мониторинга, объединения потокового процессора и БД в одной системе. Это удобнее по сравнению с необходимостью объединять отдельный потоковый процессор с отдельной БД для последующей обработки. Эта интегрированная среда помогает при отладке, обеспечивая более целостное представление о конвейере данных.

Встроенные оптимизации

В потоковых БД могут существовать встроенные оптимизации для распространенных моделей обработки потоков. Они помогают повысить производительность и надежность, а также уменьшить необходимость в сложной отладке в определенных сценариях.

Более простое развертывание

Некоторые потоковые БД разработаны с учетом упрощенного развертывания, что может облегчить процесс отладки.

SQL не панацея

SQL — это очень абстрактный язык. В некоторых случаях может быть выгодно в дополнение к SQL использовать *предметно-ориентированный язык* (Domain-Specific Language, DSL) более низкого уровня, например Kafka Streams (Streams DSL и Processor API) и Flink (DataStream API), для повышения выразительности. Пользовательские функции (User-Defined Functions, UDF) в потоковых базах данных на основе SQL могут стать смягчающим фактором, но только до определенной степени.

Для отладки и/или профилирования, особенно в ситуациях, в которых критична высокая производительность, желательно также иметь представление о фактическом логическом плане выполнения, создаваемом системой обработки потоков, например, для проверки состояния операции агрегирования, созданной оператором `GROUP BY`. Но инструментарий для проверки плана выполнения в системах потоковой обработки все еще находится в зачаточном состоянии.

Реализации потоковых баз данных

В табл. 5.1 приведен список некоторых потоковых баз данных, доступных на момент выхода книги.

Таблица 5.1. Существующие потоковые базы данных

Название	Лицензия	Реализация хранилища состояний	Варианты использования
ksqlDB	Лицензия сообщества Confluent	RocksDB (хранилище ключевых значений дерева LSM)	CQRS, push-запросы
RisingWave	Apache 2	Строчное	CQRS, push-запросы, поиск по одной записи
Materialize	Business Source License (BSL)	Строчное	CQRS, push-запросы, поиск по одной записи
Timeplus (Proton)	Apache 2	Столбцовое	Аналитические запросы push и pull

Потоковые базы данных, перечисленные в табл. 5.1, различаются по слою хранения данных, лежащему в их основе. Первая потоковая база данных, ksqlDB, использует хранилища состояний на основе RocksDB (древовидное хранилище LSM). ksqlDB поддерживает индексацию только по первичным ключам.

Хотя доступ по первичному ключу/значению обеспечивает хорошую производительность, более сложные запросы требуют полного, неиндексированного сканирования всех хранилищ состояний, что не позволяет масштабировать нетривиальные объемы данных.

Полное сканирование таблицы — это операция запроса с изучением и обработкой каждой записи или строки в наборе, таблице или базе данных. Хотя иногда полное сканирование полезно, оно часто создает различные проблемы и трудности при запросе данных. Эта процедура может быть ресурсоемкой и отнимать много времени, особенно в ходе работы с большими массивами данных. Обработка каждой записи в таблице из большого набора данных может привести к снижению производительности запросов.

Реализация слоя хранения данных потоковой базы данных определяет диапазон эффективно поддерживаемых запросов. Системы RisingWave, Materialize и Timerplus используют более похожие на базы данных реализации своих слоев хранения данных (Timerplus, например, задействует версию RTOLAP базы данных ClickHouse) с гибкими схемами индексации. Это позволяет им эффективно обслуживать большое количество разнообразных запросов.

Архитектура потоковой базы данных

Материализованные представления (таблицы изменений) в потоковой базе данных находятся в хранилище состояний. А таблицы только с добавлением — нет. Либо запрос через них проходит чисто, либо их помещают в другую конструкцию типа топика потоковой базы данных.

Чтобы лучше представить архитектуру потоковой базы данных, на рис. 5.9 показаны путь и этапы прохождения событий щелчка кнопкой мыши и данные CDC клиента.

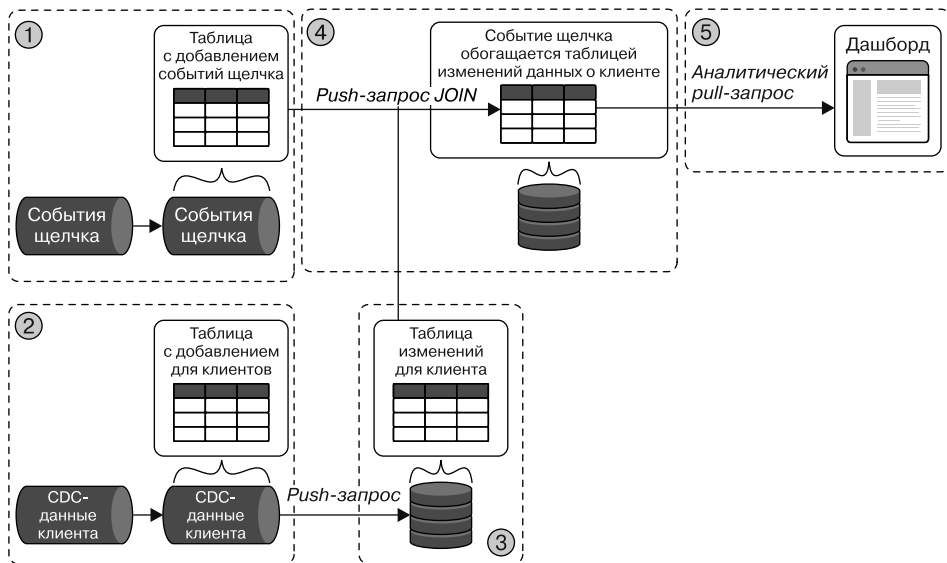


Рис. 5.9. Пути добавления и изменения потоков в потоковой базе данных

На первом этапе события щелчков попадают в топик с добавлением потоковой платформы типа Kafka. Чтобы перенести эти данные в потоковую базу данных, необходимо применить язык описания данных (Data Definition Language, DDL) для создания таблицы (см. пример 4.1).



DDL — это подмножество SQL. Этот язык описывает способы создания, изменения и удаления объектов базы данных, таких как таблицы.

В примере 5.3 создается исходная таблица в потоковой базе данных. Этот пример SQL-кода уже приводился в главе 2.

Пример 5.3. Создание исходной таблицы из топика Kafka для событий щелчка

```
CREATE SOURCE click_events (
  id integer,
  ts long, ❶
  url varchar, ❷
  ipAddress varchar, ❸
  sessionId varchar,
  referrer varchar,
  browser varchar
)
WITH (
  connector='kafka',
  topic='clicks',
  properties.bootstrap.server='kafka:9092',
  scan.startup.mode='earliest'
)
ROW FORMAT JSON;
```

❶ Временная метка.

❷ Содержит идентификатор товара, который необходимо проанализировать.

❸ Переменная `ipAddress` для идентификации клиента.

Далее мы проделаем то же самое с данными CDC клиента из примера 5.4. Обратите внимание на то, что на этом этапе данные существуют как проходной элемент в виде таблицы только с добавлениями, без хранилища состояний. У потоковых процессоров и БД нет возможности узнать, есть ли в содержимом топика данные об изменении или добавлении. Предполагается, что это данные только с добавлением, потому что все потоковые данные — это данные только с добавлением, пока они не материализуются в представление в хранилище состояний. Вот почему необходим этап 3.

Пример 5.4. Поглощение данных и создание таблицы для клиентов. Схема таблицы использует формат Debezium CDC, предоставляющий поля `before`, `after` и `op`

```
CREATE SOURCE customers (
  before ROW<id long, name varchar, email varchar, ipAddress varchar>, ❶
  after ROW<id long, name varchar, email varchar, ipAddress varchar>, ❷
  op varchar, ❸
```

```

    ts timestamp, ❷
    source <...>,
)
WITH (
    connector='kafka',
    topic='customers',
    properties.bootstrap.server='kafka:9092',
    scan.startup.mode='earliest'
)
ROW FORMAT JSON;

```

- ❶ Поле `before` содержит состояние записи до внесения изменений.
- ❷ Поле `after` содержит состояние записи после внесения изменений.
- ❸ Поле `op` указывает на тип изменения: вставка, изменение или удаление.
- ❹ Поле `ts` — это временная метка поступления изменения.

На этапе 3 (см. рис. 5.9) потоковая БД получает указание материализовать значения из таблицы с добавлением в материализованное представление. Пример 5.5 — это один из способов получения последнего состояния каждой записи.

Пример 5.5. Создание материализованного представления последних записей с помощью общего табличного выражения (Common Table Expression, CTE) и оконного режима

```

CREATE MATERIALIZED VIEW customers_mv AS
WITH ranked_customers AS (
    SELECT
        c.AFTER,
        c.op,
        c.ts,
        ROW_NUMBER() OVER (PARTITION BY c.AFTER.id ORDER BY c.ts DESC) AS rn ❶
    FROM customers AS c
)
SELECT * FROM ranked_customers WHERE rn = 1 AND op IS NOT 'D'; ❷
);

```

- ❶ Это оконный оператор разделения записей по полю `id` с последующим размещением их в порядке убывания по временной метке. Функция `ROW_NUMBER()` присваивает каждому экземпляру `id` номер строки и гарантирует, что запись 1 является самой последней записью.
- ❷ Выполняется выбор только самой последней записи, со значением 1 в переменной `rn`. Отфильтровываются также все удаленные записи со значением, отличным от D, в поле `op`.

В результате выполнения примера 5.5 должно получиться материализованное представление с последним состоянием данных о клиенте.

У некоторых потоковых БД есть возможность пропустить этап 3 при наличии пользовательского коннектора для перевода данных в формате Debezium CDC непосредственно в материализованное представление (пример 5.6).

Пример 5.6. Применение коннектора Debezium для непосредственного создания материализованного представления

```
CREATE SOURCE customers (
  before ROW<id long, name varchar, email varchar, ipAddress varchar>,
  after ROW<id long, name varchar, email varchar, ipAddress varchar>,
  op varchar,
  ts timestamp,
  source <...>,
)
WITH (
  connector='kafka-debezium-cdc', ❶
  topic='customers',
  properties.bootstrap.server='kafka:9092',
  scan.startup.mode='earliest'
)
ROW FORMAT JSON;
```

❶ Пользовательский коннектор `kafka-debezium-cdc` способен считывать данные из топика Kafka и создавать материализованное представление с отображением только последнего состояния таблицы клиента.

КОННЕКТОРЫ CDC

У каждой потоковой базы данных будут свои имена для коннекторов CDC. Не стоит ожидать, что название всегда будет `kafka-debezium-cdc`. Чтобы узнать реальное название используемого коннектора, необходимо обратиться к документации.

В некоторых потоковых БД есть собственные коннекторы CDC. Они обходят топик в потоковой платформе и подключаются непосредственно к базе данных. Эти коннекторы по-прежнему будут считывать журналы WAL и создавать копии таблиц в потоковой БД. Однако обход платформы потоковой передачи может быть ограничивающим, если данные CDC необходимо реплицировать не только в потоковую БД, но и в другие целевые БД. Более подробно варианты архитектуры для развертывания потоковых БД описаны в главе 10.

На этапе 4 (см. рис. 5.9) создается еще одно материализованное представление. В примере 5.7 таблица `CLICK_EVENTS` объединяется с таблицей `CUSTOMERS`, в результате создается материализованное представление событий щелчков, обогащенных данными клиентов.

Пример 5.7. Создание еще одного материализованного представления обогащенных событий щелчка

```
CREATE MATERIALIZED VIEW CLICK_EVENTS_ENRICHED AS
SELECT e.*, c.*
FROM CLICK_EVENTS e
JOIN CUSTOMERS c on e.ipAddress = c.ipAddress
```

Можно повторить этапы с таблицей CUSTOMERS и использовать таблицу PRODUCTS для обогащения таблицы CLICK_EVENTS информацией не только о клиентах, но и о товарах (пример 5.8).

Пример 5.8. Создание еще одного материализованного представления обогащенных событий щелчка

```
CREATE MATERIALIZED VIEW CLICK_EVENTS_ENRICHED AS
SELECT e.*, c.*, p.*
FROM CLICK_EVENTS e
JOIN CUSTOMERS c on e.ipAddress = c.ipAddress
JOIN PRODUCTS p on e.productid = p.productid
```

Итак, мы перешли к этапу 5 (см. рис. 5.9). Теперь конечные пользователи могут вызывать аналитические запросы к материализованному представлению CLICK_EVENTS_ENRICHED. Тип хранилища потоковой БД (строчное или столбцовое) будет определять сложность доступного для вызова аналитического запроса. Многие сценарии использования не требуют участия человека для инициирования запроса. Вместо этого сами приложения могут потреблять материализованные представления в режиме реального времени.

В некоторых случаях приложениям может быть удобно использовать одновременно как строчные, так и столбцовые потоковые базы данных для выполнения различных аналитических запросов с низкой задержкой. Это потребует от обеих потоковых баз данных создания копий таблиц. Поэтому, как мы уже отмечали, важно, что потоковые платформы позволяют публиковать данные CDC и подписываться на них, чтобы несколько систем могли создавать копии их таблиц.

Конвейеры ELT с потоковыми базами данных

В главе 2 мы говорили, что конвейеры данных ELT (извлечение, загрузка, преобразование) не поддерживают сценарии использования в режиме реального времени, поскольку преобразование происходит в базе данных назначения. БД в этом случае размещает потоковые данные в состоянии покоя, что заставляет применять семантику пакетной обработки для всех последующих операций.

Однако если целевая БД представляет собой потоковую базу данных, то конвейер можно считать работающим в режиме реального времени. Интеграция между

частями загрузки и преобразования из ELT реализована топиком потоковой платформы, из которой потоковая БД потребляет данные.

Существует большая экосистема с поддержкой ELT-решений, например *Dbt*¹. В сочетании с потоковыми БД эти инструменты впервые могут поддерживать ELT в режиме реального времени. А поскольку потоковые БД на первый взгляд ведут себя как базы данных, а не как потоковые процессоры, механизм ELT в них может быть реализован людьми, ранее работавшими над ELT в хранилище данных. Таким образом, многие задания ELT, которые сейчас выполняются на более поздних этапах конвейера (в хранилище или озере данных), можно перенести на слой потоковой передачи данных в режиме реального времени.

Резюме

Понятие *потоковой базы данных* объединяет потоковую обработку и базы данных. Обычно базы данных ассоциируются с пакетной обработкой данных. Следовательно, соединение потоковой обработки и баз данных также сближает потоковую обработку с пакетной, а данные в движении — с данными в состоянии покоя.

Потоковые БД возвращают в БД две конструкции, которые Мартин Клеппман извлек из БД, — журналы WAL и материализованное представление.

Механизмы обработки SQL-кода в БД до сих пор поддерживали только данные в состоянии покоя. Когда материализованное представление стало работать асинхронно в фоновом режиме, получилось дать возможность существующему SQL-коду обрабатывать данные и в состоянии покоя, и в движении. Это превращает обычную базу данных в потоковую.

Потоковые БД отличаются друг от друга реализацией слоя хранения данных. В БД *ksqlDB* задействуются хранилище *RocksDB* и индексы первичного ключа. Новые потоковые БД используют более похожий на БД слой хранения данных для эффективной поддержки большого набора запросов.

Мы также знаем, что потоковые БД поддерживают запросы *push* и *pull*. *Push*-запросы выполняются на потоковой половине потоковой БД. *Pull*-запросы выполняются на ее половине «база данных». Если потоковая часть использует часть базы данных для хранения состояния, у вас получается настоящая потоковая база данных.

¹ *Dbt* (data build tool) — это программное обеспечение с открытым исходным кодом, позволяющее инженерам и аналитикам преобразовывать, тестировать и развертывать преобразование данных с помощью SQL и Python.

Тип хранилища потоковой базы данных определяет, какой тип запросов может быть эффективно обработан. Для баз данных на основе столбцов pull-запросы могут быть аналитическими, включая быстрые агрегации. Для баз данных на основе строк pull-запросы обычно представляют собой более простые запросы на поиск, например точечные.

На рис. 5.10 показан спектр потоковых БД от строчных до столбцовых. С левой стороны находятся обычные запросы событийно-ориентированного приложения без участия человека. В правой части находятся запросы, которые обычно вызываются человеком или дашбордом.

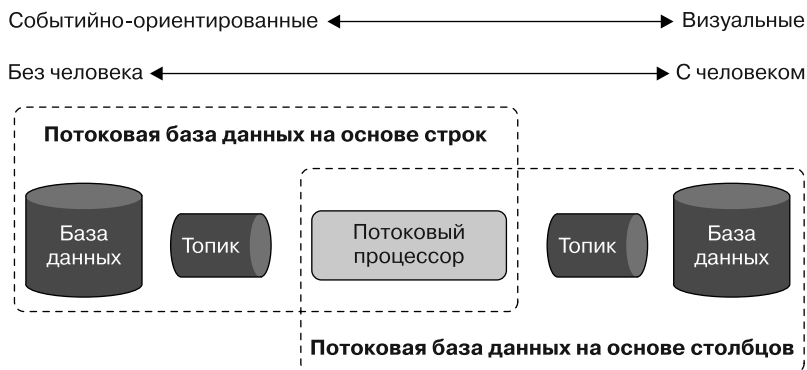


Рис. 5.10. Разнообразие потоковых баз данных

Важнейшим свойством потоковых БД выступает согласованность. В базе данных это состояние, при котором данные всегда достоверны и соответствуют заданным правилам и ограничениям. Оно гарантирует, что любая транзакция, успешная или нет, переводит базу данных из одного последовательного состояния в другое, не нарушая правил целостности. В согласованной БД все изменения данных происходят по заранее определенным правилам, что обеспечивает точность и надежность данных.

В следующей главе мы рассмотрим важность согласованности в потоковых БД, а также в потоковых процессорах.

ГЛАВА 6

Согласованность

Если вы знакомы с базами данных, то считаете согласованность само собой разумеющейся. Вам известно, что результаты запросов будут согласованы с исходными данными. Теперь представьте, что вы решились перейти по мосту из БД в мир потоковой обработки. Можете ли вы рассчитывать на аналогичные гарантии согласованности, особенно с учетом дополнительных сложностей в виде поступления данных с опозданием и не по порядку, а также с упором на низкие показатели задержки и высокую пропускную способность?

Для классических потоковых процессоров ответ будет отрицательным. Они предоставляют более слабую форму, называемую *согласованностью в конечном счете* (eventual consistency). Такой тип согласованности идеально подходит для классических сценариев обработки потоков с агрегированием данных в *оконном режиме*. Существует также возможность создавать с ним конвейеры данных с ультранизкой задержкой, очень высокой пропускной способностью и в очень больших масштабах. Но если у вас есть опыт работы в сфере баз данных, согласованность в конечном счете покажется вам запутанным и контринтуитивным вариантом. Особенно в сочетании с данными *без оконных ограничений*.

В этой главе мы используем пример — вымышленную ситуацию из банковской сферы. С его помощью продемонстрируем возможные проблемы в ходе работы с потоковыми процессорами с согласованностью в конечном счете, такими как Flink, ksqlDB и Proton, если при разработке просто следовать опыту из мира баз данных.

Интересно, что некоторые современные системы обработки потоков поддерживают более строгую форму согласованности, когда каждый результат будет правильным для подмножества входов. Это называется *внутренней согласованностью*¹. Мы применили несколько таких систем обработки потоков — RisingWave, Materialize и Pathway — для выполнения одной и той же работы. Это позволило оценить их возможности решить вымышленный пример таким образом, который хорошо отражает интуицию типичного инженера баз данных.

¹ Brandon J. Internal Consistency in Streaming Systems. 2021 (<https://oreil.ly/mwuy1>).

Далее подробно рассказывается о том, что именно может пойти не так при следовании по пути согласованности в конечном счете и почему системы обработки потоков с более надежными гарантиями согласованности могут оказаться лучше.

Наконец, мы перейдем к вопросу о том, имеет ли смысл для классических потоковых процессоров, таких как Flink, поддерживать более строгие гарантии согласованности. Другими словами, от какой части низких задержек и высокой пропускной способности им придется отказаться и стоит ли это делать?

Воображаемый пример

Эта глава создавалась на основе примера из блога Джейми Брэндона о внутренней согласованности. Мы намеренно выбрали не классический пример обработки потоков — в нем нет оконной обработки, и ему требуется синхронизация, не в полной мере доступная в классических системах обработки потоков. Выбор этого примера объясняется тем, что мы интересуемся потоковыми базами данных в условиях слияния мира потоков и баз данных. И считаем, что для реализации этого слияния на практике системы обработки потоков должны быть способны обрабатывать нетипичные, без оконной обработки, сценарии использования, сопоставимые с миром баз данных. Особенно это касается аспектов согласованности.

Представьте себе банк с десятью счетами, где одни банковские счета постоянно переводят 1 доллар на другие. Это выглядит как табл. 6.1, где каждая графа «Транзакция» — это транзакция, выполняющая дебет и кредит. В графах «Счет» и «Начальное значение» показаны счета 1, 2 и 3 с начальным значением 0 долларов. Следующая графа, «Транзакция 1», дебетует счет 1 и кредитует счет 2 и т. д. Для каждой графы, представляющей дебетовую и кредитную операции, все строки должны быть равны нулю.

Таблица 6.1. Дебетовые и кредитные операции, в долларах

Счет	Начальное значение	Транзакция 1	Транзакция 2	Транзакция 3	Транзакция 4
1	0	−1	−2	−3	−2
2	0	1	1	2	2
3	0	0	1	1	0
Сумма	0	0	0	0	0

Способ проверки согласованности заключается в том, чтобы проверить, может ли потоковый процессор вернуть ноль при суммировании остатков по всем счетам. Это и есть требуемая *инвариантность*. Любой момент, когда сумма не равна нулю, указывает на проблемы с согласованностью.

Транзакции

Мы создали пример воображаемого банка с помощью кода на языке Python в примере 6.1.

Пример 6.1. Код на языке Python для создания примера с воображаемым банком

```
import datetime, json, random, time
from kafi.kafi import Cluster

c = Cluster("local")
c.create("transactions", partitions=1)
p = c.producer("transactions")
random.seed(42)
for id_int in range(0, 10000):
    row_str = json.dumps({
        "id": id_int,
        "from_account": random.randint(0, 9),
        "to_account": random.randint(0, 9),
        "amount": 1,
        "ts": datetime.datetime.now().isoformat(sep=" ", timespec="milliseconds")
    })
    print(row_str)
    p.produce(row_str, key=str(id_int))
    time.sleep(0.01)
p.close()
```

Код создает новое сообщение в топике Kafka (с одним разделом) с названием `transactions` 10 000 раз каждые 10 мс. В нем один банковский счет, `from_account`, переводит 1 доллар на другой банковский счет, `to_account`. Образец сообщения приведен в примере 6.2.

Пример 6.2. Пример сообщения/транзакции

```
{
    "id": 42,
    "from_account": 3,
    "to_account": 0,
```

```
"amount": 1,
"ts": "2023-10-24 23:27:57.603"
}
```

Анализ транзакций

Теперь необходимо выполнить дальнейший анализ транзакций с помощью SQL-кода. Сначала создадим два представления, агрегирующих данные `credits` и `debits` счетов. Первое — это сумма денег, перечисленных на счет (`to_`), а второе — сумма денег, перечисленных с него (`from_`) (пример 6.3).

Пример 6.3. Настройка представлений `credits` и `debits`

```
CREATE VIEW credits AS
SELECT
    to_account as account,
    SUM(amount) as credits
FROM transactions
GROUP BY to_account;

CREATE VIEW debits(account, debits) AS
SELECT
    from_account as account,
    SUM(amount) as debits
FROM transactions
GROUP BY from_account;
```

Далее вычисляем баланс счета — кредиты минус дебиты:

```
CREATE VIEW balance AS
SELECT
    credits.account AS account,
    credits - debits AS balance
FROM credits
INNER JOIN debits ON credits.account = debits.account;
```

Наконец, создаем представление для суммирования остатков по всем счетам. Поскольку деньги не могут появиться ниоткуда и их невозможно потерять, эта сумма всегда должна быть равна 0. Она дает нам инвариант для проверки поведения потоковых процессоров и потоковых баз данных в отношении согласованности:

```
CREATE VIEW total(total) AS
SELECT SUM(balance) FROM balance;
```

Сравнение согласованности между системами обработки потоков

Как же ведут себя некоторые из существующих систем обработки потоков, когда сталкиваются с примером с воображаемым банком? Мы рассмотрим шесть из них:

- Flink SQL;
- RisingWave;
- ksqlDB;
- Materialize;
- Proton (Timeplus);
- Pathway.

Flink SQL

Начнем с одной из самых популярных систем обработки потоков — Flink. Поскольку нас в первую очередь интересуют потоковые базы данных на основе SQL, используем SQL-слой Flink/API *Flink SQL* (в данном случае применялась версия 1.19.0).

В системе Flink SQL сначала устанавливается соединение с исходным топиком `transactions`:

```
CREATE TABLE transactions (
  id BIGINT,
  from_account INT,
  to_account INT,
  amount DOUBLE,
  ts TIMESTAMP(3)
) WITH (
  'connector' = 'kafka',
  'topic' = 'transactions',
  'properties.bootstrap.servers' = 'localhost:9092',
  'properties.group.id' = 'transactions_flink',
  'scan.startup.mode' = 'earliest-offset',
  'format' = 'json',
  'json.fail-on-missing-field' = 'true',
  'json.ignore-parse-errors' = 'false'
);
```

Далее будут созданы представления `credits`, `debits`, `balance` и `total`:

```
CREATE VIEW credits(account, credits) AS
SELECT
  to_account as account,
  SUM(amount) as credits
FROM
  transactions
```

```
GROUP BY
    to_account;

CREATE VIEW debits(account, debits) AS
SELECT
    from_account as account,
    SUM(amount) as debits
FROM
    transactions
GROUP BY
    from_account;

CREATE VIEW balance(account, balance) AS
SELECT
    credits.account,
    credits - debits as balance
FROM
    credits,
    debits
WHERE
    credits.account = debits.account;

CREATE VIEW total(total) AS
SELECT
    SUM(balance)
FROM
    balance;
```

И в конце необходимо записать результаты из представления `total` в топик назначения Kafka с названием `total_flinksq1`:

```
CREATE TABLE total_sink (
    total DOUBLE,
    PRIMARY KEY (total) NOT ENFORCED
) WITH (
    'connector' = 'upsert-kafka',
    'property-version' = 'universal',
    'properties.bootstrap.servers' = 'localhost:9092',
    'topic' = 'total_flink',
    'key.format' = 'json',
    'value.format' = 'json',
    'properties.group.id' = 'total_flink'
);

INSERT INTO total_sink SELECT * FROM total;
```

Теперь рассмотрим график потока данных для нашего SQL-кода Flink (рис. 6.1). Слева находится исходная таблица `transactions`. Затем она разбивается на два агрегата, `credits` и `debits`. После для этих двух агрегатов выполняется операция `JOIN` в представлении `balance`, а далее рассчитывается сумма остатков по всем счетам из представления `total`.

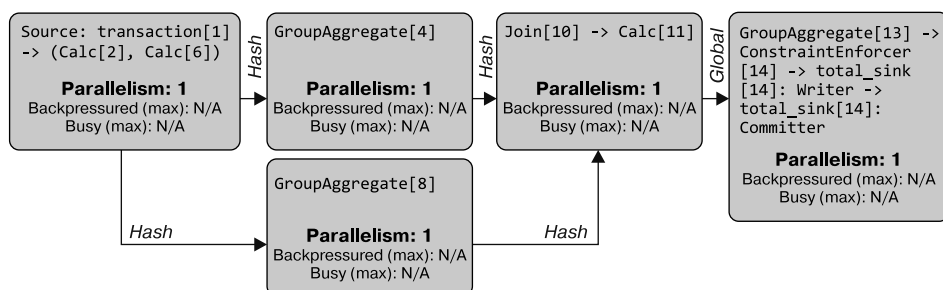


Рис. 6.1. Диаграмма потока данных SQL-кода Flink для рассматриваемого примера



Диаграмма/топология потока данных, как на рис. 6.1, используется во всех сравниваемых здесь системах потоковой обработки, включая ksqlDB, Proton, RisingWave, Materialize и Pathway.

Чтобы запустить пример во Flink SQL, мы сначала настраиваем таблицы и представления, а затем запускаем код на Python, чтобы установить 10 000 транзакций в воображаемом банке и посмотреть, что произойдет. Если опираться на опыт из сферы баз данных, то может показаться, что ничего серьезного не случится — результирующий топик должен, как и представление `total`, всегда возвращать общую сумму, равную 0.

Представим результаты, вычисленные Flink SQL из топика Kafka `total_flinksq1` (рис. 6.2).

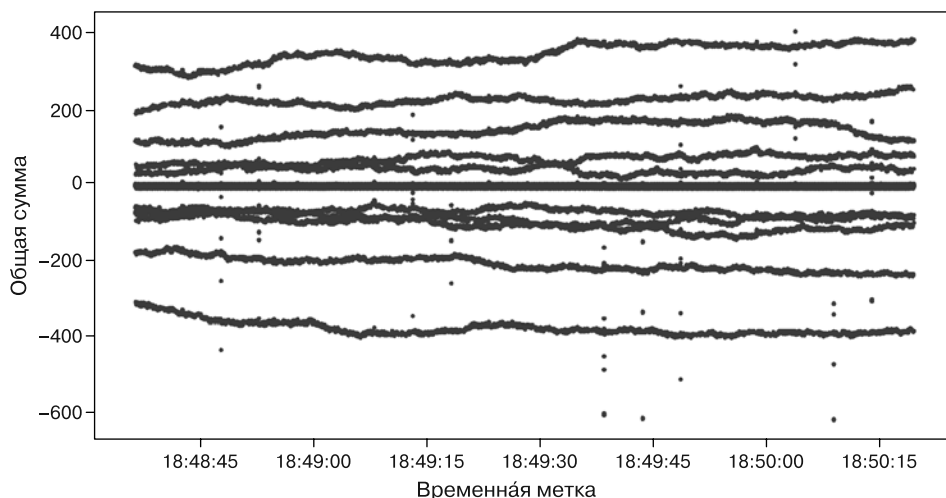


Рис. 6.2. Визуализация топика назначения Kafka `total_flinksq1`

На рис. 6.2 отображено почти 80 000 сообщений (!), колеблющихся в широком диапазоне между значениями +400 и −600 для общей суммы балансов всех счетов. Однако, когда входной поток останавливается после 10 000 сообщений, Flink SQL сходится к правильной согласованной сумме, равной 0 (табл. 6.2).

Таблица 6.2. Последние несколько сообщений в топике назначения Kafka `total_flinksq1`

Смещение	Общая сумма
79 936	−3
79 937	40
79 938	40
79 939	−2
79 940	−2
79 941	−103
79 942	−103
79 943	−1
79 944	−1
79 945	−83
79 946	−83
79 947	0

Причина такого поведения кроется в дизайне Flink SQL. При его разработке был сделан упор на поддержку согласованности в конечном счете, а не на внутреннюю, что, в свою очередь, сопутствует выбору в пользу низкой задержки. В модели согласованности в конечном счете достаточно, чтобы результат обработки потока был не всегда согласованным, а согласованным в какой-то более поздний момент времени. В нашем случае этот момент наступает, когда код Python перестает подавать новые сообщения в исходный топик `transactions`.

У модели согласованности в конечном счете есть очевидное преимущество — максимально возможное снижение задержек. Это идеальная модель для оконной обработки потоков, и именно для этого чаще всего используются Flink и Flink SQL. Но в нашем случае это приводит к возникновению дилеммы. Мы получили неограниченный, без оконной обработки, вариант применения и утратили уверенность в правильности промежуточного результата от Flink SQL, пока поток ввода не остановится. Но и прекратить входной поток в любой момент времени нельзя, по крайней мере в продуктивной среде, как нельзя запретить воображаемым клиентам совершать транзакции...

В версии 1.19 во Flink появилась оптимизация MiniBatch, или *мини-пакетная агрегация*, для операторов JOIN. При правильной настройке для конкретного сценария использования MiniBatch не только значительно повышает производительность, но и помогает достичь более высокой степени согласованности. Мы обсудим эту тему позже в данной главе.

ksqlDB

Наш следующий потоковый процессор/потоковая БД — ksqlDB компании Confluent (мы использовали версию 7.6.0). Многие считают ksqlDB первой потоковой базой данных (ее первая версия была выпущена в 2017 году). Эта глава посвящена системам обработки потоков на основе SQL, поэтому в ней мы рассматриваем только ksqlDB, а не лежащую в ее основе библиотеку Kafka Streams. Снова необходимо начать с установки соединения с исходным топиком transactions (пример 6.4).

Пример 6.4. Настройка таблицы transactions в ksqlDB

```
CREATE TABLE transactions (
  id VARCHAR PRIMARY KEY,
  from_account INT,
  to_account INT,
  amount DOUBLE,
  ts VARCHAR
) WITH (
  kafka_topic = 'transactions',
  value_format = 'json',
  partitions = 1,
  timestamp = 'ts',
  timestamp_format = 'yyyy-MM-dd HH:mm:ss.SSS'
);
```

На следующем этапе будут созданы представления `credits`, `debits`, `balance` и `total`, и каждое будет соответствовать таблице TABLE в ksqlDB:

```
CREATE TABLE credits WITH (
  kafka_topic = 'credits',
  value_format = 'json'
) AS
SELECT
  to_account AS account,
  SUM(amount) AS credits
FROM
  transactions
GROUP BY
  to_account EMIT CHANGES; ❶
```

```
CREATE TABLE debits WITH (  
    kafka_topic = 'debits',  
    value_format = 'json'  
) AS  
SELECT  
    from_account AS account,  
    SUM(amount) AS debits  
FROM  
    transactions  
GROUP BY  
    from_account EMIT CHANGES;  
  
CREATE TABLE balance WITH (  
    kafka_topic = 'balance',  
    value_format = 'json'  
) AS  
SELECT  
    credits.account AS account,  
    credits - debits AS balance  
FROM  
    credits  
    INNER JOIN debits ON credits.account = debits.account EMIT CHANGES;  
  
CREATE TABLE total WITH (  
    kafka_topic = 'total_ksqldb',  
    value_format = 'json'  
) AS  
SELECT  
    'foo',  
    SUM(balance)  
FROM  
    balance  
GROUP BY  
    'foo' EMIT CHANGES;
```

❶ Поле `EMIT CHANGES` информирует `ksqlDB`, что поступил `push`-запрос.

При создании таблицы `total` итоговые суммы остатков по всем счетам выводятся в топик назначения Kafka `total_ksqldb`.

Как и в случае с Flink SQL, сначала настраиваются таблицы в `ksqlDB`, а затем запускается Python-код для создания 10 000 транзакций. После этого происходит визуализация результатов, вычисленных `ksqlDB` из топика Kafka `total_ksqldb` (рис. 6.3).

Как и на рис. 6.2, здесь показано большое количество сообщений — почти 40 000. Их значения все еще колеблются вокруг 0 (между +100 и −100). И снова, когда входной поток останавливается после 10 000 сообщений, `ksqlDB` сходится к правильной согласованной сумме, равной 0. Это видно в последних сообщениях Kafka в топике назначения Kafka `total_ksqldb` (табл. 6.3).

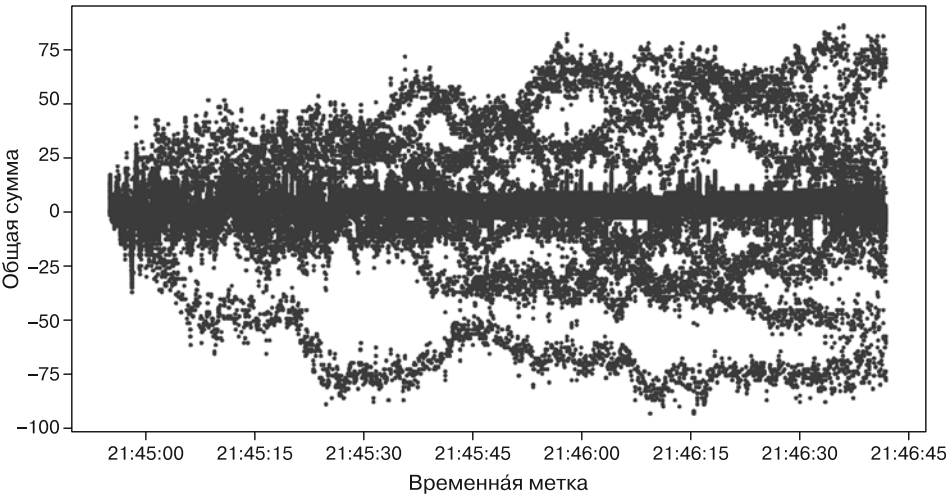


Рис. 6.3. Визуализация топика назначения Kafka с названием total_flinksql

Таблица 6.3. Последние несколько сообщений в топике назначения Kafka total_ksqldb

Смещение	Общая сумма
39 905	−6
39 907	−78
39 908	−5
39 909	68
39 910	−4
39 911	−3
39 912	−3
39 913	7
39 914	−2
39 915	−10
39 916	26
39 917	0

Причина такого поведения ksqlDB та же, что и у Flink SQL, — в ksqlDB принята модель *согласованности в конечном счете* (или, как ее называют в контексте Kafka Streams/ksqlDB, *непрерывного уточнения*). В приведенном примере система ksqlDB приводит нас к той же дилемме, что и Flink SQL, — пока входной поток не остановлен, неизвестно, какой из промежуточных результатов на самом деле правильный.

Proton (Timeplus)

В этом разделе мы рассмотрим Proton — потоковую БД с открытым исходным кодом, лежащую в основе платформы Timeplus. Настройка Proton начинается с создания потока `STREAM` для подключения к входному топику Kafka с названием `transactions` (пример 6.5).

Пример 6.5. Настройка входного потока `STREAM` в Proton

```
CREATE EXTERNAL STREAM transactions(  
    id int,  
    from_account int,  
    to_account int,  
    amount int,  
    ts datetime64  
) SETTINGS  
    type = 'kafka',  
    brokers = 'broker:29092',  
    topic = 'transactions',  
    data_format = 'JSONEachRow';
```

На следующем этапе настраиваются представления `credits`, `debits` и `balance` (пример 6.6).

Пример 6.6. Настройка представлений `credits`, `debits` и `balance` в Proton

```
CREATE EXTERNAL STREAM transactions(  
    id int,  
    from_account int,  
    to_account int,  
    amount int,  
    ts datetime64  
) SETTINGS type = 'kafka',  
    brokers = 'broker:29092',  
    topic = 'transactions',  
    data_format = 'JSONEachRow';
```

```
CREATE VIEW credits AS
SELECT
    now64() as ts,
    to_account as account,
    sum(amount) as credits
FROM
    transactions
GROUP BY
    to_account EMIT PERIODIC 100ms;
```

```
CREATE VIEW debits AS
SELECT
    now64() as ts,
    from_account as account,
    sum(amount) as debits
FROM
    transactions
GROUP BY
    from_account EMIT PERIODIC 100ms;
```

```
CREATE VIEW balance AS
SELECT
    c.account,
    credits - debits as balance
FROM
    changelog(credits, account, ts, true) AS c
    JOIN changelog(debits, account, ts, true) AS d ON c.account = d.account;
```

После создания представлений остается создать вывод потока **STREAM** (пример 6.7). Он будет передавать результаты в топик назначения Kafka с названием `total_proton` и материализованное представление `total`.

Пример 6.7. Настройка вывода потока **STREAM** и материализованного представления **MATERIALIZED VIEW total** в Proton

```
CREATE EXTERNAL STREAM total_s(total int) SETTINGS type = 'kafka',
    brokers = 'broker:29092',
    topic = 'total_proton',
    data_format = 'JSONEachRow';

CREATE MATERIALIZED VIEW total INTO total_s AS
SELECT
    sum(balance) as total
FROM
    balance;
```

В результирующем топике всего 56 сообщений, а их значения колеблются между -10 и 9 . Когда входной поток останавливается после всех 10 000 сообще-

ний, Proton сходится к правильной сумме, равной 0. Это видно в нескольких последних сообщениях в топике назначения Kafka `total_proton` (табл. 6.4).

Таблица 6.4. Последние несколько сообщений в топике назначения Kafka `total_proton`

Смещение	Общая сумма
48	9
49	9
50	9
51	9
52	9
53	−9
54	9
55	0

На рис. 6.4 представлены результаты для Proton, где явно выделяется последняя, правильная общая сумма, равная 0. Она отчетливо видна справа, потому что результирующий топик назначения относительно небольшой.

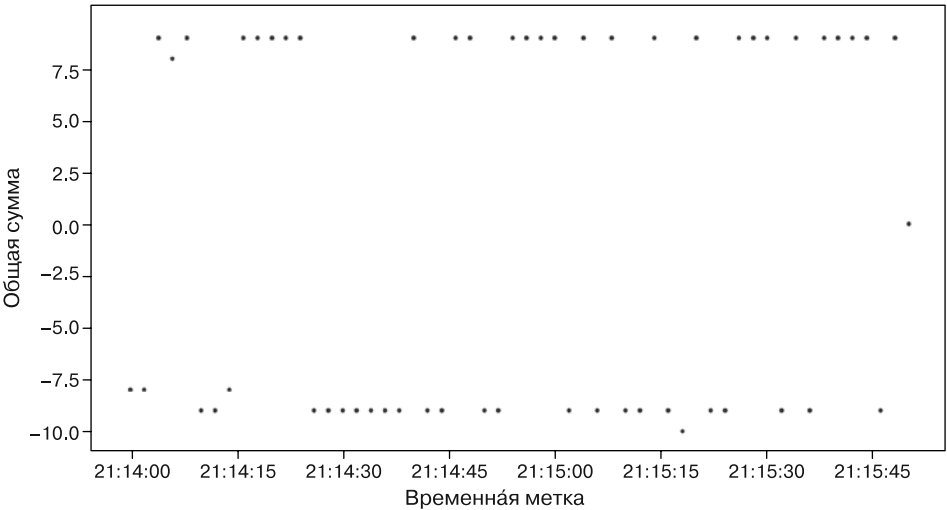


Рис. 6.4. Визуализация топика назначения Kafka с названием `total_flinksq`

Опять же, как и в случае с Flink и ksqlDB, в основе лежит модель согласованности в конечном счете, и возникающая дилемма аналогична — даже если мы получаем меньше экстремальных ошибок в промежуточных результатах, нам все равно приходится останавливать входной поток, чтобы убедиться, что в итоге получен правильный результат.

RisingWave

Теперь изучим RisingWave. Это потоковая БД, совместимая с протоколом PostgreSQL. Рассмотрим создание примера для RisingWave. Сначала создается таблица для получения данных от входного топика Kafka `transactions`:

```
CREATE TABLE IF NOT EXISTS transactions (
    id INT,
    from_account INT,
    to_account INT,
    amount INT,
    ts TIMESTAMP
) WITH (
    connector = 'kafka',
    topic = 'transactions',
    properties.bootstrap.server = 'broker:29092',
    scan.startup.mode = 'earliest',
    scan.startup.timestamp_millis = '140000000'
) ROW FORMAT JSON;
```

Далее настраиваются представления (пример 6.8).

Пример 6.8. Настройка представлений `credits`, `debits`, `balance` и `total` в RisingWave

```
CREATE VIEW accounts AS
SELECT
    from_account AS account
FROM
    transactions
UNION
SELECT
    to_account
FROM
    transactions;

CREATE VIEW credits AS
SELECT
    transactions.to_account AS account,
    SUM(transactions.amount) AS credits
FROM
    transactions
```

```
    LEFT JOIN accounts ON transactions.to_account = accounts.account
GROUP BY
    to_account;

CREATE VIEW debits AS
SELECT
    transactions.from_account AS account,
    SUM(transactions.amount) AS debits
FROM
    transactions
    LEFT JOIN accounts ON transactions.from_account = accounts.account
GROUP BY
    from_account;

CREATE VIEW balance AS
SELECT
    credits.account AS account,
    credits - debits AS balance
FROM
    credits
    INNER JOIN debits ON credits.account = debits.account;

CREATE VIEW total AS
SELECT
    sum(balance)
FROM
    balance;
```

И наконец, представление `total` передается в топик `total_risingwave`:

```
CREATE SINK total_sink
FROM
    total WITH (
        connector = 'kafka',
        properties.bootstrap.server = 'broker:29092',
        topic = 'total_risingwave',
        type = 'append-only',
        force_append_only = 'true'
    );
```

Затем в RisingWave настраиваются таблицы и представления и запускается код на языке Python для создания 10 000 транзакций. Визуализированные результаты для RisingWave показаны на рис. 6.5.

В итоге мы получили 105 сообщений в топике назначения, что немного больше, чем при использовании Proton, и намного меньше, чем при использовании Flink SQL и ksqlDB. Но что более важно, каждое из этих сообщений дает нам правильный результат, равный 0.

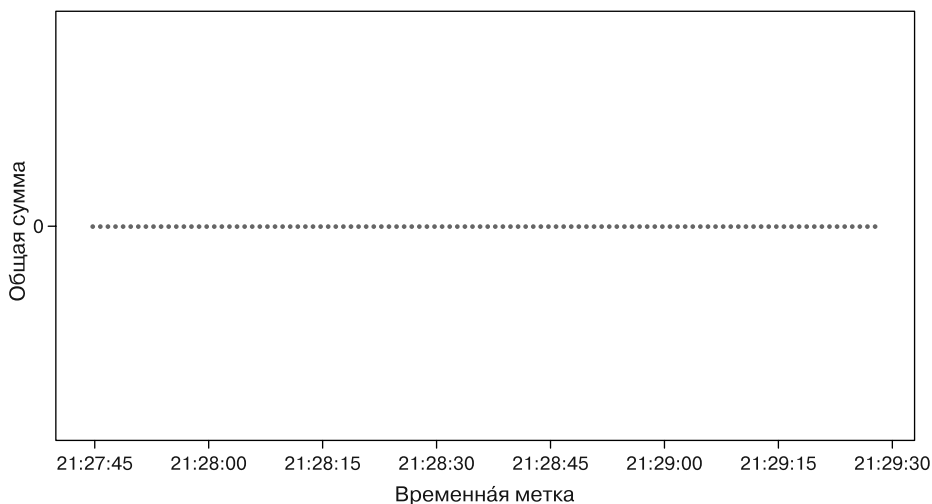


Рис. 6.5. Визуализация топика назначения Kafka с названием `total_risingwave`

Materialize

Пятая система обработки потоков из нашего списка — Materialize. Это еще одна потоковая БД, предлагающая PostgreSQL-совместимый API, как и RisingWave. Сначала создается таблица, получающая данные от входного топика Kafka `transactions`:

```
CREATE CONNECTION kafka_connection TO kafka (broker 'broker:29092');

CREATE SOURCE transactions_source
FROM
    kafka connection kafka_connection (TOPIC 'transactions', START OFFSET (0))
KEY FORMAT
    TEXT VALUE FORMAT TEXT INCLUDE KEY ENVELOPE UPSERT WITH (SIZE = '1');

CREATE VIEW transactions AS
SELECT
    ((text :: jsonb) ->> 'id') :: string AS id,
    ((text :: jsonb) ->> 'from_account') :: int AS from_account,
    ((text :: jsonb) ->> 'to_account') :: int AS to_account,
    ((text :: jsonb) ->> 'amount') :: int AS amount,
    ((text :: jsonb) ->> 'ts') :: timestamp AS ts,
    key
FROM
    transactions_source;
```

Далее настраиваются представления (пример 6.9).

Пример 6.9. Настройка представлений `credits`, `debits`, `balance` и `total` в Materialize

```
CREATE VIEW accounts AS
SELECT
    from_account AS account
FROM
    transactions
UNION
SELECT
    to_account
FROM
    transactions;

CREATE VIEW credits AS
SELECT
    transactions.to_account AS account,
    SUM(transactions.amount) AS credits
FROM
    transactions
    LEFT JOIN accounts ON transactions.to_account = accounts.account
GROUP BY
    to_account;

CREATE VIEW debits AS
SELECT
    transactions.from_account AS account,
    SUM(transactions.amount) AS debits
FROM
    transactions
    LEFT JOIN accounts ON transactions.from_account = accounts.account
GROUP BY
    from_account;

CREATE VIEW balance AS
SELECT
    credits.account AS account,
    credits - debits AS balance
FROM
    credits
    INNER JOIN debits ON credits.account = debits.account;

CREATE VIEW total AS
SELECT
    SUM(balance)
FROM
    balance;
```

И наконец, представление `total` передается в топик `total_risingwave`:

```
CREATE SINK total_sink
FROM
    total INTO kafka connection kafka_connection (TOPIC 'total_materialize')
    FORMAT JSON ENVELOPE DEBEZIUM WITH (SIZE = '1');
```

Теперь можно запустить рассмотренный пример, сначала настроив источники и представления в Materialize, а затем выполнив код на Python для создания 10 000 транзакций. Визуализированные результаты для Materialize показаны на рис. 6.6.

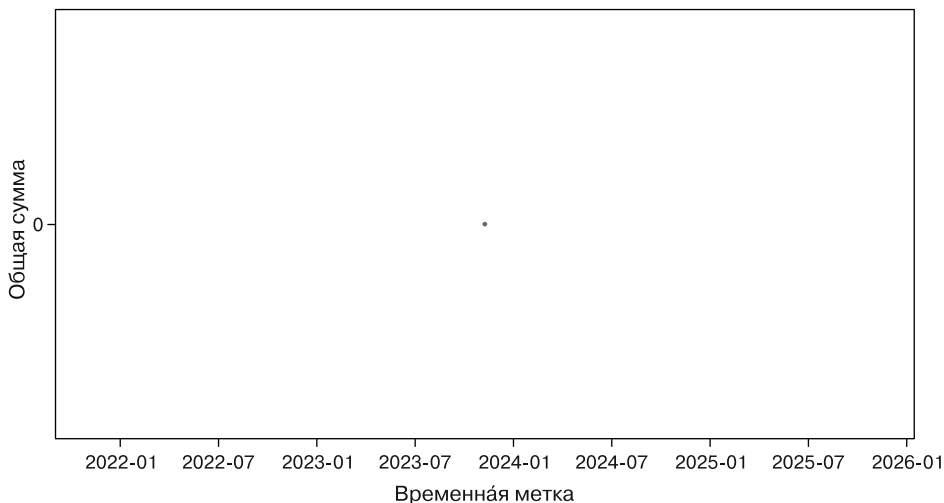


Рис. 6.6. Топик Kafka с названием `total_materialize`

При каждом запуске Materialize выводит только одно сообщение в топик назначения Kafka, и оно содержит правильный результат, равный 0. Подробнее о том, как это достигается в Materialize, расскажем в дальнейшем.

Pathway

На самом деле Pathway — это скорее библиотека обработки потоков для Python, а не полноценная потоковая БД. Мы добавили ее в эту главу, чтобы показать, что согласованности можно добиться и с помощью потокового процессора и что потоковую БД нельзя отнести к обязательным условиям.

Более того, это потоковый процессор, использующий Python в качестве императивного языка для создания потоковых конвейеров. Код на Python (пример 6.10)

содержит SQL-запросы, аналогичные тем, что используются для Flink SQL, ksqlDB, Proton, RisingWave и Materialize, за исключением того, что источник и место назначения результатов определяются на Python + SQL, а не просто на SQL.

Пример 6.10. Настройка примера на Python с помощью Pathway

```
#!/bin/python

import pathway as pw

rdkafka_settings = { ❶
    "bootstrap.servers": "localhost:56512",
    "group.id": "pw",
    "session.timeout.ms": "6000"
}

class InputSchema(pw.Schema): ❷
    id: int
    from_account: int
    to_account: int
    amount: int
    ts: str

t = pw.io.kafka.read( ❸
    rdkafka_settings,
    topic="transactions",
    schema=InputSchema,
    format="json",
    autocommit_duration_ms=1000
)

credits = pw.sql( ❹
    """
    SELECT to_account, sum(amount) as credits
    FROM T GROUP BY to_account
    """, T=t)

debits = pw.sql( ❺
    """
    SELECT from_account, sum(amount) as debits
    FROM T GROUP BY from_account
    """, T=t)

balance = pw.sql( ❻
    """
    SELECT CC.to_account, credits - debits as balance
    FROM CC
    join DD on CC.to_account = DD.from_account
    """, CC=credits, DD=debits)
```

```
total = pw.sql( ❷
    """
    SELECT sum(balance) as total FROM BB
    """, BB=balance)

pw.io.kafka.write( ❸
    total,
    rdkafka_settings=rdkafka_settings,
    topic_name='total_pathway',
    format='json')

pw.run() ❹
```

- ❶ Информация о подключении к Kafka.
- ❷ Схема потребляемых транзакций.
- ❸ `t` представляет потоковые транзакции, поступающие из Kafka.
- ❹ Таблица `credits`.
- ❺ Таблица `debits`.
- ❻ Таблица `balance` после вычитания дебетов из кредитов.
- ❼ Представление `total` будет использоваться для передачи данных в Kafka.
- ❽ Результат в топике Kafka с названием `total_pathway`.
- ❾ Асинхронный запуск потока.

Результатом выполнения этого приложения будет одна запись в топике `total_pathway` в Kafka (пример 6.11).

Пример 6.11. Единственная запись в результирующем топике `total_pathway`

```
{
    "total": 0, ❶
    "diff": 1,
    "time": 1698960910176
}
```

- ❶ Это значение записи с итоговым результатом 0.

Визуализация топика назначения (рис. 6.7) демонстрирует точно такое же поведение, как и система Materialize.

Чтобы смоделировать более реалистичные условия, мы изменили код Python из примера 6.1 так, чтобы примерно 1000 из 10 000 сообщений поступали не по порядку. Получили почти те же результаты, что и в тривиальном случае без

опоздавших/неудачных сообщений, для Flink SQL, ksqlDB и Proton и точно такие же результаты для RisingWave, Materialize и Pathway.

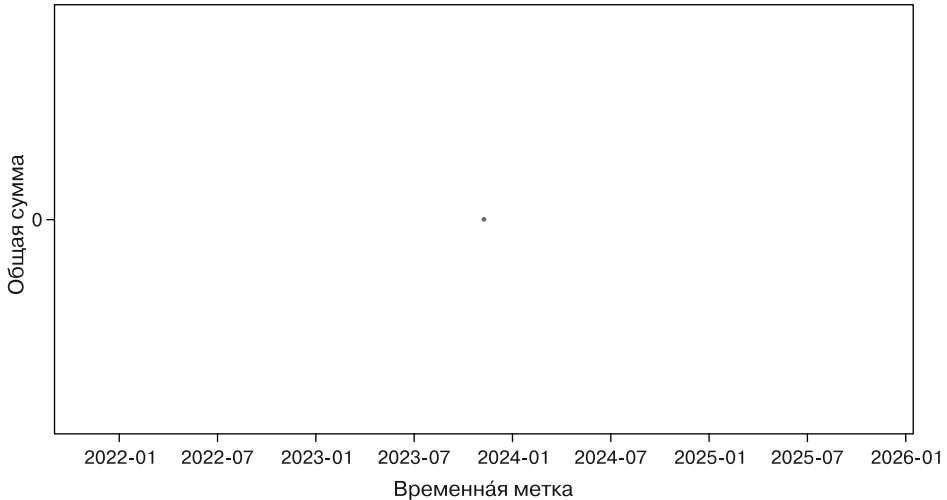


Рис. 6.7. Топик Kafka с названием balance

Выход за рамки согласованности в конечном счете

После того как вы ознакомились с работой пяти систем обработки потоков, без ответов осталось несколько вопросов.

- Почему согласованные в конечном счете Flink SQL, ksqlDB и Proton не справляются с нашим примером?
- Почему справляются внутренне согласованные RisingWave, Materialize и Pathway?
- И чему здесь можно научиться — можно ли найти обходной путь для Flink SQL, ksqlDB и Proton?

Почему согласованные в конечном счете потоковые процессоры не справляются с рассмотренным примером?

Давайте вернемся к тому, что мы делали в примере. Сначала создали два представления, `credits` и `debits`. До этого момента все было хорошо. Но как только мы соединили два представления в одно с названием `balance`, операторы `JOIN` из Flink SQL, ksqlDB и Proton не совсем корректно сопоставляли данные,

поступающие из представлений `credits` и `debits`. Чтобы понять, как это произошло, ограничимся четырьмя транзакциями.

1. Перевод одного доллара со счета 0 на счет 1.
2. Перевод одного доллара со счета 0 на счет 2.
3. Перевод одного доллара со счета 1 на счет 2.
4. Перевод одного доллара со счета 2 на счет 0.

Теперь посмотрим, что может произойти в системе обработки потоков с согласованностью в конечном счете, такой как Flink SQL, ksqlDB и Proton.

Поскольку два входа оператора `JOIN` в представлении `balance` не синхронизированы, возможен сценарий, при котором представление `credits` выдает свои результаты раньше, чем `debits`. В итоге представление `balance` объединяет свои входы следующим образом.

1. `balance` объединяет первый результат представления `credits` с первым результатом представления `debits`.
2. `balance` объединяет второй результат `credits` с первым результатом `debits`.
3. `balance` объединяет третий результат `credits` с первым результатом `debits`.
4. `balance` объединяет четвертый результат `credits` с первым результатом `debits`.

Происходящее здесь можно сравнить с *состоянием гонки*, показанным на рис. 6.8, где четыре транзакции расположены слева, а результаты в представлениях `credits` и `debits` — в средней части. Справа изображены результаты в представлении `balance` и соответствующие суммы в представлении `total`. В частях диаграммы `credits`, `debits` и `balance` есть обозначения `0:1`, которое означает «счет 0, значение 1», `2:1`, которое означает «счет 2, значение 1», и т. д., где «значение» — это `credits`, `debits` или `balance` в зависимости от положения на диаграмме.

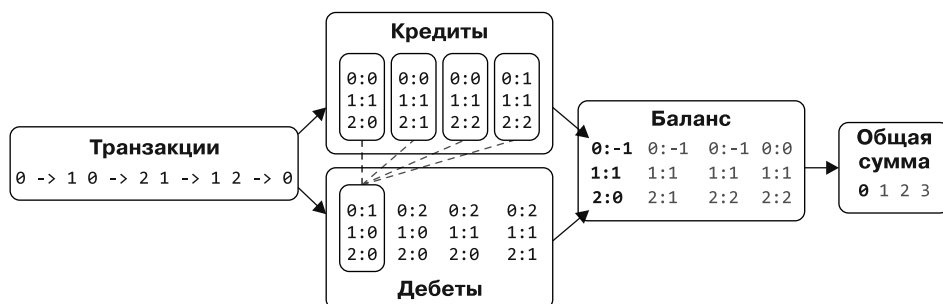


Рис. 6.8. Неправильные промежуточные результаты: `credits` выдает данные быстрее, чем `debits`

Четыре комбинации результатов из представлений `credits` и `debits` в `balance` обозначены пунктирными линиями. Три крайние правые пунктирные линии указывают на неправильные комбинации, а три крайних правых результата — на неправильные результаты в `balance` и `total`. Если первый результат (выделен полужирным шрифтом) представления `balance` (где первый результат `credits` правильно объединен с первым результатом `debits`) верен, то следующие три результата неверны и нарушают наш инвариант о том, что общая сумма баланса должна быть равна 0. Поскольку представление `credits` выдало свои результаты раньше, чем представление `debits`, итоговые неверные расчеты общей суммы дают положительные результаты (1, 2 и 3).

Для наглядности рассмотрим еще один возможный сценарий (рис. 6.9). В нем представление `debits` выдает свои результаты быстрее, чем представление `credits`.

1. `balance` объединяет первый результат представления `credits` с первым результатом представления `debits`.
2. `balance` объединяет первый результат `credits` со вторым результатом `debits`.
3. `balance` объединяет первый результат `credits` с третьим результатом `debits`.
4. `balance` объединяет первый результат `credits` с четвертым результатом `debits`.

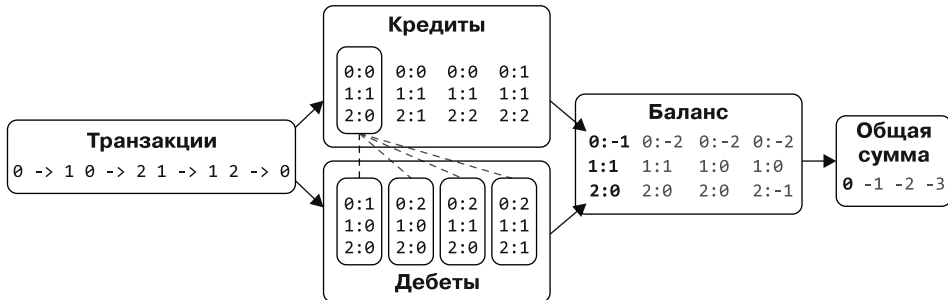


Рис. 6.9. Неправильные промежуточные результаты: `debits` выдает данные быстрее, чем `credits`

Здесь представление `debits` выдало свои результаты раньше, чем представление `credits`, и, следовательно, неправильные подсчеты общей суммы дают отрицательные результаты (−1, −2 и −3). Давайте разберемся, что могло стать причиной такого поведения.

Опережающая передача данных от немонотонных операторов

Основная цель систем потоковой обработки данных — обеспечение низкой задержки. Одно из проектных решений для ее достижения, применяемое в Flink SQL и ksqlDB, заключается в том, чтобы выдавать результаты как можно

раньше. Отметим также, что это привело передаче системами очень большого количества сообщений в топик назначения Kafka (почти 80 000 для Flink SQL и почти 40 000 для ksqlDB)¹.

Система Proton с согласованностью в конечном счете выдала гораздо меньше сообщений — 56. Для большинства классических сценариев использования потоковой обработки такое поведение вполне нормально, но для данных без оконной обработки это может стать *причиной сбоя*. В блоге Брэндона оно называется *опережающей передачей данных от немонотонных операторов*.

Опережающая передача результатов не представляет проблемы для монотонных операторов, таких как фильтры. Но на примере мы показали, что она *может стать* проблемой для немонотонных операторов. В данном случае немонотонные операторы, такие как JOIN и UNION, не могут просто объединить то, что они получают, как только они это получают, от обоих своих входов, и затем передать промежуточные результаты. Необходимо убедиться, что объединение выполняется только в отношении согласованных, синхронизированных входов.

MiniBatch в Proton, а также в качестве дополнения во Flink 1.19+ может стать средством от возникновения этой ошибки. Но об этом мы поговорим позже. В ksqlDB эту роль выполняет кэширование.

Объединение потоков без синхронизации

На самом деле причина нарушения работы систем потоковой обработки с согласованностью в конечном счете в нашем примере — это сочетание опережающей передачи от немонотонных операторов и того, что эти операторы не синхронизируют свои входы (*объединение потоков без синхронизации* в блоге Брэндона).

В данном случае транзакции нашего воображаемого банка соответствуют транзакциям БД, и они теряются, как только операция JOIN применяется к представлениям *credits* и *debits* в представлении *balance*. По сути, операторы JOIN во Flink SQL, ksqlDB и в некоторой степени в Proton просто объединяют все, что поступает из обоих представлений. И если они оказываются в состоянии гонки и любое из двух представлений предоставляет свои данные быстрее, чем другое, мы получаем неправильные/несогласованные результаты (см. рис. 6.8 и 6.9).

Распространенный ответ на необходимость синхронизации заключается в выполнении глобальной блокировки для такой формы синхронизации. Но в этом случае масштабирование будет недоступно. На самом деле использование

¹ Количество промежуточных результатов можно значительно сократить, оптимизировав конфигурации Flink SQL и ksqlDB (например, MiniBatch для Flink SQL, кэши KTable и/или интервал фиксации для ksqlDB) для конкретного случая применения. Мы просто использовали настройки по умолчанию.

глобальной блокировки для синхронизации — это только один из вариантов, самый простой и наименее подверженный масштабированию. В следующих разделах мы рассмотрим способы достижения синхронизации конкурентным и масштабируемым способом.

Как внутренне согласованные системы обработки потоков выполняют рассматриваемый пример?

Мы уже поняли, что ключ к правильному решению нашего примера — это умение сочетать потоки с синхронизацией. Интуитивно понятна необходимость убедиться в том, что объединяются только *принадлежащие друг другу* результаты из представлений `credits` и `debits` — или, другими словами, они соответствуют одной и той же транзакции в исходном топике `transactions` в Kafka.

Диаграммы с результатами топиков `total` показывают, что RisingWave (см. рис. 6.5), Materialize (см. рис. 6.6) и Pathway (см. рис. 6.7), возможно, нашли способ реализовать это. Но как?

RisingWave

В RisingWave используется концепция *барьеров*, вдохновленная *барьерами контрольных точек* во Flink. По сути, барьеры — это управляющие записи с *эпохами* (временные метки), которые периодически (например, каждую секунду) автоматически вводятся во все источники. В RisingWave барьеры используются как номера версий для данных. Операторам разрешается выдавать результат, связанный с определенной версией, только после того, как такая же версия/барьер будет получена со всех входов.

В системе Flink барьеры применяются для последовательного создания контрольных точек. В то же время в RisingWave в полной мере задействуются возможности потоковой БД, и система получает полный контроль над своим слоем хранения данных, включая контрольные точки. Таким образом, в этом отношении RisingWave может пойти дальше Flink и использовать барьеры контрольной точки не только для нее самой, но и для версий. Для этого системе потребуется адаптировать концепцию *изоляции моментальных снимков* из мира баз данных.

Давайте представим, что происходит с нашими четырьмя транзакциями и как RisingWave будет их обрабатывать. На рис. 6.10 введены барьеры в виде вертикальных линий после каждой транзакции. У каждого из барьеров есть своя версия, обозначенная подстрочными индексами (1, 2, 3, 4). Во время обработки в системе RisingWave барьеры транзакций передаются следующим операторам. Теперь при расчете представления `balance` барьеры используются для того, чтобы входные

данные из представлений *credits* и *debits* объединялись только в том случае, если они находятся перед одним барьером, то есть у них одна и та же версия.

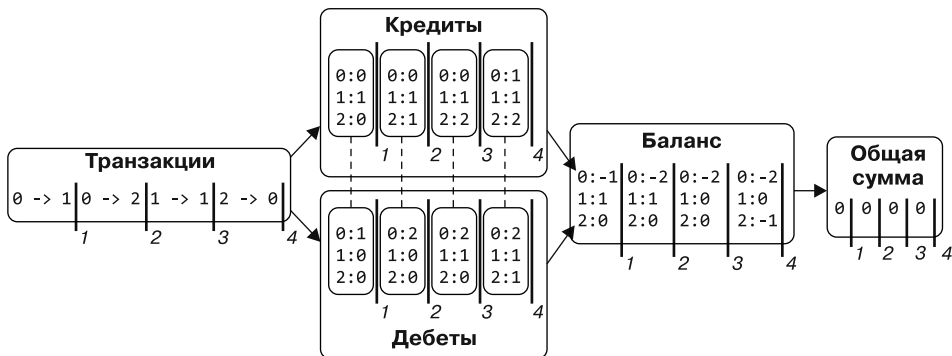


Рис. 6.10. Использование транзакционных барьеров для обеспечения согласованности

В принципе барьеры можно вводить и реже. Рассмотрим диаграмму на рис. 6.11. Здесь барьеры добавляются после каждого второго сообщения в топике *transactions*.

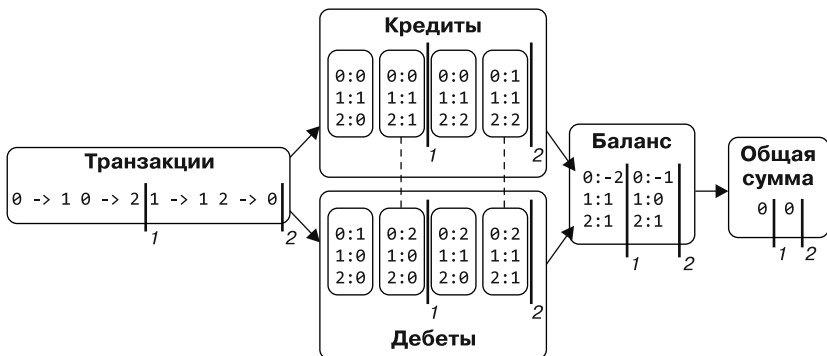


Рис. 6.11. Использование менее частых транзакционных барьеров для обеспечения согласованности



На первый взгляд барьеры напоминают водяные знаки, известные по потоковым процессорам типа Flink, поскольку они также представляют собой управляющие записи в диаграмме потока данных. Однако семантика барьеров и водяных знаков несколько различается. При наличии барьеров операторы могут выдавать результаты только после того, как они достигнут одного и того же барьера для всех входов. А при использовании водяных знаков операторы могут выполнять работу только после обнаружения такого знака. Он сигнализирует о необходимости получения всех событий до определенной временной метки.

Частота введения *барьеров эпох* в RisingWave напрямую влияет на задержки и потребление памяти. Чем чаще добавляются барьеры, тем ниже общая задержка, но это достигается за счет большего потребления памяти, поскольку хранение большего количества версий данных требует большего объема памяти.

Materialize

Система Materialize построена на *дифференциальном потоке данных* (Differential Dataflow, DD)¹. Данные в DD всегда версионированы, и все операторы DD соблюдают эти версии. Поэтому диаграмма на рис. 6.12 для Materialize очень похожа на рис. 6.10, с той лишь разницей, что данные версионированы сразу — DD не нужны никакие дополнительные концепции вроде барьеров.

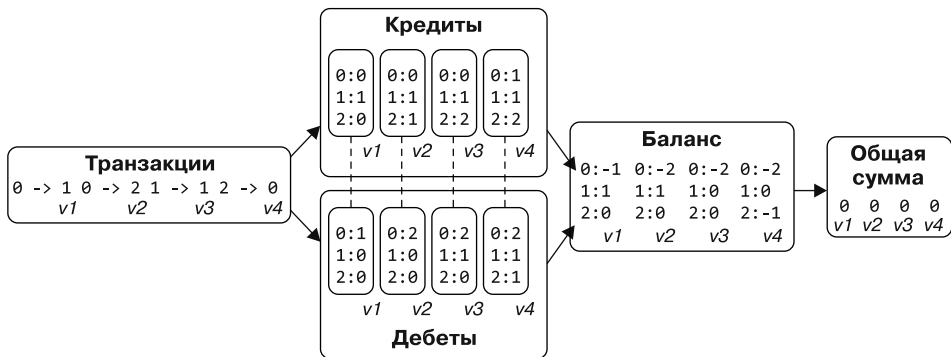


Рис. 6.12. Как DD/Materialize обеспечивает согласованность версий данных

Синхронизация операторов в DD достигается следующим образом.

1. Каждая точка отсчета сопровождается версией (на рис. 6.12 это v1, v2, v3 и v4).
2. Операторы могут объединять данные только одной версии.

Таким образом, DD также реализует форму изоляции моментальных снимков и может выполнить рассматриваемый пример.

Pathway

В основе системы Pathway тоже лежит концепция DD, следовательно, в ней также применяется версионирование для достижения синхронизации входных данных операторов. Благодаря этому Pathway может выполнить наш пример. Мы добавили описание этой системы, чтобы показать: такой формы

¹ Murray D. G. et al. Naiad: A Timely Dataflow System // Proceedings of SOSP'13: The 24th ACM Symposium on Operating Systems Principles (<https://oreil.ly/-dBrD>).

согласованности можно достичь не только с помощью потоковой базы данных, но и с библиотекой обработки потоков с внутренне согласованным базовым механизмом, например DD.

Как можно исправить системы обработки потоков с согласованностью в конечном счете, чтобы выполнить рассматриваемый пример?

Одна из ключевых особенностей внутренне согласованных систем обработки потоков — их способность синхронизировать входы двоичных немонотонных операторов, таких как UNION и JOIN. Это выполняется либо с помощью барьеров (RisingWave), либо с помощью версий (DD, Materialize, Pathway). Если говорить более абстрактно, то ключом к царству глобальной синхронизации без блокировки служат эффективные системы семантически значимых временных меток. Они позволяют раздельное выполнение в топологии потоковой обработки.

Можно ли воспользоваться этим пониманием для поиска решения, чтобы API Flink SQL также выполнил наш пример¹?

Как Flink SQL может выполнить пример

Для Flink SQL действительно существуют способы удачно выполнить наш пример. Один из них заключается в явном использовании поля временной метки `ts` в операторе WHERE представления `balance` для выполнения операции JOIN представлений `credits` и `debits` только в том случае, если поля `ts` в `credits` и `debits` совпадают². Изменения в коде Flink SQL показаны в примере 6.12.

Пример 6.12. Настройка представлений `balance` и `total` в Flink SQL с помощью поля `ts` в явном виде для синхронизации ввода оператора

```
CREATE VIEW credits(account, credits, ts) AS
SELECT
    to_account as account,
    sum(amount) as credits,
    ts
FROM
    transactions
```

¹ Мы пытались реализовать аналогичное решение для ksqlDB, но не смогли из-за ограниченного синтаксиса SQL в ksqlDB. Вероятно, можно реализовать его с помощью библиотеки Kafka Streams, используя ее низкоуровневый ProcessorAPI.

² На практике очень трудно сохранить временные метки в разных записях абсолютно одинаковыми, поэтому синхронизация по полю `ts` может оказаться ненадежным решением.

```
GROUP BY
    to_account,
    ts;

CREATE VIEW debits(account, debits, ts) AS
SELECT
    from_account as account,
    sum(amount) as debits,
    ts
FROM
    transactions
GROUP BY
    from_account,
    ts;

CREATE VIEW balance(account, balance) AS
SELECT
    credits.account,
    credits - debits as balance
FROM
    credits,
    debits
WHERE
    credits.account = debits.account
    AND credits.ts = debits.ts;
```

Благодаря этому исправлению размер топика назначения Flink SQL с названием `total_flinksql_ts` уменьшился с примерно 80 000 до 1 без каких-либо промежуточных результатов. Теперь он включает только одно сообщение с правильной суммой, равной 0, как и Materialize и Pathway (рис. 6.13).

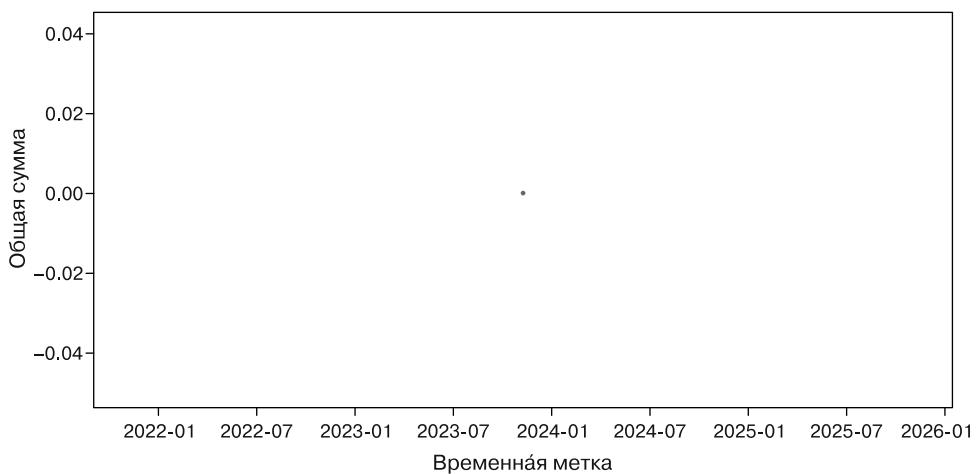


Рис. 6.13. Топик Kafka с названием `total_flinksql_ts`

Почему это решение может быть проблематичным

Выявив ключевую особенность внутренне согласованных систем обработки потоков для решения нашего примера, мы смогли решить задачу и с помощью Flink SQL. Для этого потребовалось добавить явную синхронизацию входных данных операторов по временным меткам. Так все же, является ли согласованность в конечном счете достаточно сильной для потоковой обработки? Зачем нам вообще нужна более сильная форма согласованности?

Решение Flink SQL для выполнения примера может быть проблематичным по ряду неочевидных причин. Оно отлично работает для рассматриваемого примера, но нужно учитывать следующие моменты.

- При добавлении временной метки (`ts`) к оператору `GROUP BY` в примере 6.12 было создано неограниченно растущее внутреннее хранилище состояний для агрегации. Это приведет к тому, что Flink исчерпает свою память на более позднем этапе.
- Инженеру с опытом из сферы баз данных будет трудно понять, почему его интуитивное решение создает 80 000 преимущественно неверных результатов в Flink SQL и почему ему вообще придется исправлять любой из своих идеально выглядящих интуитивно понятных кодов SQL.
- Явное «прикручивание» согласованности к уже готовой системе с согласованностью в конечном счете потребует разбирательства в каждом конкретном случае. Каждый новый практический случай может потребовать отдельного исправления.
- Подобные решения все равно могут привести к тонким несоответствиям. Предположим, у вас есть два топика ввода, `transactions1` и `transactions2`, один содержит банковские счета с 0-го по 4-й, а другой — с 5-го по 9-й. И что произойдет, если у транзакции из топика `transactions1` будет та же временная метка, что и у транзакции из `transactions2`? Тогда запретить Flink SQL комбинировать неправильные входы не получится. И снова придется искать решение для вашего исправления, например, включить идентификатор транзакции в условие операции `JOIN` и т. д.

В целом кажется, что согласованные в конечном счете системы обработки потоков часто *можно* исправить и добиться от них согласованного поведения. Но уровень этой согласованности зависит только от вас, инженера, и может измениться со следующей операцией `JOIN`, которую потребуется реализовать поверх существующего кода SQL. Согласованная потоковая работа с данными без оконной обработки *может* быть выполнена в потоковых процессорах с согласованностью в конечном счете. Но она почти гарантированно будет более трудоемкой и склонной к ошибкам. Кроме того, вряд ли она будет доступна кому-то за пределами узкого круга экспертов по потоковой обработке. Слишком легко получить противоречивые результаты, даже если на первый взгляд SQL-код выглядит совершенно нормально.

MiniBatch в системе Flink 1.19+

Во Flink 1.19 появилась семантика MiniBatch (мини-пакетной агрегации) для операций JOIN. Если активировать и настроить MiniBatch на соответствие определенным условиям использования, можно не только значительно повысить производительность Flink, но и добиться гораздо более высокого уровня согласованности.

MiniBatch — это оптимизация для буферизации входных записей с целью сокращения доступа к состоянию. Мы экспериментировали с различными конфигурациями MiniBatch и выяснили, что для примера из данной главы это очень эффективное средство от возникающей несогласованности.

Чтобы активировать MiniBatch, необходимо настроить три параметра:

```
SET 'table.exec.mini-batch.enabled' = 'true';  
  
SET 'table.exec.mini-batch.allow-latency' = '5S';  
  
SET 'table.exec.mini-batch.size' = '5000';
```

Элемент конфигурации `table.exec.mini-batch.allow-latency` устанавливает максимальную задержку оптимизации MiniBatch для буферизации входных записей операторов JOIN. `table.exec.mini-batch.size` устанавливает максимальное количество входных записей для буферизации. Агрегация MiniBatch запускается с допустимым интервалом задержки и при достижении максимального количества записей.

Мы обнаружили, что если установить значение `table.exec.mini-batch.size` равным 1, то Flink все равно выводит в топик назначения такое же количество записей (почти 80 000). И большинство из них неверны. Это неудивительно, так как данное значение настройки, по сути, отключает MiniBatch и Flink возвращается к поведению по умолчанию. Если увеличить размер пакета до 10, то количество выводимых сообщений значительно уменьшится (403). А если увеличить до 50 и более, то в топик назначения будет выведено только одно сообщение, что даст нам правильный результат, равный 0. Этот согласованный результат достигается и в том случае, если транзакции передаются не по порядку.

Таким образом, MiniBatch не только оптимизирует производительность, но и влияет на уровень согласованности Flink. Однако имейте в виду, что это не превращает Flink из согласованного в конечном счете во внутренне согласованный потоковый процессор.

Честно говоря, в документации Flink агрегация MiniBatch описывается только как оптимизация производительности. Чтобы получить преимущества в согласованности, MiniBatch необходимо настроить на правильную конфигурацию. И даже в этом случае нет гарантий достижения действительно высокого уровня согласованности во всех обстоятельствах.

Согласованность и задержка

Слон в комнате — вопрос, который вы как читатель наверняка уже задавали себе, — это вопрос о задержке. Насколько низкой задержкой системы обработки потоков с согласованностью в конечном счете необходимо пожертвовать, чтобы достичь более высокого уровня внутренней согласованности?

Прежде чем ответить на этот вопрос, рассмотрим два вида задержки.

Задержка времени обработки

Время, необходимое системе обработки потоков для получения ответа на запрос.

Общая задержка

Время, необходимое системе обработки потоков для получения *согласованного* ответа на запрос.

Задержка внутренне согласованных систем обработки потоков более высокая, если говорить о чистом времени обработки. Однако в большинстве неклассических случаев использования потоковой обработки, например, при выполнении операций JOIN над данными без оконной обработки, пользователей больше интересует общая задержка. В экстремальном случае рассматриваемого примера мы наблюдали, что все три внутренне согласованные системы обработки потоков могли прийти к согласованному результату менее чем за секунду. А в системах с согласованностью в конечном счете результат не достигается *никогда* (если только входной поток не прекратится или не будет исправлен SQL-код)...

В заключение следует сказать, что в идеальном мире новые версии Flink SQL, ksqlDB и Proton могли бы получить от своих разработчиков переключатель для включения и отключения внутренней согласованности. В таком случае пользователи могли бы легко выбрать желаемый компромисс:

- включить внутреннюю согласованность для случаев применения с неограниченными данными/без оконной обработки и там, где не требуется сверхмалая задержка;
- отключить внутреннюю согласованность и вернуться к согласованности в конечном счете для классических случаев обработки потоков с оконными данными, когда требуется сверхнизкая задержка.

Это позволит сохранить текущую модель обработки для случаев использования со сверхнизкой задержкой и в то же время даст возможность обрабатывать данные без оконной обработки более согласованно. Это позволило бы специалистам плавно переходить от базы данных к потоковой передаче, не прибегая к явному добавлению согласованности.

Резюме

Классические системы обработки потоков с низкой задержкой и высокой пропускной способностью идеально подходят для обычных случаев использования в масштабе с оконными данными. Но у них есть свои недостатки.

- Их сложно использовать в тех случаях, когда речь идет о данных без оконной обработки.
- Инженеры с опытом из сферы баз данных не могут придерживаться проверенных интуитивных представлений и формул SQL.

Мы считаем, что это два самых больших препятствия на пути широкого распространения не только систем потоковой обработки, но и потоковой передачи данных в целом. Если вы работали в сфере баз данных, стали бы вы работать с системой, которая может превратить идеально красивый SQL-код в тот непоследовательный хаос, который мы получили в рассмотренном примере? Конечно, согласованность можно «прикрыть», например усложнив SQL-код дополнительными условиями, определениями водяных знаков и т. д. Но обычно это могут надежно реализовать лишь редкие высокооплачиваемые эксперты по обработке потоков.

Внутренне согласованные системы обработки потоков, такие как RisingWave, Materialize и Pathway, обеспечивают более надежные гарантии согласованности. Они могли бы решить наш пример без дополнительных настроек и предложили бы более высокий уровень абстракции (и более целостную абстракцию) для сложных временных концепций, которые лежат в основе потоковой обработки. Таким образом, эти системы способны *демократизировать* обработку потоков для тех, кто решится перейти от баз данных к потокам, и значительно расширить рынок потоковой передачи данных в целом.

Что касается задержки по отношению ко времени обработки, то внутренне согласованные системы обработки потоков не могут превзойти системы с согласованностью в конечном счете. Но смогут, если посмотреть на более важный показатель — общую задержку.

В следующей главе мы переходим в область гибридных систем данных. Она тесно связана с потоковыми базами данных.

ГЛАВА 7

Появление других гибридных систем данных

В этой главе изучим более широкий спектр гибридных систем. Они появились в ответ на растущие требования современных событийно-ориентированных приложений реального времени. Хотя в этой книге мы определяем потоковые БД несколько в ином смысле, гибридные системы все же обладают общими качествами и свойствами для установления связи между реляционными, аналитическими и потоковыми рабочими нагрузками. Мы разберем мотивы их разработки, инновационные методы и конкретные сценарии применения, которые делают их актуальными. Что еще более важно, рассмотрим ниши, занятые этими гибридными базами данных. Эти знания дадут нам возможность выявить тенденции развития БД для предоставления аналитики в режиме реального времени.

Важно отметить, что потоковая БД — тоже пример гибридной системы. Это объясняется тем, что гибридные системы используют как минимум две точки зрения, и в случае с потоковой БД эти две точки зрения — обработка потоков и базы данных.

Понимание этих точек зрения позволит понять, какие задачи гибридные системы пытаются решить и каким образом. В данной книге потоковые БД определены с точки зрения обработки потоков следующим образом: потоковая БД — это потоковый процессор, предоставляющий свое хранилище состояний клиентам, чтобы они могли выполнять pull-запросы.

Альтернативное определение с точки зрения баз данных имеет следующий вид: потоковая БД — это БД, способная потреблять и передавать потоки, а также асинхронно формировать материализованные представления.

Такое определение гибридной системы с обеих точек зрения расширит ее доступность для других инженеров и иных вариантов использования. Примером может служить согласованность в обработке потоков — инженеры потоковых БД были вынуждены взглянуть на эту проблему с точки зрения БД, что позволило выявить недостаток согласованности в некоторых известных потоковых процессорах.

Хранилища состояний могут быть реализованы по-разному: на основе подхода с ключами-значениями, на базе строк или столбцов. Подход к реализации хранилища состояния определяет допустимые сценарии использования — они могут варьироваться от высоких требований к согласованности до аналитических запросов с необходимостью низкой задержки.

Интересно, что потоковые БД — это лишь один из примеров появления гибридных и объединенных систем. Они снижают сложность инфраструктуры и повышают доступность для разработчиков.

Плоскости данных

Давайте лучше изучим эти развивающиеся системы, создав *диаграмму Венна* (рис. 7.1). На ней показаны области существования систем реального времени. Диаграмма поможет определить различные сценарии использования (более подробно мы рассмотрим их в главе 11) и модели развертывания в аналитических сценариях реального времени.

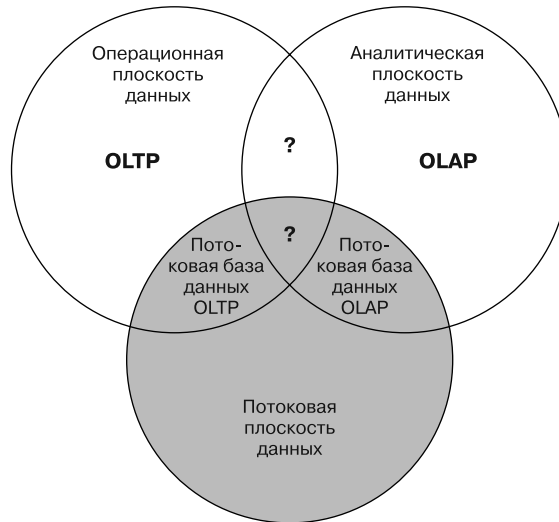


Рис. 7.1. Плоскость потоковой обработки

Диаграмма позволит рассмотреть не только потоковую передачу, но и перспективу с точки зрения БД. Например, в начале этой главы мы дали определение потоковой базы данных с точки зрения обработки потоков. А с точки зрения базы данных потоковая БД может потреблять и передавать потоки и асинхронно формировать материализованные представления.

Учет всех точек зрения позволит понять, как могут выглядеть БД следующего поколения.

Диаграмма Венна помогает лучше понять роль потоковых систем в аналитической экосистеме. Две уже известные нам плоскости данных — операционная и аналитическая (см. рис. 7.1). Но на диаграмму добавлена третья плоскость — потоковых данных. Ее существование ранее не признавали. Это единственная плоскость данных, где данные по большей части находятся в движении. Другие плоскости хранят данные в состоянии покоя. В точках пересечения потоковой плоскости с операционной и аналитической плоскостями данные хранятся в состоянии покоя и в движении. Именно здесь и располагаются потоковые БД. Другие области пересечения мы рассмотрим позже в данной главе.

Потоковая плоскость объединяет в себе операционные и аналитические аспекты обработки данных. Она собирает и обрабатывает данные в режиме реального времени, что дает возможность плавно перемещать их в аналитическую плоскость. Здесь они хранятся, анализируются и используются для обдумывания и принятия решений. Таким образом, эта плоскость выступает в роли связующего звена для ускорения принятия решений организациями. Решения будут основаны на сочетании потоковых и накопленных ранее данных.

В качестве примера приведем операционную плоскость. На ней располагаются согласованные базы данных OLTP на основе строк. Здесь находятся также приложения, использующие БД OLTP. Аналитическая плоскость включает в себя БД OLAP — согласованные в конечном счете хранилища на основе столбцов. Они оптимизированы для обслуживания аналитических запросов.

Потоковая плоскость содержит source-коннекторы для преобразования данных в состоянии покоя в данные в движении, а также sink-коннекторы записи потоковых данных в БД RTOlap для предоставления данных с низкой задержкой клиентам. В потоковой плоскости используются такие платформы, как Kafka и Kafka Connect, для репликации и предоставления потоковых данных. Потоковые процессоры с сохранением состояния и потоковые БД тоже находятся в этой плоскости.

Давайте подробнее рассмотрим только потоковую плоскость (рис. 7.2). В главе 6 вы узнали о спектре согласованности потоковых процессоров. На рисунке показана подробная версия только окружности потоковой плоскости диаграммы Венна. Здесь строго согласованные потоковые процессоры (*слева*) отделены от согласованных в конечном счете (*справа*), а типы хранилищ указаны сверху и снизу.

Потоковые данные движутся слева направо и проходят путь от операционной плоскости к аналитической. Следует помнить, что коннекторы и потоковые платформы, такие как Kafka, также расположены в потоковой плоскости.



Рис. 7.2. Потоковая плоскость

Как всегда в диаграммах Венна, самое интересное находится там, где окружности пересекаются.

Далее рассмотрим точки пересечения плоскостей операционных и аналитических данных.

Гибридная транзакционно-аналитическая база данных

Потоковые базы данных OLTP (хранилища на основе строк) объединяют потоковую обработку в потоковой плоскости с базой данных OLTP в операционной плоскости. Именно на этом пересечении находятся согласованные потоковые БД на основе строк, например RisingWave и Materialize.

Потоковые базы данных OLAP (хранилища на основе столбцов) объединяют характеристики БД OLAP в аналитической плоскости с характеристиками потоковой обработки в потоковой плоскости. Эти базы данных оптимизированы для выполнения сложных аналитических запросов с использованием индексирования и столбцового хранения данных. Они также получили характеристики согласованности в конечном счете. Система Proton располагается именно в этой области.

Пересечение операционных и аналитических (без потоковой обработки) БД представляет собой гибридную транзакционно-аналитическую обработку (HTAP) (рис. 7.3). Эти базы данных способны справляться с рабочими нагрузками OLTP и OLAP. Эта идея родилась в компании Gartner в 2014 году (<https://oreil.ly/75RaD>).

HTAP — это новая архитектура приложений, разрушающая барьер между обработкой транзакций и аналитикой. Такой подход позволяет принимать более обоснованные бизнес-решения в режиме реального времени.



Рис. 7.3. База данных HTAP — это гибридная база данных OLTP/OLAP без потоковой обработки

У вас может возникнуть вопрос: «Разве не было причины для разделения рабочих нагрузок OLTP и OLAP? Разве этот *барьер* не был создан здесь намеренно?»

База данных HTAP поддерживает два типа хранения — in-мемогу и постоянное. Дизайн БД HTAP компании Gartner позволяет выполнять аналитические запросы на «летающих» транзакциях, осуществляющих запись. База данных in-мемогу обеспечивает БД HTAP возможность выполнять оба вида рабочих нагрузок. Базы данных HTAP удовлетворяют свойствам ACID благодаря транзакциям и постоянным записям, что позволяет ей справляться с рабочими нагрузками OLTP (рис. 7.4).

Использование in-мемогу-хранилища для обслуживания аналитических запросов очень похоже на поведение потоковых БД, только HTAP не поддерживает

обработку потоков. Некоторые БД НТАР не поддерживают и асинхронные материализованные представления, что лишает их возможностей, аналогичных потоковой обработке.

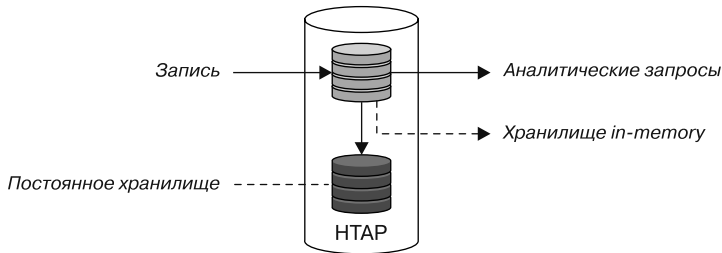


Рис. 7.4. Внутренняя архитектура НТАР определена компанией Gartner

Базы НТАР могут эффективно предоставлять простую аналитику в режиме реального времени, поскольку способны передавать запросы из своего in-мемори-хранилища непосредственно в приложение, выполняющее запись. В табл. 7.1 представлены некоторые из существующих БД НТАР, доступные на рынке на момент написания книги.

Таблица 7.1. Базы данных НТАР

Название	Поставщик	Реализация хранилища
Unistore	Snowflake	Все данные в гибридной таблице хранятся в строчном и столбцовом хранилищах. Данные при изменении синхронно изменяются в строчном хранилище и асинхронно передаются в столбцовое хранилище
SingleStoreDB	SingleStore	SingleStoreDB поддерживает два типа таблиц: на основе столбцов на диске (их называют Columnstore — тип таблицы по умолчанию для SingleStoreDB Cloud) и на основе строк в in-мемори-хранилище. Columnstore также называют Universal Storage
TiDB	PingCAP	TiDB поддерживает столбцовые и транзакционные хранилища «ключ — значение». TiKV — это распределенная транзакционная БД «ключ — значение», которая предоставляет транзакционные API с соблюдением стандарта ACID. TiFlash — это аналитическое расширение в семействе TiDB. Оно обеспечивает работу TiDB с помощью столбцового хранилища и механизма запросов с массово-параллельной архитектурой (Massively Parallel Processing, MPP)
HydraDB	Hydra	База данных с открытым исходным кодом, поддерживающая транзакционные хранилища на основе строк, называемые heap-таблицами, и компоновку хранилищ на основе столбцов, которая используется по умолчанию

Базы данных НТАР, изображенные на рис. 7.4, не соответствуют дизайну НТАР, предложенному компанией Gartner. Если следовать этой реализации хранения данных, то каждая БД НТАР должна включать в себя хранилище на основе как строк, так и столбцов. Никакой обработки на лету, для аналитических запросов применяются in-методу-транзакции.

Использование БД НТАР фактически устраняет необходимость в потоковой плоскости. Такой подход предполагает, что вся работа с данными может выполняться в режиме реального времени в БД НТАР.

У этих БД есть некоторые ограничения, не позволяющие им занять место аналитики реального времени. Это монолитные решения без возможности хранить накопленные данные, как происходит в чистых OLAP-системах. Накопленные данные могут составлять терабайты или даже петабайты, поэтому НТАР — лучшее решение, когда нет необходимости хранить такие данные. В качестве альтернативы можно создать БД OLAP и НТАР в своей инфраструктуре. И этот подход возвращает нас к описанному ранее разделению данных.

БД НТАР могут очень эффективно предоставлять аналитику приложениям в операционной плоскости. Но выполнять сложные запросы ad hoc для аналитиков данных в них затруднительно. Для этого необходимо будет извлечь агрегированную историю из OLAP-системы, чтобы уменьшить объем хранимых данных до такого, который сможет вместить НТАР. Опять же остается разделение данных, что НТАР пытается устранить.

На рис. 7.5 показаны только пересечения в середине диаграммы Венна. Здесь видно, что эти области образуют цветок с тремя лепестками — НТАР, потоковые OLTP и потоковые OLAP базы данных.

Гибридные базы данных были разработаны для решения в режиме реального времени аналитических задач, связанных с масштабируемостью и оптимизацией. Такие задачи, как правило, требуют большей или иной инфраструктуры. Увеличение инфраструктуры приводит к более сложной интеграции данных и их перемещению, прежде чем аналитика в режиме реального времени может быть предоставлена приложениям.

На данном этапе центр цветка (или пестик) еще не определен. Но уже можно сделать выводы о подходе к реализации БД в этом пестике. Но прежде необходимо обсудить другие гибридные базы данных операционной плоскости, поскольку они не так легко вписываются в нашу модель Венна.

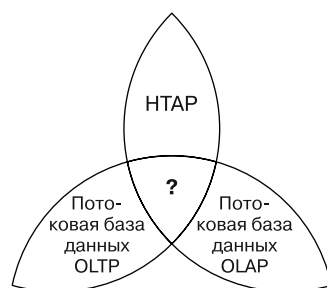


Рис. 7.5. Триада гибридных баз данных

Другие гибридные базы данных

Во время работы над книгой мы столкнулись со многими потоковыми базами данных, которые не вписывались в наши определения потоковых или БД НТАР (табл. 7.2). Эти другие гибридные БД сочетают в себе уникальные возможности для решения задач, обычно с помощью потоковых систем из потоковой плоскости.

Таблица 7.2. Другие гибридные базы данных

Название	Гибридные системы	Описание
PeerDB	БД Postgres OLTP + потоковый процессор	PeerDB — это платформа для перемещения данных на базе системы Postgres, позволяющая быстро и просто перемещать данные в Postgres и из нее. Она дает возможность синхронизировать данные, преобразовывать их и загружать в OLAP-систему. При ее использовании материализованные представления должны создаваться в БД OLTP, поэтому она не соответствует нашему определению потоковой базы данных, но все же попадает в категорию потоковых БД OLTP
Epsio	БД Postgres OLTP + внешнее асинхронное материализованное представление	Epsio подключается к существующим базам данных и постоянно обновляет результаты заданных вами запросов при каждом базовом изменении данных, не пересчитывая весь набор данных. Такой подход позволяет Epsio предоставлять мгновенные и всегда актуальные результаты по сложным запросам, значительно сокращая при этом расходы
Turso	БД SQLite OLTP + потоковая платформа	Turso позволяет разрабатывать локально и реплицировать глобально во многие точки мира, предоставляя синхронный доступ к данным, а не асинхронный, как Kafka
Redpanda	Потоковая платформа + Apache Iceberg (БД)	Разработчики могут использовать собственные механизмы обработки для запросов к данным в многоуровневом хранилище Redpanda, не перемещая их по разным системам. Это позволяет снизить нагрузку на аналитическую инфраструктуру

Мотивы создания гибридных систем

Поставщики систем обработки потоков хотят привнести в потоковую обработку опыт работы с базами данных, чтобы снять с нее клеймо слишком сложной и трудной для внедрения. Потоковая обработка часто ассоциируется со сложностью, особенно у тех, кто не очень хорошо разбирается в тонкостях обработки данных в режиме реального времени. Поставщики систем обработки потоков

стремятся восполнить этот пробел в знаниях, обеспечивая более приятный опыт работы, схожий с работой в сфере традиционных баз данных. Предполагаются упрощение API, создание удобных интерфейсов и предоставление инструментов, более интуитивно понятных широкому кругу пользователей.

Стереотип о том, что потоковую обработку данных слишком сложно внедрить, не позволяет многим организациям в полной мере задействовать аналитику данных в режиме реального времени. Поставщики систем переносят опыт работы с базами данных в потоковую обработку. Тем самым они пытаются снизить барьеры для входа в систему, делая ее более доступной и понятной для специалистов предприятий различных отраслей.

Нехватка квалифицированных инженеров по работе с данными, способных эффективно обращаться с системами потоковой обработки, также стала серьезным препятствием. Упрощение внедрения потоковой обработки за счет создания привычной среды, похожей на БД, поможет организациям применять возможности анализа данных в режиме реального времени без привлечения узких специалистов.

И наоборот, БД OLTP пытаются перенять функционал БД OLAP, чтобы лучше обслуживать аналитические запросы на операционном уровне. БД OLTP также вынуждены внедрять потоковый функционал, чтобы удовлетворить растущий спрос на аналитику данных в режиме реального времени. Эти функции позволяют избежать перехода в аналитическую плоскость, которая для многих обладает сложной и незнакомой инфраструктурой.

Во множестве организаций работа ведется в распределенной среде. В таких условиях обеспечение согласованности данных в нескольких БД и системах может оказаться непростой задачей. В БД OLTP внедряется функционал потоковой передачи данных для упрощения репликации и синхронизации данных. При использовании потоковой передачи становится возможно распространять изменения в режиме реального времени, снижая вероятность несогласованности данных.

Инженеры из сферы баз данных признают эти потребности, но не осознают, что они представляют собой характеристики потоковой обработки.

Общая цель каждой гибридной системы на диаграмме Венна на рис. 7.1 — *предоставление аналитики в режиме реального времени при менее объемной инфраструктуре и большей доступности для инженеров.*

Многие из гибридных систем основаны на БД OLTP. Это связано с их расположением ближе к пользовательскому приложению и с работой в режиме реального времени. Становится все более очевидным, что предоставление аналитики в режиме реального времени требует приближения аналитики данных

к операционной плоскости. А возможно, и полного перемещения в нее. Разработчики потоковых систем должны осознать это, чтобы лучше понять потребности клиентов и исправить репутацию слишком сложных для внедрения решений.

Влияние PostgreSQL на гибридные базы данных

Многие гибридные базы данных созданы на основе СУБД PostgreSQL (или Postgres). Она представляет собой очень популярную базу данных OLTP. Postgres и ее протокол используются во многих популярных на сегодняшний день гибридных базах данных: RisingWave, Materialize, Hydra, PeerDB и Epsio.

Популярность системы Postgres и ее сообщества можно объяснить несколькими дополнительными факторами — они перечислены в табл. 7.3.

Таблица 7.3. Факторы популярности Postgres

Фактор	Описание
Расширяемость	Расширяемая архитектура Postgres позволяет разработчикам создавать собственные типы данных, операторы и функции, что делает ее подходящей для широкого спектра приложений и отраслей
Оптимизация производительности	Сообщество вкладывает средства в оптимизацию механизма базы данных, что обеспечивает конкурентоспособную производительность и эффективную обработку запросов
Экосистема сторонних производителей	Вокруг Postgres сформировалась богатая экосистема инструментов, библиотек и расширений сторонних разработчиков, что еще больше увеличивает ее возможности и гибкость
Внедрение на предприятиях	Многие крупные организации и предприятия используют Postgres для своих критически важных приложений, что способствует росту доверия к этой системе и ее популярности
Глобальная доступность	Система Postgres не привязана к определенному региону или отрасли, что делает ее привлекательной для широкой и разнообразной базы пользователей по всему миру

Система Postgres стала популярным и долговечным проектом в мире реляционных баз данных. Это связано с сочетанием в ней принципов открытого исходного кода, доброжелательного и активного сообщества, надежных методов разработки и многофункционального механизма баз данных. Поскольку популярность Postgres продолжает расти, следует ожидать, что все больше гибридных баз данных будут предоставлять возможности работы с БД, аналогичные возможностям Postgres.

Выполнение аналитики вблизи границы плоскостей данных

Перенос аналитики в операционную плоскость — это шаг к тому, чтобы сделать данные более оперативными и доступными для конечных пользователей. Этот подход обусловлен прежде всего необходимостью снижения задержки, что, в свою очередь, дает возможность быстрее принимать решения в сценариях реального времени. Гибридные БД играют ключевую роль в этой работе, поскольку они нацелены на предоставление аналитических возможностей без необходимости дублировать всю аналитическую плоскость. Зачастую пользователям не требуется доступ ко всему хранилищу данных — они запрашивают только конкретные и актуальные данные, необходимые для принятия решений в режиме реального времени. Такая оптимизированная доставка данных обеспечивает быстрое получение ценных сведений.

Некоторые аналитические нагрузки никогда не попадут в операционную плоскость, потому что:

- агрегируются большие объемы накопленных данных из БД OLAP. Это могут быть петабайты данных, не подходящих для операционной инфраструктуры;
- обучение моделей машинного обучения требует специализированных систем, которых нет в операционной плоскости;
- требуются высокораспределенные системы с возможностью разделения данных для их обработки на основе массово-параллельной архитектуры. А это опять же не полностью поддерживается операционной плоскостью;
- нужна гибкость для выполнения запросов *ad hoc*, предоставляющих данные пользователям внутри компании. Аналитика для внешних пользователей не обладает такой гибкостью, и часто появляется необходимость запрашивать дополнительные параметры для отображения в их приложениях.

Цель создания любой системы реального времени состоит в нахождении самого простого и оптимального подхода к предоставлению аналитики конечным пользователям без огромных затрат на инфраструктуру и ресурсы. Именно гибридные БД обладают такими возможностями. Работа в режиме реального времени определяет следующее поколение гибридных БД. Сейчас все больше СУБД поддерживают функционал для работы в режиме реального времени. Следовательно, они становятся все более гибридными по своей природе.

Гибридные базы данных нового поколения

Наглядное представление о современном ландшафте систем реального времени дает рис. 7.6. На нем представлен обзор технологий и решений, используемых сегодня организациями для удовлетворения своих потребностей в обработке данных в режиме реального времени. В зонах пересечения, образующих лепестки, наиболее ярко проявляются последние тенденции и инновации в аналитике реального времени. К ним относятся HTAP, потоковые БД OLTP и потоковые БД OLAP.

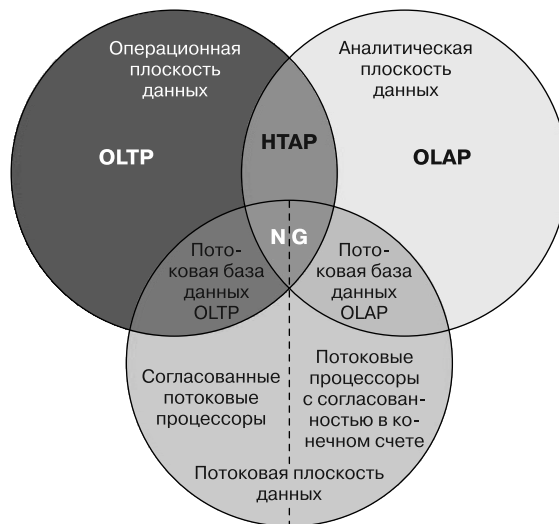


Рис. 7.6. Базы данных нового поколения

В центре рисунка обозначено следующее поколение (next generation, NG) БД реального времени. Базы данных следующего поколения будут обладать функционалом всех трех плоскостей данных, включая:

- обработку потоков с сохранением состояния;
- столбцовые хранилища для аналитических рабочих нагрузок;
- согласованность операционных рабочих нагрузок.

На момент публикации книги в области следующего поколения (см. рис. 7.6) не существовало баз данных. Базы данных в лепестках должны будут добавить

недостающий функционал, чтобы претендовать на статус нового поколения, например:

- БД HTAP могут добавить *инкрементное обслуживание представлений* (Incremental View Maintenance, IVM). Тогда эти системы будут работать аналогично потоковому процессору с сохранением состояния. Им также потребуется интеграция с потоковыми платформами, такими как Kafka. Обслуживание IVM описано в главе 11;
- потоковые БД OLTP должны обеспечивать хранение данных на основе столбцов. Этого можно добиться с помощью встроенной БД OLAP, например DuckDB. О встроенных БД OLAP расскажем в главе 8;
- потоковые БД OLAP должны добавить согласованность в потоковые процессоры, чтобы они могли лучше участвовать в реальной логике приложения.

Существующие гибридные системы находятся на переднем крае аналитики в режиме реального времени. Они предлагают организациям многофункциональные и более целостные средства для покрытия потребностей в обработке данных в динамичной и управляемой данными среде. Инновационная направленность этих гибридных систем позволяет аналитике в режиме реального времени продвигаться на новые территории. Границы между реляционными, аналитическими и потоковыми рабочими нагрузками начинают стираться, а организации могут извлекать еще большую пользу из своих активов данных.

Мы считаем, что на фоне инноваций будет происходить активное развитие гибридных систем. В каждую из них разработчики продолжают вносить новый функционал для уменьшения инфраструктуры, снижения задержек и общей сложности.

Далее мы подробнее рассмотрим дополнительные возможности существующих гибридных БД реального времени, необходимые для достижения ими статуса нового поколения.

Потоковые БД OLTP нового поколения

Потоковым БД OLTP нового поколения доступны три области для дальнейшего развития. Первая — это совершенствование моделей согласованности данных. При потоковой обработке ее необходимо повышать, чтобы система могла участвовать в логике приложения. Все больше инженеров приходят к пониманию проблем с согласованностью в потоковой обработке и осознают повышенные требования к точности при предоставлении аналитики пользователям. В связи с этим все большему количеству потоковых БД OLTP необходимо будет повышать уровень согласованности итоговой аналитики.

Вторая — улучшение доступа к данным об изменениях или журналам WAL. Для использования CDC требуются коннекторы, работающие в выделенных и распределенных кластерах. Это увеличивает сложность и объем технического сопровождения всей архитектуры. Чтобы упростить этот процесс, транзакции CDC из их журналов WAL передаются на потоковую платформу, такую как Kafka. Этот функционал уже используется такими базами данных, как PeerDB и CockroachDB.

При получении возможности передавать изменения системы БД будут избавлены от необходимости создавать коннекторы для всех возможных точек интеграции, чтобы поглощать и выводить данные. Разработка таких коннекторов требует много времени и значительных средств. Она также ограничивает систему несколькими вариантами использования, и новые появятся только в случае, если разработка еще одного коннектора станет финансово выгодной. Кроме того, некоторыми коннекторами CDC часто трудно управлять, и в результате могут возникать исключения недостатка памяти или места на диске. Самостоятельное создание событий CDC может предотвратить проблемы с помощью сторонних решений CDC (рис. 7.7).

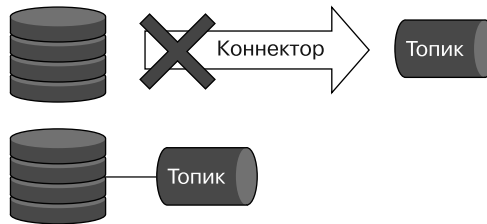


Рис. 7.7. Потоковая OLTP нового поколения будет направлять данные CDC непосредственно в топик, соответствующий требованиям *Kafka-compliant*

Третья — потоковые БД OLTP начнут внедрять столбцовые форматы хранения для аналитических рабочих нагрузок, чтобы предоставлять аналитические данные приложениям. Это не означает, что потоковые БД OLTP будут хранить все накопленные данные — они не способны хранить такие большие объемы данных. Потоковые БД OLTP должны предоставлять подмножество накопленных данных из системы OLAP либо в потоковом режиме, либо пакетным способом (пакетный подход допускается, поскольку данные не поступают в режиме реального времени). Подмножество накопленных данных будет ограничено только данными из контекста домена приложения. Такой подход гарантирует, что потоковые БД OLTP смогут поддерживать аналитические рабочие нагрузки без перегрузки требованиями к хранению накопленных данных.

Потоковые БД RTOLAP нового поколения

В существующих сейчас БД RTOLAP нет функциональных возможностей для обработки потоков. Система Proton — единственное решение, объединяющее потоковую обработку и OLAP-систему. Ее потоковый процессор обеспечивает более продвинутые функции поглощения данных и может поддерживать баланс запросов push и pull.

Большинство существующих RTOLAP сильно зависят от внешних потоковых процессоров, что увеличивает сложность и трудоемкость обслуживания всей архитектуры реального времени.

В главе 4 мы говорили о том, что авторами push-запросов зачастую выступают инженеры по обработке данных, а pull-запросы пишут аналитики данных. Координация этих двух ролей обычно затруднена, потому что чаще всего такие специалисты находятся в разных командах. Добавление потоковой обработки при поглощении данных системой сделает хранилище RTOLAP менее зависимым от внешних потоковых процессоров.

Более того, внешние потоковые процессоры, такие как Flink, часто публикуют данные таким образом, что на них могут подписаться несколько потребителей. Обычно для удовлетворения специфических потребностей аналитиков данных необходимо выполнить дополнительные преобразования в формате данных. Если эти специфические преобразования включить в push-запрос, система обработки потоков сможет также публиковать данные для одного конкретного подписчика.

Ожидается, что существующие базы данных RTOLAP перейдут на более качественную обработку данных путем добавления потоковой обработки и предоставления аналитикам возможностей push-запросов.

Базы данных HTAP нового поколения

Следующее поколение БД HTAP будет использовать возможности гибридного хранения данных за счет добавления обработки IVM. Этот метод применяется для поддержания материализованных представлений. Он асинхронно вычисляет и применяет только инкрементные изменения к представлениям и не оценивает все их содержимое заново. В предыдущих главах мы уже говорили, что асинхронное материализованное представление очень похоже на потоковую обработку.

БД HTAP благодаря внедрению IVM получают возможность преобразовывать транзакции из строчной в столбцовую форму для аналитических запросов с низкой задержкой. При этом нет необходимости выводить данные за пределы системы.

Вероятно, БД НТАР также получат возможность поглощать ограниченное количество накопленных данных из аналитической плоскости. Это обеспечит ограниченный накопленный контекст для предоставляемых в режиме реального времени аналитических данных.

Резюме

Мы подробно рассмотрели разнообразные гибридные базы данных с функциональными возможностями для работы в режиме реального времени. Этот обзор позволит вам принять взвешенное решение при выборе подходящей БД для определенных потребностей обработки данных в режиме реального времени. В этой главе была представлена диаграмма Венна для плоскостей данных. Она помогла проиллюстрировать уникальные качества существующих систем и то, как происходит их слияние в правильное решение.

Можно предположить, что слияние систем приведет к тому, что архитектуры данных станут менее распределенными и более монолитными. А монолитные системы, как правило, утрачивают гибкость и способность к масштабированию. В следующих главах мы обсудим возможности распределения систем и то, как избежать превращения гибридных систем в монолитные.

ГЛАВА 8

Zero-ETL или Near-Zero-ETL

В главе 7 мы рассказали о новых гибридных БД с альтернативными решениями для обеспечения процессов аналитики в режиме реального времени. Такие системы сокращают инфраструктуру и делают данные более доступными для аналитики. Поскольку гибридные системы выполняют слияние традиционно распределенных систем, существует предположение, что они близки к монолитным. Последние известны тем, что им не хватает модульности и масштабируемости в ходе работы с данными.

Как ни странно, разделение монолитной системы данных вернет нас к декомпозиции базы данных и выворачиванию ее наизнанку для масштабирования отдельных компонентов. Такое решение не обязательно будет негативным. Мы предлагаем поместить системы обратно в базу данных, чтобы уменьшить сложность и стоимость, которые традиционно ассоциируются с большими распределенными системами.

Подход ETL описывает способ перемещения данных из системы в систему с выполнением преобразований по пути. Мы использовали форму ETL, называемую потоковым SQL. В этой главе обсудим возможности достижения баланса сложности и масштабируемости при реализации ETL. Для этого рассмотрим существующие системы и паттерны, используемые для распределения и масштабирования рабочих нагрузок с данными.

Модель ETL

На рис. 8.1 показаны существующие решения ETL — от отсутствия ETL в БД НТАР в верхней части до распределенного решения выворачивания базы данных наизнанку в нижней части. Чем ниже решение находится на треугольнике, тем более распределенным, сложным и модульным оно становится. Наверху расположены решения более централизованные и монолитные.

В левой части треугольника находятся транзакционные базы данных, а в правой — столбцовые. В середине треугольника располагаются решения zero-ETL.

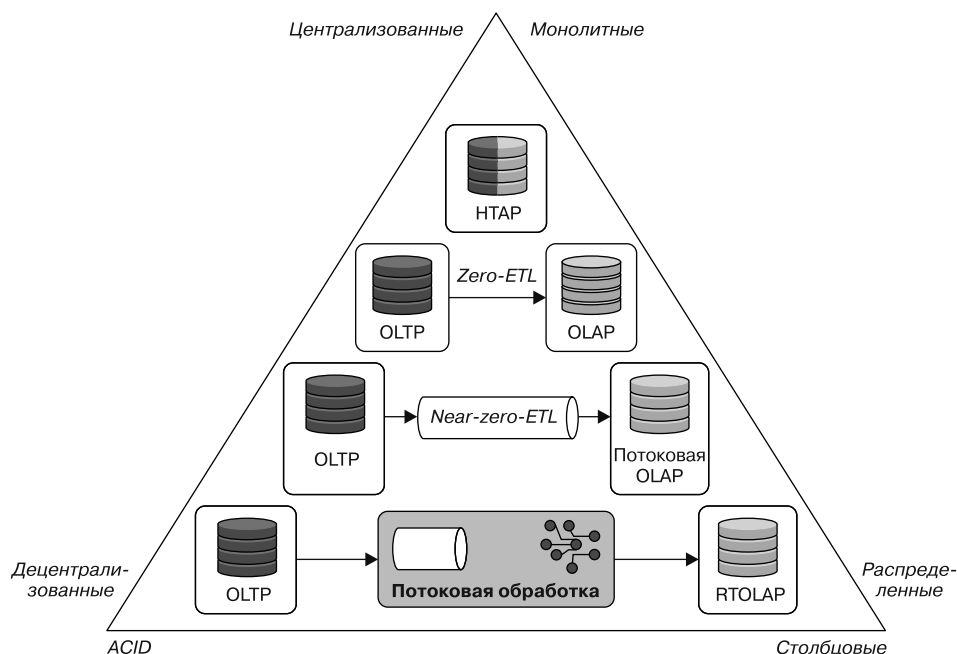


Рис. 8.1. Растущая модель ETL

Zero-ETL

Паттерн *zero-ETL* был создан компанией Amazon Web Services (AWS) для упрощения интеграции данных из БД OLTP в БД OLAP. В предложении решения *zero-ETL* определяется следующим образом:

«Набор интеграций, который устраняет или сводит к минимуму необходимость создания конвейеров данных ETL. Извлечение, преобразование и загрузка (extract, transform and load ETL) — это процесс объединения, очистки и нормализации данных из различных источников для подготовки их к анализу, обработке искусственным интеллектом (ИИ) и использованию в машинном обучении (machine learning, ML)».

AWS, *What Is Zero ETL?* (<https://oreil.ly/9-5CD>)

Zero-ETL — это подход или концепция в области интеграции данных и аналитики. Он направлен на минимизацию традиционных процессов ETL или даже устранение необходимости в них. Традиционный процесс ETL подразумевает извлечение данных из исходных систем, их преобразование в соответствии с требованиями целевой системы, а затем загрузку в целевую систему. Краткое описание новой архитектуры приведено на рис. 8.2.

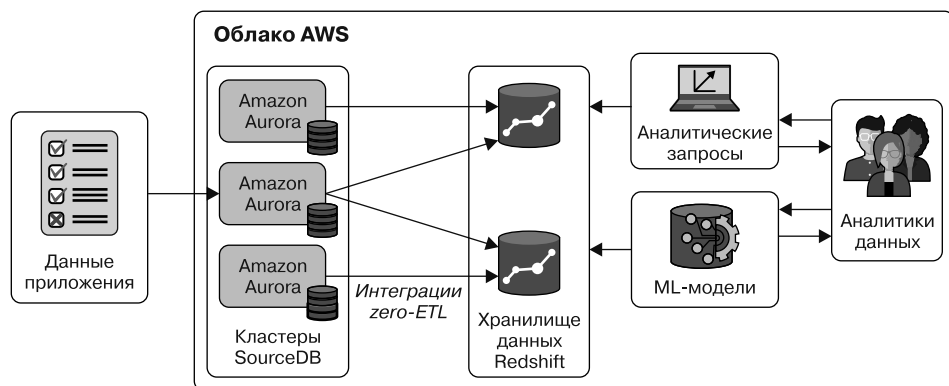


Рис. 8.2. Архитектура AWS zero-ETL для Amazon Aurora и Redshift

Решение AWS zero-ETL представляет собой управляемую интеграцию между БД OLTP Amazon с названием Aurora и хранилищем данных Amazon, называемым Redshift. Аналитику в режиме, близком к реальному времени, можно реализовать в течение нескольких минут. Это полностью управляемое решение для обеспечения доступности транзакционных данных в Redshift после их записи в Aurora.

Платформа AWS может тесно интегрировать Aurora и Redshift, поскольку ей принадлежат оба продукта баз данных, которые существуют только на этой облачной платформе. AWS может свободно создавать собственные интеграции между любыми двумя предоставляемыми ей системами. Но эти решения недоступны за пределами платформы Amazon, и это ощутимый недостаток.

Концепция zero-ETL может дать преимущества в плане гибкости, сокращения задержек и экономии средств, но она подходит не для всех сценариев. У большинства организаций, особенно со сложными требованиями к интеграции данных или нормативными ограничениями, сохраняется потребность в элементах традиционных процессов ETL.

В главе 2 мы говорили о том, что преобразование данных (также называемое ETL) в хранилище принуждает к использованию семантики пакетной обработки. А это увеличивает задержку в любом аналитическом сценарии реального времени. На рис. 8.2 точки интеграции между Aurora и Redshift не преобразуют данные. Это означает, что преобразования выполняются в Redshift. Такая проблема характерна для баз данных HTAP и решений zero-ETL. В обоих случаях необходимо запустить процесс пакетной трансформации, как только данные попадут в хранилище.

В качестве альтернативного решения применяются запросы от аналитиков данных с необходимыми преобразованиями, но это делает их очень медленны-

ми. В конечном счете без компонента обработки потоков невозможно создать материализованное представление, чтобы разделить аналитическую нагрузку между запросами push и pull.

Концепция zero-ETL бросает вызов традиционному подходу к интеграции и за определенную плату стремится снизить сложность, задержки и требования к ресурсам, связанные с ETL. В табл. 8.1 перечислены некоторые ключевые аспекты zero-ETL.

Таблица 8.1. Zero-ETL

Ключевой аспект	Описание
Интеграция данных в режиме реального времени	Минимизация пакетной обработки или отказ от нее для обеспечения интеграции данных в режиме реального времени или близком к нему. Это особенно актуально для сценариев, в которых очень важна своевременная информация
Схема при считывании (schema-on-read)	Подход «схема при считывании» не предполагает преобразования данных в заранее определенную схему в процессе ETL — данные интерпретируются в момент анализа. Это позволяет более гибко работать с разнообразными и меняющимися данными
Виртуализация данных	Использование технологий виртуализации данных, обеспечивающих единое виртуальное представление данных из различных источников без их физического перемещения или преобразования. Это позволяет избежать необходимости создавать и обслуживать отдельное хранилище данных
Обработка в базе данных	Выполнение преобразований и аналитики непосредственно в системах БД, где хранятся данные, что избавляет от необходимости извлекать и перемещать большие массивы данных для обработки
Событийно-ориентированная архитектура	Внедрение событийно-ориентированной архитектуры, в которой изменения данных вызывают немедленное обновление. Такой подход снижает зависимость от периодических пакетных процессов
Современные архитектуры данных	Использование современных архитектур данных, таких как озера данных и облачные решения, для обеспечения масштабируемых и экономически эффективных вариантов управления данными и их анализа без традиционных узких мест ETL

В конечном счете решение о применении подхода zero-ETL зависит от характера данных, требований бизнеса и доступного технологического ландшафта. Важно тщательно оценить компромиссы и выбрать подход, соответствующий целям и приоритетам организации.

Если требуется гибкость традиционного конвейера ETL, альтернативным решением может стать использование подхода near-zero-ETL.

Near-zero-ETL

В подходе near-zero-ETL сохраняется попытка ограничить инфраструктуру для компонентов ETL без потери гибкости, необходимой для поддержки сложных требований к интеграции данных. В этом случае предполагается использование систем данных с гибридными подходами.

Одним из решений служит применение БД OLTP со встроенным функционалом для отправки данных в другие системы без необходимости самостоятельно управлять коннекторами, работающими на отдельной инфраструктуре.

На рис. 8.3 показаны две базы данных со встроенным функционалом обеспечения гибкости для комплексной интеграции данных — БД OLTP и БД OLAP. Система PeerDB позволяет Postgres отправлять поток данных в топик потоковой платформы. Timeplus/Proton обеспечивает преобразование, необходимое при поглощении, перед предоставлением данных клиенту. Система Timeplus/Proton обеспечивает материализованное представление для разграничения запросов push и pull. Такой подход дает гораздо большую гибкость при выполнении быстрых аналитических запросов в режиме реального времени.



Рис. 8.3. Near-zero-ETL с использованием PeerDB и Timeplus/Proton

PeerDB

PeerDB — это решение с открытым исходным кодом для потоковой передачи данных из Postgres в хранилища данных, очереди/топики и другие системы хранения. Оно разрабатывалось с целью упростить ETL, обеспечив работу с базой данных при построении интеграции с аналитическими системами.

PEER (пир) — это соединение с базой данных, к которой PeerDB может обращаться. Пир создается с помощью команды CREATE PEER (пример 8.1).

Пример 8.1. Установка пира из другой базы данных Postgres в PeerDB

```
CREATE PEER source FROM POSTGRES WITH ❶
(
  host = 'catalog',
  port = '5432',
  user = 'postgres',
  password = 'postgres',
  database = 'source'
);
```

```

CREATE PEER sf_peer FROM SNOWFLAKE WITH ❷
(
    account_id = '<snowflake_account_id>',
    username = '<user_name>',
    private_key = '<private_key>',
    password = '<password>' -- only provide when the private key is encrypted
    database = '<database_name>',
    schema = '<schema>',
    warehouse = '<ware_house>',
    role = '<role>',
    query_timeout = '<query_timeout_in_seconds>'
);

-- Query away tables in Snowflake
SELECT * FROM sf_peer.MY_SCHEMA.MY_TABLE; ❸

```

- ❶ Создание пира к другой базе данных Postgres.
- ❷ Создание пира хранилища данных Snowflake.
- ❸ Можно выбрать любое количество таблиц из списка.

Пир дает возможность запрашивать любую таблицу в указанной базе данных и объединять ее с транзакционными данными. Доступная интеграция между системами эффективна, но она усложняется, когда базы данных расположены в удаленных друг от друга регионах. Соединения PEER действуют как запросы синхронного извлечения данных из аналитической системы. Для их выполнения необходимо, чтобы обе системы находились в одном регионе или центре обработки данных.

В других базах данных, включая потоковые БД OLTP, необходимо создавать конвейеры данных для получения данных из хранилища или БД OLAP и передачи их в БД OLTP. Создание пиров с помощью PeerDB в Postgres значительно упрощает поиск данных в аналитической плоскости. Но у такого решения есть ограничения.

Система Postgres не может хранить большие объемы данных и не оптимизирована для аналитических запросов. Поэтому строить материализованное представление с помощью PeerDB на основе Postgres будет неоптимально. Аналитические данные нужно уменьшить до размера, подходящего для емкости БД OLTP. Это означает, что push-запрос для затратных в плане системных ресурсов преобразований должен быть выполнен на внешнем потоковом процессоре. Мы обсудим этот момент позже в данной главе.

В примере 8.2 показано, как PeerDB отзеркаливает данные от одного пира к другому. Зеркало асинхронно копирует данные с исходного пира в пир назначения.

Пример 8.2. Создание ETL с операцией MIRROR из PeerDB в аналитической системе

```
CREATE MIRROR <mirror_name> [IF NOT EXISTS] FROM
    <source_peer> TO <target_peer> FOR
$$
    SELECT * FROM <source_table_name> WHERE
    <watermark_column> BETWEEN {{.start}} AND {{.end}}
$$
WITH (
    destination_table_name = '<schema_qualified_destination_table_name>',
    watermark_column = '<watermark_column>',
    watermark_table_name =
    '<table_on_source_on_which_watermark_filter_should_be_applied>',
    mode = '<mode>',
    unique_key_columns = '<unique_key_columns>',
    parallelism = <parallelism>,
    refresh_interval = <refresh_interval_in_seconds>,
    sync_data_format = '<sync_data_format>'
    num_rows_per_partition = 100000,
    initial_copy_onle = <true|false>,
    setup_watermark_table_on_destination = <true|false>,
);
```

К сожалению, операции отзеркаливания не поддерживают преобразования — их необходимо выполнить до или после зеркального отображения. Выполнение преобразований перед операцией отзеркаливания может привести к перерасходу ресурсов БД OLTP, предназначенной для обработки оперативных рабочих нагрузок. Более того, этот процесс будет выполняться в пакетном режиме, а не в режиме реального времени. Выполнение преобразований после отзеркаливания также потребует семантики пакетной обработки, если только не используется потоковая БД OLAP типа Proton.

Proton

На диаграмме Венна в главе 7 показано пересечение гибридных систем. Одной из них была потоковая БД OLAP. Proton — это база данных RTOLAP нового поколения. Она позволяет поглощать потоковые данные с учетом состояния и для получения аналитики в режиме реального времени использовать два API — асинхронный поток изменений и синхронные pull-запросы. Сложные преобразования могут быть реализованы на этапе поглощения данных для построения материализованных представлений.

На рис. 8.3 в предыдущем разделе PeerDB записывает данные в топик потоковой платформы, на который подписана БД Proton. Она может выполнять сложные push-запросы до того, как данные будут материализованы для pull-запросов.

Сначала создадим соединение PEER для потоковой платформы — в данном случае Kafka (пример 8.3).

Пример 8.3. Настройка пира к другой базе данных Postgres в PeerDB

```
CREATE PEER <eh_peer_name> FROM KAFKA WITH (
    bootstrap_server = '<bootstrap-servers>'
);

CREATE MIRROR <mirror_name> [IF NOT EXISTS] FROM
    <source_peer> TO <target_peer> FOR
$$
    SELECT * FROM <source_table_name> WHERE
    <watermark_column> BETWEEN {{.start}} AND {{.end}}
$$
WITH (
    destination_table_name = '<topic>'
);
```

В Proton создается поток для чтения из Kafka (пример 8.4).

Пример 8.4. Создание потока чтения из Kafka в Proton

```
CREATE EXTERNAL STREAM frontend_events(raw string)
SETTINGS type='kafka',
    brokers='<bootstrap-servers>',
    topic='<topic>'
```

Все потоковые БД предоставляют два режима вывода данных — синхронные pull-запросы и асинхронные push-запросы к топику. Это дает разработчикам два способа возвращения аналитики в операционную плоскость в режиме реального времени с помощью баз OLAP, встроенных в их приложения.

Встроенные OLAP

Сейчас наблюдается тенденция приближения небольших аналитических нагрузок к операционной плоскости. БД HTAP, такие как Hydra и SingleStore, предоставляют столбцовые базы данных, например, для аналитических рабочих нагрузок. Однако из-за своей ограниченной емкости эти БД не могут вместить такой объем данных, как аналитические системы Snowflake, Databricks, ClickHouse и Pinot.

И наоборот, перенос громоздких аналитических систем в операционную плоскость для более быстрого предоставления аналитики в режиме реального времени усложняет работу аналитических систем с накопленными данными. Именно эти ограничения привели к разделению между операционными и аналитическими плоскостями данных.

Альтернативным решением может стать такое сокращение объема аналитических данных, чтобы они соответствовали масштабам бизнес-домена и возможностям операционной плоскости.

На рис. 8.4 оперативные данные в режиме реального времени могут быть отправлены в топик с помощью PeerDB. Proton получает возможность захватывать и преобразовывать данные на лету с помощью push-запроса, ведущего к материализованному представлению.



Рис. 8.4. Потоковая БД OLAP сокращает аналитические данные для предоставления клиентам в операционной плоскости

Изменения материализованного представления в Proton могут быть записаны в топик для использования исходным приложением и создания локальной копии во встроенной БД OLAP, такой как DuckDB или chDB.

DuckDB

Встроенная БД OLAP DuckDB предназначена для поддержки рабочих нагрузок аналитических запросов. Встроенная — это значит работающая внутри приложения, например, в микросервисе. DuckDB предоставляет пользователям приложения возможность нарезать данные на фрагменты. Поскольку данные об изменениях поступают из Proton и потребляются микросервисом, можно создать локальную копию материализованного представления в DuckDB (пример 8.5).

Пример 8.5. Установка DuckDB для использования в микросервисе на языке Python

```
pip install duckdb
```

Пример 8.6 — это каркас микросервиса. Его можно подписать на топик Kafka и выполнить операцию UPSERT для записи в таблицу DuckDB. Затем эта таблица будет обслуживать аналитические запросы через REST API с помощью библиотеки FastAPI Python.

Пример 8.6. Псевдокод микросервиса для считывания данных из Kafka и записи в DuckDB

```
import duckdb
from threading import Thread, current_thread
from fastapi import BackgroundTasks, FastAPI
from confluent_kafka import Consumer

app = FastAPI()
duckdb_con = duckdb.connect('my_persistent_db.duckdb') ❶

def upsert(msg): ❷
    # РЕАЛИЗАЦИЯ ЛОГИКИ UPSERT

def kafka2olap(conf): ❸
    consumer = Consumer(conf)
    try:
        consumer.subscribe("my_data")
        while running:
            msg = consumer.poll(timeout=1.0)
            if msg is None: continue
            if msg.error():
                # Обработка ошибки
            else:
                upsert(msg)
    finally:
        # Закрытие потребителя для фиксации итоговых смещений
        consumer.close()

@app.on_event("startup")
async def initialize(): ❹
    conf = # Конфигурация Kafka
    thread = Thread(
        target = kafka2olap,
        args = (conf,)
    )
    thread.start()

@app.get("/my_data/")
async def read_item(id:int): ❺
    results = local_con.execute("""
        SELECT
            id,
            count(*) AS row_counter,
            current_timestamp
        FROM my_data
        where id = ?
    """, (id,)).fetchall()
```

- ❶ Создание соединения с DuckDB.
- ❷ Определение функции upsert для вставки/обновления в DuckDB.
- ❸ Определение функции для чтения из Kafka и записи в DuckDB.
- ❹ Создание асинхронного потока для чтения из Kafka и записи в DuckDB.
- ❺ Функция FastAPI для чтения DuckDB с помощью REST API.

В DuckDB есть функция для проверки наличия конфликта при вставке записи в таблицу. Оператор INSERT OR REPLACE выполнит обновление, если запись существует (пример 8.7).

Пример 8.7. Установка UPSERT для DuckDB на языке Python

```
def upsert(msg):
    # Десериализация msg, чтобы получить значения столбцов
    primary_key, col1_value, col2_value = deserialize_message(msg)

    duckdb_con.execute("""
        INSERT OR REPLACE INTO t1(id, col1, col2) VALUES(1, ?, ?) ❶
        """,
        [primary_key, col1_value, col2_value]
    );
```

❶ С помощью команды INSERT OR REPLACE можно обработать операцию UPSERT в DuckDB.

chDB

chDB тоже встраиваемая база данных OLAP. Она создана на основе СУБД ClickHouse. Последняя поддерживает операции UPSERT с помощью *механизмов обработки*, или *движков*. Один из них — движок таблиц ReplacingMergeTree, удаляющий дубликаты записей при слиянии. ReplacingMergeTree — хороший вариант для эмуляции логики upsert, когда требуется, чтобы запросы возвращали последнюю вставленную строку. Более подробно — в примере 8.8.

Пример 8.8. Движок ENGINE из ClickHouse для поддержки UPSERT

```
CREATE TABLE hackernews_rmt (
    id UInt32,
    author String,
    comment String,
    views UInt64
)
ENGINE = ReplacingMergeTree ❶
PRIMARY KEY (author, id);
```

```
SELECT *
FROM hackernews_rmt
FINAL ❷
```

❶ Движок `ReplacingMergeTree`, имитирующий поведение `upsert`.

❷ Ключевое слово `FINAL` возвращает самую последнюю запись.

В примере 8.9 показано, как создать микросервис с помощью фреймворка `Flask` и `chDB`. `Flask` — это альтернатива модели `FastAPI Python` для создания микросервисов.

Пример 8.9. Обертка микросервиса `chDB`

```
from flask import Flask, request
import chdb
import os
```

Пример сервера API `chdb` с поддержкой операций `GET/POST`, совместимый с приложением
Полный пример сервера см. на странице <https://github.com/metrico/chdb-server>

```
app = Flask(__name__, static_folder="", static_url_path="")
```

```
@app.route('/', methods=["GET"])
def clickhouse():
```

```
    query = request.args.get('query', default="", type=str)
    format = request.args.get('default_format', default="JSONCompact", type=str)
    if not query:
        return "Query not found", 400
```

```
    res = chdb.query(query, format)
    return res.bytes()
```

```
@app.route('/', methods=["POST"])
def play():
```

```
    query = request.data
    format = request.args.get('default_format', default="JSONCompact", type=str)
    if not query:
        return "Query not found", 400
```

```
    res = chdb.query(query, format)
    return res.bytes()
```

```
@app.errorhandler(404)
def handle_404(e):
    return "Not found", 404
```

```
host = os.getenv('HOST', '0.0.0.0')
port = os.getenv('PORT', 8123)
app.run(host=host, port=port)
```

Гравитация и репликация данных

Зачастую аналитические системы располагаются только в одном регионе или центре обработки данных, поскольку аналитическая инфраструктура, такая как Snowflake, стоит дорого. По этой причине все системы операционной обработки отправляют свои данные в один регион, что называется *гравитацией (притяжением) данных*.

Суть ее заключается в том, что данные обладают массой, которую трудно перемещать или реплицировать по мере увеличения их размера и важности. Гравитация влияет на такие операции, как создание данных и обмен ими, что, в свою очередь, влияет на приложения, серверы и др. В качестве типичного решения было принято реплицировать только уменьшенные объемы аналитических данных.

Если предоставлять изменения из материализованного представления системам операционной плоскости, можно распространять копии аналитики в режиме реального времени по всем регионам развертывания пользовательских приложений.

Сокращение аналитических данных

Как уменьшить объем аналитических данных, представляющих собой петабайты накопленных данных? На первый взгляд такая операция сокращения объема данных, обслуживаемых в операционной плоскости, кажется сложной. Но на самом деле это не так, потому что мы уже выполняли подобные действия. Можно использовать шаблон запросов push и pull для материализованных представлений.

При создании материализованного представления в аналитической системе из аналитической плоскости можно транслировать изменения материализованного представления в операционную плоскость. База данных Proton дает возможность реализовать этот замысел. Она может записывать изменения в материализованном представлении в топик. Затем этот топик может использоваться приложением со встроенным хранилищем OLAP для выполнения аналитических задач. Аналогичным образом потоковые БД OLTP могут потреблять данные из одного топика и предоставлять их из своего хранилища на основе строк. Однако такой способ хранения данных может увеличить задержку аналитических запросов.

Подход near-zero-ETL позволяет найти правильный баланс между сложностью и масштабируемостью. Для полноты картины давайте посмотрим, что требуется для обработки аналитических данных с помощью отдельного потокового процессора и БД OLAP в лямбда-архитектуре.

Лямбда-архитектура

Лямбда-архитектура обработки данных была разработана для сочетания пакетной и потоковой обработки данных в режиме реального времени. Это понятие ввел Натан Марц в своей книге *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*¹. Лямбда-архитектура создана как способ решения задач обеспечения надежной и масштабируемой обработки данных для приложений, работающих с большими данными. Термин «лямбда» — это название греческой буквы, которая выглядит как перевернутая буква «у» и представляет собой два пути обработки данных: пакетный и в режиме реального времени.

Лямбда-архитектура состоит из трех основных уровней.

Пакетный

Этот уровень отвечает за обработку больших объемов данных в пакетном режиме. Он предварительно вычисляет результаты по всему набору данных и хранит их на уровне пакетного обслуживания, что делает его пригодным для сложной аналитики и запросов к накопленным данным. Пакетная обработка обычно выполняется с помощью таких технологий, как Apache Hadoop, — они позволяют выполнять объемную распределенную обработку данных.

Скоростной

Скоростной уровень обрабатывает данные в режиме реального времени. Он ориентирован на обработку данных с низкой задержкой и взаимодействует с последними данными, которые еще не были обработаны пакетным уровнем. Результаты работы скоростного уровня объединяются с результатами пакетного для получения полного и актуального представления о данных. Для обработки данных в режиме реального времени на скоростном уровне обычно используются такие технологии, как Apache Storm или Apache Flink.

Предоставления

Слой предоставления объединяет результаты пакетного и скоростного уровней, обеспечивая единое представление данных. Он обслуживает обычные и аналитические запросы пользователей или приложений. Для эффективной работы с интенсивными нагрузками на чтение слой представления часто строится с помощью масштабируемых баз данных NoSQL, таких как Apache HBase или Apache Cassandra.

Преимущество лямбда-архитектуры заключается в ее способности обрабатывать данные как в пакетном, так и в режиме реального времени, обеспечивая

¹ Марц Н., Уоррен Д. Большие данные. Принципы и практика построения масштабируемых систем обработки данных в режиме реального времени. — М., 2016.

комплексное решение для аналитики больших данных. Однако поддержка двух отдельных путей обработки и управления ими может привести к возникновению сложностей, а обеспечение согласованности между пакетными и представлениями в режиме реального времени может оказаться непростой задачей. Некоторые альтернативные варианты, такие как каппа-архитектура, предлагают единый подход к потоковой и пакетной обработке, направленный на упрощение общего дизайна системы.

Возможен также вариант использования отдельного потокового процессора и базы данных OLAP. Например, можно задействовать Apache Pinot для обслуживания петабайтов накопленных данных с потоковыми данными, преобразованными с помощью Flink (или Pathway, если вы предпочитаете Python).

На рис. 8.5 представлена лямбда-архитектура. В левой части диаграммы изображен пакетный уровень, в правой — скоростной. Слой предоставления данных — Apache Pinot. В прошлом объединение потоковых данных с накопленными в хранилище было трудно осуществить, поскольку приходилось применять принцип «запрограммируй сам». Системы RTOLAP, такие как Pinot, упрощают решение сложной задачи по обеспечению единого представления всех данных.

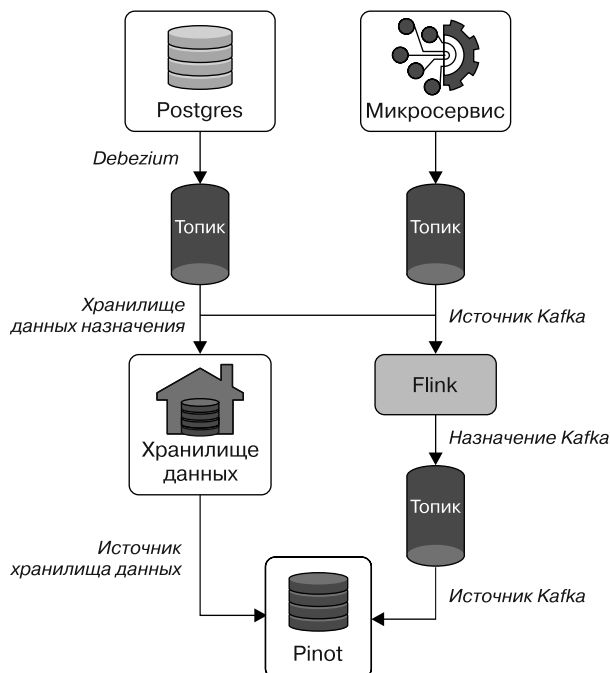


Рис. 8.5. Более сложный конвейер данных в режиме реального времени для обслуживания запросов ad hoc к потоковым и накопленным данным

Гибридные таблицы Apache Pinot

Гибридная таблица Pinot состоит из двух одноименных внутренних таблиц, одной автономной и одной в режиме реального времени. Так Pinot может объединить потоковые и накопленные данные.

Примеры 8.10 и 8.11 — это определения таблиц типа `REALTIME` и `OFFLINE` для таблицы с названием `airlineStats` соответственно.

Пример 8.10. Таблица типа `REALTIME` в Pinot

```
{
  "tableName": "airlineStats",
  "tableType": "REALTIME",
  "tenants": {},
  "segmentsConfig": {
    "timeColumnName": "DaysSinceEpoch",
    "retentionTimeUnit": "DAYS",
    "retentionTimeValue": "5",
    "replication": "1"
  },
  "tableIndexConfig": {},
  "routing": {
    "segmentPrunerTypes": [
      "time"
    ]
  },
  "ingestionConfig": { ❶
    "streamIngestionConfig": {
      "streamConfigMaps": [
        {
          "streamType": "kafka",
          "stream.kafka.topic.name": "flights-realtime",
          "stream.kafka.decoder.class.name": "org.apache.pinot.plugin.stream.
            kafka.KafkaJSONMessageDecoder",
          "stream.kafka.consumer.factory.class.name": "org.apache.pinot.plugin.
            stream.kafka20.KafkaConsumerFactory",
          "stream.kafka.consumer.prop.auto.offset.reset": "smallest",
          "stream.kafka.zk.broker.url": "localhost:2191/kafka",
          "stream.kafka.broker.list": "localhost:19092",
          "realtime.segment.flush.threshold.time": "3600000",
          "realtime.segment.flush.threshold.size": "50000"
        }
      ]
    },
    "transformConfigs": [ ❷
      {
        "columnName": "ts",
        "transformFunction": "fromEpochDays(DaysSinceEpoch)"
      }
    ]
  }
}
```

```

        "columnName": "tsRaw",
        "transformFunction": "fromEpochDays(DaysSinceEpoch)"
    }
  ],
},
"fieldConfigList": [
  {
    "name": "ts",
    "encodingType": "DICTIONARY",
    "indexTypes": [
      "TIMESTAMP"
    ],
    "timestampConfig": {
      "granularities": [
        "DAY",
        "WEEK",
        "MONTH"
      ]
    }
  }
],
"metadata": {
  "customConfigs": {}
}
}

```

- ❶ Конфигурация поглощения данных из Kafka.
- ❷ Преобразование потокового поглощения данных.

В главе 7 мы отметили, что системы RTOLAP, такие как Pinot, добавляют больше потоковой обработки с сохранением состояния и что OLAP следующего поколения будут выглядеть и работать как потоковые БД. Примером такой тенденции может служить индекс звездного дерева из Pinot (вкратце описывается в главе 3).

В примере 8.10 показаны преобразования поглощенных данных. Обычно их требуется выполнять при получении данных из системы публикации — подписки (pub-sub), такой как Kafka. Предполагается, что данные из такой системы будут доступны множеству подписчиков. Поэтому для удовлетворения их потребностей необходимо предоставлять обобщенные версии данных. Каждому потребителю придется самостоятельно выполнять любые дополнительные преобразования для своей конкретной аналитической нагрузки.

Пример 8.11. Таблица типа OFFLINE в Pinot

```

{
  "tableName": "airlineStats",
  "tableType": "OFFLINE",

```

```

"segmentsConfig": {
  "timeColumnName": "DaysSinceEpoch",
  "timeType": "DAYS",
  "segmentPushType": "APPEND",
  "segmentAssignmentStrategy": "BalanceNumSegmentAssignmentStrategy",
  "replication": "1"
},
"tenants": {},
"fieldConfigList": [
  {
    "name": "ts",
    "encodingType": "DICTIONARY",
    "indexTypes": [
      "TIMESTAMP"
    ],
    "timestampConfig": {
      "granularities": [
        "DAY",
        "WEEK",
        "MONTH"
      ]
    }
  },
  {
    "name": "ArrTimeBlk",
    "encodingType": "DICTIONARY",
    "indexes": {
      "inverted": {
        "enabled": "true"
      }
    },
    "tierOverwrites": {
      "hotTier": {
        "encodingType": "DICTIONARY",
        "indexes": {
          "bloom": {
            "enabled": "true"
          }
        }
      },
      "coldTier": {
        "encodingType": "RAW",
        "indexes": {
          "text": {
            "enabled": "true"
          }
        }
      }
    }
  }
]

```

```

"tableIndexConfig": {
  "starTreeIndexConfigs": [ ❶
    {
      "dimensionsSplitOrder": [
        "AirlineID",
        "Origin",
        "Dest"
      ],
      "skipStarNodeCreationForDimensions": [],
      "functionColumnPairs": [
        "COUNT__*",
        "MAX__ArrDelay"
      ],
      "maxLeafRecords": 10
    }
  ],
  "enableDynamicStarTreeCreation": true,
  "loadMode": "MMAP",
  "tierOverwrites": {
    "hotTier": {
      "starTreeIndexConfigs": [
        {
          "dimensionsSplitOrder": [
            "Carrier",
            "CancellationCode",
            "Origin",
            "Dest"
          ],
          "skipStarNodeCreationForDimensions": [],
          "functionColumnPairs": [
            "MAX__CarrierDelay",
            "AVG__CarrierDelay"
          ],
          "maxLeafRecords": 10
        }
      ]
    },
    "coldTier": {
      "starTreeIndexConfigs": []
    }
  },
  "metadata": {
    "customConfigs": {}
  },
  "ingestionConfig": {
    "transformConfigs": [

```

```

    {
      "columnName": "ts",
      "transformFunction": "fromEpochDays(DaysSinceEpoch)"
    },
    {
      "columnName": "tsRaw",
      "transformFunction": "fromEpochDays(DaysSinceEpoch)"
    }
  ]
},
"tierConfigs": [ ❷
  {
    "name": "hotTier",
    "segmentSelectorType": "time",
    "segmentAge": "3130d",
    "storageType": "pinot_server",
    "serverTag": "DefaultTenant_OFFLINE"
  },
  {
    "name": "coldTier",
    "segmentSelectorType": "time",
    "segmentAge": "3140d",
    "storageType": "pinot_server",
    "serverTag": "DefaultTenant_OFFLINE"
  }
]
}

```

❶ Индекс звездного дерева предварительно агрегирует накопленные данные для построения материализованного представления. Индекс звездного дерева может использоваться в таблицах Pinot в режиме реального времени и автономном режиме.

❷ У Pinot многоуровневое хранилище. Оно позволяет перемещать данные на более низкие уровни по мере их устаревания, чтобы освободить память (рис. 8.6).

В отличие от определений таблиц типа **REALTIME** с конфигурациями для чтения из топика потоковой платформы, таблицы **OFFLINE** не содержат конфигурации поглощения данных. Чтобы был возможен запрос потоковых данных в течение миллисекунд после публикации, таблицы типа **REALTIME** распределены по нескольким серверам, хранящим потоковые данные в энергозависимой памяти. Таблицы **REALTIME** и **OFFLINE** разделены и объединяются только при использовании механизма обработки запросов.

У таблиц типа **REALTIME** есть срок хранения. По его истечении данные серверов **REALTIME** выгружаются на серверы **OFFLINE**. Это позволяет сохранять емкость серверов **REALTIME**.

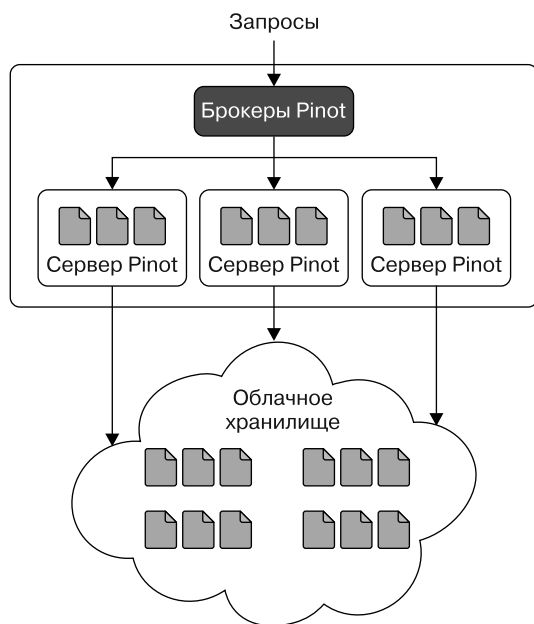


Рис. 8.6. Многоярусное хранилище Pinot

Преимущество использования БД OLAP заключается в том, что нет необходимости уменьшать аналитические данные, чтобы они поместились в объем БД OLTP или HTAP. Если уменьшить их объем, это понизит гибкость в запросах для пользователей, которым необходим доступ ко всем накопленным данным. Однако гибкость, как и остальные характеристики, сопровождается повышенной сложностью и расходами на инфраструктуру.

Благодаря многоуровневому хранению Pinot можно выгрузить старые данные на более низкие уровни, чтобы освободить больше места в таблице Pinot. Если требуется больше накопленных данных для выполнения запросов *ad hoc*, то лучше всего использовать OLAP-системы. Они могут объединять накопленные данные с поступающими в режиме реального времени.

Конфигурации конвейеров

Pinot предлагает решение для объединения накопленных данных и данных реального времени. Прежде чем добраться до Pinot, поток данных в лямбда-архитектуре получает данные из БД OLTP, например Postgres, с помощью коннектора Debezium CDC. А Debezium захватывает транзакции изменений из многих типов транзакционных баз данных. В примере 8.12 показана конфигурация Debezium Postgres.

Пример 8.12. Создание source-коннектора Kafka во Flink

```
{
  "name": "postgres",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "tasks.max": "1",
    "database.hostname": "0.0.0.0",
    "database.port": "5432",
    "database.user": "postgres",
    "database.password": "postgres",
    "database.dbname": "postgres",
    "topic.prefix": "dbserver1",
    "schema.include.list": "inventory"
  }
}
```

Этот коннектор записывает данные об изменениях в топик. Можно использовать Flink SQL в качестве потокового процессора для преобразования данных. В примере 8.13 показано, как Flink SQL получает потоковые данные из Kafka.

Пример 8.13. Создание source-коннектора Kafka во Flink

```
CREATE TABLE KafkaSource (
  `id` BIGINT,
  `col1` STRING,
  `col2` STRING,
  `ts` TIMESTAMP(3) METADATA FROM 'timestamp'
) WITH (
  'connector' = 'kafka',
  'topic' = 'my_data',
  'properties.bootstrap.servers' = 'localhost:9092',
  'properties.group.id' = 'abc',
  'scan.startup.mode' = 'earliest-offset',
  'format' = 'json'
)
```

После получения данных из топика можно выполнить преобразования с помощью SQL. Во Flink это называется конвейером. После завершения преобразования данные можно записать обратно в топик с помощью кода из примера 8.14.

Пример 8.14. Создание таблицы назначения Kafka во Flink SQL

```
CREATE TABLE KafkaSink (
  `user_id` BIGINT,
  `col1` STRING,
  `col2` STRING,
  `ts` TIMESTAMP(3) METADATA FROM 'timestamp'
) WITH (
  'connector' = 'kafka',
  'topic' = 'my_data_transformed',
```

```
'properties.bootstrap.servers' = 'localhost:9092',  
'properties.group.id' = 'testGroup',  
'scan.startup.mode' = 'earliest-offset',  
'format' = 'json'  
)
```

Pinot позволяет выполнять ad hoc-запросы к данным из топика в режиме реального времени, а также к накопленным данным из хранилища данных или объектного хранилища, например Amazon S3. Важно знать, что сценарии с выполнением запросов ad hoc к накопленным данным и данным реального времени не предназначены для пользовательских приложений. Запросы ad hoc используются аналитиками данных, которые работают в компании и получают информацию из данных. Такие функциональные возможности нельзя предоставлять многочисленным сторонним пользователям — они будут слишком сильно нагружать OLAP-систему.

Резюме

К недостаткам подхода zero-ETL можно отнести его неспособность обеспечить аналитику в режиме реального времени, тенденцию навязывать семантику пакетной обработки для push-запросов, а также отсутствие материализованных представлений. Компенсировать эти недостатки можно с помощью представленных альтернативных вариантов с поддержкой подхода near-zero-ETL. Они также позволяют выполнять аналитику в режиме реального времени. Например, можно применить систему Proton для сокращения накопленных данных и данных в режиме реального времени и передать их в материализованное представление. После этого представление можно записать в топик Kafka. Для создания локальной копии во встроенной базе данных OLAP нам потребовалось создать потребителя Kafka на операционном уровне. Этот же подход можно реализовать с использованием БД HTPA вместо встроенных OLAP.

Паттерны из этой главы сильно зависят от потоковой плоскости диаграммы Венна. В следующей главе мы более подробно рассмотрим потоковую плоскость и то, как ее можно использовать в таких архитектурных паттернах, как сетка данных (data mesh).

Плоскость потоковой обработки

В предыдущей главе мы рассмотрели существующие в современной экосистеме системы обработки в режиме реального времени и представили три плоскости данных: операционную, аналитическую и потоковую. Операционная и аналитическая плоскости работают преимущественно с данными в состоянии покоя, делая акцент на статической информации. А потоковая плоскость выделяется как единственная с данными в движении.

В этой главе мы рассмотрим потоковую плоскость и то, как архитекторы и инженеры данных могут начать размышлять о ее роли в упрощении аналитики в режиме реального времени.

Мы постоянно связываем термины «асинхронный», «данные в движении» и «потоковая передача», рассматривая их как взаимозаменяемые выражения. Они представляют собой синонимы длительных процессов, выполняющих непрерывные преобразования для упрощения и ускорения получения аналитических данных в режиме реального времени для приложений.

На рис. 9.1 показана только часть диаграммы Венна, и мы будем ссылаться на нее на протяжении всей главы.

В этой главе мы рассмотрим несколько ключевых тем, помогающих понять тонкости обработки данных в режиме реального времени на потоковой плоскости. В центре внимания находится концепция *аналитики в операционной плоскости*, или *операционной аналитики*. Она рассматривает практику выполнения аналитических нагрузок рядом с приложениями в операционной плоскости. Кроме того, мы рассмотрим аспекты *локальности данных* и выясним, как географическая близость данных к среде их обработки влияет на общую *производительность системы*. Очень важно изучить эту концепцию для оптимизации использования ресурсов и минимизации задержек в сценариях потоковой аналитики.



Рис. 9.1. Потоковая плоскость диаграммы Венна (см. главу 7)

Еще одна важная тема этой главы — *репликация данных*. Это основополагающая стратегия в потоковой плоскости. Она предполагает дублирование данных на нескольких узлах или в глобальных регионах, что способствует *децентрализации*. Мы рассмотрим концепцию *продуктов данных*, вдохновленную сеткой данных. Обсудим, как можно использовать аналитику в режиме реального времени для создания ценных информационных продуктов, которые можно тиражировать по всему миру для потребления в разных местах.

Изучив эти важные темы, читатели поймут многогранные проблемы и возможности потоковой плоскости и предложат решения для создания устойчивых и эффективных систем в рамках глобального ландшафта потоковой аналитики.

Гравитация данных

Представьте себе данные в виде планеты. По мере накопления массы данные притягивают сервисы и приложения так же, как гравитация притягивает Луну к Земле и не дает вам улететь в космос. Например, вы создаете данные, когда публикуете сообщение в социальных сетях. Ваше сообщение было отправлено в сервис, и он привлекает к взаимодействию ваших друзей и создает еще больше данных.

В типичной архитектуре данных без учета потоковой плоскости операционная плоскость будет напрямую передавать данные в аналитическую. Представьте себе операционную плоскость как луны на орбите вокруг плотной планеты — аналитической плоскости. Луны операционной плоскости передают данные на планету. По мере того как все больше лун операционной плоскости будут отправлять данные на планету аналитической плоскости, планета превратится в монолитную систему накопленных данных. На рабочие нагрузки начинает влиять размер данных, что приводит к возникновению задержек.

Ранее мы говорили, что движение данных из операционной плоскости в аналитическую представляет собой однонаправленный нисходящий поток. Заставить данные перемещаться в обратном направлении сложно. Для рассмотрения этого процесса с другой точки зрения подойдет идея гравитации данных.

На рис. 9.2 показана типичная архитектура данных без учета потоковой плоскости. Операционная плоскость представлена в виде внешних узлов, публикующих данные в аналитическую плоскость, представленную в виде Земли.

По мере того как все больше систем операционной обработки передают свои данные в аналитическую плоскость, она превращается в монолитную систему

накопленных данных. Рабочие нагрузки на аналитическом уровне начинают ощущать влияние размера данных, что увеличивает задержку аналитики — система уже не может обеспечить обработку в режиме реального времени.

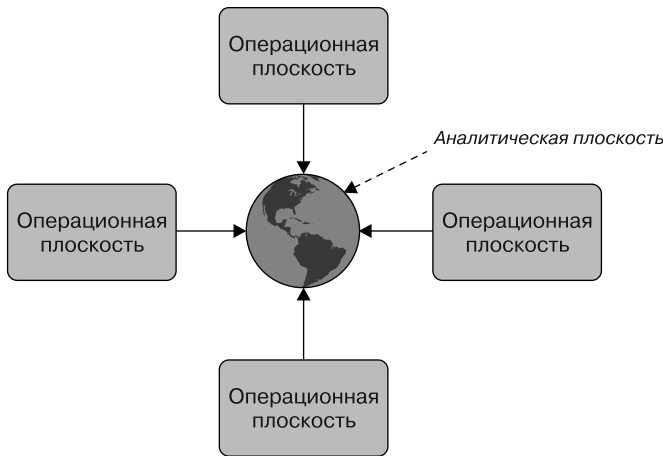


Рис. 9.2. Влияние гравитации данных на инфраструктуру и данные

Совсем другое дело, если архитектура данных включает в себя потоковую плоскость. Луны операционной плоскости вместо того, чтобы передавать данные непосредственно на планету, передают их на спутники внутри потоковой плоскости. Эти спутники позволяют проводить аналитику ближе к лунам операционной плоскости, одновременно доставляя данные на планету. Так гасится воздействие силы гравитации. Потоковая плоскость обеспечивает текучесть, необходимую для возврата данных в операционную плоскость для пользовательской аналитики.

На рис. 9.3 данные в материализованных представлениях потоковой плоскости представлены в виде спутников на орбите планеты (или вокруг аналитической плоскости). Из этих материализованных представлений можно принимать данные в режиме реального времени, как прямые телевизионные программы со спутника. Подобно просмотру телевизионных программ в прямом эфире, данные в потоковой плоскости могут предоставляться глобально.

Потоковая плоскость обеспечивает средства для реализации аналитики ближе к операционной плоскости, при этом продукты данных, составляющие эту аналитику, доставляются в глобальном масштабе. Она снижает эффект гравитации данных, обеспечивая аналитическую плоскость инкрементными данными для архивирования и масштабной пакетной обработки.

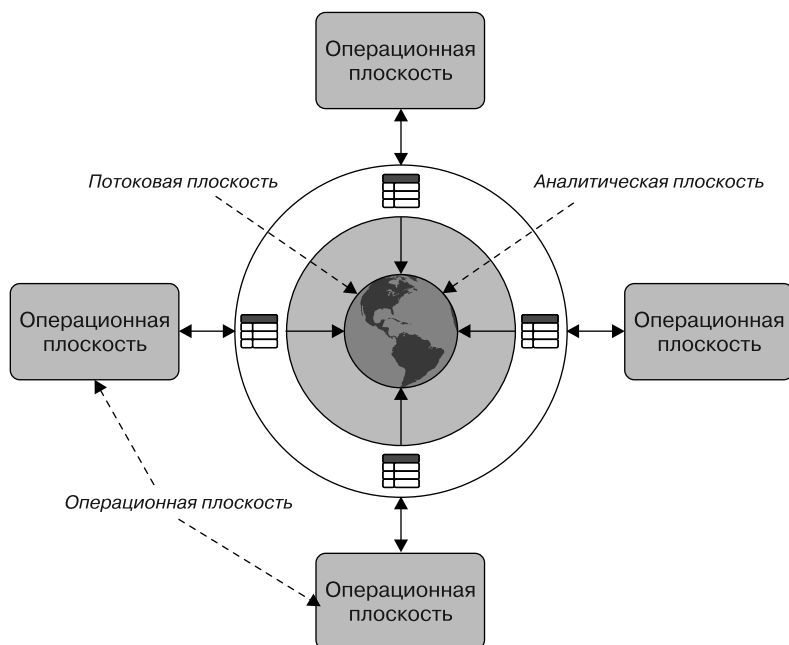


Рис. 9.3. Материализованные представления вращаются вокруг земного шара с помощью репликации

Компоненты потоковой плоскости

На рис. 9.4 показаны многочисленные актуальные для потоковой плоскости решения и системы. На протяжении всей главы мы будем использовать эту диаграмму для описания систем потоковой плоскости и того, как они способствуют децентрализации данных и выполнению аналитики в режиме реального времени.

На противоположных сторонах рис. 9.4 размещены уже знакомые операционная и аналитическая плоскости. Они разделены потоковой плоскостью в виде облака. Та базируется на двух компонентах: потоковой платформе (например, Kafka) и source-/sink-коннекторах. Они обозначены двунаправленной стрелкой. Это основа, из которой другие компоненты потоковой плоскости потребляют данные в режиме реального времени.

Потоковые процессоры, БД RTOLAP и потоковые БД считывают данные из Kafka для преобразования и предоставления аналитических данных в режиме реального времени. Потоковые процессоры и потоковые БД также могут записывать аналитические данные обратно в Kafka, что позволяет им создавать копии этих данных в других плоскостях.

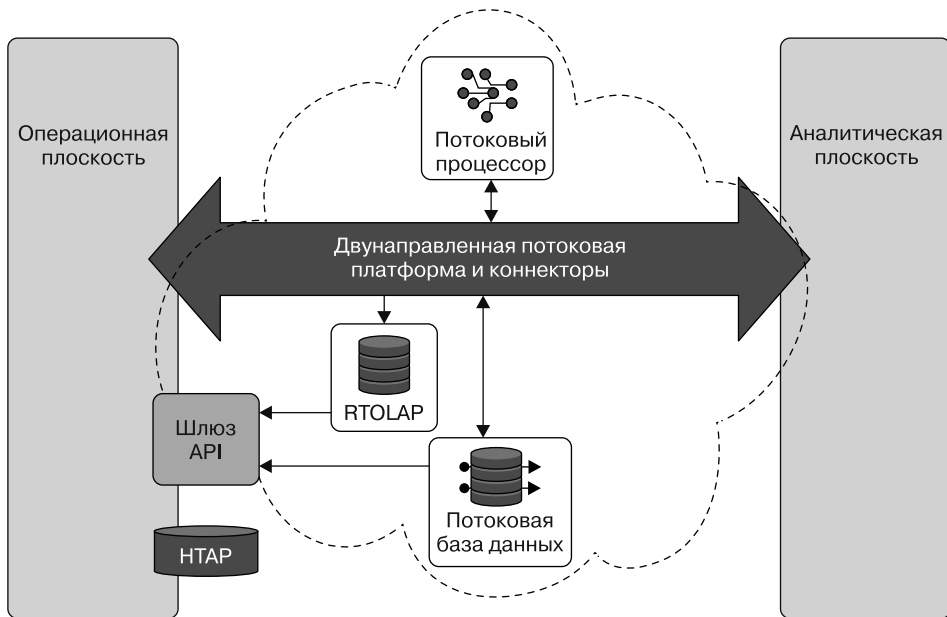


Рис. 9.4. Плоскость потоковой обработки

Потоковая плоскость делится на согласованную и согласованную в конечном счете потоковую передачу и системы реального времени. Системы в этой плоскости мигрируют в соответствующие позиции согласованности и выполняют рабочие нагрузки в своих плоскостях потребления. Слева от потоковой плоскости находится операционная аналитика, справа — внутренняя аналитика. Последняя допускает применение согласованности в конечном счете и имеет доступ ко всем накопленным данным.

Подводя итог, можно сказать, что к компонентам потоковой плоскости относятся:

- потоковые платформы, такие как Kafka;
- коннекторы source и sink;
- потоковые процессоры, такие как Flink;
- базы данных RTOLAP;
- потоковые базы данных.

При проектировании инфраструктуры данных для аналитики в режиме реального времени архитекторы должны учитывать согласованность и тех, кто потребляет аналитику.

Инфраструктура потоковой плоскости

При построении инфраструктуры операционной, аналитической и потоковой плоскостей архитекторы должны учитывать необходимость создания отдельной инфраструктуры для каждой из них. Наличие выделенной инфраструктуры для операционной и аналитической плоскостей всегда было стандартной практикой. Поскольку концепция потоковой плоскости была впервые представлена в этой книге, для архитекторов может оказаться неочевидным выделение для нее инфраструктуры. Они могут подумать о развертывании потоковых систем в существующих плоскостях, тем более что эти системы обычно не хранят данные в течение длительного времени.

Существует множество причин для выделения отдельной инфраструктуры для различных систем. В табл. 9.1 приведена лишь часть из них.

Таблица 9.1. Причины выделения инфраструктуры для систем данных

Причина	Описание
Масштабируемость	По мере роста объемов данных архитекторам необходимо разрабатывать инфраструктуру, способную масштабироваться для обработки возросших объемов данных без ущерба для производительности. Для этого необходимо выбрать соответствующее оборудование, базы данных и другие компоненты с возможностью беспрепятственного масштабирования по горизонтали или вертикали
Производительность	Выделенная инфраструктура позволяет архитекторам оптимизировать производительность для конкретных задач, связанных с данными. При этом учитываются скорость обработки данных, производительность запросов и общая отзывчивость системы
Надежность и доступность	Архитекторы разрабатывают инфраструктуру, чтобы обеспечить надежность и доступность систем данных по мере необходимости. К этим решениям относятся резервирование, механизмы обхода отказа и планы аварийного восстановления, призванные свести к минимуму время простоя и потерю данных
Безопасность	Защита конфиденциальных данных — один из главных приоритетов. Выделенная инфраструктура позволяет архитекторам применять надежные меры безопасности, включая контроль доступа, шифрование и регулярный аудит для защиты данных от несанкционированного доступа и киберугроз
Интеграция	Во многих организациях системы данных должны интегрироваться с различными приложениями, сервисами и платформами. Выделенная инфраструктура способствует интеграции без границ, обеспечивая эффективный обмен данными и их использование в организации

Причина	Описание
Соответствие нормативным требованиям	В зависимости от отрасли могут существовать нормативные требования к хранению и обработке данных. Архитекторы должны проектировать инфраструктуру в соответствии с этими нормами, чтобы избежать юридических проблем и штрафов
Экономическая эффективность	Выделение определенных ресурсов инфраструктуры для систем данных позволяет архитекторам оптимизировать расходы. Они могут выбрать наиболее экономичные решения для хранения данных, вычислительные устройства и сетевые компоненты, отвечающие конкретным потребностям систем обработки данных
Руководство данными	Инфраструктура играет решающую роль в обеспечении соблюдения политики руководства данными. Архитекторы могут разрабатывать системы с отслеживанием и обеспечением качества, целостности и согласованности данных, гарантируя их надежность и точность

Выделение инфраструктуры связано с эффективным управлением данными, их хранением, обработкой и доступностью. Поточковая плоскость не хранит данные в течение длительного времени, а обрабатывает и перемещает их при увеличении масштаба. Выделять инфраструктуру для потоковых систем необходимо, чтобы обеспечить эффективное, безопасное и надежное управление данными в движении с поддержкой различных функций и стратегических целей компании.

Любые данные в потоковой плоскости эфемерны — они временные или недолговечные. Текучесть потоковой плоскости позволяет сохранять свежесть данных и ограничивает эффект гравитации данных. Ее вычислительные возможности обеспечивают аналитическую нагрузку на операционную плоскость.

Операционная аналитика

Операционная аналитика подразумевает сбор, обработку и анализ данных в непосредственной близости от источника их формирования, как правило, в пользовательском приложении или рядом с ним, а не только в системах аналитической плоскости. Операционная аналитика подразумевает выполнение аналитики рядом с приложениями или микросервисами, генерирующими данные. Для этого применяется способность потоковой плоскости запускать асинхронные процессы в масштабе.

У вас может возникнуть вопрос: «Зачем выполнять аналитические рабочие нагрузки на операционной плоскости?» Вопрос вполне резонен, тем более что существует множество причин для разделения этих рабочих нагрузок (о них мы рассказывали в главе 3).

Приведем некоторые из причин для переноса аналитических рабочих нагрузок в операционную плоскость.

Принятие решений в режиме реального времени

Операционная аналитика позволяет организациям получать информацию из данных в режиме реального времени. Это очень важно для принятия быстрых и обоснованных решений, влияющих на текущую деятельность.

Повышенная эффективность

Внедряя аналитические возможности в системы операционной обработки, организации могут оптимизировать процессы и сократить необходимость непосредственного вмешательства специалистов. Автоматизированная аналитика в рамках операционных рабочих процессов позволяет повысить эффективность и сократить время обработки данных.

Улучшение клиентского опыта или персонализация

Аналитика в режиме реального времени позволяет организациям персонализировать взаимодействие с клиентами на основе их текущего поведения и предпочтений. Это может значительно повысить общий уровень клиентского опыта.

Проактивное решение проблем и предиктивная аналитика

Операционная аналитика часто включает в себя предиктивное моделирование, помогая организациям выявлять потенциальные проблемы до их возникновения. Такой проактивный подход дает возможность своевременно вмешаться и решить проблему.

Экономия средств и оптимизация ресурсов

Интегрируя аналитику в системы операционной обработки, организации могут оптимизировать распределение человеческих ресурсов, запасов товара или оборудования. Это поможет экономить средства и повышать эффективность использования ресурсов.

В целом перенос аналитических нагрузок в операционную плоскость вызван необходимостью принятия решений в режиме реального времени, требованием к обеспечению гибкости и стремлением извлекать полезные сведения непосредственно в контексте текущих бизнес-операций. Такая интеграция позволяет организациям приближать свой подход к ориентированному на данные варианту и оперативно реагировать на быстро меняющиеся условия в операционной среде.

Инфраструктура операционной плоскости недостаточно емкая для хранения всех накопленных данных из инфраструктуры аналитической плоскости. У аналитических рабочих нагрузок в операционной плоскости будет ограниченный

доступ к накопленным данным. Системы операционной обработки также ограничены в масштабах, поэтому, даже если у них будет доступ ко всем накопленным данным, они смогут использовать лишь небольшую их часть.

Получить накопленные данные из аналитической плоскости для использования системами в операционной плоскости для аналитических рабочих нагрузок может быть непросто. Эффект гравитации данных очень значителен и распространяется не только на данные. Он относится и к приложениям обработки данных. Из-за этого создание решения для операционной аналитики может оказаться запутанным. О возможностях получения накопленных данных мы расскажем в главе 10.

Несмотря на эти ограничения, приближение аналитики к приложению и источнику данных делает ее полезным подходом для развивающейся аналитической среды реального времени. Вспомним пример с воображаемым банком из главы 6 — потоковые процессоры или БД, применяемые в операционной аналитике, должны быть согласованными. Вероятность асинхронной обработки потоков транзакций, потребляемых приложением в режиме реального времени, чрезвычайно высока. Последствия несогласованности могут оказаться серьезными и привести к утрате доверия к приложению, как показано в главе 6. Фрэнк Макшерри (ученый, работающий в области информатики, и главный научный сотрудник компании Materialize.io) рассказывает о трех характеристиках, формирующих доверие к потоковым данным¹.

Отзывчивость

Подразумевает синхронный интерактивный доступ к аналитическим данным, измеряется задержкой запросов, QPS и конкурентностью (количество конечных пользователей).

Свежесть данных

Подразумевает то, насколько близки к реальному времени результаты аналитики. Свежесть данных — это показатель их ценности с течением времени. На рис. 9.5 масштаб оси X (время) соотносится со сценарием использования. Он может измеряться в часах, минутах, секундах или миллисекундах.

Согласованность

О согласованности мы подробно говорили в главе 6.

На рисунке поле реального времени содержит данные, которые не потеряли своей ценности из-за времени. Масштаб оси X (время) определяет типы используемых

¹ McSherry F. A Guided Tour Through Materialize's Product Principles // Materialize, September 22, 2023 (<https://oreil.ly/jqfL1>).

систем. Если шкала измеряется в календарных или рабочих днях, достаточно применить пакетную обработку. Если масштаб не превышает часа, необходимо задействовать потоковые системы, развернутые на инфраструктуре потоковой плоскости. Если шкала находится в пределах 8 часов, потоковые системы подготовят вас к более агрессивным срокам в будущем.

Потоковая плоскость предоставляет операционной все необходимое для аналитической обработки в режиме реального времени с помощью потоковых баз данных и потоковых процессоров. Она предоставляет аналитику в режиме реального времени в виде табличных структур для приложений и их пользователей. Более того, общая перспектива потоковой плоскости из сетки соединений и таблиц напоминает потоковую БД. Идеи реализации операционной аналитики напоминают архитектуру сетки данных. Она способствует децентрализации данных и обращает вспять эффект гравитации данных.

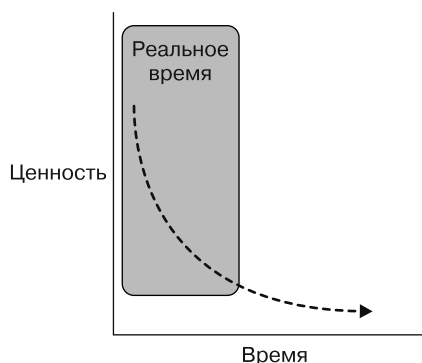


Рис. 9.5. Определение реального времени основано на том, насколько быстро или медленно деградирует ценность данных с течением времени

Сетка данных

Сетка данных (data mesh) — это концептуальная схема архитектуры данных, представленная Жамаком Дехгани в 2019 году. Сетка данных отходит от традиционных централизованных моделей и выступает за децентрализованный подход, в котором данные рассматриваются как продукт. При этом право собственности распределяется между различными доменами или подразделениями организации. Каждый домен отвечает за получение данных с сервера самостоятельно. Команды, работающие с данными, функционируют как команды, работающие с продуктами. Они управляют полным жизненным циклом обрабатываемых данных, включая их качество, документацию и доступность. Инфраструктура создана для самообслуживания и дает возможность командам доменов самостоятельно получать доступ к своим данным и управлять ими. *Федеративное руководство вычислениями* обеспечивает соблюдение общих стандартов и политик, позволяя при этом осуществлять локальный контроль в каждом домене. Этот подход направлен на решение задач масштабируемости в крупных организациях, повышение гибкости и улучшение общего качества данных.

Столпы сетки данных

Сеть данных включает в себя четыре ключевых принципа, или столпа. Они служат основой для разработки и внедрения децентрализованной масштабируемой архитектуры данных.

Децентрализованное владение данными, ориентированное на домен

В этом компоненте особое внимание уделяется распределению прав собственности на данные между различными доменами или подразделениями организации. Вместо формирования централизованной команды для работы с данными отдельные домены берут на себя ответственность за создание и использование данных в своем конкретном контексте. Такой подход приводит управление данными в соответствие с организационной структурой, способствуя обеспечению автономности и подотчетности.

Данные как продукт

Второй компонент рассматривает данные как продукт с собственным жизненным циклом, стандартами качества и документацией. Команды для работы с данными функционируют как команды для работы с продуктами, отвечая за полное управление производимыми данными. Сюда входят обеспечение качества данных, предоставление надлежащей документации и обеспечение легкого доступа к данным внутри организации. Отношение к данным как к продукту способствует изменению мышления в направлении предоставления высококачественных, пригодных для использования данных.

Инфраструктура данных с самообслуживанием

Третий компонент направлен на создание инфраструктуры данных с возможностью самообслуживания. Она позволяет командам домена автономно получать доступ к собственным данным и управлять ими. Реализация такой инфраструктуры предполагает предоставление инструментов, платформ и услуг для обработки данных. При их использовании команды доменов не будут прибегать к помощи централизованных команд для работы с данными. Инфраструктура самообслуживания обеспечивает гибкость и сокращает количество узких мест, позволяя командам более оперативно реагировать на конкретные потребности в данных.

Федеративное руководство вычислениями

В четвертом компоненте рассматривается применение федеративного руководства вычислениями. Эта концепция работы предполагает установление с помощью федеративных советов общих стандартов и политик при сохранении местной автономии в каждом домене. Таким образом обеспечивается баланс между соблюдением общеорганизационных стандартов и обеспечением

отдельным доменам гибкости при управлении данными в соответствии с их специфическими требованиями. Федеративное руководство вычислениями помогает поддерживать согласованность и соответствие нормативным требованиям в организации.

Эти четыре столпа в совокупности формируют основу структуры сетки данных, давая организациям руководящие принципы для создания масштабируемых и гибких архитектур данных.

Сеть данных децентрализует владение данными и рассматривает их как продукт, что показывает стремление преодолеть ограничения централизованных моделей, способствуя повышению автономности, эффективности и улучшению руководства. Такой подход признает динамичный и развивающийся характер управления данными. Это говорит о необходимости для организаций адаптироваться к меняющимся требованиям и использовать данные в качестве стратегического актива.

Сеть данных фокусируется в первую очередь на децентрализации данных, в том числе аналитических. Аналогично операционная аналитика направлена на перенос аналитических рабочих нагрузок в операционную плоскость. Это влечет за собой унаследованный побочный эффект децентрализации данных и аналитических нагрузок.

Приближение аналитических рабочих нагрузок к операционной плоскости соответствует концепциям сетки данных, способствуя децентрализованному владению данными и повышая автономность команд доменов. Основная идея такого перемещения заключается в том, чтобы предоставить операционным командам прямой доступ к аналитике, что позволит им получать информацию и принимать решения на основе данных в режиме реального времени.

Это перемещение происходит в контексте сетки данных следующим образом.

Децентрализованное владение данными, ориентированное на домен

В сетке данных владение данными распределяется между различными доменами или подразделениями организации. Приближение аналитических рабочих нагрузок к операционной плоскости расширяет этот принцип, позволяя операционным командам анализировать данные, генерируемые в их домене, и владеть ими. Такая децентрализация способствует более гибкому и контекстно-зависимому подходу к аналитике, поскольку операционные команды глубже понимают свои данные.

Данные как продукт

Отношение к данным как к продукту подразумевает, что их не только собирают и хранят, но также анализируют и потребляют в рамках общего жизненного цикла продукта данных. Приближение аналитических рабочих

нагрузок к операционной плоскости гарантирует, что полученные в результате аналитики знания будут легко интегрированы в операционные процессы. Операционные команды становятся ответственными за весь жизненный цикл своих данных, включая их анализ, интерпретацию и применение для достижения бизнес-результатов.

Инфраструктура данных с самообслуживанием

Для выполнения аналитических задач часто требуются специализированные инструменты и платформы. Благодаря приближению этих рабочих нагрузок к операционной плоскости операционные команды получают большую самостоятельность при доступе к аналитическим инструментам и их использовании. Это снижает зависимость от централизованных команд для работы с данными и позволяет операционным командам немедленно выполнять операции анализа, генерировать идеи и совершенствовать аналитические процессы без значительной внешней поддержки.

Федеративное руководство вычислениями

Аналитические рабочие нагрузки на операционном уровне должны соответствовать общим стандартам и политикам руководства, обеспечивая при этом локальную автономию. Федеративное руководство вычислениями необходимо для обеспечения согласованности, соответствия нормативным требованиям и безопасности. Этот принцип гарантирует, что, хотя операционные команды обладают автономией в своей аналитической работе, существуют общие стандарты, поддерживающие целостность и надежность данных организации.

В итоге можно сказать, что согласование аналитических нагрузок с операционной плоскостью в контексте сетки данных поддерживает основные принципы децентрализации, автономии и отношения к данным как к продукту. Оно направлено на улучшение реакции аналитики на операционные потребности. Это способствует развитию культуры принятия решений на основе данных в различных доменах организации.

Трудности применения сетки данных

Внедрение сетки данных сопряжено с трудностями, вызванными необходимостью значительно изменять культуру производства, повышенной технической сложностью, организационной разобщенностью и нехваткой навыков. Культурный сдвиг предполагает переход от централизованного управления к децентрализованной модели. В ней особое внимание уделяется владению данными и сотрудничеству с учетом доменов организации. При создании инфраструктуры самообслуживания данных в соответствии с принципами сетки данных возникают технические проблемы, часто вызванные интеграцией

с существующими системами. Очень важно преодолеть организационную разобщенность и наладить межфункциональное сотрудничество, а также устранить пробелы в навыках и обеспечить ориентацию команд на продукт. Качество данных и руководство ими создают проблемы обеспечения баланса между локальной автономией и всеобщими стандартами. Решающее значение имеют эффективная коммуникация и координация. А постепенное внедрение практики сети данных при поддержке существующей инфраструктуры требует тщательного планирования.

Несмотря на многообещающие перспективы, внедрение сетки данных может столкнуться с определенными препятствиями. Они возникают по причине высоких требований к уровню знаний, необходимости применения современных инструментов и инфраструктуры для самостоятельного обслуживания.

*Роланд Меертенс и др. Статья InfoQ AI, ML and Data Engineering Trends Report. Сентябрь 2023 года
(<https://oreil.ly/MyvA2>)*

Потенциальные преимущества в виде повышения гибкости и улучшения качества данных подталкивают организации к успешному внедрению сетки данных, невзирая на перечисленные трудности.

Упрощенный и привычный подход к сетке данных может облегчить ее принятие и упростить внедрение. С этим помогут потоковые базы данных. Они обеспечивают привычный подход к данным, позволяя повысить их доступность и ускорить внедрение архитектуры сетки данных. Потоковые БД дают возможность использовать продукты данных в различных областях по всему миру, ускоряют итеративную разработку и быстро совершенствуют решения в области данных.

Потоковая сетка данных с потоковой плоскостью и потоковыми БД

Потоковая сетка данных реализует все основные принципы сетки данных, но при этом задействует потоки в режиме реального времени. Она позволяет выполнять анализ в режиме реального времени для всех потребителей домена. Команды доменов при использовании потоковых БД могут создавать продукты потоковых данных, причем от них не требуется глубоко понимать концепции потоковой обработки — вместо этого они могут применять свои знания о базах данных и для потоковых БД¹.

¹ Более подробную информацию можно найти в книге *Streaming Data Mesh* (O'Reilly) и статье *Kafka and RisingWave* (<https://oreil.ly/8LFpA>).

В главе 7 мы определили потоковую БД с точки зрения базы данных как БД, способную потреблять и передавать потоки, а также асинхронно выполнять материализованные представления.

Потоковые БД обладают уникальной способностью выдавать данные в виде потоков. Их, в свою очередь, могут потреблять другие потоковые БД. Теперь есть возможность создать сеть соединений между потоковыми БД для обмена данными. И напомним, что потоковую плоскость можно рассматривать как потоковую БД, охватывающую все домены предприятия.

На рис. 9.6 показаны цель потоковой сетки данных и то, как она реализуется с помощью потоковой плоскости. Потоковые БД строят материализованные представления на основе реплицированных данных из других доменов. Они также передают свои продукты данных для различных доменов.

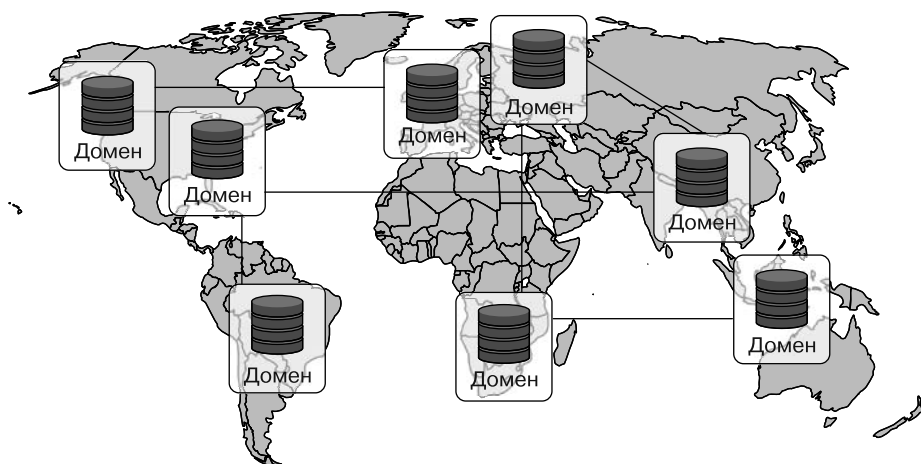


Рис. 9.6. Потоковая плоскость использует потоковые БД для потребления и производства продуктов данных, а также применяет коннекторы и потоковые платформы для репликации продуктов данных и аналитики

Локальность данных

Потребление данных локально в сетке данных влияет как на производительность, так и на безопасность. С точки зрения производительности локальное потребление обеспечивает более эффективный и быстрый доступ к данным. Поскольку при необходимости данные реплицируются из удаленных доменов и хранятся в локальном, команды могут минимизировать задержки и оптимизировать обработку данных для конкретных сценариев использования. Таким образом можно повысить производительность аналитических рабочих нагрузок,

обработки данных в режиме реального времени и других операций на основе данных. Это связано с тем, что команды будут адаптировать свою инфраструктуру данных в соответствии с локальными требованиями к производительности. Кроме того, локальное потребление облегчает масштабирование обработки данных, поскольку команды доменов могут самостоятельно масштабировать свою инфраструктуру в зависимости от конкретных потребностей, не затрагивая другие домены.

С точки зрения безопасности локальное потребление данных соответствует принципу федеративного руководства вычислениями в сетке данных. Меры безопасности можно реализовать на уровне домена, что позволяет каждому домену применять собственные политики безопасности и средства контроля доступа. Кроме того, команды доменов могут сосредоточиться на защите своих информационных активов, не ставя под угрозу безопасность всей организации. При реализации мер безопасности на уровне домена и поддержке общих глобальных стандартов сетка данных улучшает баланс между локальной автономией и всеобщими требованиями безопасности организации.

Системы репликации становятся одним из краеугольных камней сетки данных для получения данных в целях локального использования в домене.

Репликация данных

Репликация играет ключевую роль в реализации потоковой сетки данных, обеспечивая доставку данных в точки назначения для локального потребления. Данные в потоковой сетке распределены между различными доменами, и механизмы репликации в режиме реального времени имеют решающее значение для поддержания согласованности и надежности данных.

Кроме того, репликация поддерживает требования к масштабируемости потоковой сетки данных. Поскольку объемы данных и требования к их обработке в разных доменах различны, репликация позволяет эффективно распределять рабочие нагрузки обработки данных. Благодаря репликации соответствующих потоков данных в необходимые домены организации могут оптимизировать использование ресурсов и минимизировать задержки. Это позволяет каждому домену самостоятельно масштабировать свою инфраструктуру в зависимости от конкретных операционных потребностей. Такая масштабируемость очень важна для учета колебаний скорости передачи данных и обеспечения быстрой и эффективной адаптации потоковой сетки данных к изменяющимся требованиям бизнеса. Репликация в потоковой плоскости выполняется путем создания сетки связанных потоковых платформ. Например, в Kafka есть механизм Mirror Maker 2.0 (ММ2). Он дает возможность зеркально отображать топики одного кластера Kafka на другой, удаленный кластер Kafka.

В примере 9.1 приводится образец конфигурации для ММ2. В данном случае механизм будет отзеркаливать топики между кластерами Kafka.

Пример 9.1. Настройка Mirror Maker 2.0 для зеркального отображения топики между двумя кластерами Kafka

```
# Указание любого количества псевдонимов кластера
clusters = source, destination ❶

# Информация о соединениях для каждого кластера
# Здесь это разделенные запятыми пары хост:порт для каждого кластера.
# "A_host1:9092, A_host2:9092, A_host3:9092"
# И вы можете увидеть точное имя хоста в Ambari > Hosts
source.bootstrap.servers = kafka-source1:9092,kafka-source2:9092,kafka-
source3:9092
destination.bootstrap.servers = kafka-dest1:9092,kafka-dest2:9092,kafka-
dest3:9092

# Включение и настройка отдельных потоков репликаций
source->destination.enabled = true

# Регулярное выражение для определения того, какие топики
# будут реплицироваться.
source->destination.topics = foo ❷
groups=.*
topics.blacklist="*.internal,___.*"

# Установка коэффициента репликации для вновь созданных
# территориально удаленных топики
replication.factor=3

checkpoints.topic.replication.factor=1
heartbeats.topic.replication.factor=1
offset-syncs.topic.replication.factor=1

offset.storage.replication.factor=1
status.storage.replication.factor=1
config.storage.replication.factor=1
```

❶ Имена для исходного и целевого кластеров Kafka.

❷ Топики для отзеркаливания в целевом кластере Kafka.

ММ2 отзеркаливает топики только между двумя кластерами Kafka. Потоковая плоскость может состоять из нескольких кластеров Kafka (возможно, 1–2 на регион), где располагаются домены. Один экземпляр ММ2 должен быть развернут для дополнительных регионов с требованием потребления данных в режиме реального времени локально.

ММ2 обычно реализуется в виде коннектора, работающего в кластере Kafka Connect. Мы уже говорили, что потоковая плоскость построена на фундаменте потоковых платформ и коннекторов. Именно эти компоненты создают потоковую сетку данных.

Альтернативой ММ2 может служить проприетарное решение Cluster Linking (CL), разработанное компанией Confluent. CL соединяет кластеры Kafka и отзеркаливает топики между ними. В отличие от ММ2, для работы CL не требуется кластер Kafka Connect. ММ2 и CL предлагают решения для поддержания данных в движении, чтобы минимизировать эффект гравитации данных.

Резюме

Потоковые БД нивелируют значительную часть сложности потоковой плоскости с помощью простой и привычной работы с базами данных. Любой операционный домен может участвовать в сетке данных, производить и потреблять данные в режиме реального времени вместе с другими доменами. И для этого не требуются глубокие знания о потоковой обработке. В следующей главе мы рассмотрим различные модели развертывания. Они послужат образцом для удовлетворения ваших аналитических потребностей в режиме реального времени.

Модели развертывания

В этой главе мы рассмотрим несколько моделей развертывания для различных сценариев применения. Ключевым аспектом нашего исследования будет разбор ситуаций выгодного использования потоковых БД и ситуаций, когда альтернативные подходы могут стать более подходящим решением. Мы будем учитывать все не рассмотренные ранее атрибуты потоковой обработки, включая согласованность, типы рабочих нагрузок, форматы хранения и идеи о потоковой плоскости, представленные в главе 9.

Сосредоточимся на раскрытии различных архитектурных паттернов, способствующих выполнению аналитики в режиме реального времени в потоковой плоскости. Динамичная природа потоковой плоскости предполагает уникальный набор стратегий для эффективного использования ее потенциала — она способна поддержать широкий спектр задач в режиме реального времени.

- На одном конце спектра находятся сценарии взаимодействия с приложением, развернутым в операционной плоскости. Эти потоковые решения требуют согласованности, но их доступ к накопленным данным ограничен.
- На другом конце спектра находятся сценарии работы только с аналитикой в режиме реального времени, не требующие взаимодействия с логикой приложения. Эти потоковые решения могут быть согласованными в конечном счете и иметь доступ ко всем накопленным данным.

Конечно, среди этого многообразия можно встретить разные степени согласованности и уровни доступа к накопленным данным. Мы охватим весь аналитический спектр в режиме реального времени — от левого (согласованные/внутренне согласованные решения) до правого (только аналитические).

Важно подчеркнуть, что решения, рассматриваемые в этой главе, будут использовать потоковые решения с настройками по умолчанию от производителя. Это особенно важно, когда речь идет о согласованности. Для потоковых БД и процессоров с согласованностью в конечном счете допускается эмулировать более сильные гарантии согласованности. Например, можно добавить дополнительные

поля и включить их в логику операций объединения. Реализация этого подхода требует более глубоких, выходящих за рамки тем этой книги знаний о потоковой платформе. Однако, если инженеры знакомы с БД, но впервые сталкиваются с потоковой обработкой, они будут ожидать более высокой степени согласованности, когда не требуется ее имитировать или «прикручивать» в последующем. На основе этих знаний можно сделать вывод, что более согласованные решения обеспечат максимальную пользу при меньших усилиях.

Согласованная потоковая база данных

При необходимости можно создать потоковую БД для выполнения сложной асинхронной/потоковой обработки, которая будет участвовать в логике приложения. Более того, поскольку это еще и база данных, можно запрашивать результаты асинхронного процесса непосредственно из потоковой БД. Применение согласованной потоковой БД упростит использование инфраструктуры, а необходимость в получении глубоких знаний в области потоковой обработки отпадает. Этим требованиям отвечают системы RisingWave и Materialize.

На рис. 10.1 согласованная потоковая БД, такая как RisingWave или Materialize, демонстрирует свои гибридные характеристики, находясь на границе между потоковой и операционной плоскостями.



Рис. 10.1. Операционная аналитика с использованием согласованной потоковой БД

Сплошные стрелки на рис. 10.1 обозначают потоковые данные, а пунктирные — взаимодействие между приложением и базами данных в режиме чтения и записи. Согласованная потоковая БД потребляет данные из БД OLTP и выполняет аналитические преобразования с помощью потоковой обработки. Результаты сохраняются в строчном хранилище потоковой БД.

Такое решение обеспечивает приложению возможность записывать данные непосредственно в БД OLTP и считывать их из потоковой БД. А также позволяет разделить ресурсы для чтения и записи данных.

При использовании потоковой БД допускается потреблять и агрегировать данные из нескольких БД OLTP и отправлять преобразованные данные в хранилище данных или Lakehouse (аналитическая плоскость). Хранилище будет использоваться для внутренней бизнес-аналитики.

Этот подход окажется ограниченным, если пользовательскому приложению потребуются накопленные данные из хранилища или Lakehouse. Из-за гравитации данных будет затруднительно получить накопленные данные из аналитической плоскости.

Согласованная потоковая БД достигает своих пределов, когда требуются сложные аналитические запросы. Такие решения лучше подходят для столбцовых баз данных. Сводная таблица плюсов и минусов этого решения выглядит следующим образом.

Плюсы	Минусы
• Обеспечивает свежесть данных в масштабе миллисекунд. • Существует разделение ресурсов чтения и записи, которые можно независимо и явно масштабировать в большую или меньшую стороны. • Можно объединить данные из нескольких БД OLTP (даже от разных производителей) в единую согласованную потоковую БД. • Преобразования могут выполняться инкрементно, до отправки данных в аналитическую плоскость. • В потоковой БД поддерживаются запросы push и pull с использованием одного и того же механизма/интерфейса запросов	• Отсутствует столбцовое хранение данных, лучше подходящее для более быстрых аналитических запросов. • Сами согласованные потоковые БД не могут получить накопленные данные из аналитической плоскости — требуется помочь им это сделать. • Могут возникнуть трудности, когда размер данных становится слишком большим

Это решение следует использовать для случаев, когда требуется согласованная обработка потоков с участием в логике приложения. Но в то же время не должно быть необходимости в слишком большом количестве накопленных данных из аналитической плоскости. Это также отличный способ агрегировать данные из нескольких БД OLTP разных приложений.

Согласованный потоковый процессор и RTOLAP

В качестве альтернативы можно использовать согласованный потоковый процессор, например Pathway, если вы предпочитаете выводить поток в столбцовую базу данных. Такие процессоры могут выполнять push-запросы в операционной плоскости и участвовать в логике приложения. Они записывают свои данные в потоковую платформу типа Kafka, а база данных RTOLAP (например, Pinot) получает их оттуда и в дальнейшем предоставляет клиентам.

На рис. 10.2 показано, как преобразовывать данные в потоковом процессоре и записывать результаты в хранилище данных/Lakehouse и БД RTOLAP. Система RTOLAP также может считывать накопленные данные из хранилища данных/Lakehouse (аналитическая плоскость), объединять их с данными в режиме реального времени от согласованного потокового процессора и предоставлять операционной плоскости в виде запросов с низкой задержкой. Плюсы и минусы этого решения приведены в таблице.

Плюсы	Минусы
• Свежесть данных может выражаться в масштабе от миллисекунд до секунд. • Аналитика, ориентированная на пользователя, может включать весь накопленный контекст без сохранения его в операционной инфраструктуре. • Столбцовый формат базы данных RTOLAP обеспечивает быструю аналитическую нагрузку для приложения. • RTOLAP и потоковый процессор могут повторно использовать множество топигов на потоковой платформе. • Согласованный потоковый процессор может играть двойную роль: участвовать в бизнес-логике приложения и готовить аналитику в режиме реального времени	• Запросы push и pull разделяются между потоковым процессором и БД OLAP соответственно. Для разделения этих запросов часто требуются отдельные инженеры и строгая координация между ними. На практике такой подход может оказаться трудно-выполнимым

Это решение применяется, если в вашем случае требуется, чтобы большая часть накопленных данных или все они были доступны для пользовательской аналитики, а согласованный потоковый процессор участвовал в логике приложения.

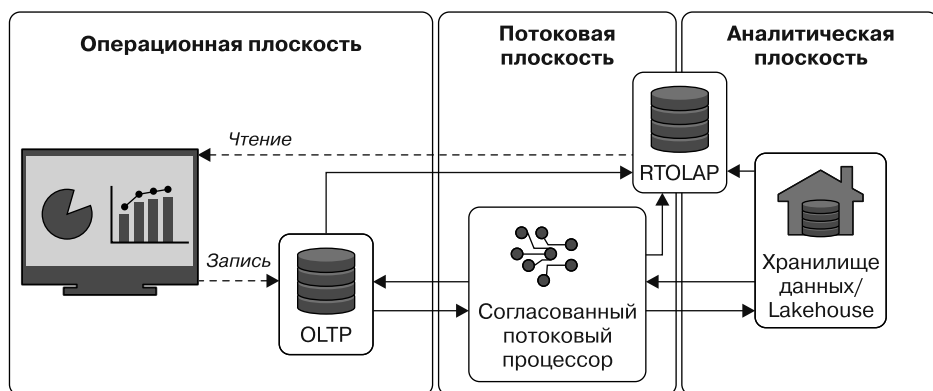


Рис. 10.2. Согласованный потоковый процессор

Потоковая БД OLAP с согласованностью в конечном счете

Если наличие отдельного потокового процессора и БД OLAP делает инфраструктуру слишком сложной, то использование потоковой БД OLAP, такой как Proton, — отличный способ объединить все аналитические рабочие нагрузки в одном решении. Но из-за согласованности в конечном счете ей не следует участвовать в логике приложения. На рис. 10.3 движение данных между OLTP и потоковой БД однонаправленное. Приложения могут использовать преимущества столбцового хранилища OLAP для запросов с малой задержкой.

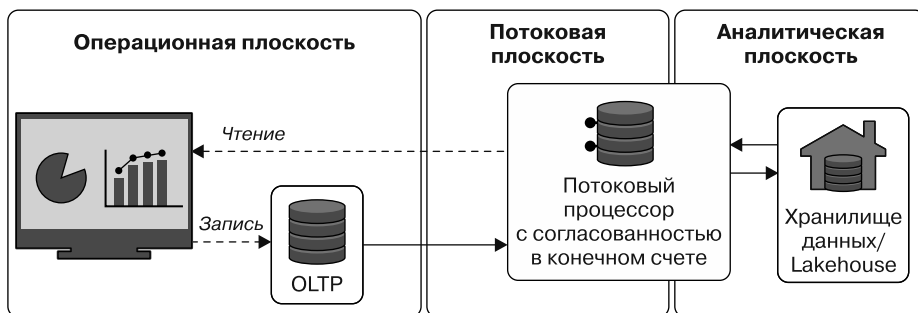


Рис. 10.3. Потоковая БД OLAP с согласованностью в конечном счете

Поскольку система Proton представляет собой потоковую БД, у нее есть возможность записывать результаты потоковой обработки в Kafka. Это дает

возможность другим базам данных потреблять аналитический поток для создания копий в других БД. Но так как в Proton встроена СУБД ClickHouse (RTOLAP) в виде столбцового хранилища, система может обслуживать аналитические запросы с низкой задержкой. Функционал вывода аналитики в Kafka в качестве дополнения дает возможность распространять аналитику в другие регионы более глобально в режиме реального времени. Рассмотрим плюсы и минусы этого подхода.

Плюсы	Минусы
• Свежесть данных в масштабе миллисекунд и секунд. • Предоставляет больше накопленных данных или даже их все, обеспечивая больший контекст для данных в режиме реального времени. • Proton может передавать аналитические изменения, чтобы разрабoтчики имели возможность создавать копии аналитических результатов. • Более простое решение с объединением технологий потоковой обработки и OLAP. • Предоставляет единый SQL-движок для создания запросов push и pull. • Менее громоздкое решение	• Только с согласованностью в конечном счете. Не может участвовать в логике приложения

Наибольшее преимущество использования потоковой БД OLAP заключается в возможности сбалансировать запросы push и pull в рамках единого механизма запросов и интерфейса. В этом случае для работы требуется только один инженер, в отличие от решения с отдельным потоковым процессором и БД OLAP. Proton предлагает простое решение для выполнения аналитики в режиме реального времени.

Это решение применяется для того, чтобы снизить сложность инфраструктуры и инженерных решений. Оно также позволит получить доступ к большому количеству накопленных данных для выполнения аналитики, ориентированной на пользователя.

Потоковый процессор с согласованностью в конечном счете и RTOLAP

Это решение, пожалуй, самое распространенное на сегодняшний день при предоставлении аналитики в режиме реального времени. Для его реализации часто используются Flink и БД RTOLAP, например Pinot (рис. 10.4).

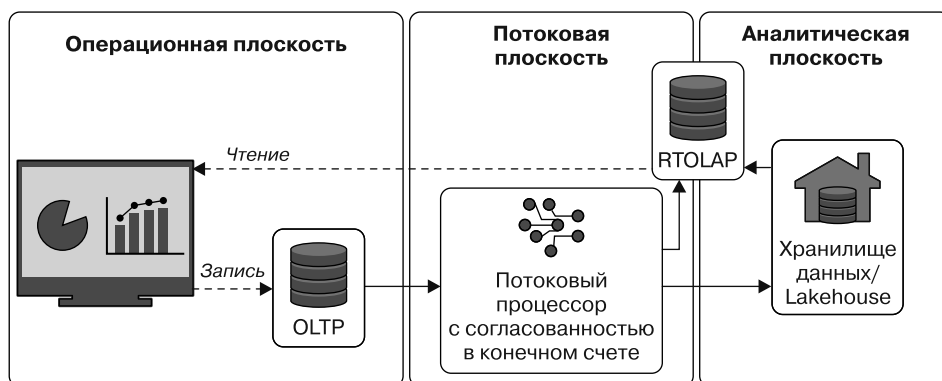


Рис. 10.4. Потоковый процессор с согласованностью в конечном счете и RTOLAP

Это решение уже доказало свою эффективность во многих крупномасштабных приложениях, работающих в режиме реального времени.

Плюсы	Минусы
• Может быть достигнута свежесть данных в масштабе от миллисекунд до секунд. • Накопленные данные и данные реального времени можно объединить для получения полного представления об аналитике	• Это сложное и громоздкое решение, способное привести к увеличению стоимости. • Поскольку Flink — это фреймворк с согласованностью в конечном счете, он не должен принимать участие в логике приложения. • Поскольку потоковый процессор и механизмы выполнения OLAP в режиме реального времени разделены, данное решение не обеспечивает единый SQL-движок для запросов push и pull. Это может привести к увеличению инженерного и организационного напряжения

Примените это решение, если в вашем случае требуется больше накопленных данных для пользовательской аналитики в приложении. Согласованность не должна вызывать серьезных опасений при обогащении потоков фактов потоками измерений.

Потоковый процессор с согласованностью в конечном счете и HTAP

В случаях, когда требуется расположить аналитические рабочие нагрузки вблизи операционной плоскости или в ней самой, удобным решением может стать использование БД HTAP совместно с БД OLTP. Для сбора накопленных данных,

которые будут отправлены в хранилище данных или Lakehouse, можно добавить потоковый процессор с согласованностью в конечном счете (рис. 10.5).

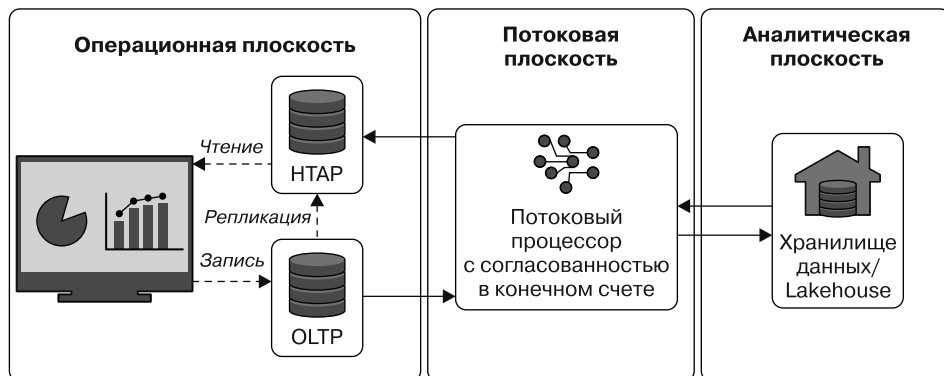


Рис. 10.5. Потоковый процессор с согласованностью в конечном счете и HTAP

Потоковый процессор может получать ограниченное количество накопленных данных из аналитической плоскости и предоставлять их в БД HTAP. Благодаря столбцовому формату она способна обслуживать аналитические запросы с низкой задержкой. Объем накопленных данных в БД HTAP и/или его время хранения обычно ограничены. В таблице приведены плюсы и минусы решения.

Плюсы	Минусы
• Обеспечивает свежесть данных в масштабе миллисекунд. • У баз данных HTAP столбцовое хранение данных для быстрых аналитических запросов. • Низкая инфраструктурная сложность	• Ограниченное количество накопленных данных. • Сложность возрастает при реализации сохранения накопленных данных из БД HTAP. • Потоковый процессор не может принимать участие в логике приложения

Примените это решение, если в вашем случае данные должны быть свежими в течение миллисекунд и требуется лишь небольшое подмножество накопленных данных для пользовательской аналитики.

ksqlDB

В главе 6 мы обсуждали гарантии согласованности, предоставляемые ksqlDB (непрерывное уточнение, аналогичное согласованности в конечном счете). ksqlDB основана на базовой JVM-библиотеке Kafka Streams для развертывания внутри

микросервисов, которые разворачиваются в операционной плоскости как часть бэкенда приложения.

Мы рекомендуем использовать Kafka Streams и ksqlDB только для более простых операций обработки потоков. Сложно правильно выполнить операции JOIN, особенно при объединении потоков только для добавления и таблиц, аналогичных журналам изменений. ksqlDB поддерживает только часть синтаксиса и семантики SQL (например, в этой СУБД нет операций JOIN, выполняемых таблицей с самой собой, и вложенных JOIN). Риск реализации несогласованной логики очень высок, даже несмотря на возможность использования ksqlDB, если в вашем распоряжении есть команда опытных специалистов по обработке потоков.

В результате наилучшими сценариями применения ksqlDB станут более простые операции потоковой обработки на Kafka (ksqlDB поддерживает только Kafka в качестве источника и назначения данных). Эти операции подготавливают данные для аналитических точек назначения, таких как хранилища или озера данных. Можно также использовать материализованные представления ksqlDB для выполнения отдельных запросов, например, для точек назначения с поддержкой только пакетной обработки без возможности применения полноценного, постоянно работающего потребителя Kafka. В этом случае ksqlDB берет на себя роль потребителя с возможностью фильтровать и предварительно обрабатывать данные для системы, работающей только в пакетном режиме.

Плюсы	Минусы
• Свежесть данных. • Возможности потоковой обработки. • Позволяет точечным запросам, использующим материализованные представления (TABLE), поддерживать только пакетные точки назначения	• Поддерживает только Kafka в качестве источника/назначения. • Требуется высокий уровень знаний в области обработки потоков. • Сложные операции обработки потоков трудно реализовать корректно. • Нет поддержки полного синтаксиса и семантики языка SQL

Инкрементное обслуживание представлений

Решения с поддержкой инкрементного обслуживания представлений (IVM), такие как Feldera, PeerDB или Epsio, также могут использоваться для поддержки пакетных точечных запросов к предварительно обработанным свежим данным. В отличие от ksqlDB, эти решения более тесно интегрированы с операционными базами данных, такими как PostgreSQL. Кроме того, им не требуется использовать брокер Kafka в качестве промежуточного слоя.

В то же время эти решения позволяют выполнять более сложную предварительную обработку данных с применением полной и последовательной семантики языка SQL. Но системы на основе Kafka остаются менее гибкими. По сути, вы ограничены теми источниками и точками назначения данных, которые поддерживаются соответствующим поставщиком. А при использовании Kafka в качестве промежуточного слоя выбор источников и точек назначения значительно расширится.

Решения IVM способствуют расширению представления ИТ-организаций об асинхронной и непрерывной обработке данных. Для таких компаний они могут стать шлюзом в мир потоковых технологий, потоковой обработки и потоковых баз данных. Такие решения сохраняют способность работать привычным образом (например, PeerDB и Epsio) или даже изнутри экосистемы Postgres, но способ работы близок к потоковым технологиям. Это может стать необходимым для интеграции более крупных архитектур в больших организациях.

Плюсы	Минусы
• Свежесть данных. • Полный синтаксис и семантика SQL. • Согласованность. • Доступность точечных запросов. • Внедрение потоковых аспектов в мир баз данных	• Ограничение источников/точек назначения, поддерживаемых соответствующим поставщиком

Обертка сторонних данных Postgres Multicorn

Multicorn — это расширение для PostgreSQL 9.1+, призванное сделать разработку *обертки сторонних данных* (Foreign Data Wrapper, FDW) более удобной, позволяя программистам использовать язык программирования Python. Этот инструмент дает им возможность создавать в операционной плоскости FDW для баз данных не из системы Postgres (например, для БД RTOLAP, таких как Apache Pinot) и делать данные доступными в БД PostgreSQL.

Если ваши базы данных на операционном уровне созданы на основе Postgres, то FDW станет хорошим вариантом сближения операционной и аналитической плоскостей для Postgres. Преимуществом будет отсутствие сложностей, связанных с использованием, например, потоковой платформы Kafka и потоковой обработки между плоскостями данных. Однако, как и в случае с IVM в предыдущем разделе, проблема Multicorn заключается в том, что у этого инструмента лишь один поставщик. В крупных организациях в 99 % случаев в операционной плоскости используется множество технологий баз данных — не только Postgres. Следовательно, вам придется сделать Postgres центральной операционной базой данных,

что на практике недостижимо для больших организаций. А использование потоковой платформы, такой как Kafka, в качестве промежуточного слоя поможет, если вы можете позволить себе преодолеть связанные с этим решением сложности.

Плюсы	Минусы
• Полный синтаксис и семантика SQL. • Согласованность. • Доступ к базам данных OLTP (Postgres), другим БД OLTP, не относящимся к Postgres, и БД OLAP (в режиме реального времени) в экосистеме Postgres (операционная плоскость)	• Требование использовать Postgres в качестве центральной технологии БД (непрактично при развертывании в больших масштабах/в крупных организациях)

Когда следует использовать потоковые процессоры на основе кода

Обработку и запрос данных можно реализовать и не на основе потоковых БД, а с помощью других технологий. Классические технологии обработки потоков, такие как Kafka Streams и Flink, по-прежнему полезны. Особенно актуальны они для «тяжелых» потоковых приложений, таких как обнаружение мошенничества, или внутри архитектуры микросервисов. Обычно такие приложения располагаются на операционном уровне.

За последние годы значительно расширился набор технологий для обработки потоков на основе кода. Теперь, помимо классических технологий, можно выбрать Quix Streams (Python, C#), Bytewax (Python) и Pathway (Python). Что интересно, Bytewax и Pathway основаны на том же базовом механизме обработки потоков, что и система Materialize (поток данных с временными метками/DD). Другие новые и интересные технологии в этом сегменте — Deephaven и основанный на Python фреймворк GlassFlow.

Когда следует использовать технологии Lakehouse/Streamhouse

Технологии Lakehouse (например, Delta Tables от Databricks, Apache Iceberg или Apache Hudi) тоже становятся все больше ориентированными на потоковые технологии и предлагают функции потоковой передачи и/или обработки потоков. В платформе Databricks этот функционал предлагается изначально (Spark Structured Streaming). Разработчики платформы Ververica совместно с разработчиками Apache Paimon представили новую архитектуру Streamhouse.

Она обеспечивает еще более тесную интеграцию потоковой и обычной обработки в озере данных. Чуть позже компания Confluent анонсировала функцию Tableflow для представления данных в виде таблиц Iceberg на своей потоковой платформе Confluent Cloud на основе Kafka. Как и Paimon, Confluent использует фреймворк Flink для плавной интеграции потоковой и пакетной обработки.

Стартап Streambased.io предлагает решение, похожее на Tableflow. Разработчики создали свое решение для запросов к топикам Kafka. С одной стороны, оно тесно связано с Kafka, а с другой — предлагает SQL-интерфейс, похожий на базу данных, на основе механизма запросов Trino SQL. Если вы используете движок Trino на Kafka без сервиса Streambased, производительность SQL-запросов может быть очень низкой, поскольку механизм запросов считывает данные из топиков Kafka напрямую. Streambased добавляет индекс на основе фильтра Блума к топикам Kafka. Этот индекс используется для ускорения выполнения запросов на основе Trino в 10 раз и более (на сайте компании заявлено ускорение в 39 раз).

В отличие от БД RTOLAP, таких как Pinot, Druid или ClickHouse, Streambased не копирует данные Kafka, а только создает дополнительные индексы. И эти индексы занимают гораздо меньше места на диске, чем реальные данные. Дополнительная производительность от индексирования позволяет запрашивать топики Kafka почти напрямую с очень низкой задержкой. Это может стать еще одним решением для обеспечения нисходящей совместимости потоковой и пакетной обработки — можно запрашивать топики Kafka через REST или JDBC (Java Database Connectivity), как если бы они были таблицами базы данных. С появлением этих решений потоковая обработка может стать гораздо более доступной, поскольку она будет практически невидимой для конечного пользователя.

По сути, все эти разработки увеличивают гравитацию данных в потоковой плоскости. Большой объем обработки будет выполняться ближе к потоку, или данным в движении, а меньший объем обработки должен быть реализован на поздних этапах в хранилище/озере данных. Если повышение свежести/снижение задержки данных не ставится в качестве главной задачи, обработку данных можно легко перенести из потоковой плоскости обратно в аналитическую (и наоборот, если снижение задержки становится более важным).

Технологии кэширования

Конечно, здесь есть из чего выбрать. Если требуется чрезвычайно низкая задержка, даже более низкая, чем у потоковой платформы, такой как Kafka, можно рассмотреть инструменты кэширования, например Redis или Hazelcast. Их можно расположить и на операционной, и на потоковой плоскости в зависимости

от того, насколько тесно они интегрированы с потоковой платформой. Такие решения, как платформа Hazelcast, предлагают функции, схожие с обработкой потоков. Для этого в них может применяться SQL-подобный синтаксис.

Где выполнять обработку и запросы в общем случае

За время работы над этой книгой мы рассмотрели большое количество технологий. Некоторые из них располагаются в операционной плоскости, некоторые — в аналитической, некоторые — в потоковой, но большинство — где-то между. Вы, читатель, скорее всего, практикующий специалист. Вам поставлена определенная задача, и вы можете прочитать эту книгу, чтобы получить новые идеи для ее решения. Можем ли мы, авторы, помочь вам в этом? Есть ли какой-то более общий совет для выбора правильного метода и технологии, а также способов их распространения и масштабирования?

Конечно, самый безопасный ответ на подобные вопросы: «Зависит от ситуации». Так и есть, но мы бы хотели быть немного более полезными. В этом разделе вы узнаете, от какой ситуации все зависит и каких последствий можно ожидать от выбора тех или иных технологий для решения конкретной задачи.

Четыре вопроса «Где?»

Какие факторы будут наиболее важными при выборе технологии, когда вы размышляете о решении определенной задачи или сценарии использования? Вернемся к нашей карте, или ландшафту, пространства данных. Оно было представлено в виде диаграммы Венна с пересекающимися операционной, аналитической и потоковой плоскостями. Эта карта может подтолкнуть нас к более серьезным размышлениям о точках на ней и попыткам начать поиски с этих точек.

В результате, когда вы рассматриваете свою задачу или сценарий применения, постарайтесь задать себе следующие четыре вопроса «Где?».

1. Где располагается мой сценарий использования?
2. Где находятся данные, необходимые для его реализации?
3. Где их обрабатывать?
4. Где их можно запросить?

Аналитический сценарий использования

На рис. 10.6 представлен типичный пример. Рассмотрим аналитический сценарий, когда данные, изначально поступающие из систем операционной обработки, необходимо запросить в приложении при создании дашборда для бизнес-задач. Следовательно, ответ на первый вопрос («Где располагается мой сценарий использования?»): «В аналитической плоскости». Предположим, что на второй вопрос тоже легко ответить — источник данных находится на потоковой платформе в потоковой плоскости. На диаграмме следует обратить внимание на стрелку от источника данных (ответ на вопрос 2) к сценарию использования (ответ на вопрос 1).



Рис. 10.6. Спектр точек на плоскости данных

Давайте зададим два следующих вопроса «Где?»: где обрабатывать данные и где их можно запросить? Обычно в современном мире ответ звучит так: «Делать и то и другое на аналитическом уровне». Например, в хранилище данных типа Snowflake или технологическом решении Lakehouse типа Databricks.

Вероятно, такой простой (и, возможно, дорогостоящий) ответ вам не подойдет. Нам он точно не подходит, потому что на вопросы 3 и 4 не существует единого ответа. На самом деле можно дать целый ряд ответов.

Сосредоточимся на прямой линии, или пути, от вопроса 2 к вопросу 1. Этот путь обозначает множество ответов на вопросы 3 и 4. Конечно, можно давать простые ответы и обрабатывать и запрашивать данные на аналитической плоскости, как показано на рис. 10.7.



Рис. 10.7. Обработка и формирование запросов на аналитическом уровне

Однако стоит рассмотреть и другие варианты. Например, на вопрос 3 («Где обрабатывать данные?») можно ответить технологией с возможностью обработки на потоковой плоскости. Допустим, можно предложить потоковый процессор типа Flink или потоковую БД. А запрос данных (вопрос 4) можно даже перенести в плоскость потоковой передачи при условии использования потоковой БД, как показано на рис. 10.8.

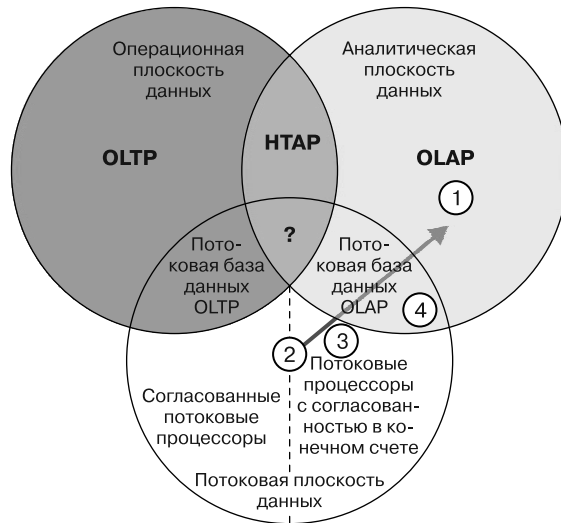


Рис. 10.8. Выполнение обработки и запросов ближе к потоковой плоскости либо на ней

Таким образом, путь от местоположения вашей задачи к источнику данных для нее определяет множество или набор возможных точек, а значит, и технологий для ее решения. Обработку и запрос данных можно выполнять в любом месте на этом пути, но у вашего выбора всегда есть и положительные, и отрицательные последствия.

Последствия

Каковы же эти последствия? Теоретически чем дальше от источника данных вы их обрабатываете (вопрос 3) и запрашиваете (вопрос 4), тем больше недостатков получаете. Это связано с гравитацией данных, описанной в предыдущих главах. Например, если вы ответите на вопросы 3 и 4, как на рис. 10.7, и выполните оба действия на аналитической плоскости, близкой к аналитическому сценарию использования, то будет затруднительно:

- поддерживать свежесть данных, поскольку они должны сначала попасть в аналитическую плоскость, прежде чем их можно будет обработать и затем запросить;
- совместно использовать обработанные данные из-за того, что обработка выполняется слишком близко к сценарию использования;
- выполнять обработку инкрементно. В аналитической плоскости преобладает пакетная обработка. Чтобы этот процесс был успешным, придется многократно просматривать одни и те же данные. А это неэффективно, и процесс может оказаться дорогостоящим, особенно когда применяются облачные технологии хранения данных.

Почему же подавляющее большинство коллективов в индустрии данных до сих пор так поступает? Почему ответы на вопросы 3 и 4 чаще всего звучат так: «Сделаем это в аналитической плоскости»? Почему для этого очень распространенного сценария применения притягательность потоковой плоскости остается такой низкой, а аналитической плоскости — такой высокой в свете перечисленных недостатков?

Мы считаем, что одна из главных причин такого необъяснимого принятия архитектуры с длинным списком недостатков заключается в сохраняющейся сложности потоковой обработки, а также запросов к потокам. И эта сложность объясняется как минимум тремя причинами.

- Требуется большой опыт работы в области потоковой обработки, а зачастую его просто нет.

- Даже если опыт есть, потоковая обработка все равно будет требовать больших затрат — настолько больших, что они перевесят потенциальную экономию от более ранней и инкрементной обработки.
- Даже имея опыт работы с потоковыми данными, трудно добиться стабильных результатов при обработке потоков.

Чем раньше начнется обработка, тем более свежими будут данные, тем большим количеством обработанных данных вы сможете поделиться и тем более инкрементно получится обрабатывать данные. Но до появления потоковых БД применение потоковой обработки часто было нецелесообразным по весьма прагматичным причинам.

Потоковые БД способны изменить ситуацию, поскольку они работают гораздо лучше обычных БД. Например, Materialize и RisingWave даже используют Postgres Wire Protocol. Специалисты по MongoDB могут без труда применять новый механизм обработки потоков в этой СУБД. Таким образом, в обоих случаях требуется гораздо меньший опыт работы с потоками. В принципе можно привлечь к работе уже имеющихся специалистов по базам/хранилищам данных. Это снизит стоимость внедрения потоковой обработки, ведь не нужно будет нанимать дополнительных сотрудников.

Что касается четвертого вопроса «Где?» о запросе данных, то потоковые БД нового поколения даже предлагают возможность почти прямого запроса потоковых данных, обеспечивая материализованные представления. При постоянном обновлении они освежают данные, а использование внутри системы инкрементной потоковой обработки позволяет снизить стоимость реализации сценария применения, поскольку обрабатывать, а затем материализовывать нужно только новые данные. Выполнение запросов к материализованным представлениям избавляет от необходимости снова и снова кропотливо просматривать одни и те же старые точки данных. При использовании запросов с высокой частотой это может снизить требования к вычислениям и, следовательно, значительно уменьшить стоимость.

В результате потоковые БД могут сыграть решающую роль во внедрении потоковой передачи данных и способны увеличить объем данных в потоковой плоскости. Во многих сценариях использования обработку (вопрос 3) можно перенести в потоковую плоскость, что было невозможно до появления потоковых баз данных.

Если объединить эту мысль с недавним появлением таких технологий, как Streamhouse (Ververica) и Tableflow (Confluent), гравитация данных в потоковой плоскости еще больше возрастет, а границы между плоскостями на диаграмме Венна продолжают размываться.

Резюме

В этой главе были рассмотрены различные модели развертывания и архитектуры потоковых баз данных и других гибридных технологий, перемещающихся между операционной, аналитической и потоковой плоскостями, а также плюсы и минусы построения архитектуры на их основе.

Мы продолжили обсуждение технологий, которые не очень тесно связаны с основной темой этой книги, но все же достаточно близки к ней, чтобы о них стоило упомянуть. Среди них были решения для обработки потоков на основе кода, новые разработки в области потоковой передачи данных в хранилища Lakehouse и технологии кэширования.

Мы обобщили свои рекомендации по моделям развертывания с помощью набора вопросов «Где?». Эти вопросы были сформулированы на основе диаграммы Венна, чтобы помочь вам очертить множество возможностей для размещения систем обработки и запросов ваших данных.

Мы пришли к выводу, что потоковые БД могут привести к демократизации потоковой обработки и использования потоков в целом. В результате они способны значительно увеличить объем данных в потоковой плоскости. Их влияние усиливается с ростом функционала у поставщиков потоковых (Streamhouse от Ververica, Tableflow от Confluent и т. д.) и непотоковых (MongoDB, Databricks и т. д.) решений. Все это еще больше помогает в позиционировании поставщиков в пересекающихся с потоковой плоскостью участках диаграммы Венна.

В следующей, последней главе мы будем развивать захватывающую мысль о слиянии потоковых технологий и баз данных.

Будущее данных в режиме реального времени

В пещере, куда вы боитесь войти, хранится сокровище, которое вы ищете.

Джозеф Кэмпбелл

Глубоко изучив в этой главе варианты развертывания потоковых баз данных, мы немного шире взглянем на будущее данных в режиме реального времени. Этот взгляд сформирован одной из центральных тем книги — ускоренным слиянием потоковой передачи данных и баз данных. Потоковые БД — одно из проявлений этой тенденции. Но есть еще много достойных внимания доказательств.

Мы начнем с графовых БД и их постоянного продвижения в сферу потоковых данных, например Memgraph, thatDot, а затем, после прорыва ChatGPT в сфере генеративного ИИ (GenAI), перейдем к суперпопулярным векторным базам данных, например Milvus и Weaviate. Продолжим путешествие по сближающимся сферам потоковой обработки и баз данных. Предложим инструменты для привнесения одного из центральных аспектов потоковых БД, а именно материализованных представлений, или IVM, в классические БД — Feldera, PeerDB и Epsio. В конце главы мы рассмотрим потоковую функциональность известных поставщиков баз данных, таких как MongoDB. После плавно перейдем в аналитическую плоскость с такими хранилищами данных, как BigQuery, Redshift и Snowflake, поскольку они тоже постепенно расширяют свои потоковые возможности. В заключение обсудим слияние потоковых и Lakehouse-архитектур, обусловленное появлением систем Apache Iceberg, Apache Hudi, Delta Lake и Apache Paimon. Это один из самых многообещающих макротрендов не только в области потоковых технологий, но и в пространстве больших данных в целом.

Слияние плоскостей данных

Перед началом путешествия необходимо определить контекст, еще раз взглянув на диаграмму Венна из главы 7 (рис. 11.1).



Рис. 11.1. Диаграмма Венна для баз данных нового поколения

На диаграмме показано слияние трех плоскостей данных:

- операционной;
- аналитической;
- потоковой.

В первых разделах данной главы мы сосредоточимся на операционной плоскости, то есть на базах данных, которые чаще всего используются именно в операционной среде. Мы подробнее остановимся на том, как различные виды операционных БД пробивают себе дорогу в заштрихованную часть потоковой плоскости, где операционная и потоковая плоскости пересекаются.

После этого рассмотрим текущие разработки поставщиков из аналитической плоскости, включая хранилища данных (например, Snowflake) и хранилища Lakehouse (например, Databricks), по дополнению инструментария функционалом потоковой обработки. Тем самым мы приблизимся к целевой заштрихованной области. Особенно intriguing в этом контексте выглядит продолжающееся слияние архитектур Lakehouse и потоковых данных. В ближайшие годы это станет одной из главных тем в пространстве больших данных и, следовательно, будет кульминацией главы.

Графовые базы данных

Графовые базы данных используют базовые структуры графов для выполнения запросов к графам, которые было бы трудно эффективно реализовать в реляционной базе данных. Они стали важной частью развития парадигмы NoSQL в 2000-х и 2010-х годах, что значительно расширило пространство баз данных. Популярные графовые БД — Neo4J, ArangoDB и TigerGraph. Графовые БД, безусловно, относятся к операционной плоскости данных. Обычно их задействуют для операционных приложений, и в этом их главное отличие от *графовых вычислительных механизмов*, место которых — в аналитической плоскости.

Вот типичные варианты использования таких БД.

Социальные сети

Графовые БД отлично справляются с моделированием и запросами к социальным сетям и широко применяются, например, в LinkedIn. Обычно узлы представляют пользователей, а ребра — отношения между ними, например: «пользователь А подписан на пользователя Б» или «пользователь А — друг пользователя Б».

Рекомендации

Здесь моделируются и запрашиваются отношения между пользователями и продуктами/контентом для получения персонализированных рекомендаций на основе предпочтений пользователя.

Графы знаний

В этом случае графы используются для моделирования отношений между фрагментами знаний, чтобы обеспечить более точный семантический поиск.

Обнаружение мошенничества

В сочетании с ИИ и моделями ML точность механизмов обнаружения мошенничества можно заметно повысить за счет использования графовых БД для улучшения анализа моделей связей между сущностями.

Цепочка поставок

Здесь узлы представляют собой, например, местоположение, товары и организации, а ребра — связи и перемещения в цепочке поставок. В этом контексте графовые БД могут применяться для значительной оптимизации цепочек поставок.

Хотя такие известные графовые БД, как Neo4J и ArangoDB, можно подключить к потоковым платформам, например, с помощью Kafka Connect, новые поставщики, такие как Memgraph и thatDot, начали предлагать функционал для еще более тесной интеграции.

Memgraph

Memgraph предлагает функционал прямого подключения к потоковым платформам, таким как Kafka и Pulsar, для получения данных. Он также предоставляет поддержку так называемых *модулей трансформации* для преобразования входящих сообщений в целях их корректного потребления. Модули трансформации могут использовать API на языках C или Python.

В примере 11.1 показано, как Memgraph позволяет напрямую получать данные из Kafka.

Пример 11.1. Прямое поглощение данных из Kafka в потоки Memgraph

```
CREATE KAFKA STREAM <stream name>
  TOPICS <topic1> [, <topic2>, ...]
  TRANSFORM <transform procedure>
  [CONSUMER_GROUP <consumer group>]
  [BATCH_INTERVAL <batch interval duration>]
  [BATCH_SIZE <batch size>]
  [BOOTSTRAP_SERVERS <bootstrap servers>]
  [CONFIGS { <key1>: <value1> [, <key2>: <value2>, ...]}]
  [CREDENTIALS { <key1>: <value1> [, <key2>: <value2>, ...]}];

START STREAM <stream name> [BATCH_LIMIT <count>] [TIMEOUT <milliseconds>];
```

В примере 11.2 показан модуль трансформации, использующий API Python.

Пример 11.2. Использование API Memgraph Python для преобразования входящих сообщений из потоковой платформы

```
import mgp

@mgp.transformation
def transformation(context: mgp.TransCtx,
                  messages: mgp.Messages
                  ) -> mgp.Record(query=str, parameters=mgp.Nullable[mgp.Map]):
    result_queries = []

    for i in range(messages.total_messages()):
        message = messages.message_at(i)
        payload_as_str = message.payload().decode("utf-8")
        result_queries.append(mgp.Record(
            query=f"CREATE (n:MESSAGE {{timestamp: '{message.timestamp()}', payload:
                '{payload_as_str}', topic: '{message.topic_name()}'}})",
            parameters=None))

    return result_queries
```

thatDot/Quine

Разработчики из стартапа thatDot создали Quine — граф (базу данных) с открытым исходным кодом. Quine позволяет объединять множество потоков событий в общий граф, запрашивать сложные связи между событиями и выполнять действия в режиме реального времени. В характеристике продукта Quine отсутствует выражение «база данных» — компания называет свою технологию потоковым графом для конвейеров данных. Однако в этой книге Quine вполне может считаться потоковой БД (на основе графов).

- Эта система на основе графов предлагает обработку потоков (standing query), чтение из них и запись результатов обработки в потоки.
- Она предоставляет материализованные представления (графы).

Quine дает возможность обрабатывать графы в гораздо больших масштабах, чем существующие графовые БД, такие как Neo4J или TigerGraph. Это связано с тем, что система полностью распределенная и основана на концепции *акторов* (внутренняя реализация на основе библиотеки Scala *Pekko*, форка Akka 2.6.1). Она также предоставляет подключаемый уровень хранения данных с поддержкой RocksDB, MapDB (локально) и Cassandra (удаленно).

В примере 11.3 показан вариант использования поглощения потока событий на стороне сервера для подключения к потоку изменений страниц в Википедии.

Пример 11.3. Получение потока событий на стороне сервера о редактировании страниц в Википедии с помощью Quine

```
curl -X "POST" "http://127.0.0.1:8080/api/v1/ingest/wikipedia-revision-create" \
  -H 'Content-Type: application/json' \
  -d '${
    "format": {
      "query": "CREATE ($that)",
      "parameter": "that",
      "type": "CypherJson"
    },
    "type": "ServerSentEventsIngest",
    "url": "https://stream.wikimedia.org/v2/stream/mediawiki.revision-create"
  }'
```

На следующем этапе (пример 11.4) будет создан запрос на поглощение данных. Мы применили язык графовой базы данных Cypher из Neo4J, чтобы загрузить отдельные события в узлы потоковой графовой БД.

Пример 11.4. Запрос на поглощение данных, написанный на языке Cypher, для загрузки отдельных событий в узлы графовой базы данных Quine

```
MATCH (revNode),(pageNode),(dbNode),(userNode),(parentNode)
WHERE id(revNode) = idFrom('revision', $that.rev_id)
      AND id(pageNode) = idFrom('page', $that.page_id)
      AND id(dbNode) = idFrom('db', $that.database)
      AND id(userNode) = idFrom('id', $that.performer.user_id)
      AND id(parentNode) = idFrom('revision', $that.rev_parent_id)

SET revNode = $that,
    revNode.bot = $that.performer.user_is_bot,
    revNode:revision

SET parentNode.rev_id = $that.rev_parent_id

SET pageNode.id = $that.page_id,
    pageNode.namespace = $that.page_namespace,
    pageNode.title = $that.page_title,
    pageNode.comment = $that.comment,
    pageNode.is_redirect = $that.page_is_redirect,
    pageNode:page

SET dbNode.database = $that.database,
    dbNode:db

SET userNode = $that.performer,
    userNode.name = $that.performer.user_text,
    userNode:user

CREATE (revNode)-[:TO]->(pageNode),
      (pageNode)-[:IN]->(dbNode),
      (userNode)-[:RESPONSIBLE_FOR]->(revNode),
      (parentNode)-[:NEXT]->(revNode)
```

После создания графа из источника потока Quine позволяет задать *постоянный запрос* (standing query) для отслеживания потока на предмет обнаружения определенных закономерностей. На их основе могут быть выполнены обновление самого графа путем создания новых узлов или ребер, запись результатов в Kafka или Amazon Kinesis или отправка результатов в Webhook. Одной из важнейших функциональных возможностей постоянных запросов в Quine стало отсутствие требований указания временных окон, как, например, во Flink. Такие запросы по сути соответствуют потоковой обработке на основе графов.

В примере 11.5 показан постоянный запрос. Он состоит из шаблона и вывода. Шаблон определяет, что необходимо сопоставить, а вывод — какие действия предпринять. Постоянные запросы тоже написаны на языке Cypher.

Пример 11.5. Постоянный запрос на языке Cypher сопоставляет шаблон и выполняет заданные действия (с помощью файла назначения)

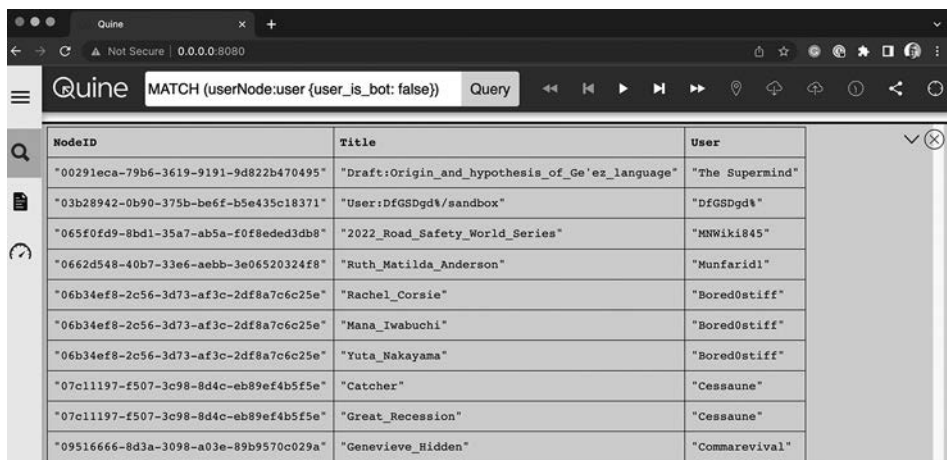
```
{
  "pattern": {
    "query": "MATCH (n)-[:has_father]->(m) WHERE exists(n.name) AND exists(m.name)
    RETURN DISTINCT strId(n) AS kidWithDad",
    "type": "Cypher"
  },
  "outputs": {
    "file-of-results": {
      "path": "kidsWithDads.jsonl",
      "type": "WriteToFile"
    }
  }
}
```

Чтобы получить последние десять узлов событий `revision-create`, можно запросить Quine, как показано в примере 11.6.

Пример 11.6. Получение из Quine последних десяти узлов событий `revision-create`

```
MATCH (userNode:user {user_is_bot: false})-[:RESPONSIBLE_FOR]->(revNode:revision
{database: 'enwiki'})
RETURN DISTINCT strId(userNode) as NodeID,
       revNode.page_title as Title,
       revNode.performer.user_text as User
LIMIT 10
```

Результат этого запроса приведен на рис. 11.2.



NodeID	Title	User
"00291eca-79b6-3619-9191-9d822b470495"	"Draft:Origin_and_hypothesis_of_Ge'ez_language"	"The Supermind"
"03b28942-0b90-375b-be6f-b5e435c18371"	"User:DfGSDgd8/sandbox"	"DfGSDgd8"
"065f0fd9-8bd1-35a7-ab5a-f0f8eded3db8"	"2022_Road_Safety_World_Series"	"MNWiki845"
"0662d548-40b7-33e6-aebb-3e06520324f8"	"Ruth_Matilda_Anderson"	"Munfarid1"
"06b34ef8-2c56-3d73-af3c-2df8a7c6c25e"	"Rachel_Corsie"	"Bored0stiff"
"06b34ef8-2c56-3d73-af3c-2df8a7c6c25e"	"Mana_Iwabuchi"	"Bored0stiff"
"06b34ef8-2c56-3d73-af3c-2df8a7c6c25e"	"Yuta_Nakayama"	"Bored0stiff"
"07c11197-f507-3c98-8d4c-eb89ef4b5f5e"	"Catcher"	"Cessaune"
"07c11197-f507-3c98-8d4c-eb89ef4b5f5e"	"Great_Recession"	"Cessaune"
"09516666-8d3a-3098-a03e-89b9570c029a"	"Genevieve_Hidden"	"Commarevival"

Рис. 11.2. Результат запроса «не бот»

Memgraph, а особенно Dot/Quine, находятся в гуще событий слияния графовых БД и потоковой обработки. Следовательно, они могут служить прекрасным примером слияния операционной и потоковой плоскостей.

Векторные базы данных

В двух словах, *векторные базы данных* оптимизированы для хранения и запроса векторов, то есть списков чисел фиксированной длины (обычно от 50 до 1500 на вектор). Запросы в векторной базе данных зачастую сводятся к поиску сходства. Он реализуется алгоритмами поиска *приближенных ближайших соседей* (Approximate Nearest Neighbor, ANN). У векторов из векторной БД высокая размерность, и они представляют признаки неструктурированных данных, таких как текст, звук и видео. Среди популярных векторных БД можно отметить Milvus, Weaviate, Pinecone, Vespa и Qdrant. Интересно, что все большее количество существующих баз данных также используют функциональность векторных, например ClickHouse, Rockset, PostgreSQL, Cassandra, Elastic, Redis и SingleStore.

Идея превращения неструктурированных данных в векторные включения восходит к «языковой игре» Людвига Витгенштейна¹.

В 2010-х годах два успешных подхода, Word2vec центра Google Research и GloVe Стэндфордского университета², продвинули векторные включения в мейнстрим и косвенно привели к появлению новой категории векторных БД. Большие языковые модели (Large Language Model, LLM), такие как GPT-модели OpenAI, также могут генерировать векторные включения. Вот сценарии использования векторных баз данных.

Рекомендации

Поиск по сходству может использоваться в рекомендательных системах. Например, его применяют в интернет-магазинах, чтобы находить товары, похожие на те, что искал пользователь. Поиск сходства может быть даже мультимодальным, например поиск текстового и графического сходства.

Обнаружение мошенничества

Здесь поиск по сходству используется для выявления, например, мошеннических операций.

¹ Wittgenstein L. Philosophical Investigations. — New York: MacMillan, 1958 (1953).

² Mikolov T. et al. Efficient Estimation of Word Representations in Vector Space, 2013; Pennington J. et al. GloVe: Global Vectors for Word Representation, 2014.

Чат-боты/генеративный ИИ

Векторные базы данных полезны, например, для классификации намерений, применения генерации с дополненной выборкой (Retrieval-Augmented Generation, RAG) и т. д.

Векторные базы данных, как и графовые, чаще всего используются для операционных приложений и поэтому прочно привязаны к операционной плоскости. Теперь, возвращаясь к рис. 11.1, мы задаемся вопросом: как векторные базы данных переходят в пересечение операционной и потоковой плоскостей?

Все более распространенной становится реализация потокового поглощения данных в векторные БД, например, использование Kafka Connect для таких векторных баз данных, как Pinecone и Milvus. Но есть и другие точки соприкосновения векторных БД и потоковой обработки. Например, СУБД Milvus 2.x была полностью переработана после версий Milvus 1.x и даже включала в себя потоковую платформу. В то же время ряд баз данных RTOLAP, таких как ClickHouse и Rockset, все чаще добавляет поддержку векторного поиска.

Milvus 2.x — потоковая передача как основа

По сравнению с первыми, довольно монолитными single-binary-версиями система Milvus 2.x прошла процедуру полного архитектурного редизайна. Ее создали с учетом горизонтальной масштабируемости и подготовили к работе в облаке. Очень интересным аспектом новой архитектуры стало добавление хранилища сообщений или брокера журналов. Этот инструмент отвечает за хранение потоковых данных, выполнение надежных асинхронных запросов, уведомление о событиях и возврат результатов запросов. Он также обеспечивает целостность инкрементных данных при восстановлении узлов обработчика после сбоя системы. Milvus придерживается принципа «журнал как данные». В этой системе ведется физическая таблица, но надежность данных гарантирована благодаря постоянному ведению журнала и моментальным снимкам журналов (рис. 11.3).

Система Milvus 2.x может похвастаться также унифицированной пакетной и потоковой обработкой. Это стало возможным благодаря реализации лямбда-архитектуры с интеграцией обработки инкрементных и накопленных данных. В Milvus применяется новый механизм водяных знаков для разбиения неограниченных данных (потока) на ограниченные окна. Система разделяет потоковые данные на несколько пакетов сообщений по времени записи или события и ведет временную шкалу, чтобы пользователи могли запрашивать данные по времени.

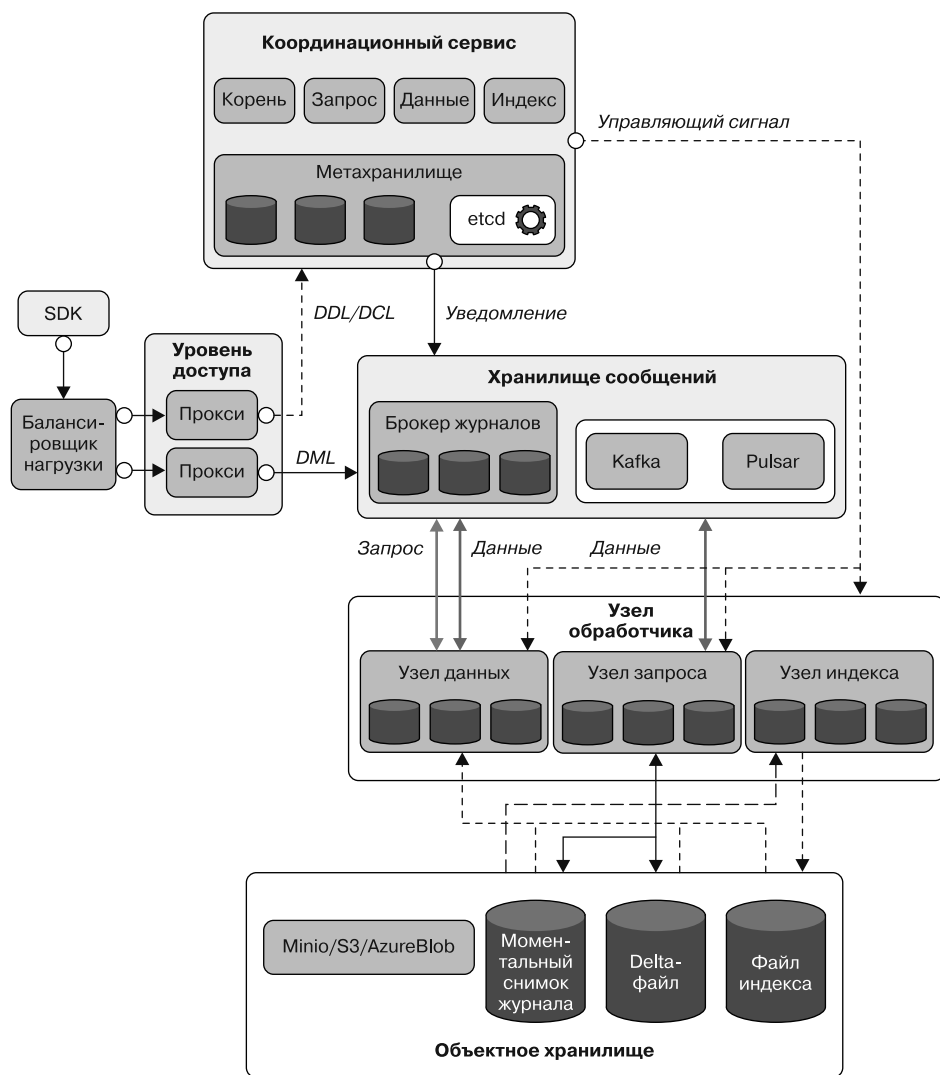


Рис. 11.3. Архитектура Milvus 2.x с потоковой обработкой (хранилищем сообщений) в качестве основы

Совокупность функционала и возможностей Milvus 2.x показывает, как дизайн новых баз данных может глубоко проникнуть в потоковую плоскость. В архитектуре Milvus 2.x потоковая передача/публикация-подписка стала основой всей системы. Она гарантирует масштабируемость и устойчивость векторной базы данных. Будет интересно увидеть, как другие векторные БД последуют этому примеру в ближайшем будущем.

Базы данных RTOLAP — добавление векторного поиска

Все больше поставщиков баз данных добавляют функционал векторного поиска в свои действующие продукты. В качестве примера можно привести MongoDB, Elastic и SingleStore. Этой тенденции следуют и базы данных RTOLAP, такие как ClickHouse и Rockset.

ClickHouse

В ClickHouse применяется тип массива столбцов (`Array<Float32>`) для моделирования встраивания векторов. Эта БД также предоставляет функции для вычисления расстояния между вектором поиска и значениями столбца, такими как `cosineDistance` и `L2Distance`. Еще ClickHouse предлагает специально оптимизированные алгоритмы поиска ANN, такие как индексы Annoy (Approximate Nearest Neighbors Oh Yeah).

В отличие от специализированных векторных БД, ClickHouse и другие БД RTOLAP сочетают векторный поиск с дополнительной фильтрацией или агрегированием метаданных. В качестве примера можно привести векторный поиск по изображениям без защиты авторских прав, где он сочетается с фильтрацией на основе метаданных об авторских правах.

SQL-запрос ClickHouse, объединяющий векторный поиск и фильтрацию метаданных, приведен в примере 11.7.

Пример 11.7. Сочетание векторного поиска и фильтрации метаданных в ClickHouse

```
SELECT
    url,
    caption,
    L2Distance(image_embedding, [<embedding>]) AS score
FROM laion_100m
WHERE (width >= 300) AND (height >= 500) AND (copyright = '') AND similarity > 0.3
ORDER BY score ASC
LIMIT 10
FORMAT Vertical
```

Как видите, ClickHouse позволяет легко сочетать фильтрацию метаданных (здесь — ширина и высота искомых изображений, а также метаданные об авторских правах) с векторным поиском (с использованием функции `L2Distance`).

Rockset

Rockset — еще одна база данных RTOLAP с поддержкой векторного поиска — более простого варианта поиска К ближайших соседей (K-nearest neighbors, KNN) и ANN. В Rockset индекс ANN можно настроить, как показано в примере 11.8.

Пример 11.8. Настройка индекса ANN в Rockset

```
CREATE SIMILARITY INDEX book_catalog_embeddings_ann_index
ON FIELD commons.book_dataset:book_embedding DIMENSION 1536 as
'faiss::IVF256,Flat';
```

Здесь создается индекс `book_catalog_embeddings_ann_index`. Мы указываем размерность входных векторов и тип создаваемого индекса — в данном случае инвертированный индекс файла (Inverted File Index, IVF) из библиотеки Faiss.

Запрос к Rockset с использованием этого индекса показан в примере 11.9.

Пример 11.9. Использование векторного поиска для создания запросов в Rockset

```
SELECT
    book_dataset.title,
    book_dataset.author
FROM
    book_dataset ds
JOIN book_metadata m ON ds.isbn = m.isbn
WHERE
    m.publish_date > DATE(2010, 12, 26)
    and m.rating >= 4
    and m.price < 50
ORDER BY
    APPROX_DOT_PRODUCT(:target_embedding, ds.book_embedding) DESC
LIMIT
    30
```

Самое интересное здесь находится в выражении `ORDER BY`, где поиск по сходству используется для упорядочивания результатов запроса на основе сходства их содержимого.

Инкрементное обслуживание представлений (IVM)

С помощью IVM материализованные представления в реляционной базе данных поддерживаются в актуальном состоянии инкрементным способом. В этом случае изменения постоянно вычисляются и применяются к представлениям, содержимое представлений не пересчитывается полностью. IVM может обновлять материализованные представления более эффективно, чем повторное вычисление, поскольку за один раз изменяются только небольшие части представления.

Существуют два варианта выбора времени обслуживания представления: немедленное и отложенное. При использовании *немедленного обслуживания*

представления обновляются в транзакции изменения их базовой таблицы. При *отложенном* варианте представления обновляются после фиксации транзакции. Например, это происходит в ответ на прямую команду `REFRESH MATERIALIZED VIEW` от пользователя. Она будет выполняться периодически в фоновом режиме или только при обращении к представлению.

IVM в немедленной форме обслуживания — это еще одна дорога, пересекающая операционную и потоковую плоскости. За последние несколько лет в этой области появилось множество решений, например `pg_ivm` (модуль для PostgreSQL), Hydra, PeerDB, Epsio, а также в некоторой степени Feldera/DBSP.

pg_ivm

Модуль `pg_ivm` добавляет в PostgreSQL материализованные представления с немедленным обслуживанием. Материализованные представления немедленно обновляются в триггерах `AFTER` при изменении базовой таблицы. Рассмотрим пример 11.10.

Пример 11.10. Создание немедленно обслуживаемого материализованного представления в PostgreSQL с помощью `pg_ivm`

```
SELECT create_immv('myview', 'SELECT * FROM mytab');
```

Преимущество модуля `pg_ivm` в том, что он идеально интегрирован с базовой БД, поэтому нет необходимости создавать и поддерживать дополнительную внешнюю систему.

В то же время тесная связь `pg_ivm` с оперативной БД (PostgreSQL) также означает, что ему приходится делить ресурсы (вычислительные, память) с самой БД. Поскольку инкрементное обслуживание материализованных представлений может быть дорогостоящим, разумнее вынести его в отдельную систему. В тех случаях, когда формат столбцовых таблиц будет более полезен, таблица PostgreSQL на основе строк может стать одной из причин возникновения проблемы с производительностью.

Hydra

База данных Hydra поддерживает традиционные материализованные представления. В ней применяются таблицы со строками и столбцами, а также инкрементные материализованные представления на основе модуля `pg_ivm`. В примере 11.11 показано, как настроить в Hydra материализованные представления на основе строк и столбцов.

Пример 11.11. Создание немедленно обслуживаемого материализованного представления в Hydra с помощью `pg_ivm`

```
CREATE TABLE heap_table (...) USING heap;  
CREATE TABLE columnar_table (...) USING columnar;
```

Дополнительная гибкость Hydra означает, что эту систему можно использовать в сценариях, требующих строчной структуры таблиц, и в сценариях со столбцовым хранением данных. Можно также создать внешнюю таблицу в PostgreSQL и синхронизировать ее с Hydra с имитацией подхода zero-ETL.

Epsio

Epsio — это инструмент для инкрементного обслуживания материализованных представлений. Он подключается к существующим БД PostgreSQL и постоянно и инкрементно обновляет результаты запросов при изменении базовых данных. Epsio работает в инкрементном режиме и никогда не пересчитывает весь набор данных, что позволяет ему мгновенно предоставлять актуальные результаты по сложным запросам. Epsio поддерживает большой набор синтаксиса SQL, включая большинство типов операций JOIN, общих табличных выражений (СТЕ), подзапросов, операций GROUP BY и т. д.

Как Epsio работает на практике? В примере 11.12 определяется простое материализованное представление Epsio.

Пример 11.12. Определение материализованного представления в Epsio

```
CALL epsio.create_view('epsio_view',  
  'SELECT SUM(SALARY), d.name FROM employee_salaries e  
    JOIN departments d on e.department_id = d.id  
    GROUP BY d.name');
```

В примере 11.13 показано, как работает запрос к материализованному представлению (точно так же, как это происходит непосредственно в базе данных PostgreSQL).

Пример 11.13. Запрос к материализованному представлению в Epsio

```
SELECT * FROM epsio_view;
```

Существует несколько преимуществ использования отдельной системы, такой как Epsio, вместо прямого обращения к PostgreSQL.

- Материализованные представления в Epsio постоянно инкрементно обновляются, так что нет необходимости пересчитывать весь набор данных.
- На производительность исходной базы данных не влияют обслуживание ее собственных материализованных представлений или непрерывные запросы.

Feldera

Feldera — это платформа непрерывной аналитики. Она дает пользователям возможность выполнять непрерывные запросы непосредственно на потоковых данных. По сути, она обрабатывает запросы и выдает результаты непрерывно — при каждом изменении инкрементно пересчитывает результаты запроса и только потом отправляет измененные результаты на свои выходы.

В некотором смысле это платформа ETL, работающая в режиме реального времени на основе потоковой обработки данных. Она находится под управлением движка DBSP (Database Stream Processor). Во многих аспектах Feldera пересекается с потоковыми БД, такими как Materialize или RisingWave, но с акцентом на передачу данных от источников к целям и без поддержки материализованных представлений. Ее также можно сравнить с системой чисто потоковой обработки, такой как Flink, но с дополнительными гарантиями согласованности, как, например, в Materialize. DBSP во многом похож на базовый механизм потоковой обработки Materialize — дифференциальный поток данных (DD).

Рассмотрим работу платформы Feldera на практике. Сначала в примере 11.14 объявляется несколько таблиц ввода.

Пример 11.14. Объявление таблиц ввода в Feldera

```
create table VENDOR (  
    id bigint not null primary key,  
    name varchar,  
    address varchar  
);  
  
create table PART (  
    id bigint not null primary key,  
    name varchar  
);  
  
create table PRICE (  
    part bigint not null,  
    vendor bigint not null,  
    price decimal  
);
```

В объявлениях не указываются конкретные источники данных. Записи для таблиц VENDOR, PART и PRICE могут поступать из потока Kafka, базы данных или HTTP-запроса. Позже SQL-программа может быть инстанцирована источником данных (или даже несколькими) в пользовательском интерфейсе Feldera.

На втором этапе можно писать непрерывные запросы к исходным данным. Это показано в примере 11.15.

Пример 11.15. Написание непрерывных запросов в Feldera

```
-- Самая низкая цена на каждую деталь у всех поставщиков.
create view LOW_PRICE (
    part,
    price
) as
    select part, MIN(price) as price from PRICE group by part;

-- Самая низкая доступная цена каждой детали, а также сведения о детали и поставщике.
create view PREFERRED_VENDOR (
    part_id,
    part_name,
    vendor_id,
    vendor_name,
    price
) as
    select
        PART.id as part_id,
        PART.name as part_name,
        VENDOR.id as vendor_id,
        VENDOR.name as vendor_name,
        PRICE.price
    from
        PRICE,
        PART,
        VENDOR,
        LOW_PRICE
    where
        PRICE.price = LOW_PRICE.price AND
        PRICE.part = LOW_PRICE.part AND
        PART.id = PRICE.part AND
        VENDOR.id = PRICE.vendor;
```

В Feldera запросы записываются в виде представлений SQL. Их можно определить в терминах таблиц и других представлений. Представление `PREFERRED_VENDOR` в примере 11.15 задано в терминах представления `LOW_PRICE`. Такая гибкость позволяет Feldera выражать даже глубоко вложенные запросы.

PeerDB

PeerDB — это инструмент с открытым исходным кодом для потоковой передачи данных из PostgreSQL в хранилища данных (Snowflake, BigQuery), очереди (Azure Event Hubs) и системы хранения (S3, GCS). Как и Feldera, PeerDB не предлагает обслуживание материализованных представлений напрямую. Но добиться аналогичного эффекта можно путем синхронизации базы данных PostgreSQL в режиме реального времени, например, со Snowflake. И тогда материализованное представление размещается в хранилище Snowflake.

У PeerDB есть четыре режима репликации:

- на основе журналов;
- на основе курсора (временной метки или целого числа);
- на основе режима XMIN;
- потоковый запрос.

PeerDB в режимах на основе журнала, курсора и режима XMIN получает изменения из таблиц PostgreSQL (исходный пир) и переносит их в целевой пир. В отличие от аналогичных инструментов CDC, PeerDB заявляет о своей гораздо большей производительности (быстрее в 10 раз). PeerDB также поддерживает репликацию потоковых запросов для более сложных требований к репликации. В этом режиме можно выполнять сложные преобразования данных исходного пира перед отправкой их целевому.

Рассмотрим пример работы PeerDB. Первым шагом станет создание исходного и целевого пилов (пример 11.16).

Пример 11.16. Создание исходного и целевого пилов в PeerDB

```
CREATE PEER source FROM POSTGRES WITH
(
    host = 'catalog',
    port = '5432',
    user = 'postgres',
    password = 'postgres',
    database = 'source'
);

CREATE PEER target FROM POSTGRES WITH
(
    host = 'catalog',
    port = '5432',
    user = 'postgres',
    password = 'postgres',
    database = 'target'
);
```

В примере 11.17 выполняется запуск CDC на основе журналов с помощью создания зеркала.

Пример 11.17. Настройка CDC на основе журналов в PeerDB

```
CREATE MIRROR cdc_mirror FROM source TO target
WITH TABLE MAPPING (public.test:public.test);
```

Это зеркало теперь обеспечивает репликацию всех DML-команд (INSERT, UPDATE, DELETE) из источника в точку назначения и использует сопоставление таблиц.

Настройка потокового запроса немного сложнее (пример 11.18).

Пример 11.18. Настройка потокового запроса в PeerDB

```
CREATE MIRROR qrep_mirror FROM source TO target
FOR $$
    SELECT id, hashint4(c1) hash_c1, hashint4(c2) hash_c2, md5(t) AS hash_t
    FROM test WHERE id BETWEEN {{.start}} AND {{.end}}
$$ WITH (
    watermark_table_name='public.test',
    watermark_column='id',
    num_rows_per_partition = 10000,
    destination_table_name='public.test_transformed',
    mode='append'
);
```

Потоковый запрос маскирует столбцы `c1`, `c2` и `t` с помощью хеширования перед отправкой в точку назначения.

Обертывание данных и Postgres Multicorn

В главе 7 мы рассказывали о повсеместном распространении системы PostgreSQL. Этот процесс объясняется несколькими факторами, такими как открытый исходный код и наличие сообщества разработчиков. Кроме того, одним из ключевых факторов выступает исключительная расширяемость, например добавление модуля `pg_ivm`. Это означает, что СУБД можно настраивать и расширять за пределы основной функциональности системы, добавляя следующие возможности.

Обертка FDW

Обеспечивает плавную интеграцию с внешними источниками данных, превращаясь в центральный узел разнообразных данных.

Расширение Multicorn

Упрощает создание пользовательских оберток FDW на языке Python, что еще больше снижает барьер для интеграции данных. Этот функционал в сочетании с развитым сообществом, способствующим разработке расширений, дает возможность пользователям адаптировать СУБД PostgreSQL к конкретным потребностям. Все это в сумме делает систему ценным инструментом для многих областей применения.

Обертки FDW в PostgreSQL позволяют получать доступ к данным, находящимся вне системы, как если бы это была обычная таблица в базе данных. FDW обеспечивают возможность объединять данные с помощью привычных запросов SQL из различных источников, таких как другие базы данных, файлы, API, потоковые платформы и базы данных на основе столбцов.

Multicorn — это расширение Postgres для создания пользовательских FDW с помощью языка Python. Оно обеспечивает удобную для пользователя структуру, абстрагируясь от сложных низкоуровневых деталей реализации FDW. При использовании Multicorn можно сосредоточиться на специфической логике взаимодействия с внешним источником данных, оставив взаимодействие с Postgres фреймворку. FDW и Multicorn совместно обеспечивают гибкую интеграцию данных и оптимизацию разработки для работы с различными источниками данных в среде PostgreSQL.

Обсудим процесс реализации аналитики в режиме реального времени с помощью PostgreSQL, расширения Multicorn и базы данных OLAP.

1. Установка PostgreSQL и Multicorn.

- А. Установка PostgreSQL. Убедитесь, что в вашей системе установлена и работает СУБД PostgreSQL. Инструкции по установке можно найти на официальном сайте PostgreSQL (<https://postgresql.org>).
- Б. Включение расширения Multicorn. После того как PostgreSQL настроен, необходимо включить расширение Multicorn с помощью следующей команды:

```
CREATE EXTENSION IF NOT EXISTS multicorn;
```

2. Реализация конвейера поглощения данных.

- А. Установка соединения. Создайте соединение между своим приложением и БД PostgreSQL с помощью соответствующей библиотеки, например `psycopg2` для Python или `pg` для Node.js.
- Б. Получение данных в режиме реального времени. Реализуйте в своем приложении логику для сбора данных в режиме реального времени. Этот процесс может включать в себя:
 - потоковую передачу данных. Используйте библиотеки/фреймворки поглощения данных в режиме реального времени из различных источников, например Apache Kafka или очереди сообщений RabbitMQ;
 - протоколы WebSocket/Server-Sent Event (SSE). Устанавливайте соединения с клиентами в режиме реального времени и получайте обновления данных через протоколы WebSocket или SSE;
 - вызовы API. Если данные поступают из внешних API, периодически выполняйте вызовы API для получения и обновления базы данных.

3. Очередь, обработка и хранение данных.

- А. Очередь данных. Рассмотрите возможность использования временных таблиц или очередей в базе данных PostgreSQL для буферизации поступающих данных перед обработкой.

- Б. Обработка в in-мемогу-хранилище. Для ускорения обработки данных в режиме реального времени изучите in-мемогу-базы данных или материализованные представления в PostgreSQL. Однако для больших массивов данных это может оказаться неприемлемым.
 - В. Преобразование и хранение данных. В своем приложении или с помощью триггеров в PostgreSQL преобразуйте полученные данные в нужный формат и сохраните их в соответствующих таблицах. Это могут быть таблицы измерений для хранения статических атрибутов и таблицы фактов для хранения показателей/метрик.
4. Подключение к базе данных OLAP.
- А. Выбор OLAP-решения. Выберите БД OLAP, например Apache Druid, Apache Kylin или ClickHouse, соответствующую вашим потребностям и объему данных.
 - Б. Установка соединения. Используйте предоставленные библиотеки/коннекторы из выбранного вами OLAP-решения для подключения к базе данных из приложения.
 - В. Периодическая передача данных. Настройте запланированные задачи или используйте триггеры в PostgreSQL для периодической передачи обработанных данных из вашей БД PostgreSQL в БД OLAP. Благодаря этому база данных OLAP постоянно обновляется самыми свежими данными для эффективных аналитических запросов.

Не забудьте учесть и следующие аспекты.

Обработка ошибок и ведение журнала

Внедрите надлежащие механизмы обработки ошибок для обеспечения согласованности и качества данных на протяжении всего конвейера обработки.

Мониторинг и масштабирование

Следите за производительностью системы и БД OLAP по мере увеличения объема данных. При необходимости масштабируйте инфраструктуру, чтобы справиться с потоком данных в режиме реального времени.

Безопасность

Обеспечьте безопасность конвейера данных в критически важных точках с помощью механизмов аутентификации, авторизации и шифрования.

Помните, что это общие рекомендации, а конкретные детали реализации будут зависеть от выбранных вами инструментов, технологий и специфических требований к аналитике в режиме реального времени.

Классические базы данных

Постоянной темой этой главы и всей книги стало продолжающееся слияние потоковой обработки и баз данных. Другими словами, мы обсуждаем унификацию данных в движении с данными в состоянии покоя. Одним из последних показателей этого слияния будет новая функциональная возможность *Atlas Stream Processing* из уже классической NoSQL-базы данных MongoDB. Инструмент Atlas Stream Processing дополняет MongoDB возможностями обработки потоков. Теперь система приблизилась к сущности потоковой БД. На диаграмме Венна (см. рис. 11.1) точка слияния находится на пересечении операционной и потоковой плоскостей — именно здесь и располагаются потоковые БД, такие как Materialize и RisingWave.

Atlas Stream Processing добавляет в MongoDB функции обработки потоков, способные считывать данные с потоковых платформ, таких как Kafka, и записывать обработанные данные обратно на потоковую платформу. Кроме того, Atlas Stream Processing позволяет использовать коллекции MongoDB в качестве материализованных представлений обрабатываемых данных. Примерная архитектурная схема представлена на рис. 11.4.

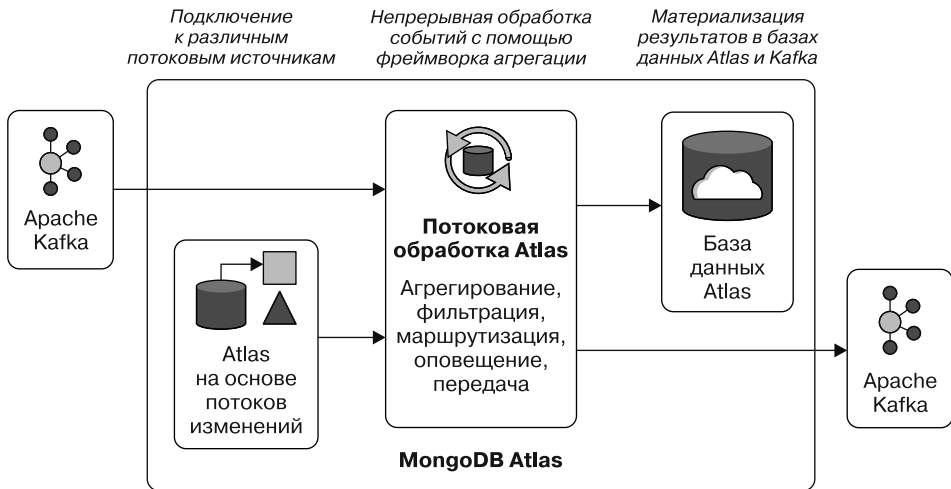


Рис. 11.4. Архитектура потоковой обработки MongoDB Atlas

Поскольку система Atlas Stream Processing встроена в MongoDB, она обеспечивает плавную интеграцию использования данных в движении (посредством обработки потоков) и данных в состоянии покоя (как и прежде, с помощью коллекций MongoDB). Схожесть сообщений (обычно в формате JSON) и документов

MongoDB (также в формате, похожем на JSON) делает интеграцию еще более плавной. И гораздо более плавной, чем при интеграции потоковой обработки с реляционной моделью, как в потоковых БД на основе SQL из главы 5. Однако Atlas Stream Processing все еще находится на ранней стадии развития, о чем можно судить по матрице возможностей, размещенной на домашней странице MongoDB. Например, не поддерживаются никакие операции JOIN. Пока неясно, как этот инструмент войдет на рынок потоковой обработки данных, где уже существуют такие признанные решения, как Flink. И как он сможет стать прямым конкурентом таких потоковых баз данных, как Materialize и RisingWave.

Рассмотрим Atlas Stream Processing на практике. JavaScript-код с API MongoDB Query из примера 11.19 демонстрирует, как запросить топик Kafka и передать результаты в коллекцию MongoDB.

Пример 11.19. Запрос топика Kafka и передача результатов в коллекцию MongoDB с помощью Atlas Stream Processing

```
// Определение источника из реестра соединений
var source = { $source: {
  connectionName: 'kafkaprod',
  topic: 'stocks'
} }

// Создание нескольких других этапов
var match = { $match: { 'exchange': 'NYSE' } }

// Создание точки назначения
var sink = { $merge: {
  into: {
    connectionName: 'mongoprod',
    db: 'StockDB',
    col: 'TransactionHistory'
  }
} }

// Попытка запуска
var myProcessor = [source, match, sink]
sp.process(myProcessor)
```

Здесь уровень `source` подключается к топикам Kafka `stocks`, уровень `match` сопоставляется только с теми сообщениями Kafka, у которых поле `exchange` совпадает со строкой `NYSE`, а уровень `sink` переносит совпавшие сообщения в материализованное представление, то есть в коллекцию MongoDB с названием `TransactionHistory`. Последняя строка, начинающаяся с выражения `sp.process`, фактически запускает потоковый процессор.

В другом примере показано использование Atlas Stream Processing для оконных агрегаций (пример 11.20).

Пример 11.20. Выполнение оконной агрегации в Atlas Stream Processing

```
// Определение переворачивающегося окна
{
  $tumblingWindow: {
    interval: {
      size: NumberInt(60), unit: 'second'},
    pipeline: [{
      $group: {
        _id: '$ip_source',
        count_connection_reset: { $sum: 1 }
      }
    }]
  }
},

// Результат представлен в упрощенном виде
{
  _id: '127.0.0.1',
  count_connection_reset: 60,
  _stream_meta: {
    sourceType: 'kafka',
    windowStartTimestamp: 2023-05-18T17:07:00.000Z,
    windowEndTimestamp: 2023-05-18T17:08:00.000Z
  }
}
```

Во фрагменте кода из примера 11.20 сначала определяется переворачивающееся окно в 60 с. Оно используется для подсчета сообщений в топике Kafka.

Хранилища данных

Потоковая передача данных проникает не только в базы, но и в хранилища данных. В этом разделе мы рассмотрим трех основных игроков: BigQuery, Redshift и Snowflake. Они расположены в пересекающейся области между аналитической и потоковой плоскостями на диаграмме Венна (см. рис. 11.1).

BigQuery

Сервис BigQuery компании Google поддерживает потоковое поглощение через свой набор SDK. Как и в случае с Redshift, это позволяет BigQuery обеспечивать аналитику, работающую в режиме, близком к работе в реальном времени, где низкая задержка имеет первостепенное значение. Данные, поступающие в BigQuery через потоковое поглощение, становятся доступными для анализа в режиме реального времени уже через несколько секунд после первой потоковой вставки в таблицу.

В отличие от Redshift, в BigQuery потоковое поглощение реализовано на основе SDK для Java и Python, а не SQL. В примере 11.21 показан код Python для вставки табличных данных с описанным подходом.

Пример 11.21. Потоковое поглощение (вставка строк) в BigQuery с помощью SDK на основе Python

```
def stream_data(dataset_name, table_name, json_data):
    bigquery_client = bigquery.Client()
    dataset = bigquery_client.dataset(dataset_name)
    table = dataset.table(table_name)
    data = json.loads(json_data)

    # Обновление таблицы для получения схемы.
    table.reload()

    rows = [data]
    errors = table.insert_data(rows)

    if not errors:
        print('Loaded 1 row into {}:{}'.format(dataset_name, table_name))
    else:
        print('Errors:')
        pprint(errors)
```

Google Cloud Platform (GCP) также предлагает интеграцию BigQuery со своим продуктом Cloud Dataflow на основе Apache Beam. Данные можно записать в BigQuery с помощью Cloud Dataflow, а данные из BigQuery — передать в Cloud Dataflow. Кроме того, недавно компания Google анонсировала сервис Apache Kafka for BigQuery. С его помощью будет легко развернуть Apache Kafka в связке с BigQuery бессерверным способом.

Redshift

Сервис Redshift способен поглощать данные с низкой задержкой из Kinesis или Kafka (Amazon Managed Service for Apache Kafka, или Amazon MSK) и помещать их в материализованные представления. Последние можно настроить с помощью операторов SQL, и они станут *посадочной площадкой* для данных из входного потока. Данные обрабатываются по мере поступления — например, значения JSON из потоков данных Kinesis или топикив Kafka могут быть обработаны и сопоставлены со столбцами данных материализованных представлений.

Эта функция позволяет Redshift использовать аналитику в режиме, близком к работе в реальном времени. В этом случае данные, поступающие непрерывным потоком, должны быть обработаны в течение короткого времени после их создания. В качестве примера можно привести устройства Интернета вещей (IoT), данные телеметрии системы или потоки цифрового следа. Непрямое

получение данных от потоковой платформы с помощью сервиса Kinesis Data Firehose выполняется с предварительным размещением их в хранилище S3. А прямое поглощение потоковых данных приводит к снижению сложности и уменьшению задержки.

Рассмотрим работу потокового поглощения в Redshift на практике. Самое главное — настроить материализованное представление для получения данных, например, из топика Kafka (пример 11.22).

Пример 11.22. Создание материализованного представления для получения данных из топика Kafka в Redshift

```
CREATE MATERIALIZED VIEW MyView AUTO REFRESH YES AS
SELECT
    kafka_partition,
    kafka_offset,
    kafka_timestamp_type,
    kafka_timestamp,
    kafka_key,
    JSON_PARSE(kafka_value) as Data,
    kafka_headers
FROM
    MySchema."mytopic"
WHERE
    CAN_JSON_PARSE(kafka_value);
```

Материализованные представления и потоковое поглощение в Redshift можно сравнить с материализованными представлениями в потоковых БД, таких как Materialize и RisingWave, особенно при использовании операции `AUTO REFRESH` вместо `MANUAL REFRESH`. Но поскольку базовая архитектура Redshift построена не на потоковой обработке, говорить о полном совпадении технологий нельзя. В отличие от потоковых БД, здесь обновление происходит периодически, а не постоянно, что увеличивает задержку Redshift по сравнению с потоковой БД, работающей в полностью инкрементном режиме.

Snowflake

В последнее время компания Snowflake вкладывает много средств в создание функциональных возможностей для потоковой обработки. В результате платформа Snowflake теперь предоставляет следующие возможности на основе потоковой передачи данных.

Непрерывная загрузка данных

Речь идет о потоковом поглощении данных, аналогичном Redshift и BigQuery. Для этого Snowflake предоставляет инструменты Snowpipe Streaming и Snowflake Connector for Kafka Connect.

Непрерывное преобразование данных

В Snowflake появилась функция динамических таблиц для декларативной реализации автоматизированных конвейеров данных. Они упрощают поглощение данных в сочетании с их обработкой.

Отслеживание изменений данных

Эта функция реализует CDC для обычных таблиц Snowflake и динамических таблиц.

Сначала внимательнее рассмотрим Snowpipe Streaming для потокового поглощения данных. Платформа Snowflake, как и BigQuery, предоставляет SDK (для Java) для реализации этого процесса. Таким образом, для осуществления потокового поглощения пользователю необходимо разработать Java-приложение. В примере 11.23 показан упрощенный пример того, как это выглядит на практике.

Пример 11.23. Потоковое поглощение (вставка строк) в Snowflake с помощью SDK на базе Java

```
[package, imports]
```

```
public class SnowflakeStreamingIngestExample {
    [setup]
    // Вставка строк в канал (с помощью API insertRows)
    final int totalRowsInTable = 1000;
    for (int val = 0; val < totalRowsInTable; val++) {
        Map<String, Object> row = new HashMap<>();

        // c1 соответствует имени столбца в таблице
        row.put("c1", val);

        // Вставка строки с текущим токеном смещения
        InsertValidationResponse response = channel1.insertRow(row,
            String.valueOf(val));
        if (response.hasErrors()) {
            // Если исключение возникло, оно будет просто выброшено
            // или с ошибочной строкой можно сделать все что угодно
            throw response.getInsertErrors().get(0).getException();
        }
    }

    // При необходимости можно проверить токен смещения,
    // зарегистрированный в Snowflake, чтобы убедиться, что все зафиксировано
    final int expectedOffsetTokenInSnowflake = totalRowsInTable - 1;
    // Токен смещения на основе 0
    final int maxRetries = 10;
    int retryCount = 0;
```

```

do {
    String offsetTokenFromSnowflake = channel1.getLatestCommittedOffsetToken();
    if (offsetTokenFromSnowflake != null
        && offsetTokenFromSnowflake.equals(String.valueOf
            (expectedOffsetTokenInSnowflake))) {
        System.out.println("SUCCESSFULLY inserted " + totalRowsInTable + " rows");
        break;
    }
    retryCount++;
} while (retryCount < maxRetries);

[close]
}
}
}

```

В Snowflake появились динамические таблицы для декларативного построения конвейеров поглощения данных с помощью SQL-кода. Динамические таблицы во многом похожи на материализованные представления потокового поглощения в Redshift или потоковых БД, таких как Materialize и RisingWave. И здесь, как и в случае с Redshift, базовая архитектура сформирована на основе пакетной, а не потоковой обработки данных. Таким образом, даже автоматическое обновление динамических таблиц все равно происходит периодически, а не строго непрерывно, как в потоковых БД. И это приводит к гораздо более высоким задержкам.

Lakehouse

Сейчас наблюдается тенденция быстрого роста количества потоковых механизмов обработки (движков) и систем потоковой обработки данных. Они предлагают более эффективную поддержку озер данных/хранилищ Lakehouse. Потоковые механизмы, такие как Confluent, Redpanda и WarpStream, переходят к предоставлению потоковых данных непосредственно из своего «холодного» слоя объектного хранилища. Для этого в них используются открытые форматы таблиц, такие как Apache Iceberg. Разработки Kafka с открытым исходным кодом тоже движутся в этом направлении (<https://oreil.ly/nEBfP>). А детище Apache Flink с названием Apache Paimon переносит потоковую обработку на основе Flink в озеро данных — так называемое хранилище Streamhouse. Компания Confluent уже начала предлагать поддержку Iceberg непосредственно на собственной потоковой платформе Confluent Cloud/Kora. Ее специалисты также ввели термин «*мультимодальный поток*», чтобы описать способность платформы предлагать данные как в потоковом, так и в табличном формате. Благодаря этой функциональной возможности, названной Tableflow, Flink можно использовать для

доступа как к потоковым, так и к табличным/пакетным данным. С данными на уровне Delta Lake, еще одном открытом формате таблиц, тоже можно работать с помощью потоковой обработки (Spark Structured Streaming).

В текущем разделе мы продолжим исследовать эту форму слияния аналитической и потоковой плоскостей (см. диаграмму Венна на рис. 11.1) и закончим обсуждением настоящего и будущего взаимоотношений потоковых технологий и хранилища Lakehouse.

Delta Lake

Как и два других наиболее популярных открытых формата таблиц, Apache Iceberg и Apache Hudi, Delta Lake основан на формате столбцовых файлов Parquet. В дополнение к файлам Parquet он предлагает журнал транзакций на основе файлов для реализации транзакций с требованием ACID и масштабируемой обработки метаданных. Поскольку этот уровень совместим с API Apache Spark (Spark и Delta Lake по большей части разработаны компанией Databricks), Delta Lake также тесно интегрирован с API Spark Structured Streaming для потоковой и пакетной обработки данных в больших масштабах. Обратите внимание на то, что Spark Structured Streaming использует микропакеты, поэтому, строго говоря, это не чистая потоковая обработка.

Открытый формат таблиц в основе Delta Lake носит название Delta Table. Такие таблицы могут служить как источниками, так и точками назначения, а в сочетании с API Spark Structured Streaming можно реализовать потоковую обработку на уровне Delta Lake. В примере 11.24 показан простой вариант реализации.

Пример 11.24. Использование Delta Lake (таблиц Delta Table) в качестве источника с помощью API Spark Structured Streaming

```
spark.readStream.format("delta")
    .load("/tmp/delta/events")

import io.delta.implicits._
spark.readStream.delta("/tmp/delta/events")
```

В этом примере таблица Delta Table `/tmp/delta/events` считывается как поток с помощью API Spark Structured Streaming.

Теперь можно добавлять запросы с произвольной логикой обработки потоков, и запрос будет обрабатывать все данные, находящиеся в таблице и поступающие в дальнейшем.

Аналогично работает запись обработанных данных из Spark Structured Streaming в Delta Table (пример 11.25).

Пример 11.25. Использование Delta Lake (таблиц Delta Table) в качестве точки назначения API Spark Structured Streaming

```
events.writeStream
    .outputMode("append")
    .option("checkpointLocation", "/tmp/delta/events/_checkpoints/")
    .toTable("events")
```

Apache Paimon

Apache Paimon стремится быть похожим на Delta Lake. Но вместо использования микропакетов, как в Delta Lake/Spark Structured Streaming, он построен на чистой обработке потоков на базе Apache Flink. Главный поставщик Paimon, компания Ververica, назвал свое решение *Streamhouse* (в противоположность Lakehouse).

Paimon не использует Parquet в качестве основного формата столбцовых данных, как делают Delta Lake, Iceberg и Hudi. Вместо этого файлы создаются в собственном формате на основе структуры LSM-дерева (Log-Structured Merge). Paimon поддерживает чтение этих файлов системой Flink и другими вычислительными системами, такими как Apache Hive, Apache Spark и Trino. Платформа Paimon создана на базе потоковой архитектуры, но поддерживает и пакетный режим чтения и записи данных.

В примере 11.26 показано, как создать таблицы `customers` и `Orders` в Paimon. Причем последняя основана на исходном топике Kafka (для краткости мы опустили настройку каталога).

Пример 11.26. Создание таблицы `customers` и временной таблицы `Orders` на основе топика Kafka в Apache Paimon

```
CREATE TABLE customers (
    id INT PRIMARY KEY NOT ENFORCED,
    name STRING,
    country STRING,
    zip STRING
);

INSERT INTO customers ...

CREATE TEMPORARY TABLE Orders (
    order_id INT,
    total INT,
    customer_id INT,
    proc_time AS PROCTIME()
) WITH (
    'connector' = 'kafka',
    'topic' = '...',
```

```

        'properties.bootstrap.servers' = '...',
        'format' = 'csv'
        ...
    );

```

Теперь к этой таблице можно обратиться, например, с помощью запроса поиска JOIN (пример 11.27).

Пример 11.27. Обращение к таблице Orders с помощью запроса поиска JOIN в Apache Paimon

```

SELECT o.order_id, o.total, c.country, c.zip
FROM Orders AS o
JOIN customers
FOR SYSTEM_TIME AS OF o.proc_time AS c
ON o.customer_id = c.id;

```

Apache Iceberg

Apache Iceberg — это очень популярный открытый формат таблиц. Он разработан в компании Netflix, а позже его поддерживали многие производители, такие как Spark, Flink, Presto, Trino, Hive, Impala, StarRocks, Snowflake, Doris и Pig. Поточковые платформы, такие как Redpanda, WarpStream и Kafka с открытым исходным кодом, также планируют добавить поддержку Iceberg/Parquet для прямого доступа к своим «холодным» данным через API Iceberg. В Confluent Cloud/Kora уже начали предоставлять топики Kafka в виде таблиц Iceberg с помощью платформы Tableflow.

Потоковые базы данных также следуют этой тенденции. В системе RisingWave уже предусмотрена поддержка Iceberg в качестве точки назначения. Это показано в примере 11.28, где данные точки назначения вставляются в таблицу Iceberg (RisingWave также поддерживает режим только добавления).

Пример 11.28. Передача таблицы s1_table в таблицу Apache Iceberg в RisingWave

```

CREATE SINK s1_sink FROM s1_table
WITH (
    connector = 'iceberg',
    warehouse.path = 's3a://my-iceberg-bucket/path/to/warehouse',
    s3.endpoint = 'https://s3.ap-southeast-1.amazonaws.com',
    s3.access.key = '${ACCESS_KEY}',
    s3.secret.key = '${SECRET_KEY}',
    database.name='dev',
    table.name='table',
    primary_key='seq_id'
);

```

Поскольку API Spark Structured Streaming тоже поддерживает Iceberg, в примере 11.29 представлен сценарий, в котором данные вносятся в таблицу Iceberg с помощью Python и Spark.

Пример 11.29. Использование Spark Structured Streaming для поглощения данных таблицей Apache Iceberg

```
df.writeStream \
    .format("iceberg") \
    .outputMode("append") \
    .trigger(processingTime=WINDOW_SIZE) \
    .option("path", table_id) \
    .option("fanout-enabled", "true") \
    .option("checkpointLocation", checkpointPath) \
    .start()
```

Таблицы Iceberg также могут служить источниками для API Spark Structured Streaming (пример 11.30).

Пример 11.30. Использование Spark Structured Streaming для считывания данных из таблицы Apache Iceberg

```
spark.readStream \
    .format("iceberg") \
    .load(basePath) \
    .start()
```

Apache Hudi

В компании Uber разработали еще один популярный открытый формат таблиц с названием Apache Hudi. Он также поддерживает ряд механизмов запросов, таких как Apache Spark, Apache Impala, Apache Hive, Presto и Trino. В отличие от Iceberg, в Hudi есть функционал *инкрементного запроса*. Он создан на основе поддержки потоков изменений на уровне записей. Этот механизм можно задействовать для переноса данных из Hudi, скажем, в Spark Structured Streaming (пример 11.31).

Пример 11.31. Использование Spark Structured Streaming для считывания данных из таблицы Apache Hudi

```
spark.readStream \
    .format("hudi") \
    .load(basePath) \
    .start()
```

Данные, скажем, из API Spark Structured Streaming тоже можно передать в Hudi, аналогично работе с Iceberg в примере 11.30. Код для сценария с Hudi показан в примере 11.32.

Пример 11.32. Использование Spark Structured Streaming для записи данных в таблицу Apache Hudi

```
df.writeStream
    .format("hudi") \
    .options(**hudi_streaming_options) \
    .outputMode("append") \
    .option("path", baseStreamingPath) \
    .option("checkpointLocation", checkpointLocation) \
    .trigger(once=True) \
    .start()
```

OneTable или XTable

OneTable или XTable¹ — это новый открытый формат таблиц. Его разработали с целью сделать различные открытые форматы таблиц совместимыми. В этом смысле это формат метаоткрытой таблицы. В настоящее время OneTable или XTable поддерживает Apache Iceberg, Apache Hudi и Delta Lake.

Этот инструмент особенно полезен для компаний, использующих одновременно несколько форматов открытых таблиц, таких как Iceberg и Delta Lake. XTable предоставляет уровень абстракции, позволяющий не копировать данные между различными хранилищами. Поддержка формата таблиц Apache Paimon тоже будет доступна в XTable, поскольку это инструмент с открытым исходным кодом.

Взаимоотношения потоковой обработки и хранилищ Lakehouse

Redpanda, WarpStream и решения Kafka с открытым исходным кодом (<https://oreil.ly/LkExc>) прокладывают путь к все более тесной интеграции потоковых платформ, потоковой обработки и хранилищ Lakehouse. Они предлагают доступ к «холодным» данным из многоуровневых хранилищ (или в случае WarpStream — из его единственного уровня) напрямую с использованием форматов столбцовых таблиц (например, Iceberg/Parquet) и разработке Streamhouse из Apache Paimon.

¹ С сентября 2024 года проект носит название Apache XTable. — *Примеч. пер.*

Интеграция хранилищ Lakehouse и потоковой обработки часто ограничивается потоковым поглощением данных. И пока что изменений в этой ситуации не наблюдается. Более тесная интеграция потоковых платформ и открытых форматов таблиц, которая намечается в ближайшем будущем, довольно сильно изменит такое положение вещей. Одним из возможных результатов развития этих решений станет возможность переноса данных, хранящихся в Lakehouse, на потоковые платформы. Такой шаг позволит избежать дополнительного копирования/поглощения с потоковой платформы в актуальное хранилище Lakehouse. Это также означает, что потоковые платформы, а не хранилища данных будут все чаще становиться единым источником истины для данных. Или же такие технологии, как Tableflow компании Confluent, могут предложить два API — один потоковый и событийный, другой пакетный и табличный — для доступа к одним и тем же недублированным данным непосредственно на потоковой платформе.

В целом многие сервисы данных SaaS идут по пути использования облачных объектных хранилищ в качестве хранилищ нижнего уровня или многоуровневых. При использовании многоуровневых хранилищ предприятия могут разгрузить более «холодные» данные, что способствует сокращению расходов на хранение. Многие экстраполировали эту идею и предложили не применять сам интерфейс SaaS, а использовать облачное объектное хранилище в качестве интерфейса для предоставления и хранения данных. Например, если ваше решение SaaS предоставляет базу данных и выгружает данные в объектное хранилище для дешевого «холодного» хранения, почему оно не может просто считывать данные непосредственно из этого хранилища? И точно так же база данных SaaS должна уметь считывать данные из облачного объектного хранилища. Почему бы тогда не использовать уже имеющиеся в облачном объектном хранилище данные для предоставления их клиентам базой данных SaaS?

Как все это отразится на ключевой технологии из книги — потоковых базах данных? Мы видим в этом серьезную возможность для их развития. Если потоковые платформы, такие как Kafka, будут все больше и больше превращаться в единый источник истины, конвейеры обработки данных, ранее выполнявшие только в хранилищах Lakehouse, также смогут переместиться на потоковые платформы.

Для всех сценариев с обработкой огромных объемов данных, особенно для операционных приложений с приоритетом низкой общей задержки, системы потоковой обработки и потоковые БД обеспечат гораздо более высокую производительность и огромную экономию средств по сравнению с такими пакетными системами, как хранилища данных (например, Snowflake) или Lakehouse (например, Databricks).

Резюме

Мы проделали долгий путь. Вы узнали, как на потоковые технологии повлиял ранее реализованный функционал баз данных. Поняли, что, вывернув базу данных наизнанку, можно масштабировать отдельные части и выйти за пределы возможностей одной базы данных. Самое главное, вы узнали, что возвращение этих технологий в БД помогло консолидировать инфраструктуру и упростить интерфейс для инженеров без потери масштабируемости, полученной от декомпозиции БД.

Мы наблюдаем множество признаков продолжающегося слияния потоковых и пакетных технологий (баз данных/озер данных/хранилищ Lakehouse). Оно берет свое начало в увеличивающемся пересечении операционных и потоковых плоскостей данных. И теперь все больше поставщиков графовых и векторных БД предлагают функции для более прямой интеграции с потоковыми платформами. А некоторые из них также обеспечивают поддержку инкрементного обслуживания материализованных представлений. Такие известные поставщики баз данных, как MongoDB, даже частично превращаются в полноценные потоковые базы данных.

Еще более интересным можно считать продолжающееся сближение аналитической плоскости с потоковой. Решения для хранилищ данных, такие как BigQuery, Redshift и Snowflake, предлагают все больше интересных функций для прямой поддержки потоковой передачи данных. Однако наиболее интересным на данный момент стало слияние потоковой обработки и хранилищ Lakehouse. Здесь мы наблюдаем активное движение с обеих сторон. Оно направлено на органичное объединение потоковой обработки с архитектурой Lakehouse, что приведет к появлению новых концепций, таких как мультимодальные потоки под управлением открытых форматов таблиц, например Apache Iceberg. По нашим наблюдениям, такое стечение обстоятельств может стать серьезным стимулом для внедрения чисто потоковых решений для обработки потоков и, конечно же, потоковых баз данных.

На момент написания этой книги мы находились в самом начале пути. И нам не терпится узнать, как на самом деле будет развиваться история о слиянии операционной, аналитической и потоковой плоскостей.

Об авторах

Хьюберт Дюлей — инженер по системам и данным в компании StarTree. Имеет более 20 лет опыта работы в области больших и быстрых данных, а также MLOps. Хьюберт консультировал многие финансовые учреждения, медицинские организации и телекоммуникационные компании, предлагая простые решения задач с данными. Является автором книги *Streaming Data Mesh* (O'Reilly).

Ральф М. Дебусманн — бывший исследователь в области ИИ/обработки естественного языка, ставший инженером-программистом и архитектором программных решений. Он 15 лет проработал в компаниях SAP, Bosch и Forecasty.AI/BASF SE, после чего в 2023 году начал работать в Migros-Genossenschafts-Bund (Цюрих, Швейцария), где занимает должность инженера Enterprise Kafka. В 2006 году Ральф получил докторскую степень по информатике со специализацией на обработке естественного языка и ИИ (Саарбрюккенский и Эдинбургский университеты).

Иллюстрация на обложке

Животное на обложке — дунайский лосось (*Hucho hucho*). Это крупная рыба семейства лососевых, обитающая в бассейне реки Дунай и на Балканах. Там известна также под названием «младница». Данный вид относится к мигрирующим и нерестится в мелких пресных водах. Длина взрослой особи может достигать до полутора метров, а вес колеблется от 25 до 45 кг. Эти рыбы питаются другими рыбами и насекомыми.

В 2008 году дунайский лосось был занесен в Красную книгу как вид, находящийся под угрозой исчезновения из-за утраты среды обитания. Многие из животных, изображенных на обложках книг издательства O'Reilly, находятся под угрозой исчезновения, и все они важны для нашего мира.

Иллюстрация на обложке выполнена Карен Монтгомери на основе черно-белой гравюры из книги «Королевская естественная история» Лидеккера. Дизайн серии разработали Эди Фридман, Элли Фолькхаузен и Карен Монтгомери.

Алфавитный указатель

А

Аналитическая обработка в режиме
реального времени 12
Атомарность транзакций 69

Б

Барьеры контрольных точек 149

В

Веб-граница 110
Векторные базы данных 240
Внутренняя согласованность 124
Водяные знаки 150
Выразительность SQL-кода 111

Г

Гибридная транзакционно-
аналитическая обработка 64
Гравитация (притяжение) данных 186
Графовые базы данных 235

Д

Двунаправленная (полнодуплексная)
связь 78
Диаграмма Венна 159
Дифференциальный поток данных 52

З

Задержка времени обработки 156
Захват изменений данных 33

И

Изоляция транзакций 69
Индекс звездного дерева 67

Индексирование 72

Инкрементное обслуживание
представлений 170

Инкрементные изменения 84

К

Количество запросов в секунду 71
Коннектор 35
Контрольная точка 24

Л

Локальность данных 197

М

Материализованные представления 29
Механизм обработки (движок) 184
Механизм упреждающей записи 22
Мини-пакетная агрегация 132
Модель публикации и подписки 25
Модуль трансформации 236
Моментальный снимок 34

Н

Немедленное обслуживание
представления 244

О

Обертка сторонних данных 224
Обработка транзакций в режиме
реального времени 12
Обрезка данных 72
Общая задержка 156
Общее табличное выражение 119
Однонаправленная связь 77

Операционная аналитика 197
Операционная база данных 38
Опережающая передача
 результатов 148
Отложенное обслуживание
 представления 245

П

Паттерн zero-ETL 175
Пир (PEER) 178
Платформы потоковой обработки 25
Полное сканирование таблицы 116
Постоянный запрос
 (standing query) 238
Поток изменений 53
Потоки только для добавлений 53
Потоковая сетка данных 210
Потоковый процессор 50
Поток цифровых следов
 пользователя 30
Предметно-ориентированное
 проектирование 31
Предметно-ориентированный
 язык 115
Преобразования с состоянием 43
Приближенные ближайшие
 соседи 240
Протокол HTTP 77
Протокол WebSocket 78

Р

Раздел потоковой платформы 25
Репликация данных 198, 212

С

Сетка данных, data mesh 206
Сжатый топик 93
События 31
События изменений 31

События, отправляемые сервером 77
Совместимость с браузерами 77
Согласованность в конечном счете 124
Согласованность транзакций 69
Соглашение об уровне услуг 22
Состояние гонки 146
Средства разработки программного
 обеспечения 51
Столбцовый формат базы данных 70
Строчный формат базы данных 70
Сущность 32

Т

Таблицы изменений 95
Таблицы только с добавлением 95
Топик 25
Топики изменений 95
Топики только с добавлением 95
Традиционные представления 82
Транзакции 31
Транзакция 23

У

Устойчивость транзакций 69

Ф

Федеративное руководство
 вычислениями 206
Формат потока событий 77

Х

Хранилище состояний 42

Ч

Частые опросы (поллинг) 76

Я

Язык описания данных 117

A

ACID 69

Approximate Nearest
Neighbor, ANN 240

C

Change Data Capture, CDC 33

Common Table Expression, CTE 119

CRUD 68

D

Data Definition Language, DDL 117

Differential Dataflow, DD 52

Domain-Driven Design, DDD 31

Domain-Specific Language, DSL 115

F

Foreign Data Wrapper, FDW 224

H

Hybrid transactional/analytical
processing, HTAP 64

I

Incremental View Maintenance, IVM 170

O

OnLine Analytical Processing, OLAP 12

OnLine Transaction Processing, OLTP 12

Q

Queries Per Second, QPS 71

S

Server-Sent Events, SSE 77

Service Level Agreement, SLA 22

Sink-коннекторы 35

Software Development Kit, SDK 51

Source-коннекторы 35

U

Upsert 91

W

Write-Ahead Log, WAL 22

КРОК

СОЗДАЕМ НАСТОЯЩЕЕ,
ИНТЕГРИРУЕМ БУДУЩЕЕ



croc.ru

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.

